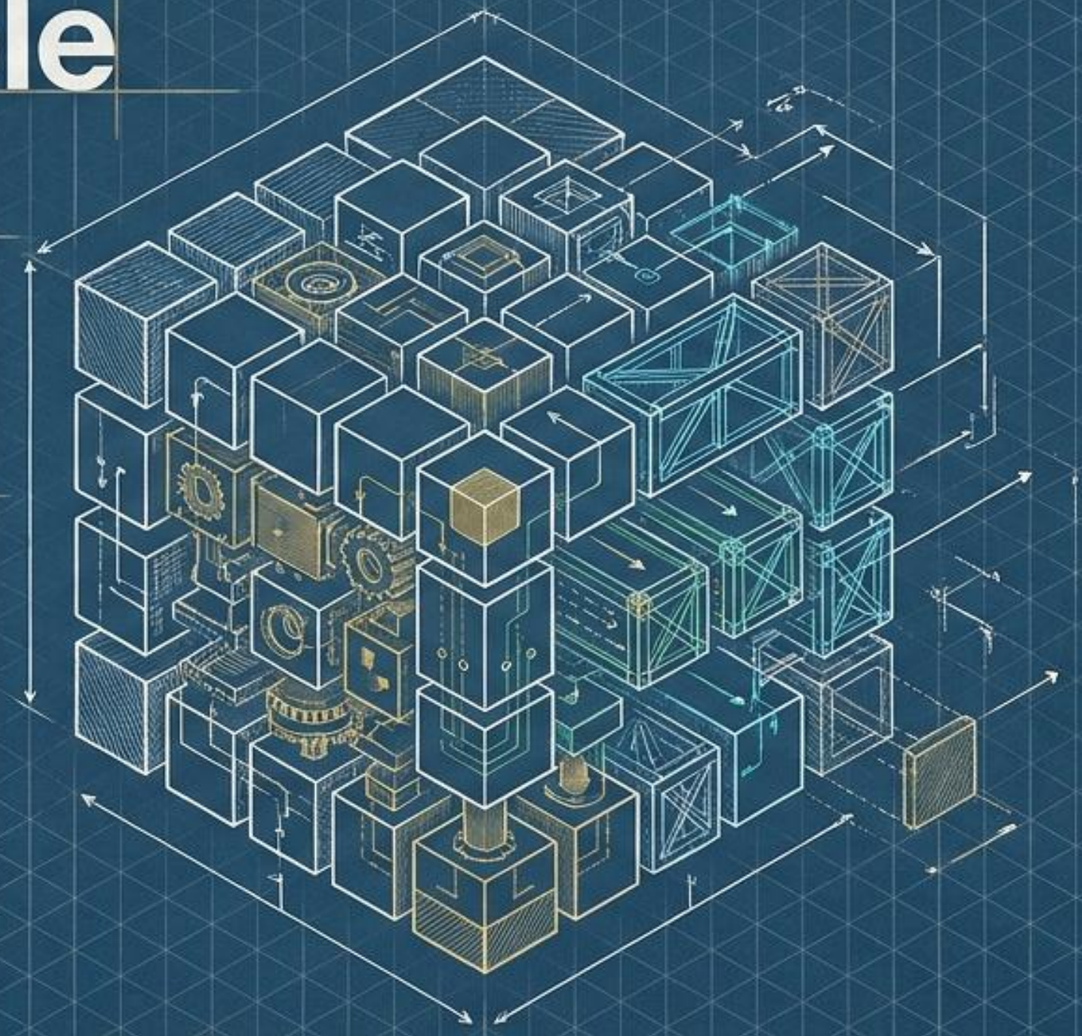
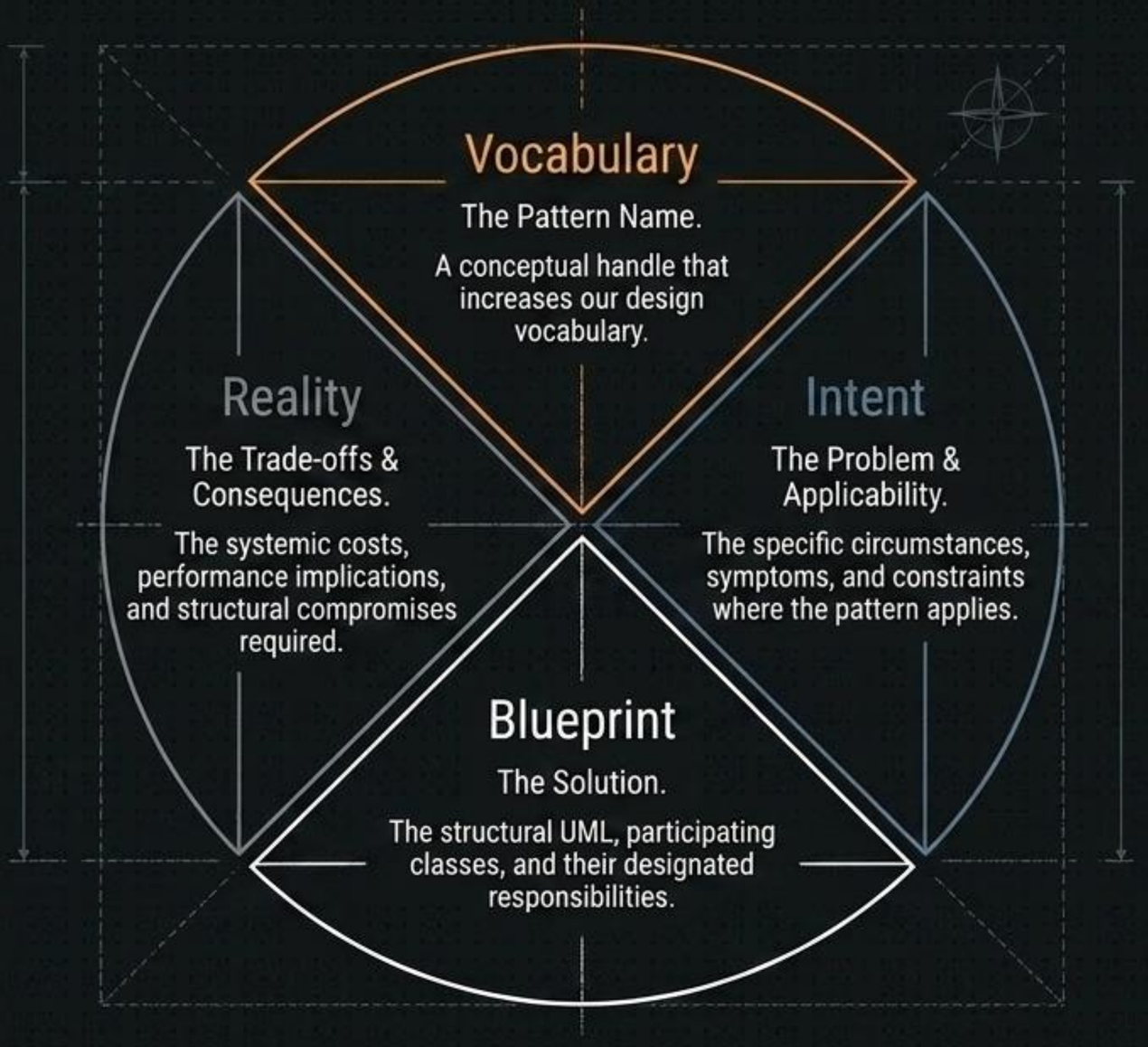
The Architecture of Reusable Software

Core Concepts and Classifications of Design Patterns



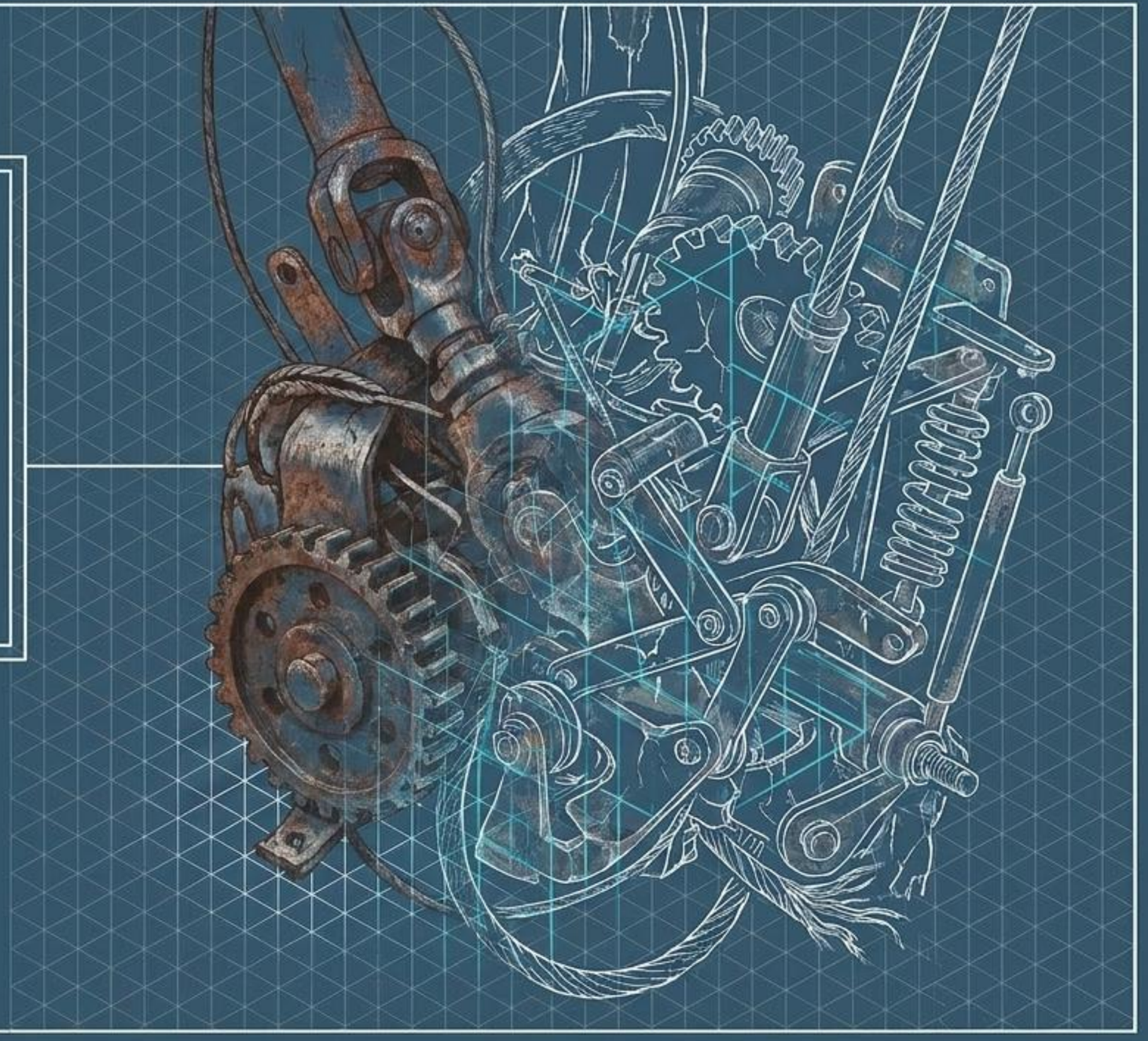
The Anatomy of a Design Pattern



THE BLUEPRINT SOLUTION

Core Definition Panel

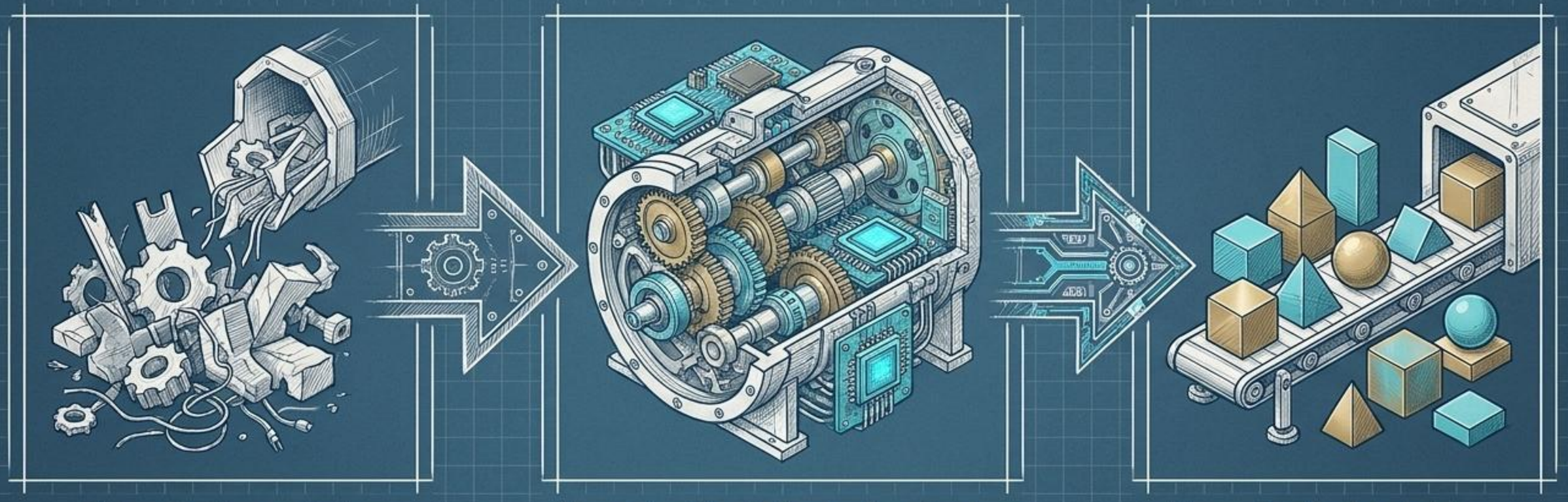
Design Patterns are typical solutions to commonly occurring problems in software design. They are not off-the-shelf code you can copy and paste, but conceptual blueprints that can be customized to solve specific design problems in your architecture.



Key Insight

Originating from the collective experience of top-tier designers, these patterns provide a shared vocabulary, allowing developers to create flexible, elegant, and reusable object-oriented systems without rediscovering the solutions themselves.

The Developer’s Ecosystem



Phase 1: Messy Code

Legacy systems burdened by technical debt and code smells.

Phase 2: The Refactoring Engine

The systematic process of improving code using general principles (like SOLID) to untangle the mess without creating new functionality.

Phase 3: Design Patterns

The target state. Refactored code eventually crystalizes into these proven architectural blueprints.

The Gang of Four Classification Matrix

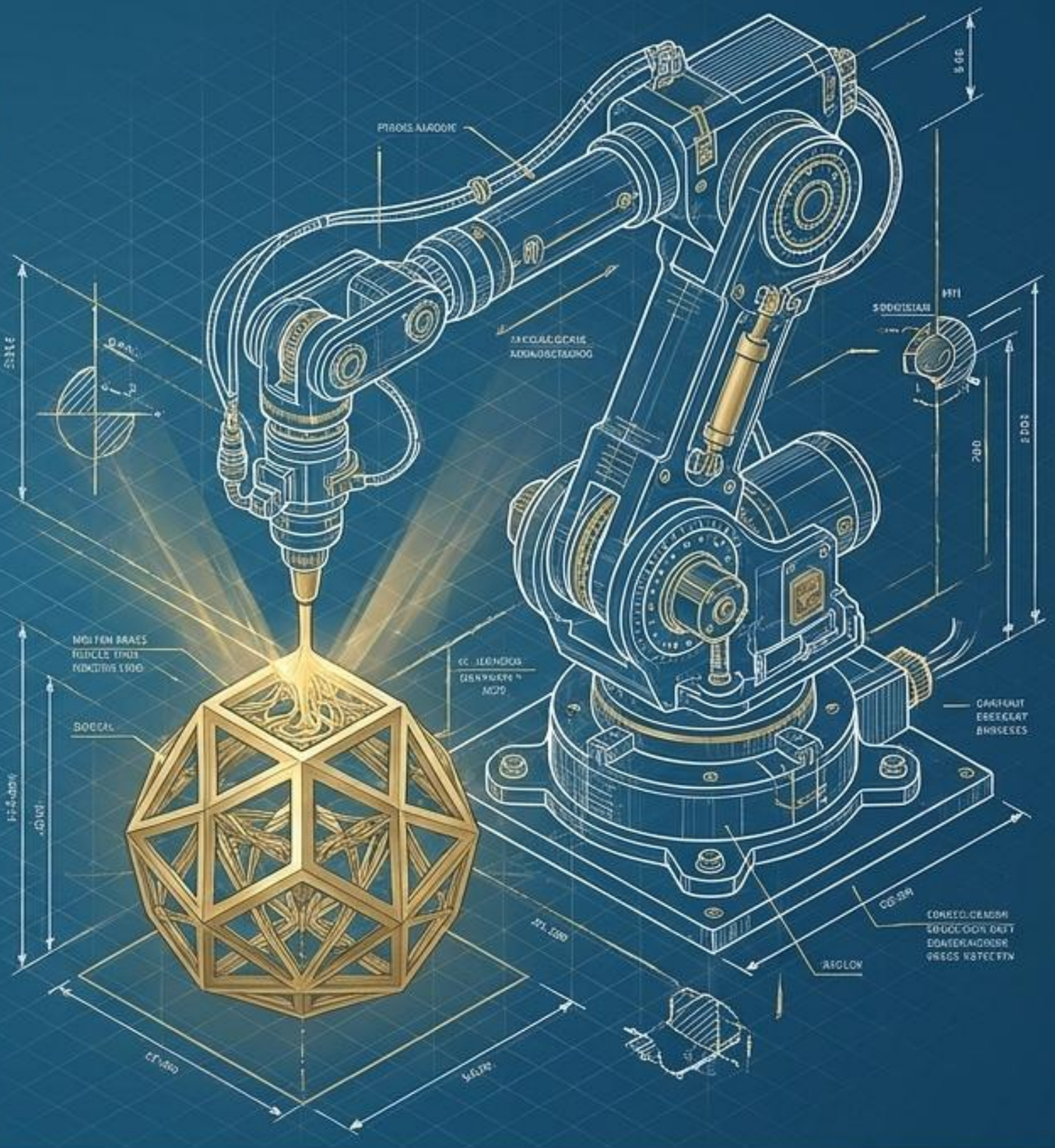
Creational	Structural	Behavioral
Core Intent: How objects are made.	Core Intent: How objects are assembled.	Core Intent: How objects communicate.
What it Abstracts: The instantiation process.	What it Abstracts: Object composition and interfaces.	What it Abstracts: Control flow and assignment of responsibilities.
Real-World Analogy: A high-tech manufacturing plant.	Real-World Analogy: Architectural scaffolding and load-bearing trusses.	Real-World Analogy: A complex switchboard or neural network.

Pillar 1: Creational Patterns

Creational patterns separate the system from how its objects are created, composed, and represented.

The Why

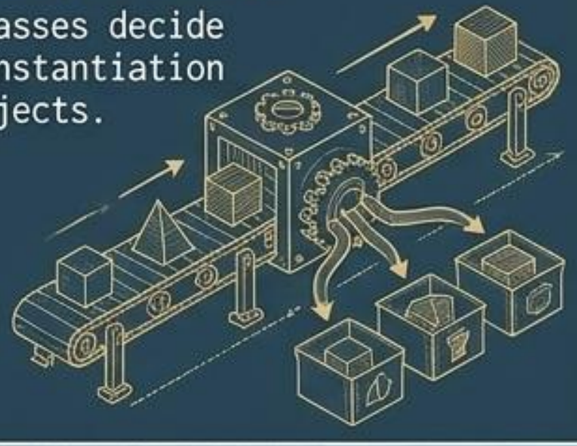
By hiding the complex logic of creation, these patterns make systems independent of specific classes, allowing the software to scale in complexity without tightly coupling the instantiation process to the core business logic.



The Creational Catalog

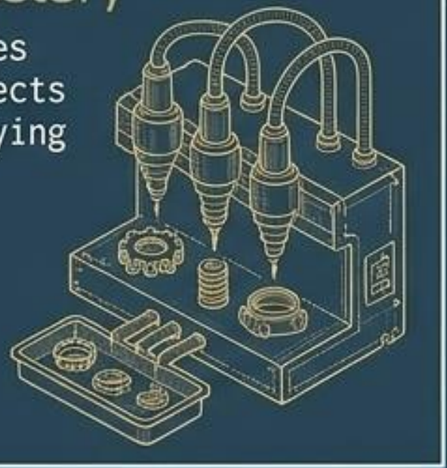
Factory Method

Subclasses decide the instantiation of objects.



Abstract Factory

Creates families of related objects without specifying concrete classes.



Builder

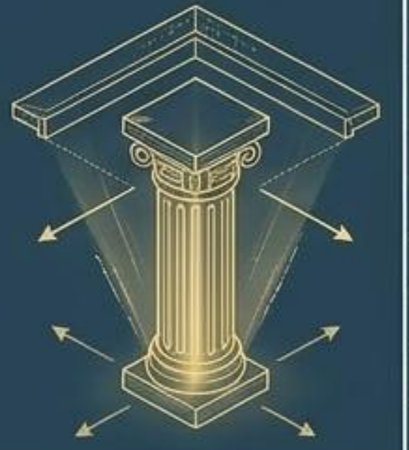
Step-by-step construction of complex objects.

Prototype

Creating new objects by cloning an existing instance.

Singleton

Ensures a class has only one single, globally accessible instance.



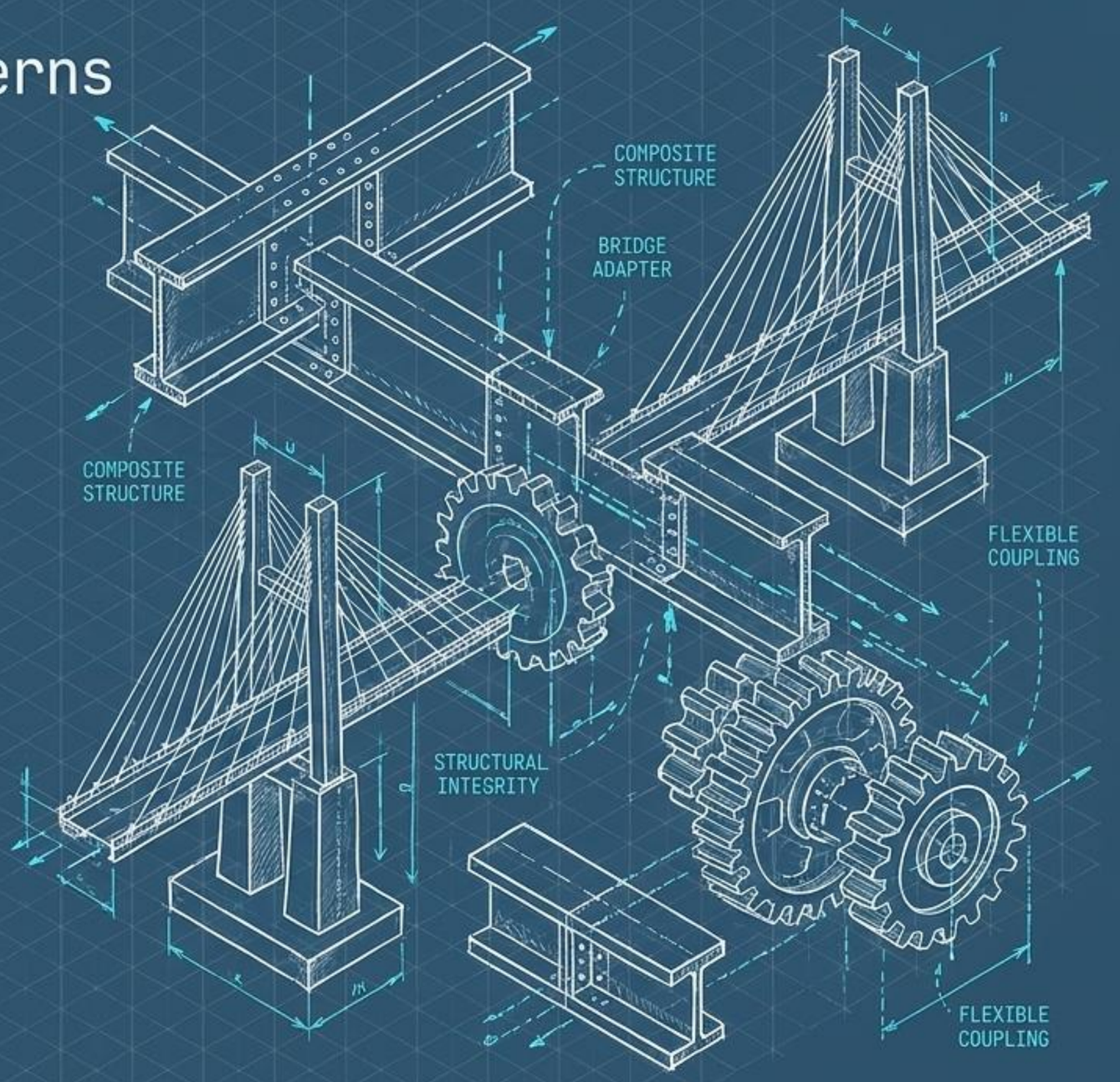
Pillar 2: Structural Patterns

Structural patterns explain how to assemble objects and classes into larger, more complex structures.

The Why

They ensure that even as individual pieces of a system grow and change, the overarching structure remains flexible, efficient, and structurally sound.

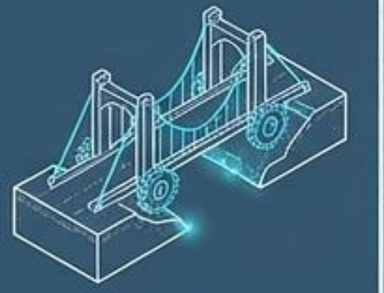
They are the glue and the scaffolding of object-oriented architecture.



The Structural Catalog

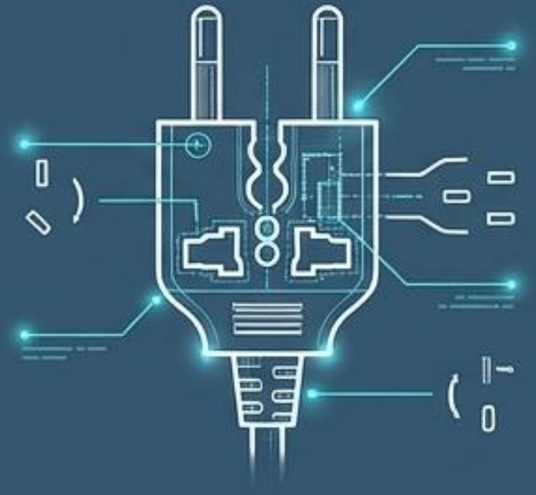
Bridge

Separates abstraction from implementation.



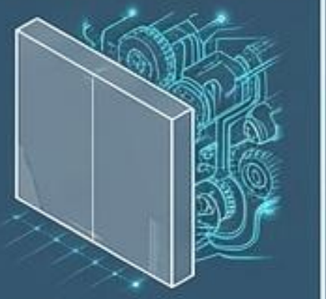
Adapter

Makes incompatible interfaces collaborate.



Facade

Simplified interface to a complex subsystem.

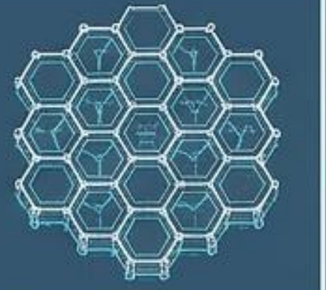


Composite

Composes objects into tree structures.

Flyweight

Shares state to support massive numbers of objects efficiently.

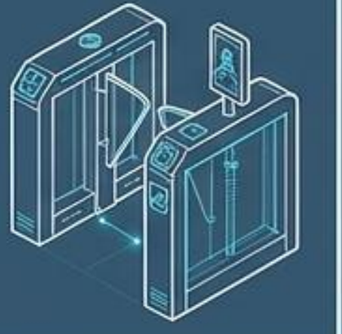


Decorator

Attaches new behaviors dynamically.

Proxy

Placeholder to control access to another object.

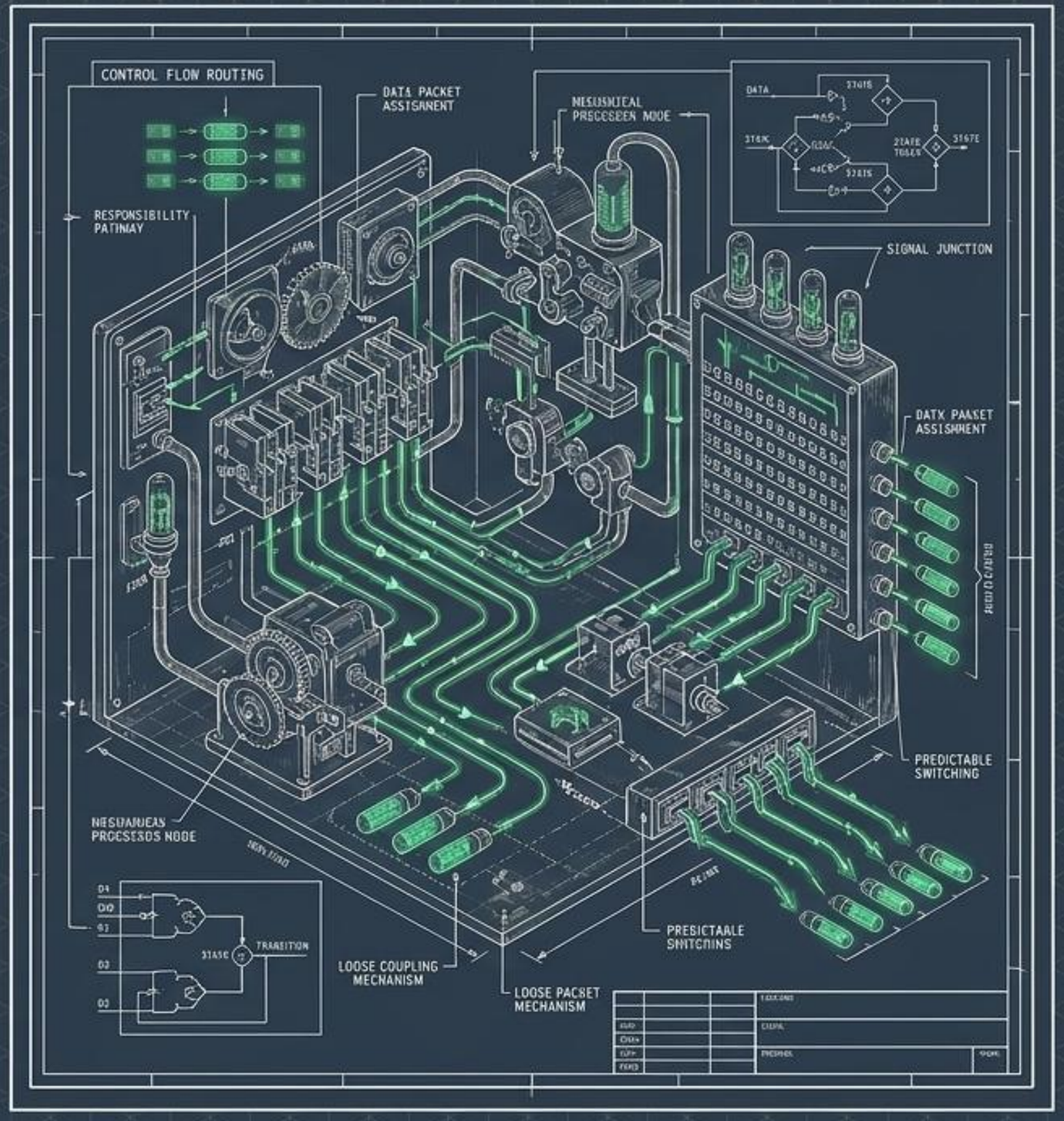


Pillar 3: Behavioral Patterns

Behavioral patterns focus entirely on algorithms and the assignment of responsibilities between objects.

The Why

They dictate not just what objects are, but how they talk to each other. They untangle the communication web, ensuring that control flow between distinct entities remains predictable, scalable, and loosely coupled.



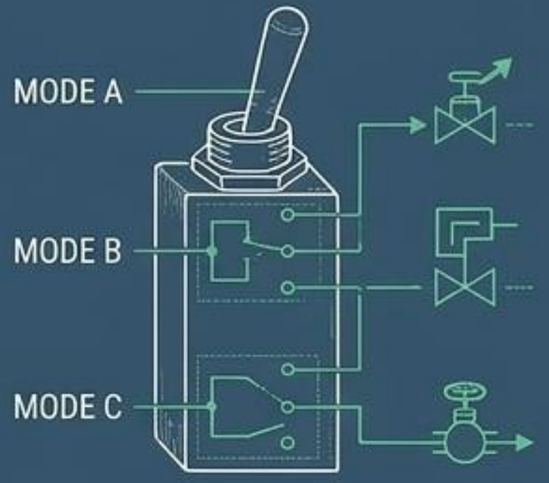
The Behavioral Catalog

Observer

Defines a subscription mechanism to notify objects of state changes.

State

Alters an object's behavior when its internal state changes.

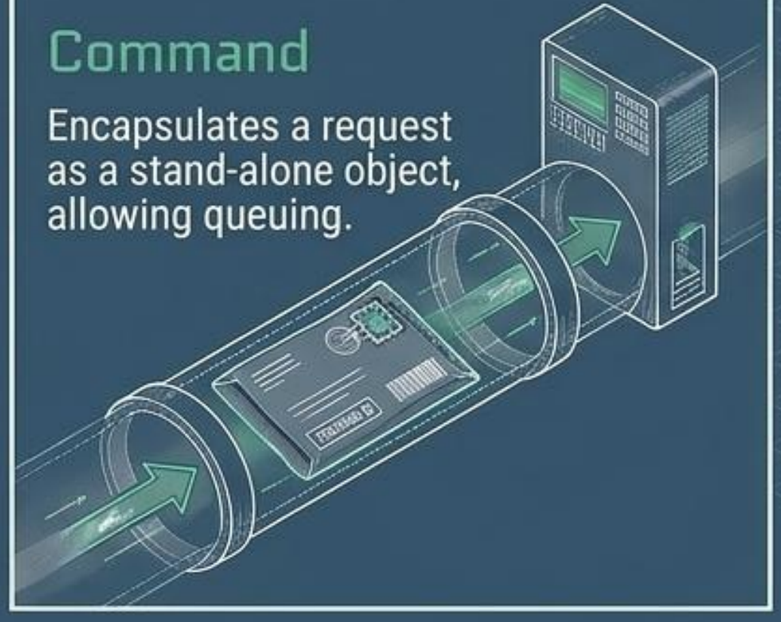


Strategy

Encapsulates a family of algorithms, making them interchangeable.

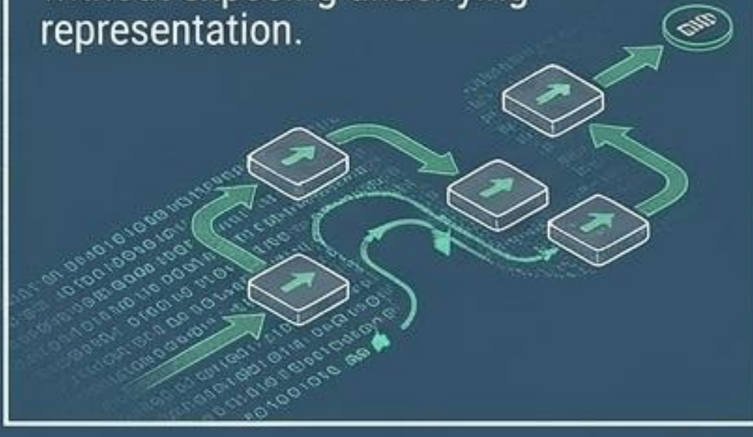
Command

Encapsulates a request as a stand-alone object, allowing queuing.

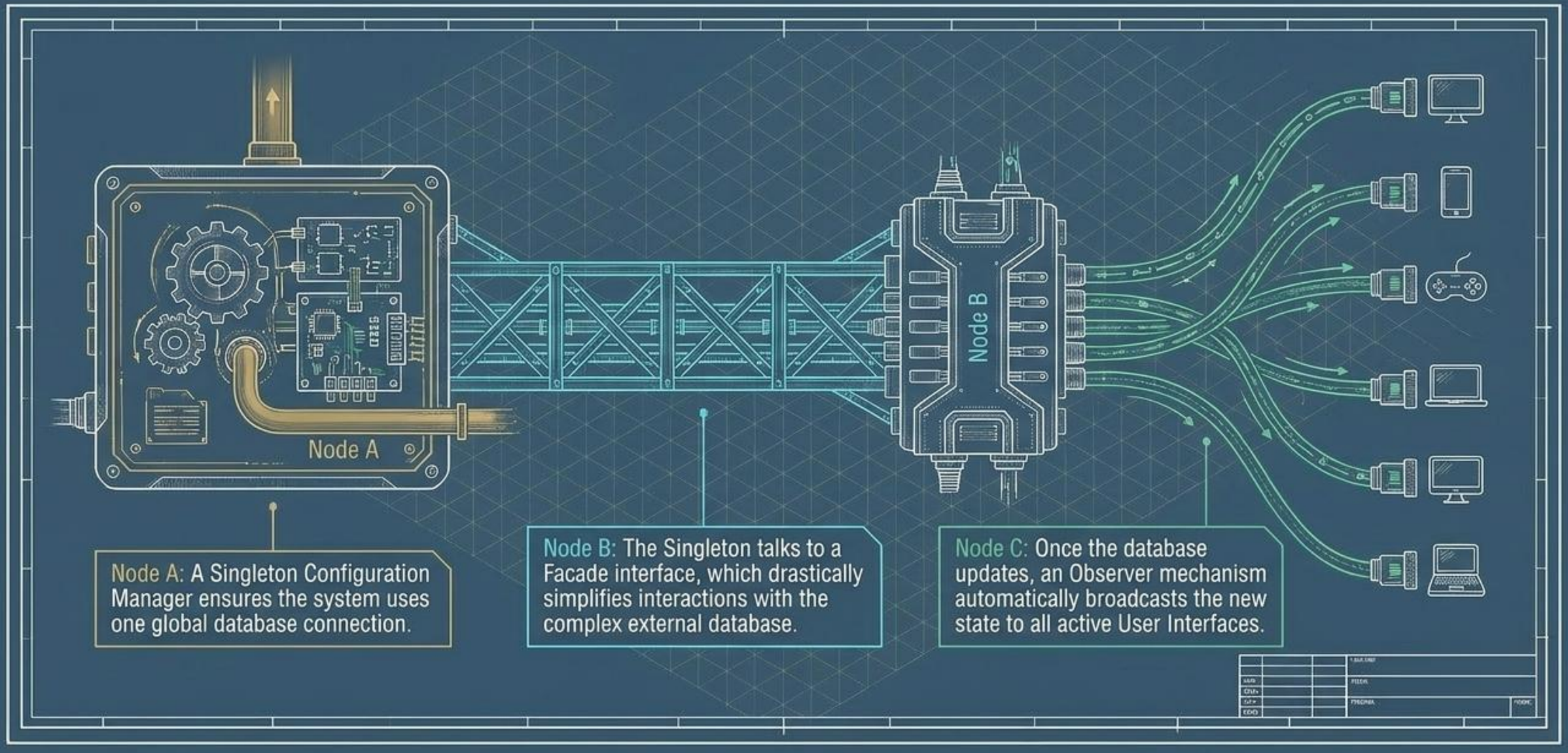


Iterator

Traverses elements of a collection without exposing underlying representation.



Patterns in Concert



Node A: A Singleton Configuration Manager ensures the system uses one global database connection.

Node B: The Singleton talks to a Facade interface, which drastically simplifies interactions with the complex external database.

Node C: Once the database updates, an Observer mechanism automatically broadcasts the new state to all active User Interfaces.

The True Value of Patterns

A Shared Vocabulary

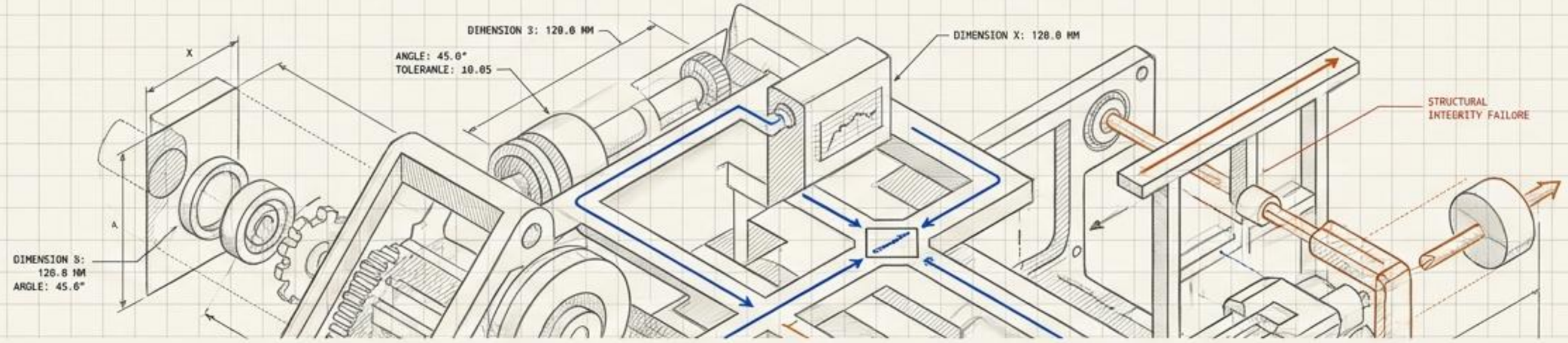
Saying “Just use a Factory” instantly communicates a dozen complex architectural decisions in four words.

Eradication of Reinvention

Leveraging 20 years of collective software engineering experience to solve recurring problems immediately.

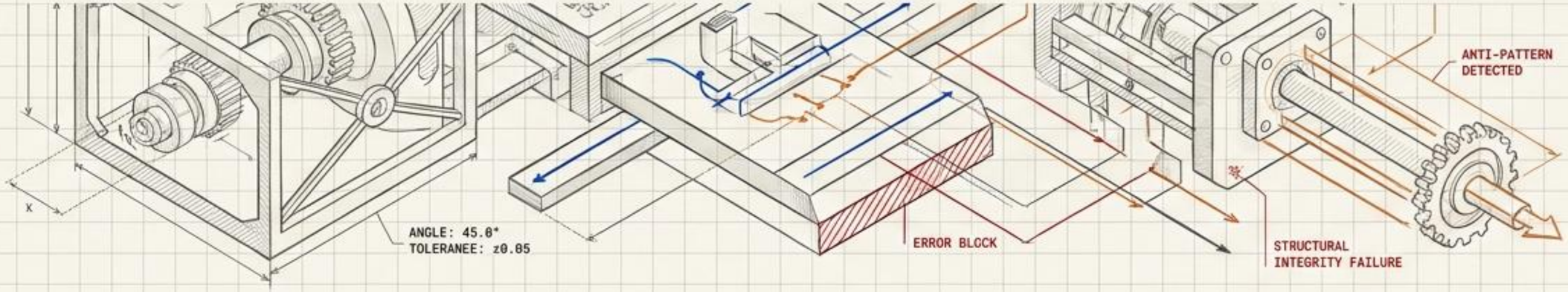
Resilient Architecture

Future-proofing codebases against changing requirements through intentional, reusable object-oriented design.

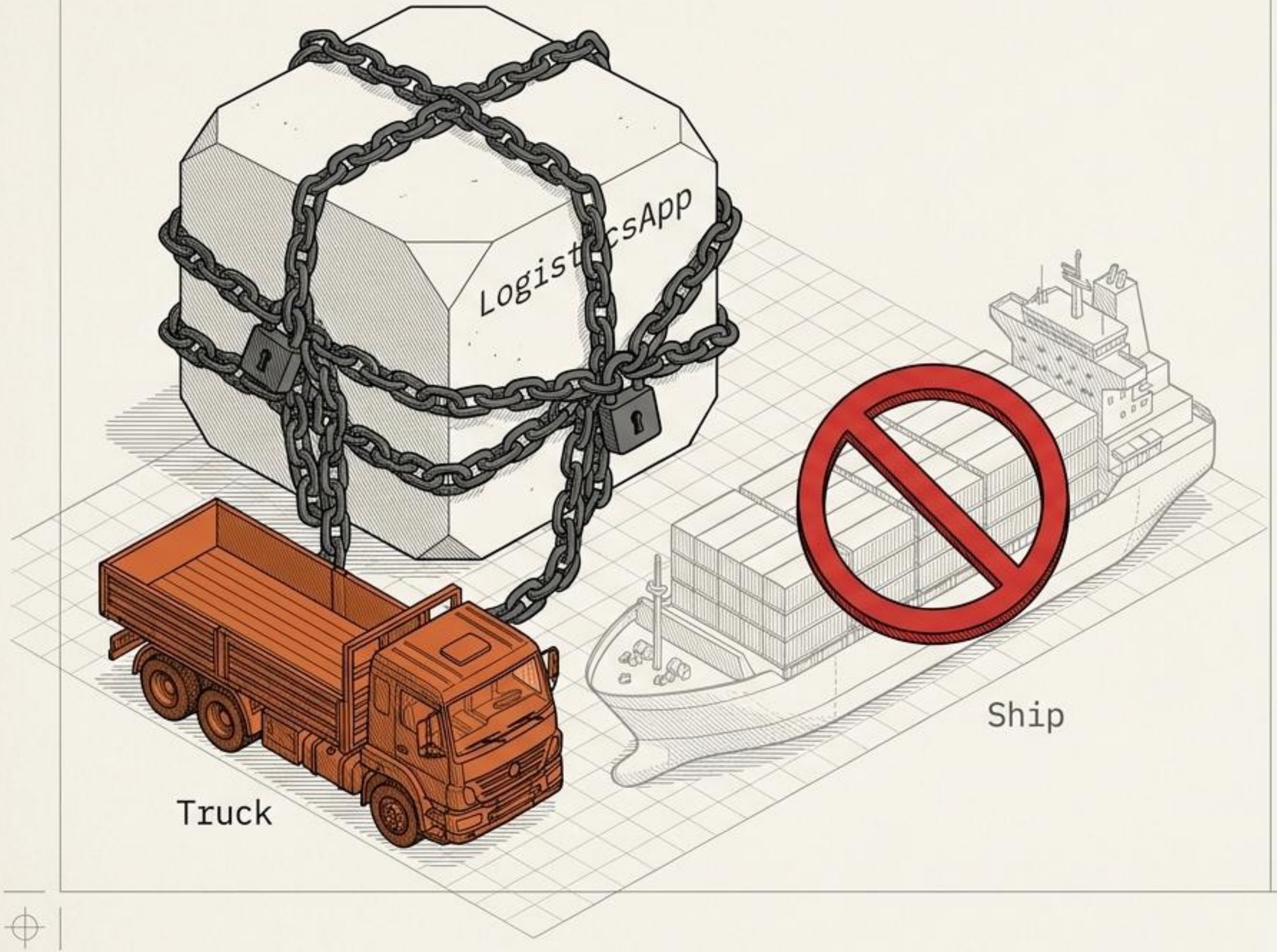


CREATIONAL DESIGN PATTERNS

Architecting Object Creation: A Diagnostic Blueprint



FACTORY METHOD • ABSTRACT FACTORY • BUILDER • PROTOTYPE • SINGLETON



The Rigid Dependency

SYMPTOM

Core business logic is entangled with the new keyword.

CONSTRAINT

The system is brittle. Adding new product types requires modifying existing, tested code.

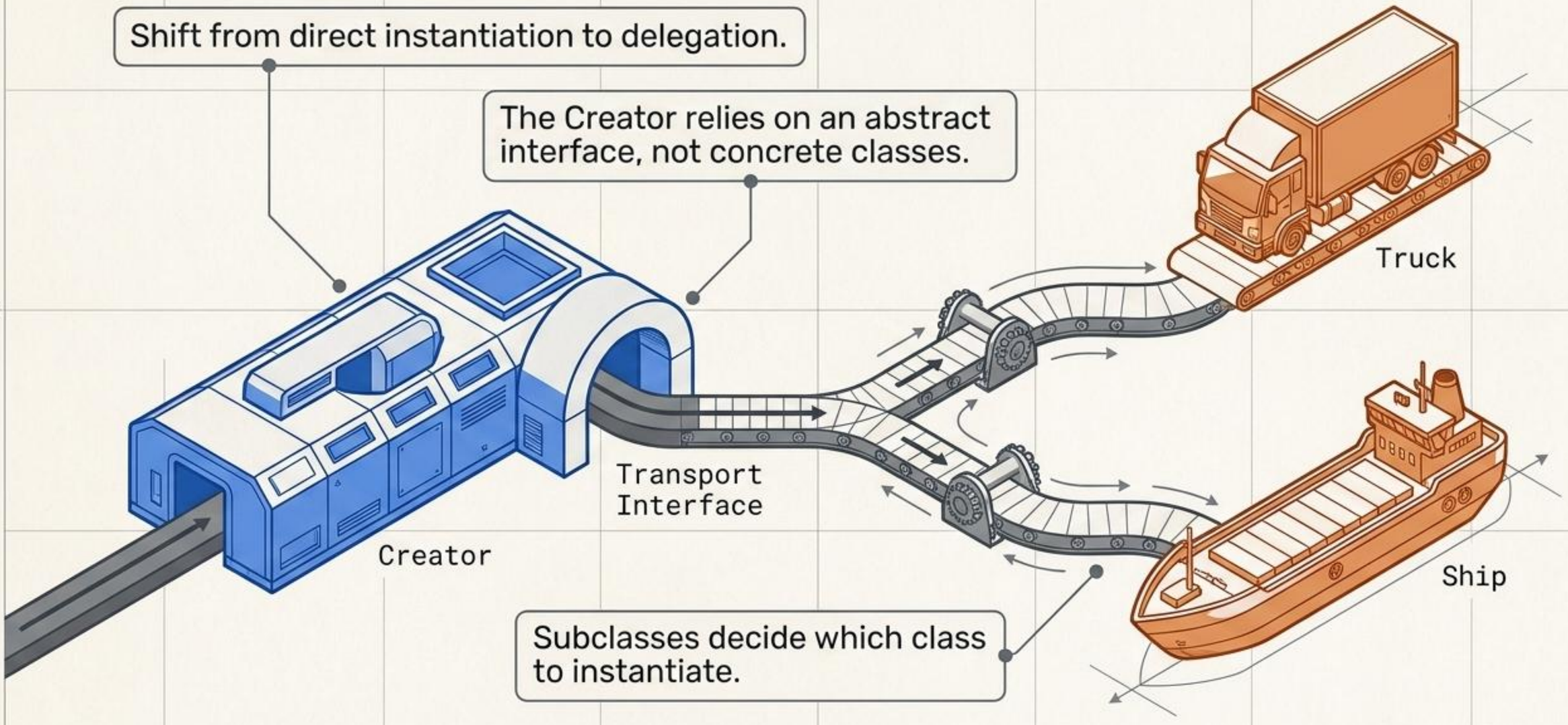
RESULT

Closed to extension, violating the Open/Closed Principle.

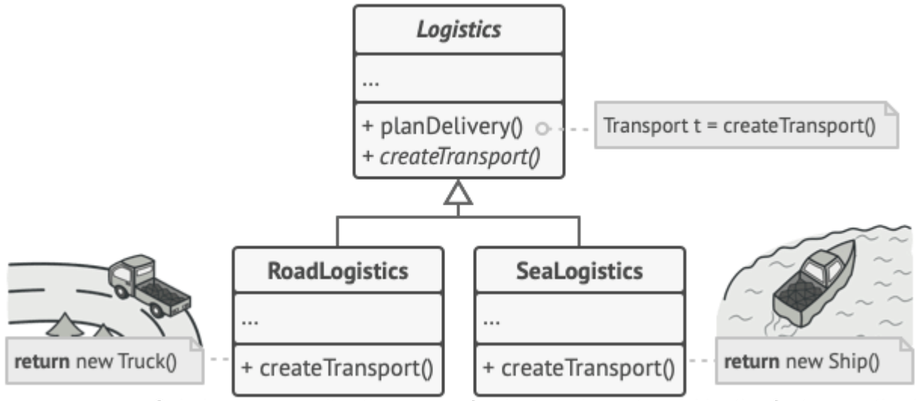
The Virtual Constructor

Shift from direct instantiation to delegation.

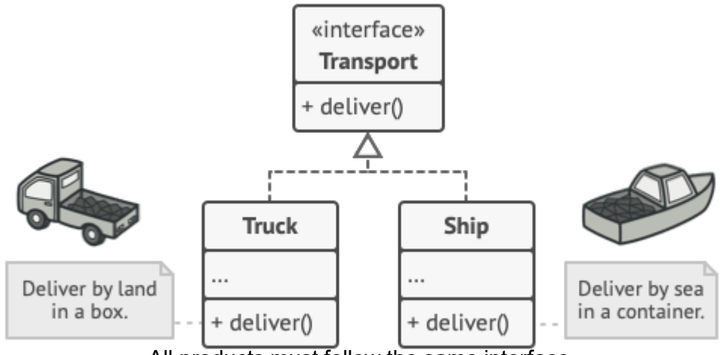
The Creator relies on an abstract interface, not concrete classes.



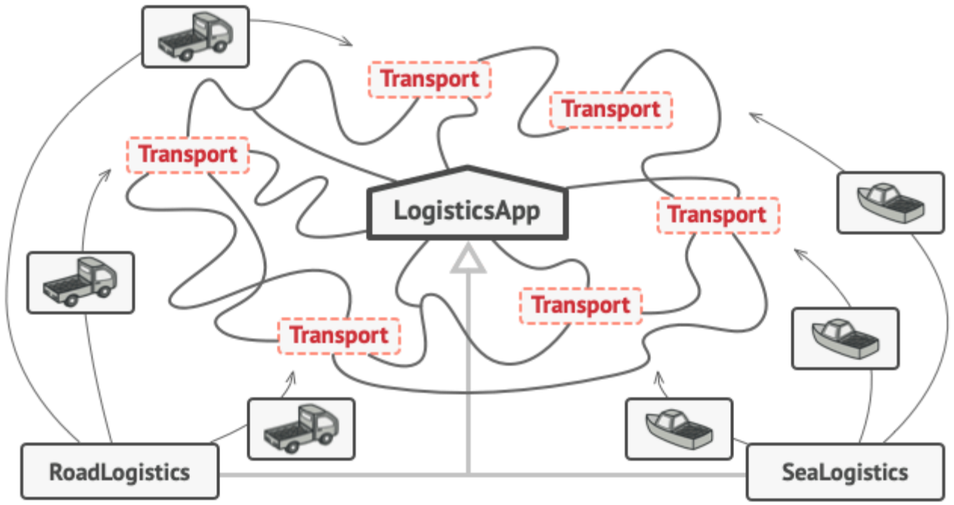
Subclasses decide which class to instantiate.



Subclasses can alter the class of objects being returned by the factory method.

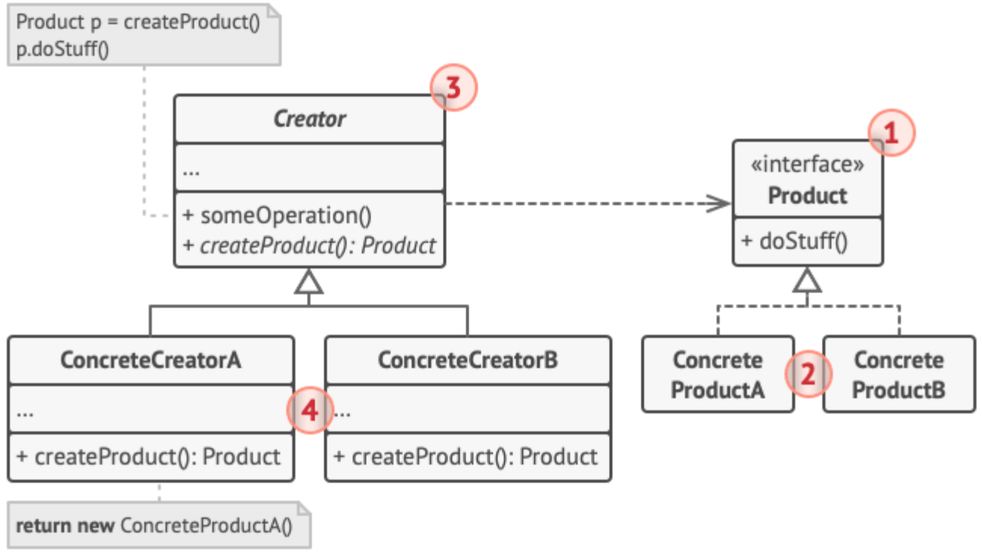


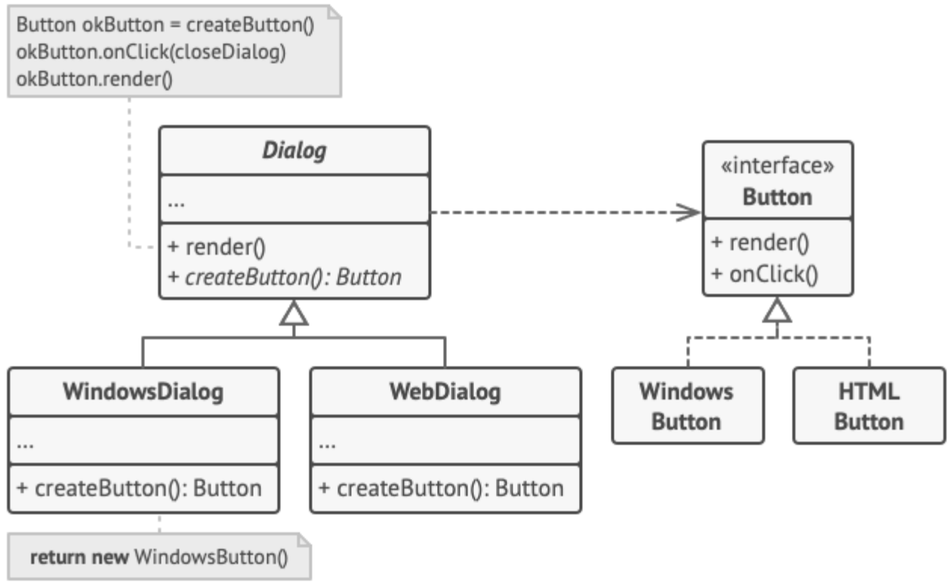
All products must follow the same interface.



As long as all product classes implement a common interface, you can pass their objects to the client code without breaking it.

Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.





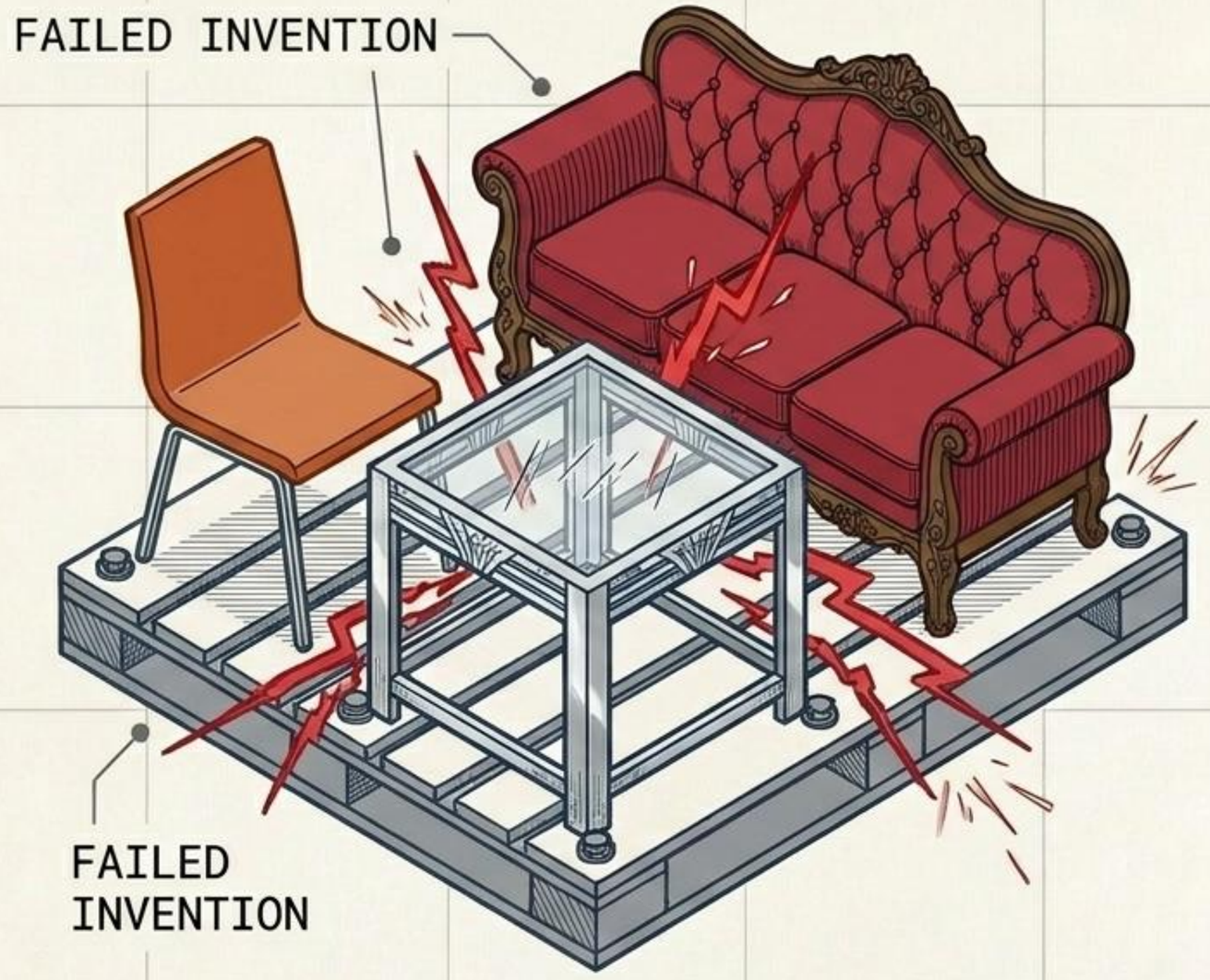
The cross-platform dialog example.

```
// The creator class declares the factory
method that must
// return an object of a product class.
The creator's subclasses
// usually provide the implementation of
this method.
class Dialog is
    // The creator may also provide some
default implementation
    // of the factory method.
    abstract method createButton():Button

    // Note that, despite its name, the
creator's primary
    // responsibility isn't creating
products. It usually
    // contains some core business logic
that relies on product
    // objects returned by the factory
method. Subclasses can
    // indirectly change that business
logic by overriding the
    // factory method and returning a
different type of product
    // from it.
    method render() is
        // Call the factory method to
create a product object.
        Button okButton = createButton()
        // Now use the product.
        okButton.onClick(closeDialog)
        okButton.render()
// Concrete creators override the factory
method to change the
// resulting product's type.
class WindowsDialog extends Dialog is
    method createButton():Button is
        return new WindowsButton()

class WebDialog extends Dialog is
    method createButton():Button is
        return new HTMLButton()

// The product interface declares the
operations that all
// concrete products must implement.
interface Button is
    method render()
    method onClick(f)

// Concrete products provide various
implementations of the
// product interface.
class WindowsButton implements Button is
    method render(a, b) is
        // Render a button in Windows
style.
    method onClick(f) is
        // Bind a native OS click event.

class HTMLButton implements Button is
    method render(a, b) is
        // Return an HTML representation
of a button.
    method onClick(f) is
        // Bind a web browser click
event.

class Application is
    field dialog: Dialog

    // The application picks a creator's
type depending on the
    // current configuration or
environment settings.
    method initialize() is
        config =
readApplicationConfigFile()

        if (config.OS == "Windows") then
            dialog = new WindowsDialog()
        else if (config.OS == "Web") then
            dialog = new WebDialog()
        else
            throw new Exception("Error!
Unknown operating system.")

    // The client code works with an
instance of a concrete
    // creator, albeit through its base
interface. As long as
    // the client keeps working with the
creator via the base
    // interface, you can pass it any
creator's subclass.
    method main() is
        this.initialize()
        dialog.render()
```

```
// The creator class declares the factory method that must
// return an object of a product class. The creator's subclasses
// usually provide the implementation of this method.
class Dialog is
    // The creator may also provide some default implementation
    // of the factory method.
    abstract method createButton():Button

    // Note that, despite its name, the creator's primary
    // responsibility isn't creating products. It usually
    // contains some core business logic that relies on product
    // objects returned by the factory method. Subclasses can
    // indirectly change that business logic by overriding the
    // factory method and returning a different type of product
    // from it.
    method render() is
        // Call the factory method to create a product object.
        Button okButton = createButton()
        // Now use the product.
        okButton.onClick(closeDialog)
        okButton.render()


// Concrete creators override the factory method to change the
// resulting product's type.
class WindowsDialog extends Dialog is
    method createButton():Button is
        return new WindowsButton()

class WebDialog extends Dialog is
    method createButton():Button is
        return new HTMLButton()


// The product interface declares the operations that all
// concrete products must implement.
interface Button is
    method render()
    method onClick(f)
```

```
// The product interface declares the operations that all
// concrete products must implement.
interface Button is
    method render()
    method onClick(f)

// Concrete products provide various implementations of the
// product interface.
class WindowsButton implements Button is
    method render(a, b) is
        // Render a button in Windows style.
    method onClick(f) is
        // Bind a native OS click event.

class HTMLButton implements Button is
    method render(a, b) is
        // Return an HTML representation of a button.
    method onClick(f) is
        // Bind a web browser click event.


class Application is
    field dialog: Dialog

    // The application picks a creator's type depending on the
    // current configuration or environment settings.
    method initialize() is
        config = readApplicationConfigFile()

        if (config.OS == "Windows") then
            dialog = new WindowsDialog()
        else if (config.OS == "Web") then
            dialog = new WebDialog()
        else
            throw new Exception("Error! Unknown operating system.")

    // The client code works with an instance of a concrete
    // creator, albeit through its base interface. As long as
    // the client keeps working with the creator via the base
    // interface, you can pass it any creator's subclass.
    method main() is
        this.initialize()
        dialog.render()
```

The Mismatched Family



SYMPTOM

Instantiating related objects from different families randomly throughout the codebase.

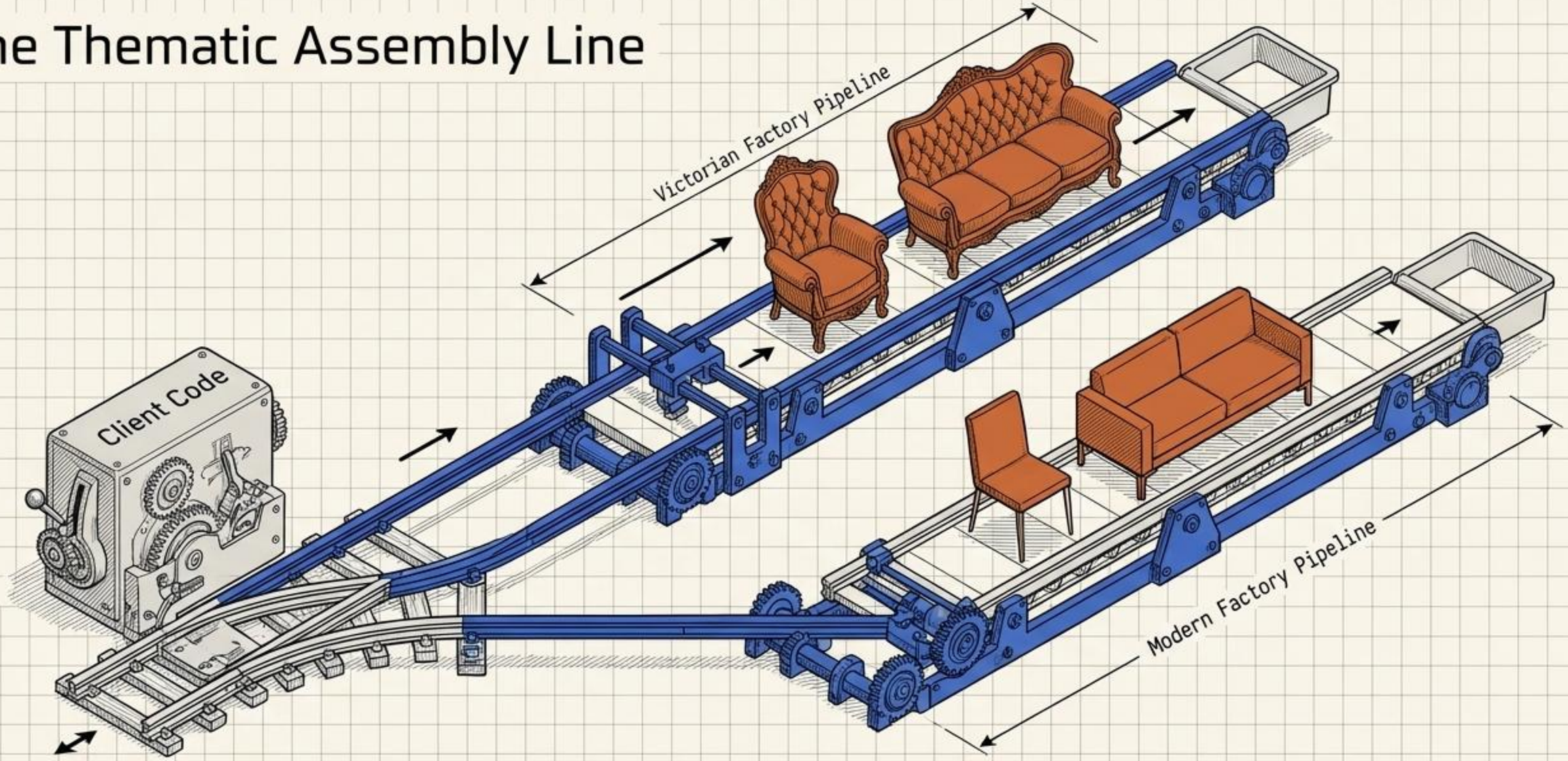
CONSTRAINT

Products must be guaranteed to match their counterparts, but concrete classes are hardcoded.

RESULT

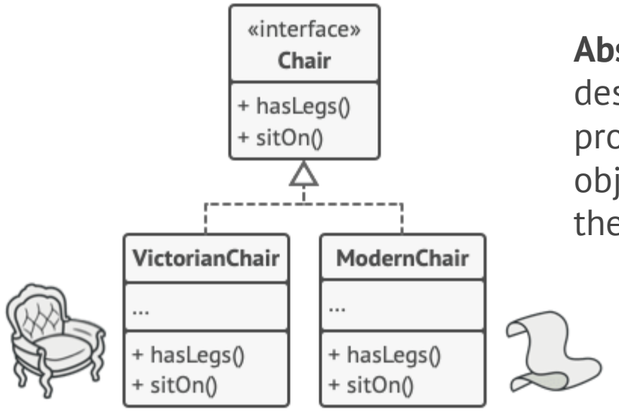
Visual and functional inconsistencies; updating a product family breaks client code.

The Thematic Assembly Line



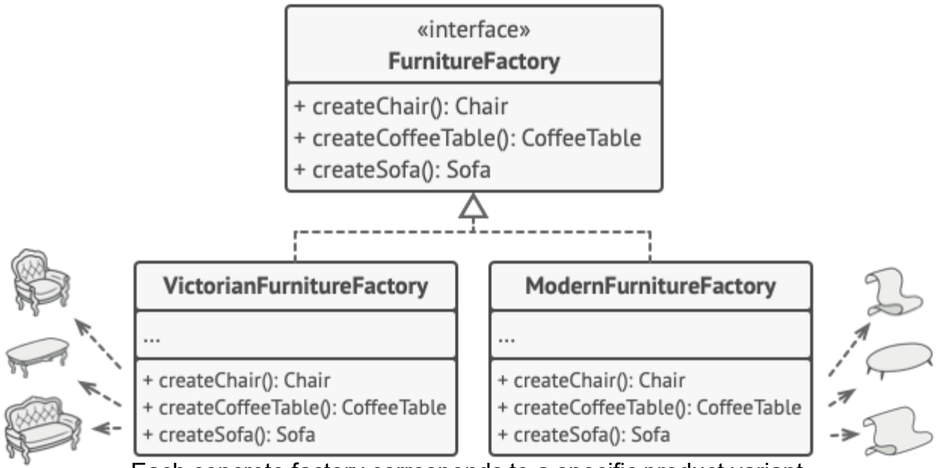
ARCHITECTURAL LABEL	

Encapsulate individual factories into a single “super-factory” interface. The client code works with any concrete factory/ product variant through their abstract interfaces, guaranteeing 10000% compatibility.

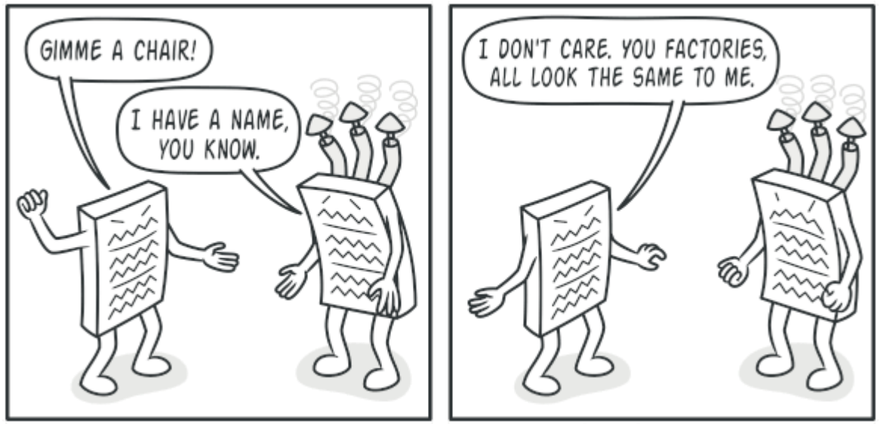


Abstract Factory is a creational design pattern that lets you produce families of related objects without specifying their concrete classes.

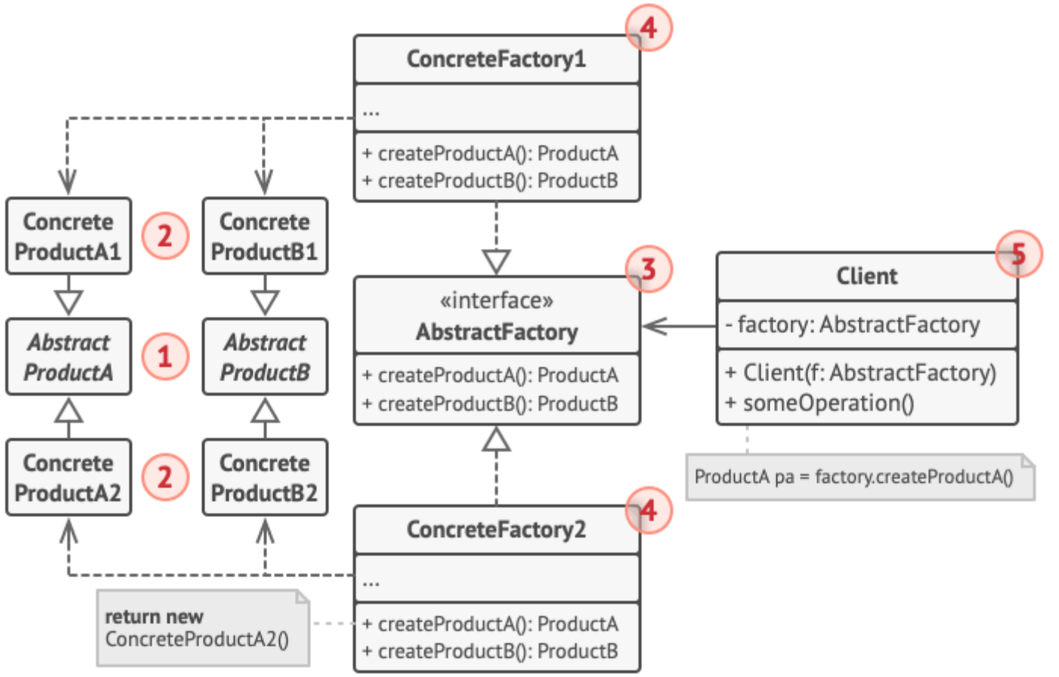
All variants of the same object must be moved to a single class hierarchy.

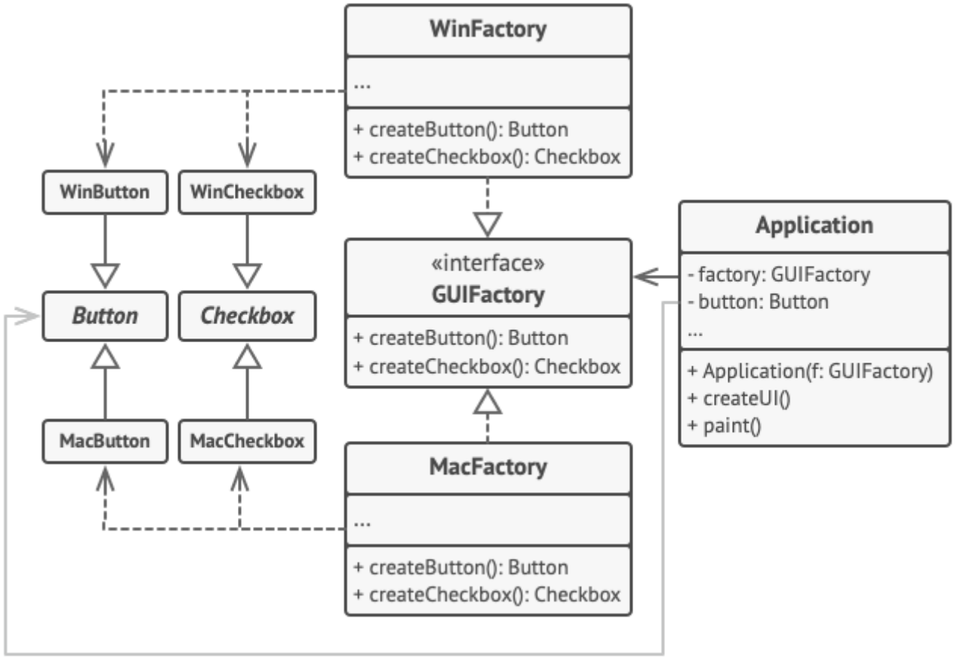


Each concrete factory corresponds to a specific product variant.



The client shouldn't care about the concrete class of the factory it works with.



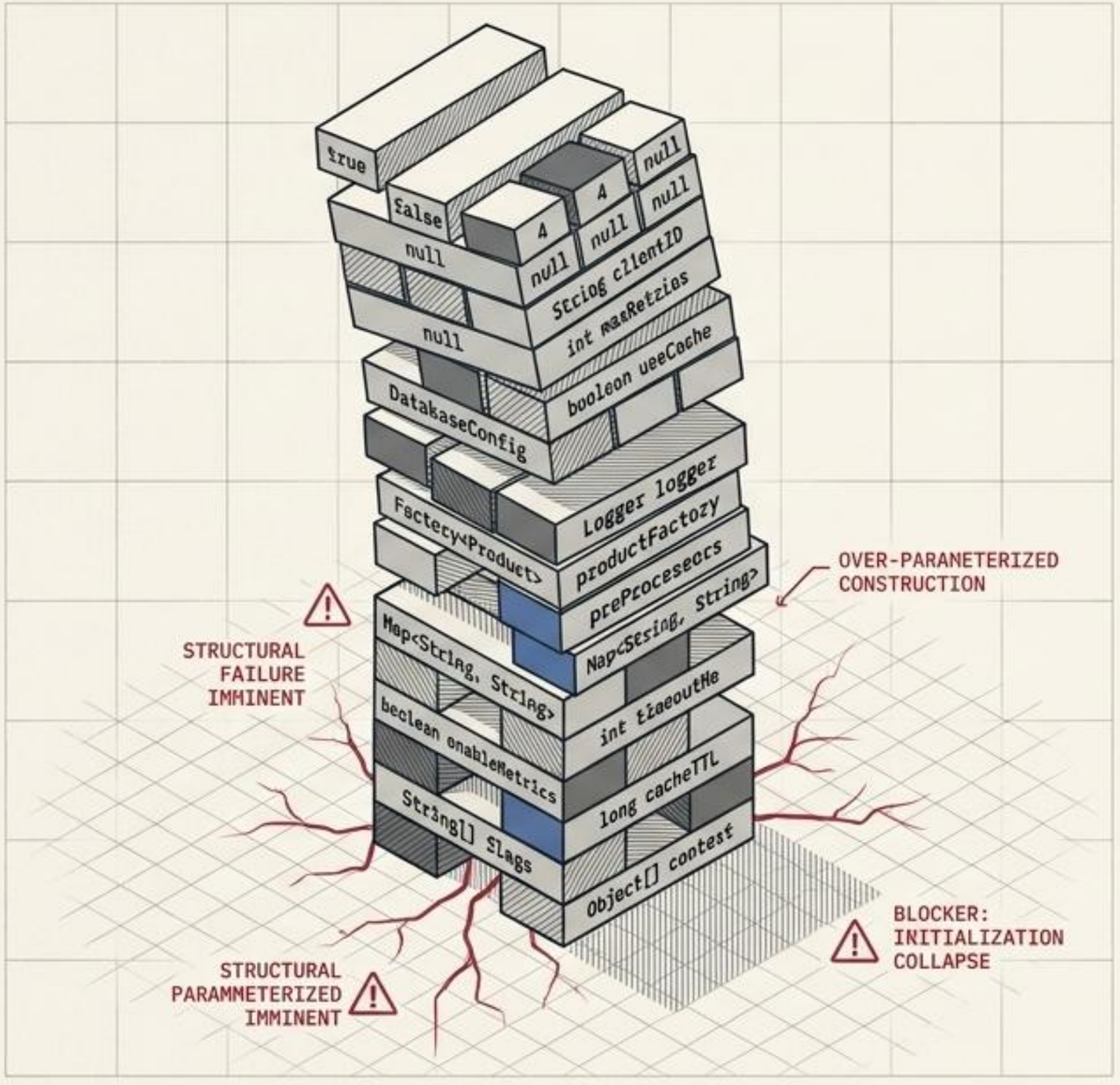


The cross-platform UI classes example.

```
// The abstract factory interface declares a set of methods that
// return different abstract products. These products are called
// a family and are related by a high-level theme or concept.
// Products of one family are usually able to collaborate among
// themselves. A family of products may have several variants,
// but the products of one variant are incompatible with the
// products of another variant.
interface GUIFactory is
    method createButton():Button
    method createCheckbox():Checkbox


// Concrete factories produce a family of products that belong
// to a single variant. The factory guarantees that the
// resulting products are compatible. Signatures of the concrete
// factory's methods return an abstract product, while inside
// the method a concrete product is instantiated.
class WinFactory implements GUIFactory is
    method createButton():Button is
        return new WinButton()
    method createCheckbox():Checkbox is
        return new WinCheckbox()

// Each concrete factory has a corresponding product variant.
class MacFactory implements GUIFactory is
    method createButton():Button is
        return new MacButton()
    method createCheckbox():Checkbox is
        return new MacCheckbox()
```

```
// Each distinct product of a product family should have a base
// interface. All variants of the product must implement this
// interface.
interface Button is
    method paint()

// Concrete products are created by corresponding concrete
// factories.
class WinButton implements Button is
    method paint() is
        // Render a button in Windows style.

class MacButton implements Button is
    method paint() is
        // Render a button in macOS style.

// Here's the base interface of another product. All products
// can interact with each other, but proper interaction is
// possible only between products of the same concrete variant.
interface Checkbox is
    method paint()

class WinCheckbox implements Checkbox is
    method paint() is
        // Render a checkbox in Windows style.

class MacCheckbox implements Checkbox is
    method paint() is
        // Render a checkbox in macOS style.


// The client code works with factories and products only
// through abstract types: GUIFactory, Button and Checkbox. This
// lets you pass any factory or product subclass to the client
// code without breaking it.
class Application is
    private field factory: GUIFactory
    private field button: Button
    constructor Application(factory: GUIFactory) is
        this.factory = factory
    method createUI() is
        this.button = factory.createButton()
    method paint() is
        button.paint()


// The application picks the factory type depending on the
// current configuration or environment settings and creates it
// at runtime (usually at the initialization stage).
class ApplicationConfigurator is
    method main() is
        config = readApplicationConfigFile()

        if (config.OS == "Windows") then
            factory = new WinFactory()
        else if (config.OS == "Mac") then
            factory = new MacFactory()
        else
            throw new Exception("Error! Unknown operating system.")

        Application app = new Application(factory)
```

The Telescoping Monolith



SYMPTOM

The Telescoping Constructor anti-pattern—dozens of parameters, mostly nulls.

CONSTRAINT

Creating a complex object requires massive initialization code buried inside a monster subclass hierarchy or gigantic constructor.

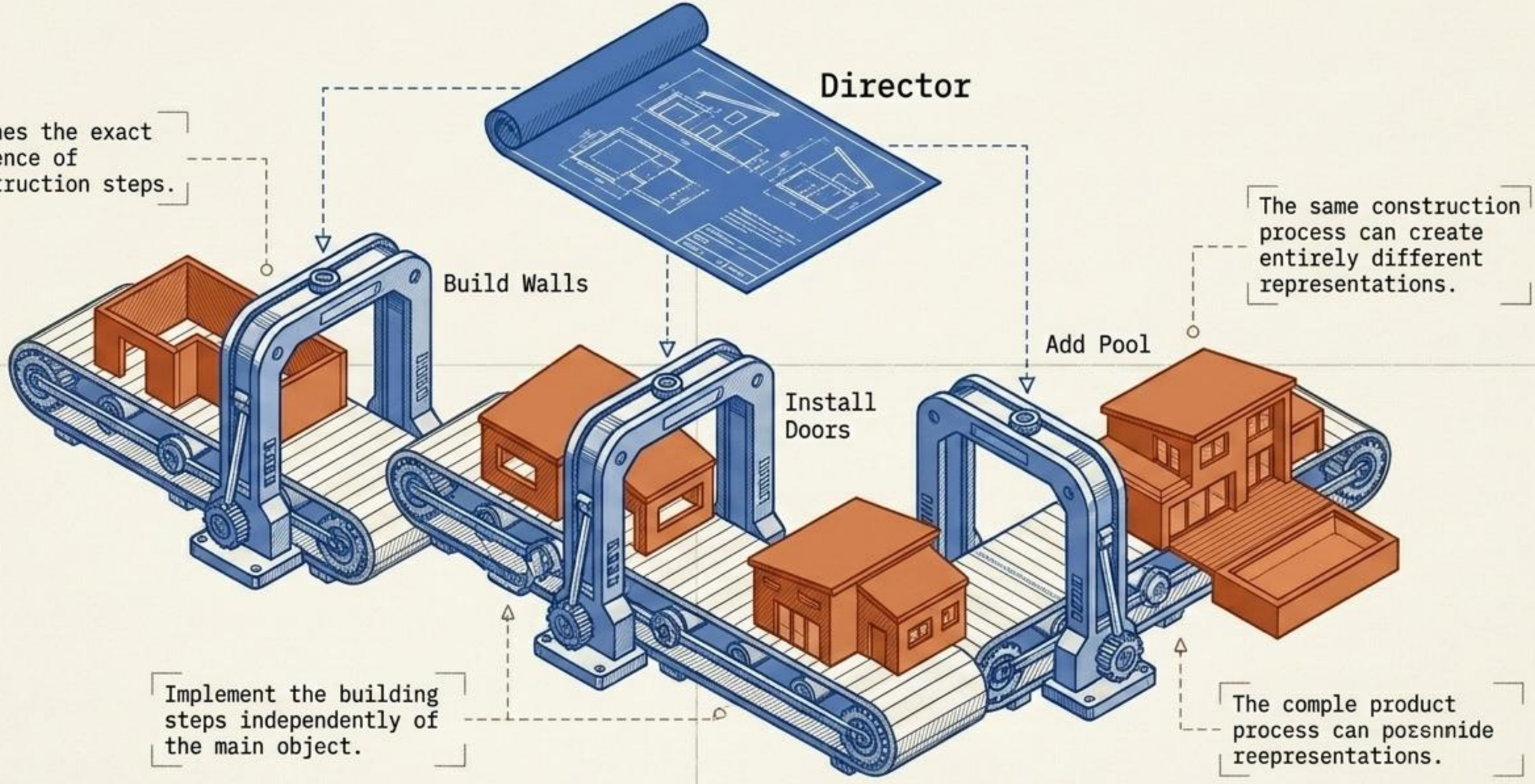
RESULT

Unreadable, error-prone initialization that cannot be easily varied.

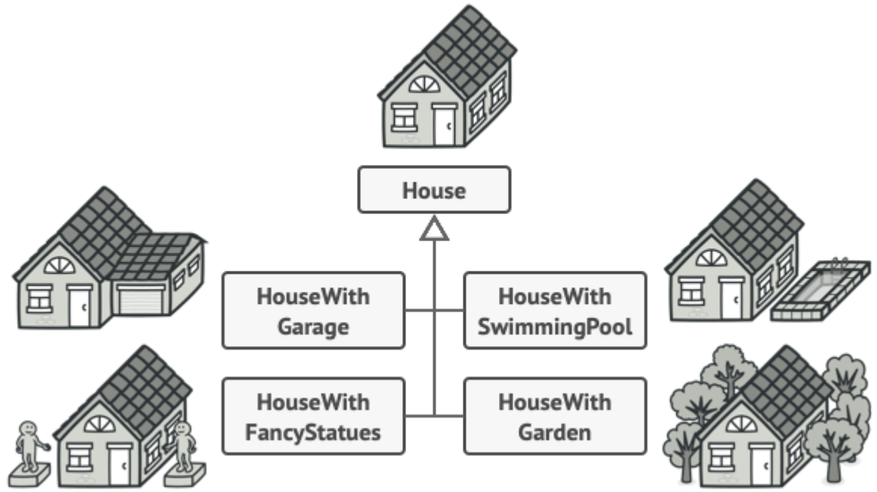
ARCHITECTURAL NOTE

This anti-pattern often necessitates a builder or factory to abstract away complex initialization, improving readability and maintainability by reducing the cognitive load of construction.

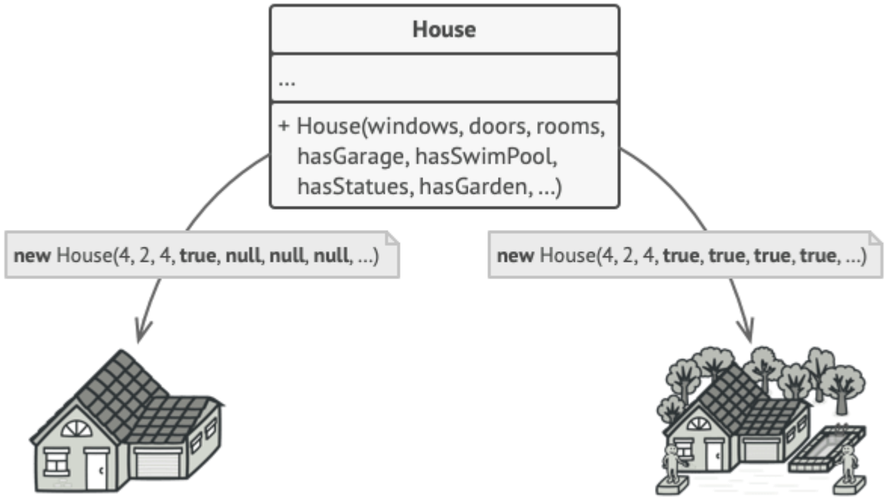
Orchestrated Assembly Steps



Builder is a creational design pattern that lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code.

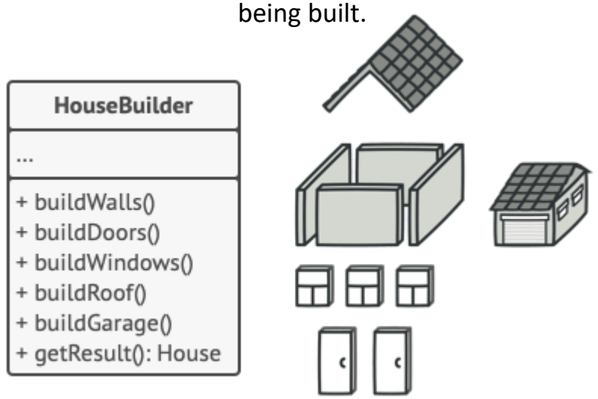


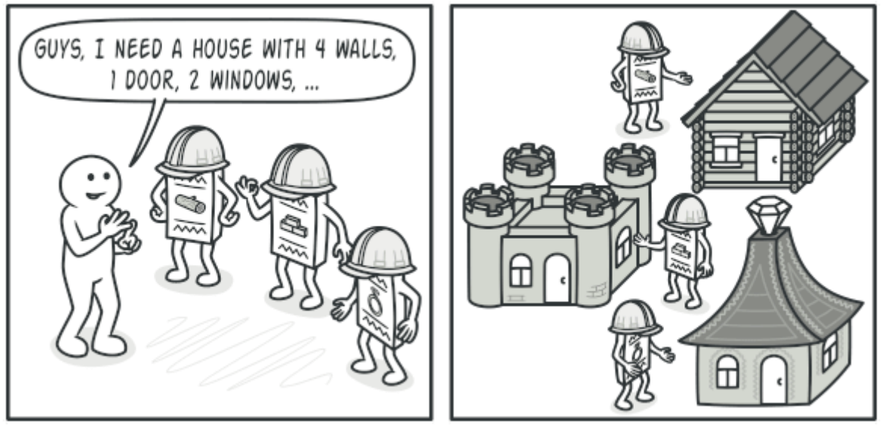
You might make the program too complex by creating a subclass for every possible configuration of an object.



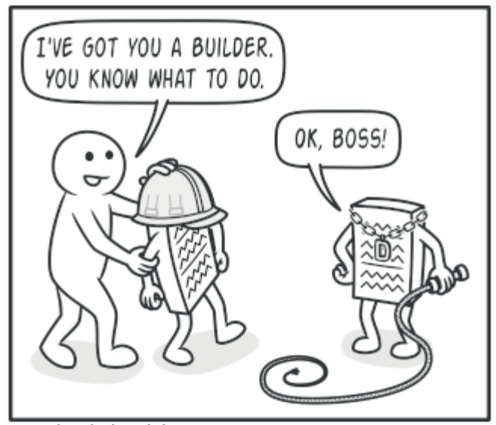
The constructor with lots of parameters has its downside: not all the parameters are needed at all times.

The Builder pattern lets you construct complex objects step by step. The Builder doesn't allow other objects to access the product while it's being built.

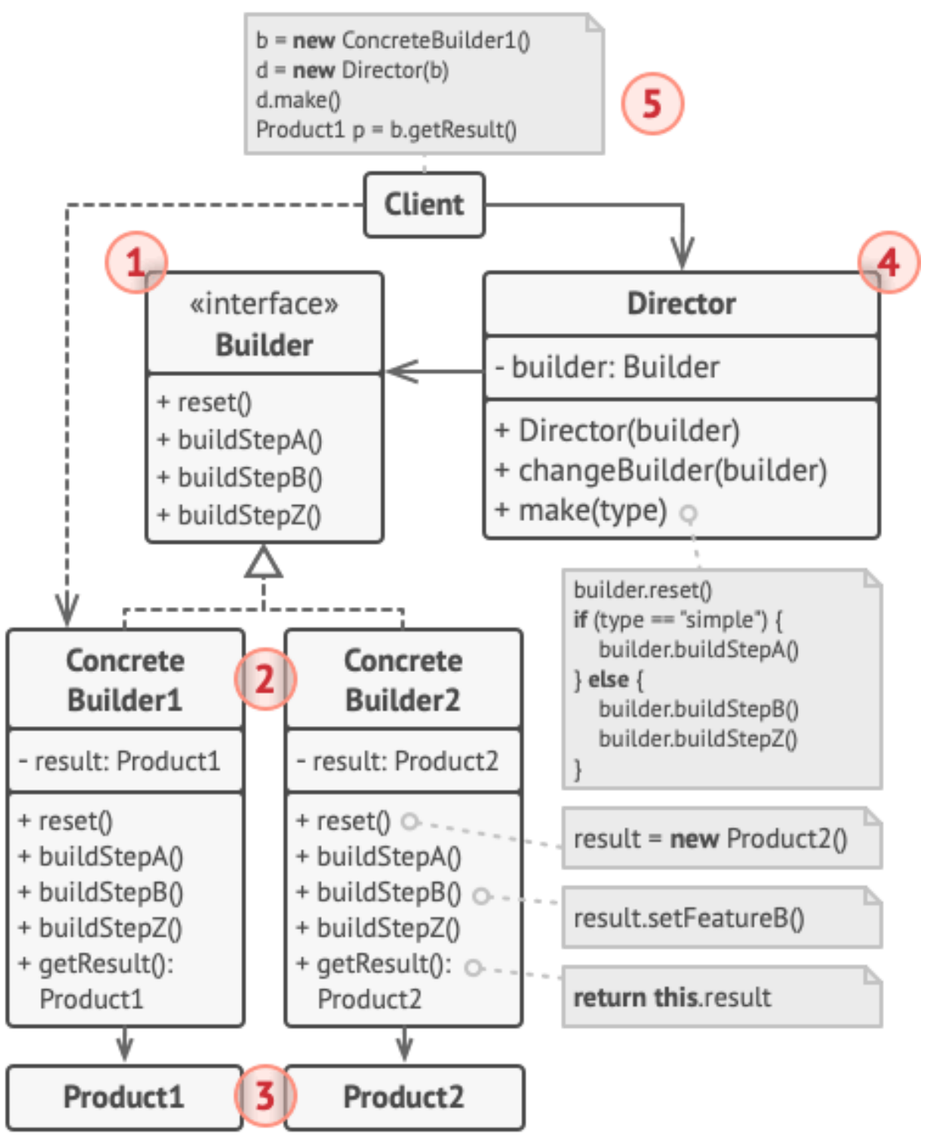


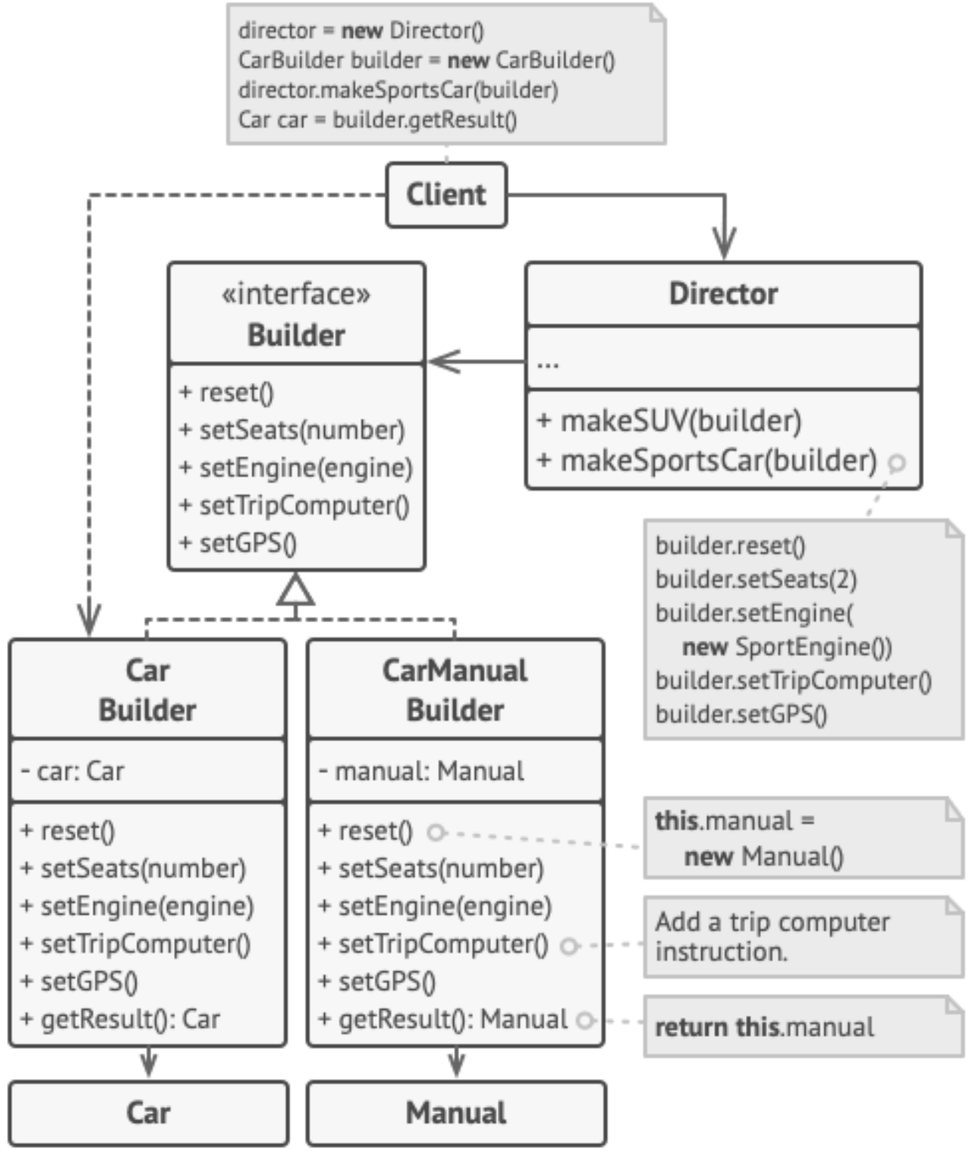


Different builders execute the same task in various ways.



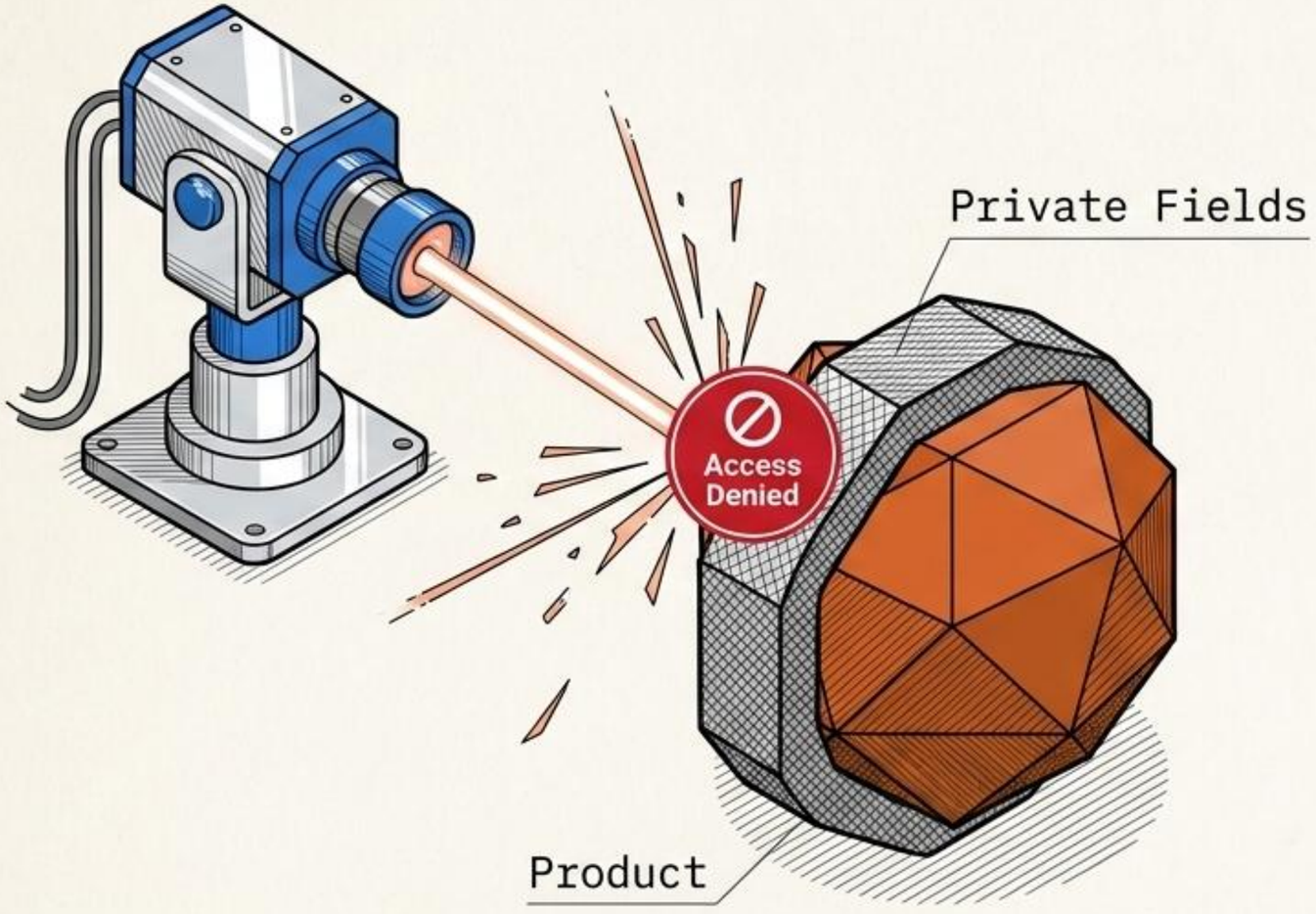
The director knows which building steps to execute to get a working product.


b = new ConcreteBuilder1()
d = new Director(b)
d.make()
Product1 p = b.getResult()
5
Client
1
«interface»
Builder
+ reset()
+ buildStepA()
+ buildStepB()
+ buildStepZ()
4
Director
- builder: Builder
+ Director(builder)
+ changeBuilder(builder)
+ make(type)
builder.reset()
if (type == "simple") {
builder.buildStepA()
} else {
builder.buildStepB()
builder.buildStepZ()
}
Concrete
Builder1
2
Concrete
Builder2
- result: Product1
- result: Product2
+ reset()
+ buildStepA()
+ buildStepB()
+ buildStepZ()
+ getResult():
Product1
+ reset()
+ buildStepA()
+ buildStepB()
+ buildStepZ()
+ getResult():
Product2
result = new Product2()
result.setFeatureB()
return this.result
Product1
3
Product2


director = new Director()
CarBuilder builder = new CarBuilder()
director.makeSportsCar(builder)
Car car = builder.getResult()
Client
«interface»
Builder
+ reset()
+ setSeats(number)
+ setEngine(engine)
+ setTripComputer()
+ setGPS()
Director
...
+ makeSUV(builder)
+ makeSportsCar(builder)
builder.reset()
builder.setSeats(2)
builder.setEngine(
new SportEngine())
builder.setTripComputer()
builder.setGPS()
Car
Builder
- car: Car
+ reset()
+ setSeats(number)
+ setEngine(engine)
+ setTripComputer()
+ setGPS()
+ getResult(): Car
CarManual
Builder
- manual: Manual
+ reset()
+ setSeats(number)
+ setEngine(engine)
+ setTripComputer()
+ setGPS()
+ getResult(): Manual
this.manual =
new Manual()
Add a trip computer
instruction.
return this.manual
Car
Manual

```
// Using the Builder pattern makes sense only when your
products
// are quite complex and require extensive configuration. The
// following two products are related, although they don't
have
// a common interface.
class Car is
    // A car can have a GPS, trip computer and some number of
    // seats. Different models of cars (sports car, SUV,
    // cabriolet) might have different features installed or
    // enabled.

class Manual is
    // Each car should have a user manual that corresponds to
    // the car's configuration and describes all its features.

// The builder interface specifies methods for creating the
// different parts of the product objects.
interface Builder is
    method reset()
    method setSeats(...)
    method setEngine(...)
    method setTripComputer(...)
    method setGPS(...)

// The concrete builder classes follow the builder interface
and
// provide specific implementations of the building steps.
Your
// program may have several variations of builders, each
// implemented differently.
class CarBuilder implements Builder is
    private field car:Car

    // A fresh builder instance should contain a blank product
    // object which it uses in further assembly.
    constructor CarBuilder() is
        this.reset()

    // The reset method clears the object being built.
    method reset() is
        this.car = new Car()

    // All production steps work with the same product
instance.
    method setSeats(...) is
        // Set the number of seats in the car.

    method setEngine(...) is
        // Install a given engine.

    method setTripComputer(...) is
        // Install a trip computer.

    method setGPS(...) is
        // Install a global positioning system.

    // Concrete builders are supposed to provide their own
    // methods for retrieving results. That's because various
    // types of builders may create entirely different
products
    // that don't all follow the same interface. Therefore
such
    // methods can't be declared in the builder interface (at
    // least not in a statically-typed programming language).
    //
    // Usually, after returning the end result to the client,
a
    // builder instance is expected to be ready to start
    // producing another product. That's why it's a usual
    // practice to call the reset method at the end of the
    // `getProduct` method body. However, this behavior isn't
    // mandatory, and you can make your builder wait for an
    // explicit reset call from the client code before
disposing
    // of the previous result.
    method getProduct():Car is
        product = this.car
        this.reset()
        return product

// Unlike other creational patterns, builder lets you
construct
// products that don't follow the common interface.
class CarManualBuilder implements Builder is
    private field manual:Manual

    constructor CarManualBuilder() is
        this.reset()

    method reset() is
        this.manual = new Manual()

    method setSeats(...) is
        // Document car seat features.

    method setEngine(...) is
        // Add engine instructions.

    method setTripComputer(...) is
        // Add trip computer instructions.

    method setGPS(...) is
        // Add GPS instructions.

    method getProduct():Manual is
        // Return the manual and reset the builder.


// The director is only responsible for executing the building
// steps in a particular sequence. It's helpful when producing
// products according to a specific order or configuration.
// Strictly speaking, the director class is optional, since
the
// client can control builders directly.
class Director is
    // The director works with any builder instance that the
    // client code passes to it. This way, the client code may
    // alter the final type of the newly assembled product.
    // The director can construct several product variations
    // using the same building steps.
    method constructSportsCar(builder: Builder) is
        builder.reset()
        builder.setSeats(2)
        builder.setEngine(new SportEngine())
        builder.setTripComputer(true)
        builder.setGPS(true)

    method constructSUV(builder: Builder) is
        // ...
// The client code creates a builder object, passes it to the
// director and then initiates the construction process. The
end
// result is retrieved from the builder object.
class Application is

    method makeCar() is
        director = new Director()

        CarBuilder builder = new CarBuilder()
        director.constructSportsCar(builder)
        Car car = builder.getProduct()

        CarManualBuilder builder = new CarManualBuilder()
        director.constructSportsCar(builder)

        // The final product is often retrieved from a builder
        // object since the director isn't aware of and not
        // dependent on concrete builders and products.
        Manual manual = builder.getProduct()
```

The Costly From-Scratch Build



SYMPTOM

Relying on external classes to duplicate an object.

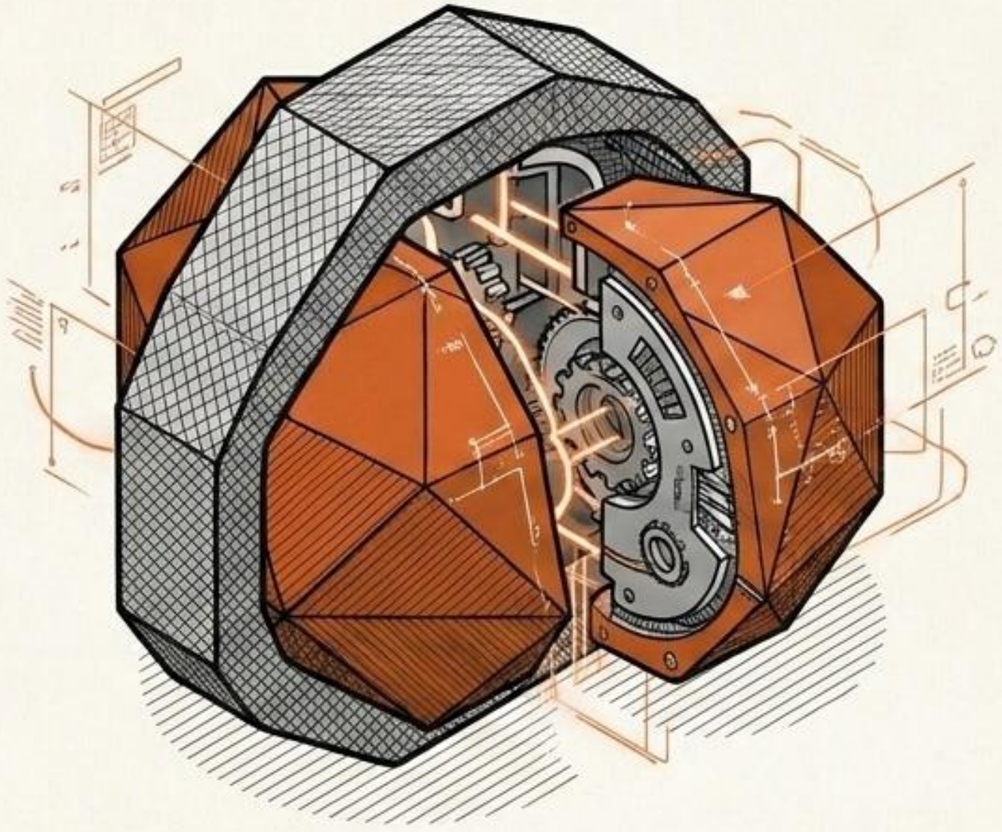
CONSTRAINT

External code cannot see or copy an object's private state or fields. Furthermore, initializing a complex object from scratch consumes massive CPU cycles.

RESULT

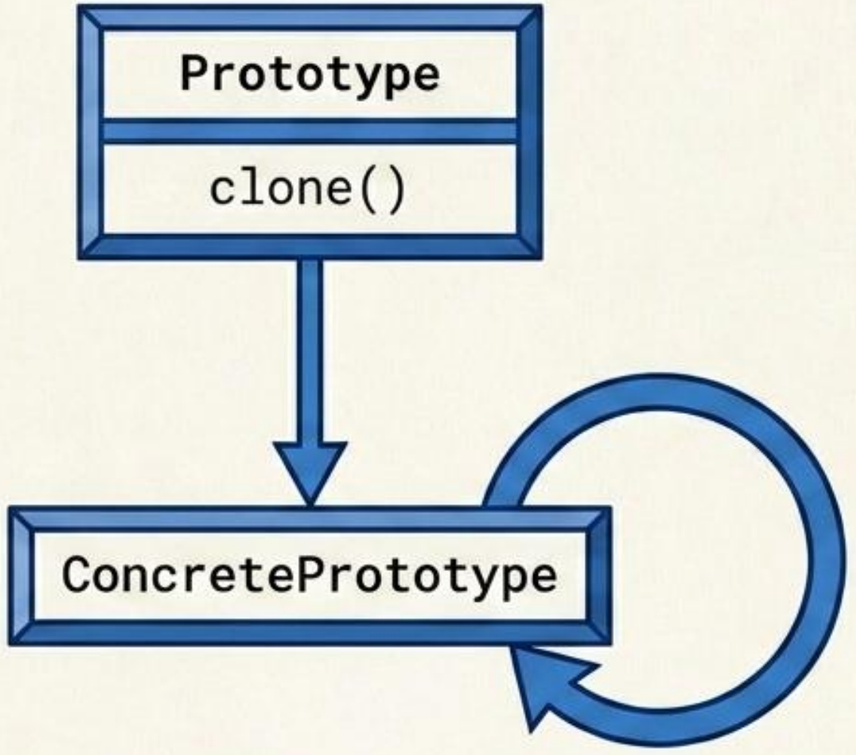
Incomplete copies and tight coupling to concrete classes.

Cellular Mitosis

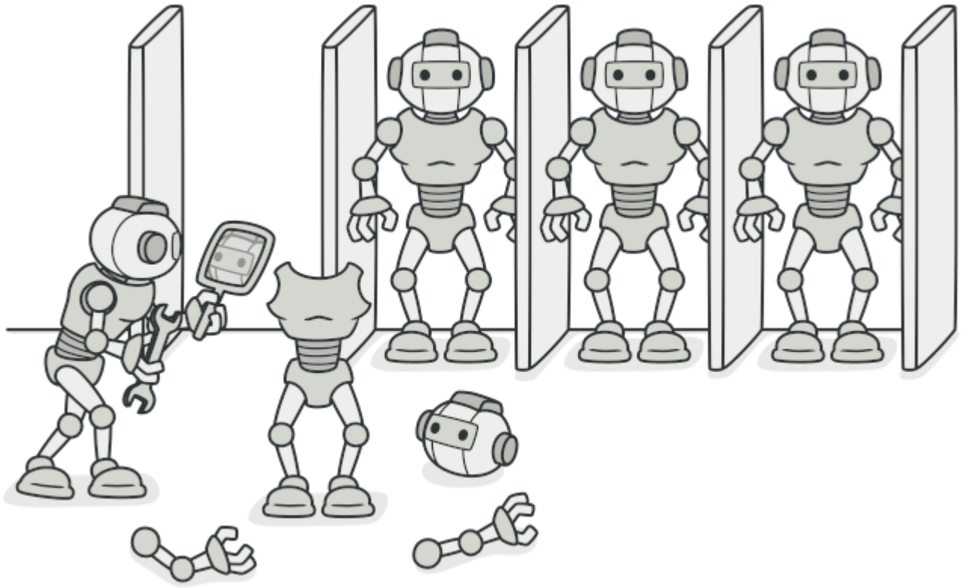


Delegate the cloning process to the actual objects being cloned. An object with full access to its own state creates an exact replica of itself without exposing its internal structure to the client.

Prototype Schematic

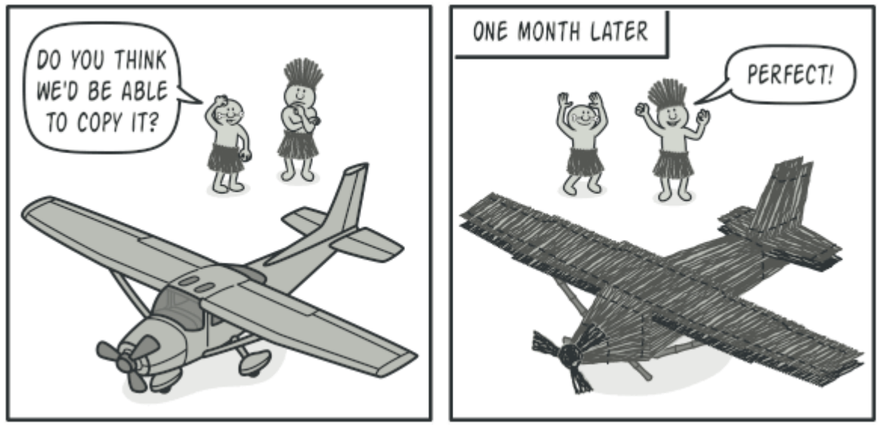


A unified cloning interface lets you copy objects without coupling your code to the classes of those objects.

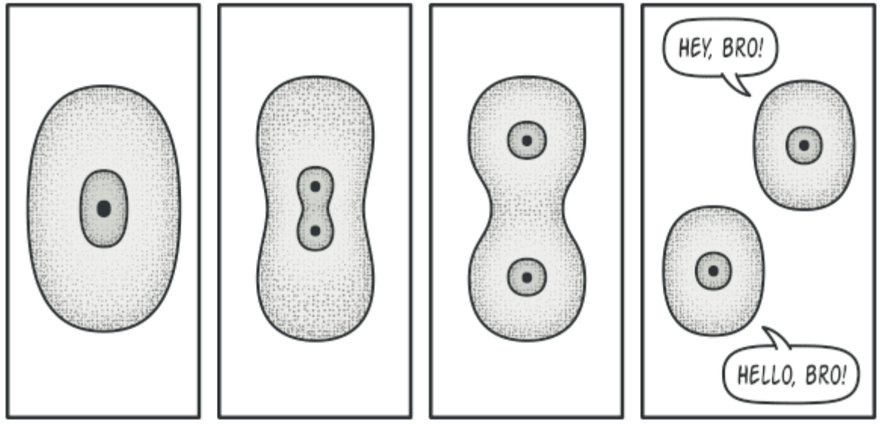


Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.

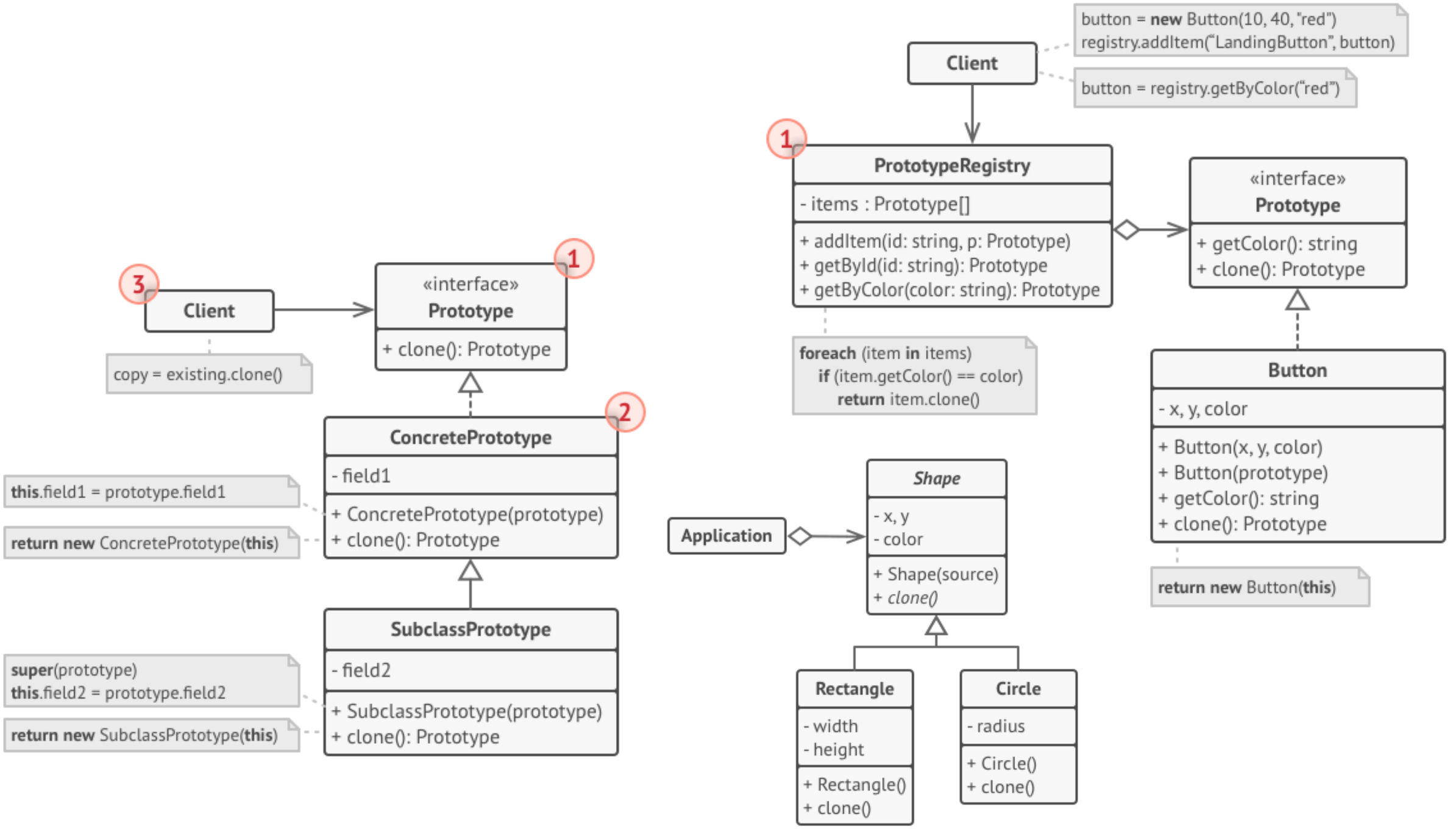
Pre-built prototypes can be an alternative to subclassing.



*Copying an object "from the outside" **isn't** always possible.*



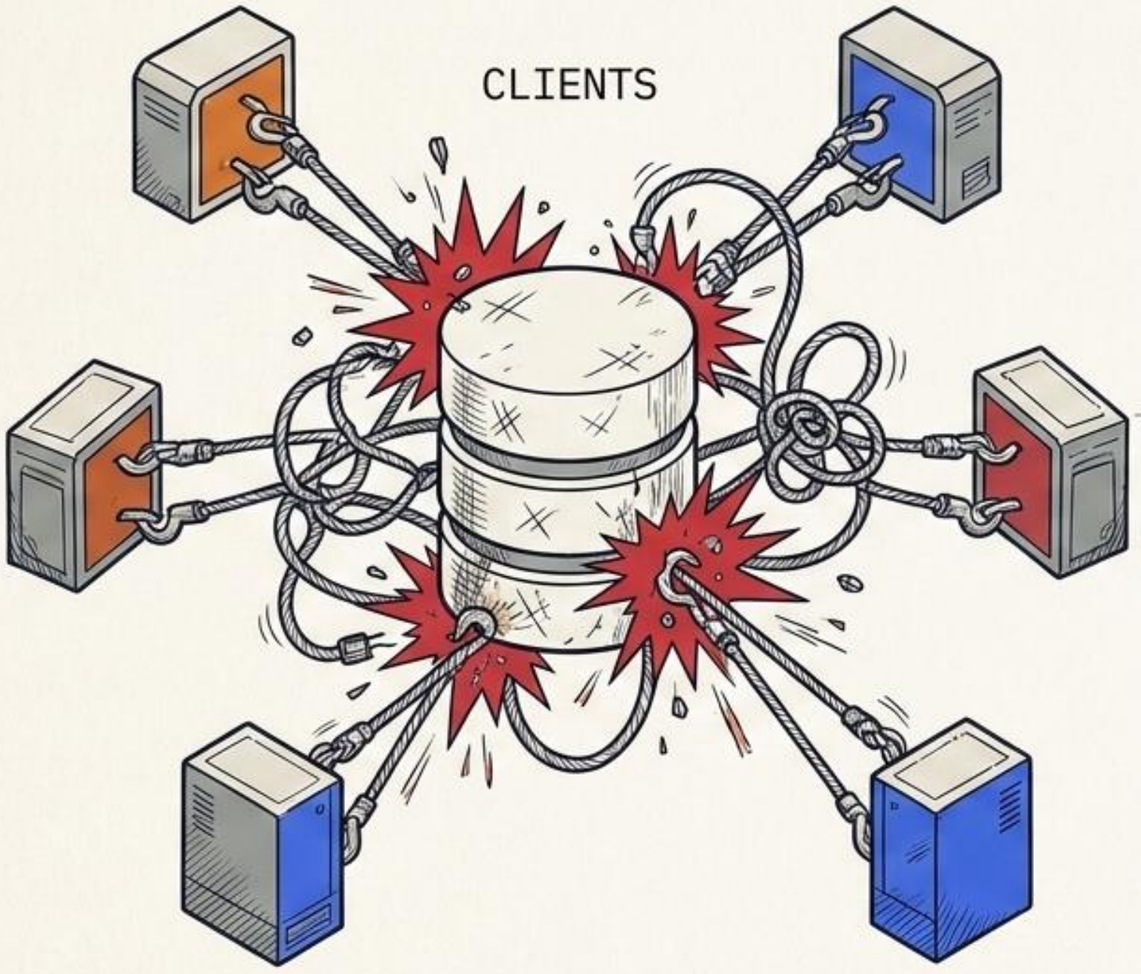
The division of a cell.



Cloning a set of objects that belong to a class hierarchy.

```
// Base prototype.
abstract class Shape is
    field X: int
    field Y: int
    field color: string

    // A regular constructor.
    constructor Shape() is
        // ...

    // The prototype constructor. A fresh object is
initialized
    // with values from the existing object.
    constructor Shape(source: Shape) is
        this()
        this.X = source.X
        this.Y = source.Y
        this.color = source.color

    // The clone operation returns one of the Shape
subclasses.
    abstract method clone():Shape


// Concrete prototype. The cloning method creates a
new object
// in one go by calling the constructor of the
current class and
// passing the current object as the constructor's
argument.
// Performing all the actual copying in the
constructor helps to
// keep the result consistent: the constructor will
not return a
// result until the new object is fully built; thus,
no object
// can have a reference to a partially-built clone.
class Rectangle extends Shape is
    field width: int
    field height: int

    constructor Rectangle(source: Rectangle) is
        // A parent constructor call is needed to
copy private
        // fields defined in the parent class.
        super(source)
        this.width = source.width
        this.height = source.height

    method clone():Shape is
        return new Rectangle(this)


class Circle extends Shape is
    field radius: int

    constructor Circle(source: Circle) is
        super(source)
        this.radius = source.radius

    method clone():Shape is
        return new Circle(this)


// Somewhere in the client code.
class Application is
    field shapes: array of Shape

    constructor Application() is
        Circle circle = new Circle()
        circle.X = 10
        circle.Y = 10
        circle.radius = 20
        shapes.add(circle)

        Circle anotherCircle = circle.clone()
        shapes.add(anotherCircle)
        // The `anotherCircle` variable contains an
exact copy
        // of the `circle` object.

        Rectangle rectangle = new Rectangle()
        rectangle.width = 10
        rectangle.height = 20
        shapes.add(rectangle)

    method businessLogic() is
        // Prototype rocks because it lets you
produce a copy of
        // an object without knowing anything about
its type.
        Array shapesCopy = new Array of Shapes.

        // For instance, we don't know the exact
elements in the
        // shapes array. All we know is that they are
all
        // shapes. But thanks to polymorphism, when
we call the
        // `clone` method on a shape the program
checks its real
        // class and runs the appropriate clone
method defined
        // in that class. That's why we get proper
clones
        // instead of a set of simple Shape objects.
        foreach (s in shapes) do
            shapesCopy.add(s.clone())

        // The `shapesCopy` array contains exact
copies of the
        // `shape` array's children.
```

The Chaotic Resource Grab



SYMPTOM

Multiple instances of a resource-heavy class colliding, destroying shared state.

CONSTRAINT

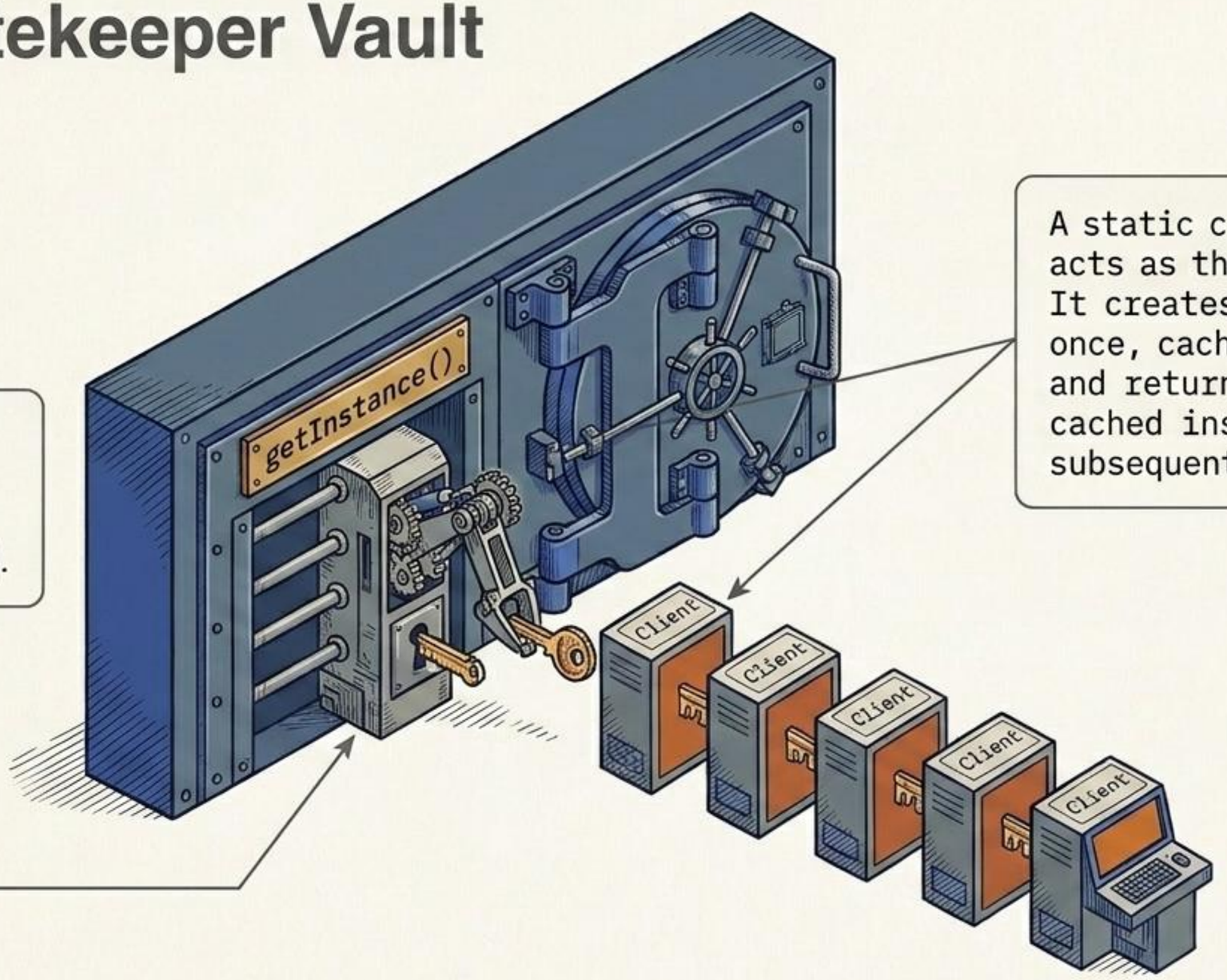
You need strict control over a shared resource (like a database or file system), but you also need a global access point that won't be overridden.

RESULT

Race conditions, corrupted data, and unpredictable system behavior.

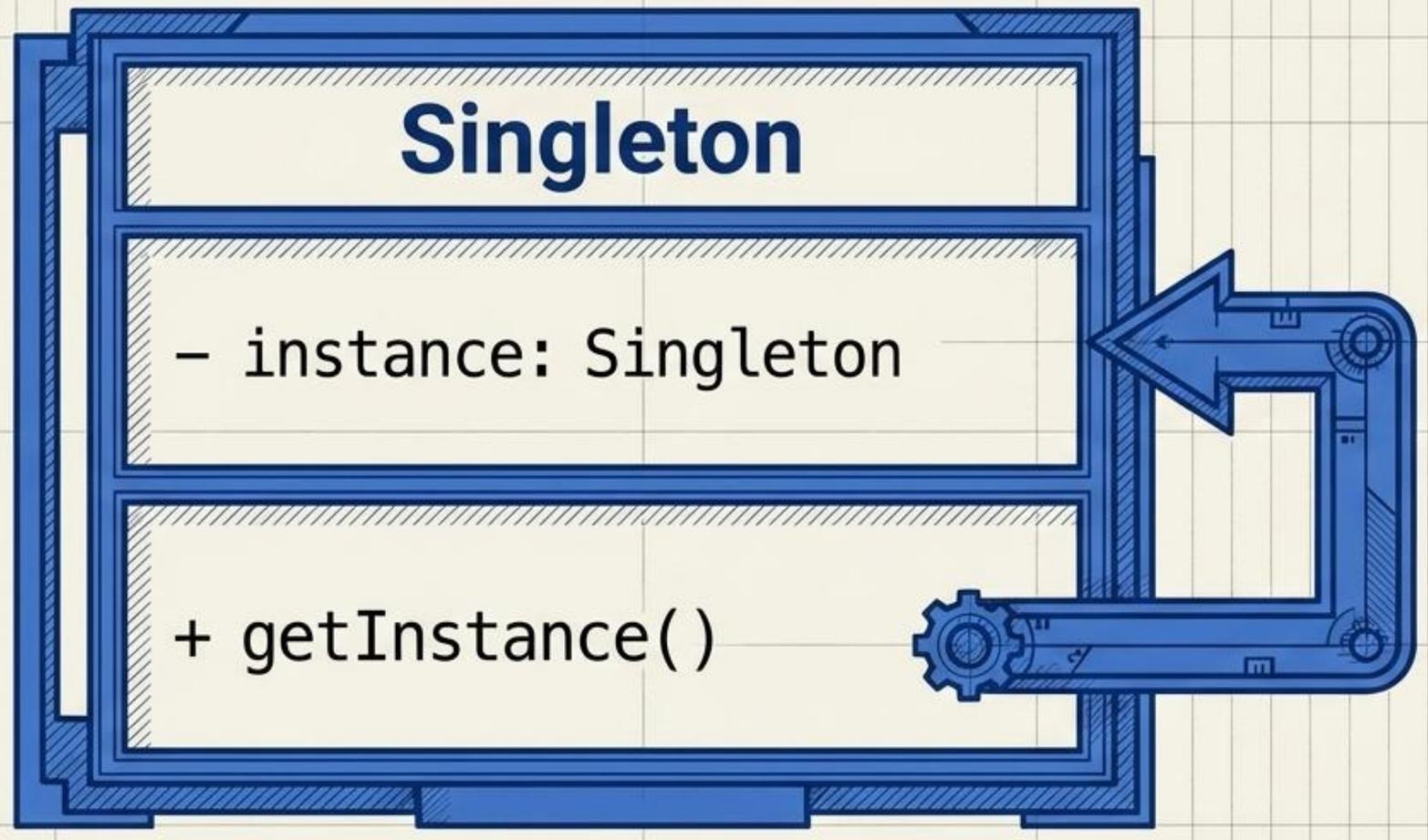
The Gatekeeper Vault

The default constructor is hidden (made private).

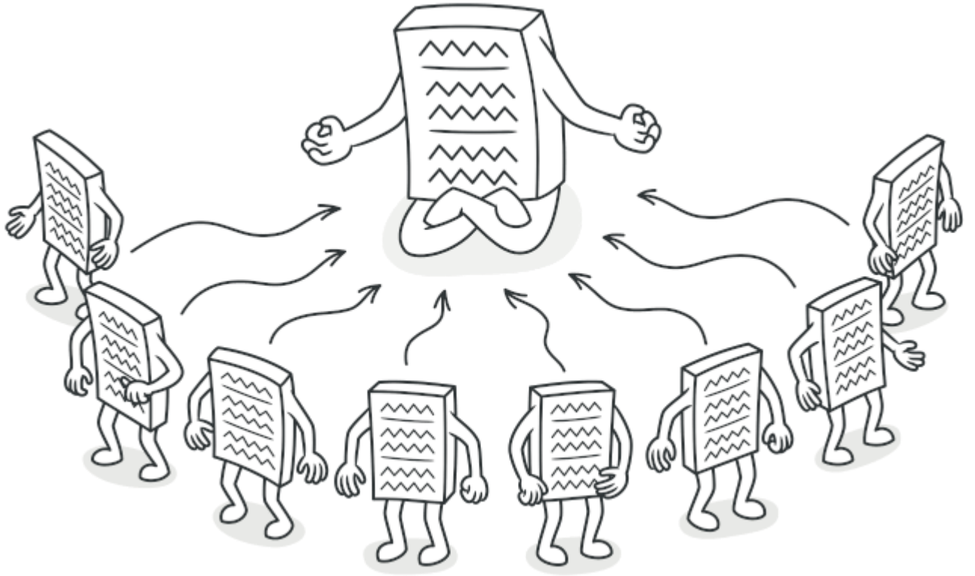


A static creation method acts as the gatekeeper. It creates the object once, caches it, and and returns that same cached instance for all subsequent calls.

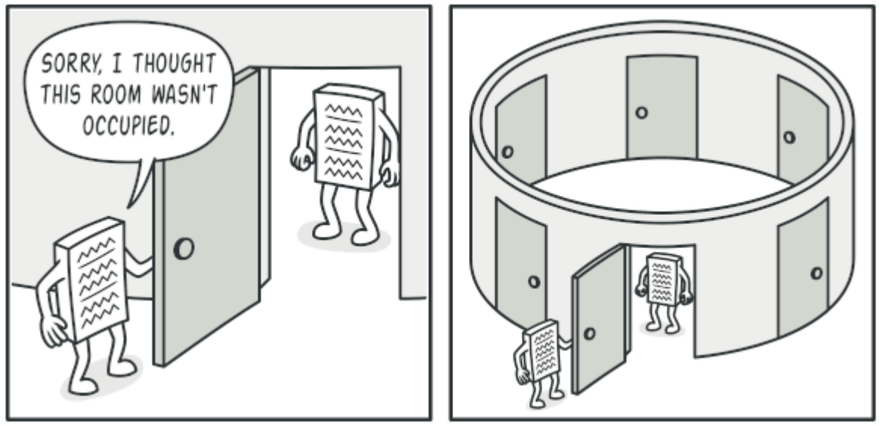
Singleton Schematic

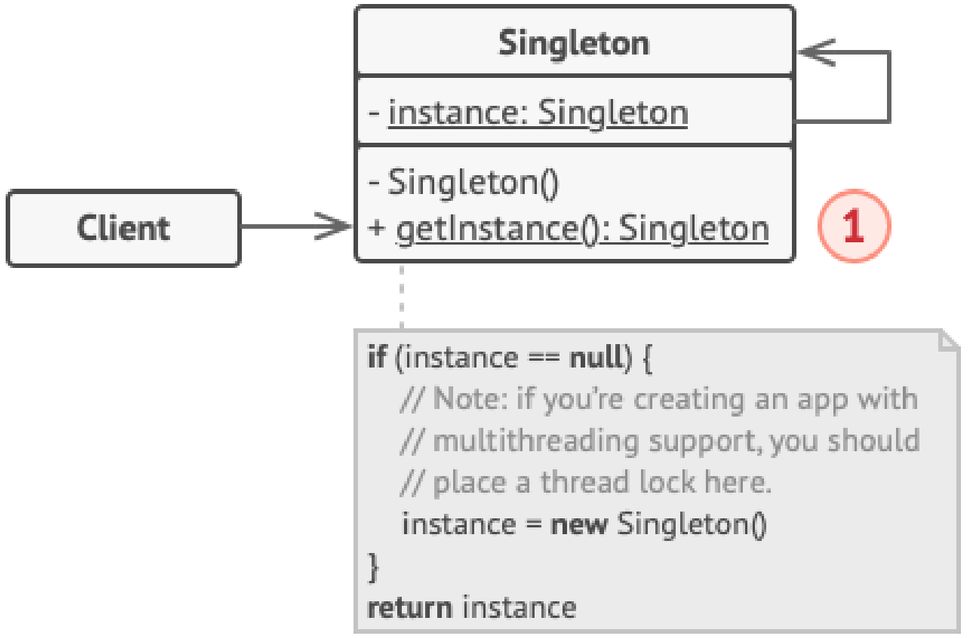


Solves two problems at once: Guarantees a single instance exists, and provides a global, strictly controlled access point to it.



Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.

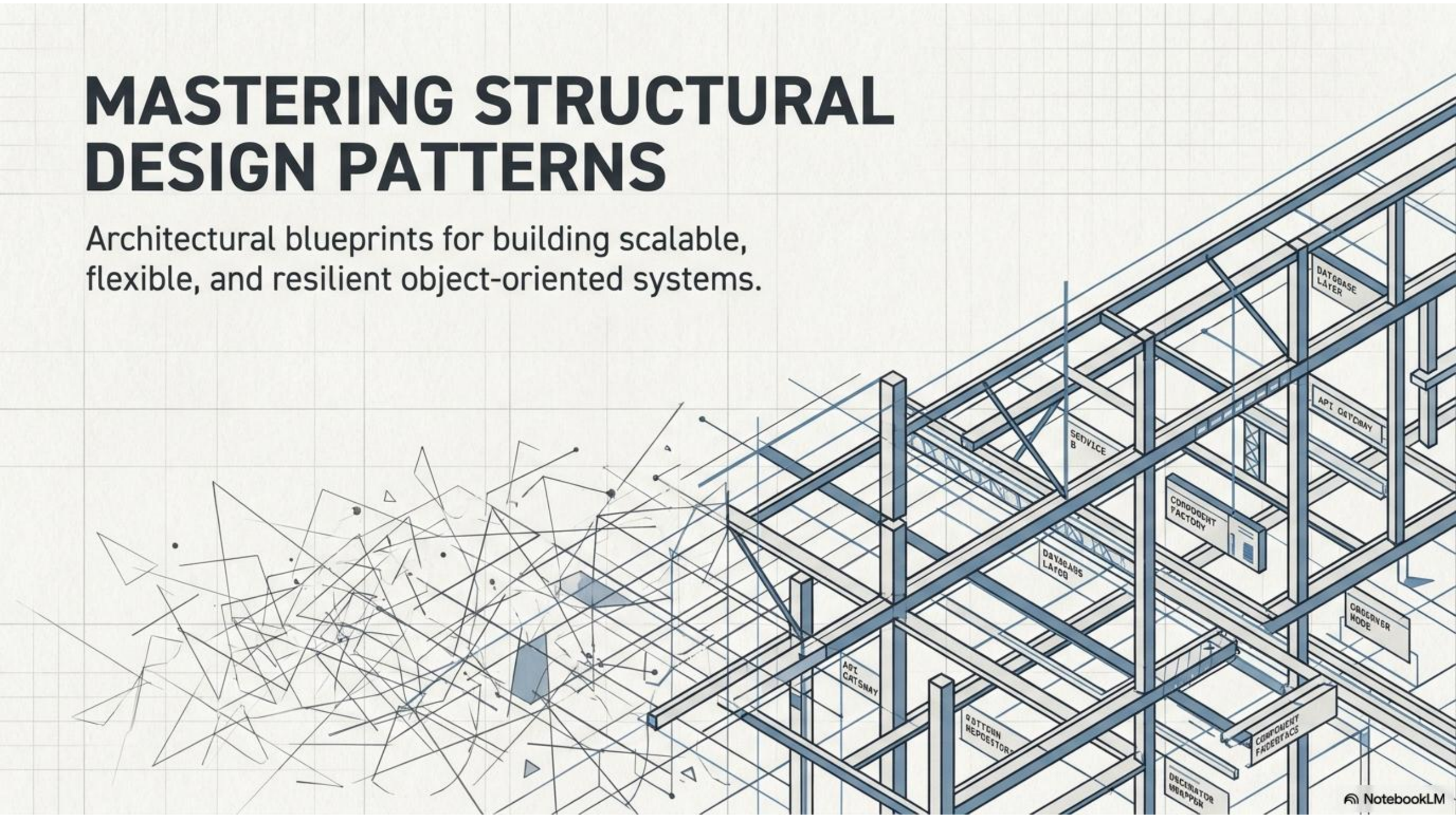




```
// The Database class defines the `getInstance` method that
lets
// clients access the same instance of a database connection
// throughout the program.
class Database is
    // The field for storing the singleton instance should be
    // declared static.
    private static field instance: Database

    // The singleton's constructor should always be private to
    // prevent direct construction calls with the `new`
    // operator.
    private constructor Database() is
        // Some initialization code, such as the actual
        // connection to a database server.
        // ...

    // The static method that controls access to the singleton
    // instance.
    public static method getInstance() is
        if (Database.instance == null) then
            acquireThreadLock() and then
                // Ensure that the instance hasn't yet been
                // initialized by another thread while this one
                // has been waiting for the lock's release.
                if (Database.instance == null) then
                    Database.instance = new Database()
        return Database.instance

    // Finally, any singleton should define some business logic
    // which can be executed on its instance.
    public method query(sql) is
        // For instance, all database queries of an app go
        // through this method. Therefore, you can place
        // throttling or caching logic here.
        // ...

class Application is
    method main() is
        Database foo = Database.getInstance()
        foo.query("SELECT ...")
        // ...
        Database bar = Database.getInstance()
        bar.query("SELECT ...")
        // The variable `bar` will contain the same object as
        // the variable `foo`.
```

MASTERING STRUCTURAL DESIGN PATTERNS

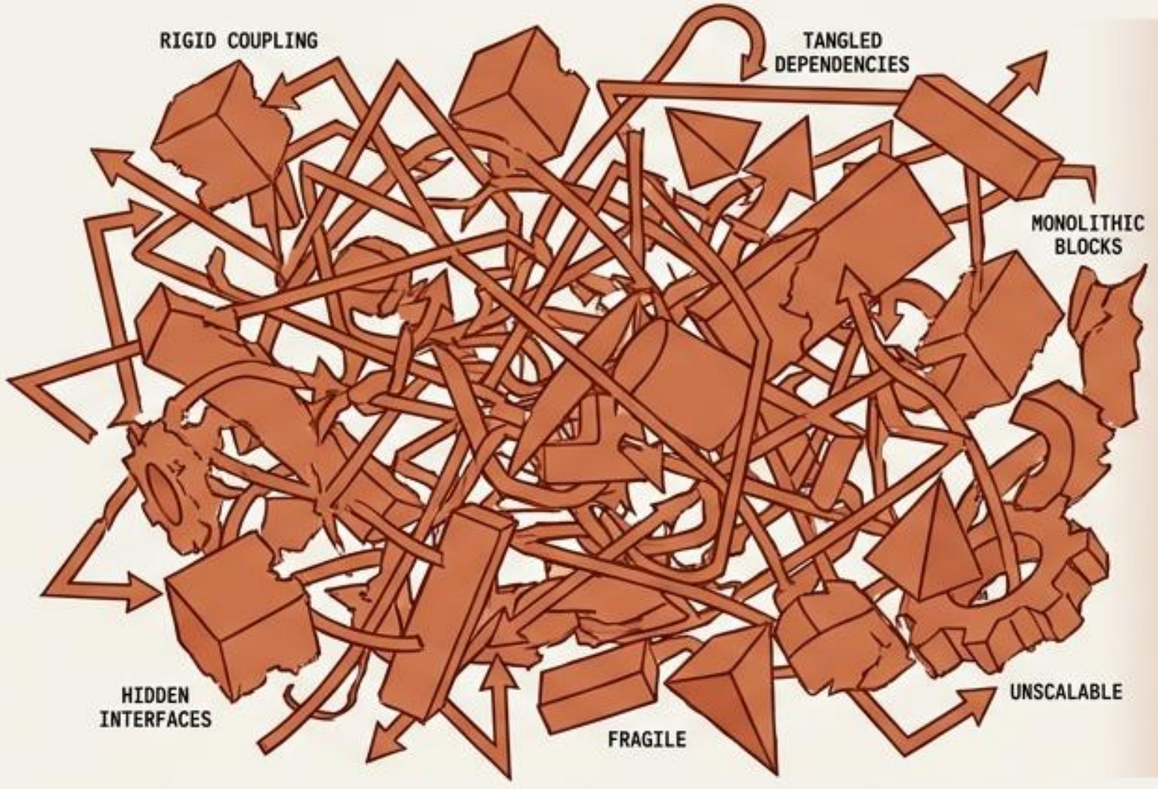
Architectural blueprints for building scalable, flexible, and resilient object-oriented systems.



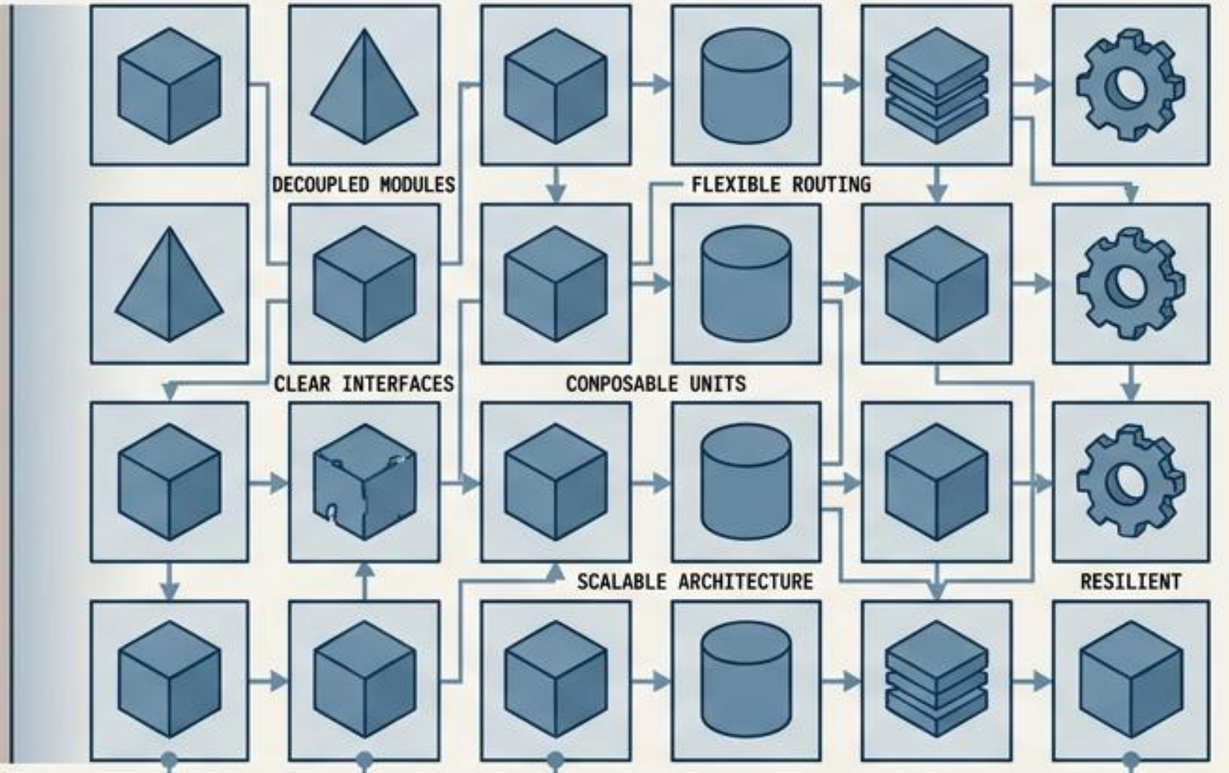
THE ANATOMY OF COMPOSITION

Structural patterns define how to assemble objects into larger, flexible architectures.
They resolve systemic friction by focusing on three core architectural principles.

SYSTEMIC FRICTION



STRUCTURAL COMPOSITION



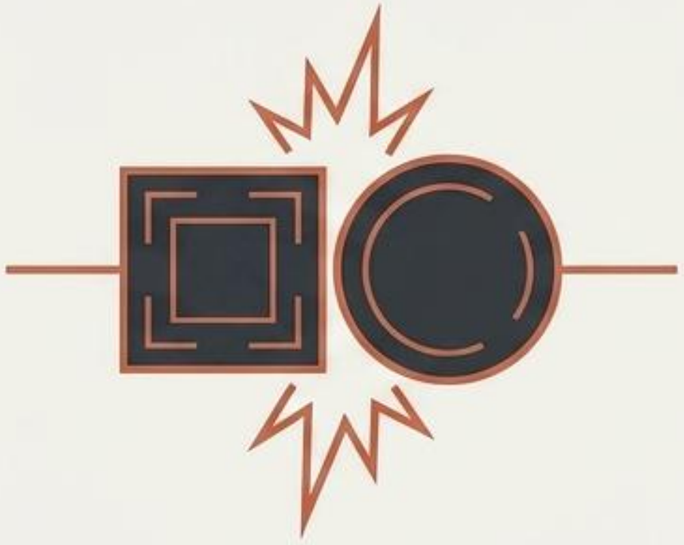
1. DECOUPLING:
Separating rigid interfaces from their underlying implementations.

2. COMPOSITION:
Building complex, scalable behaviors from simple, isolated pieces.

3. WRAPPING:
Enhancing objects dynamically without altering their core DNA.

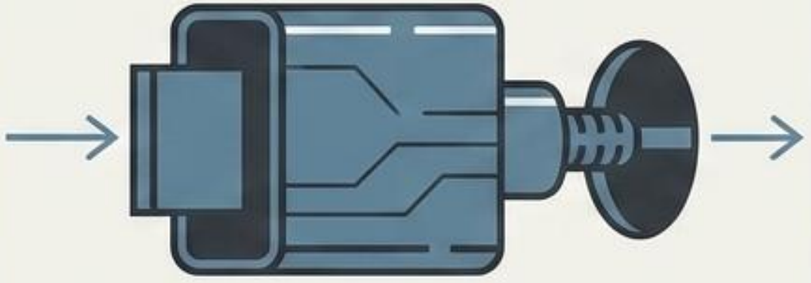
THE FRICTION

Incompatible interfaces prevent distinct systems from communicating.

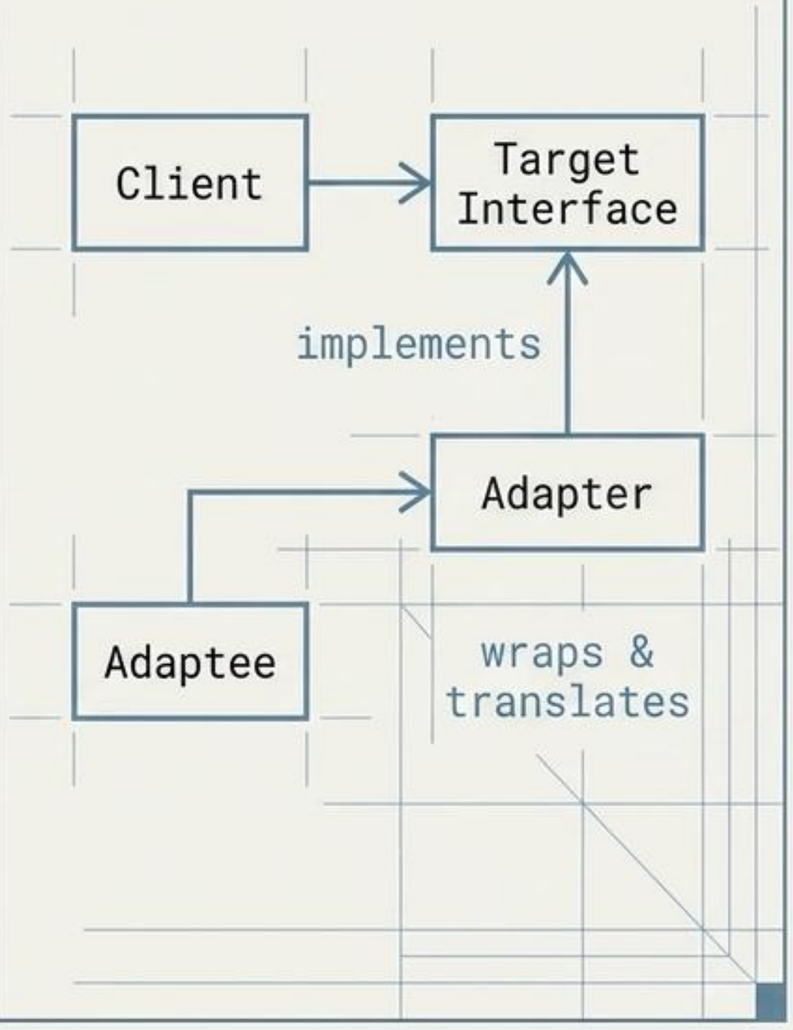


THE MECHANISM

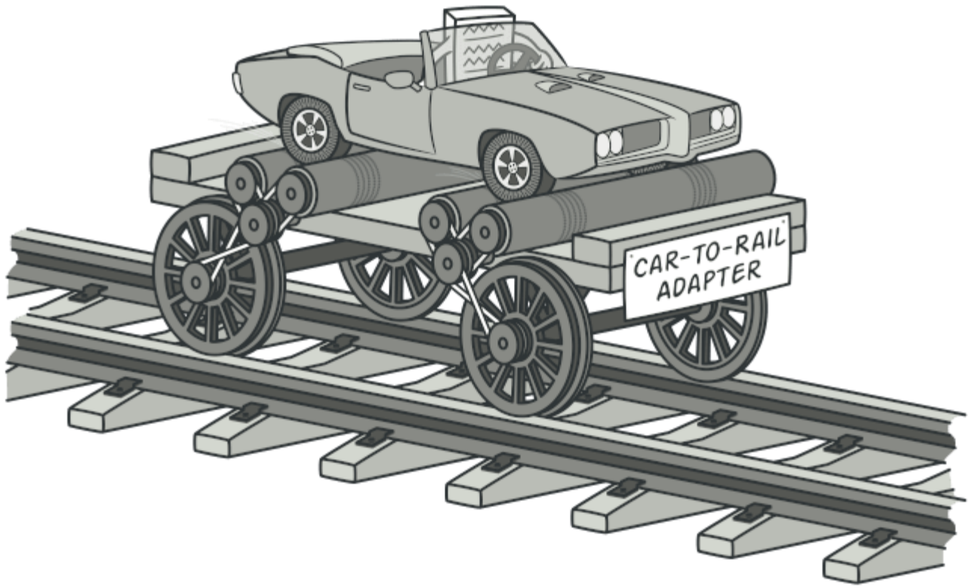
Create a wrapper class that catches calls from the client and translates them into a format the existing system understands.

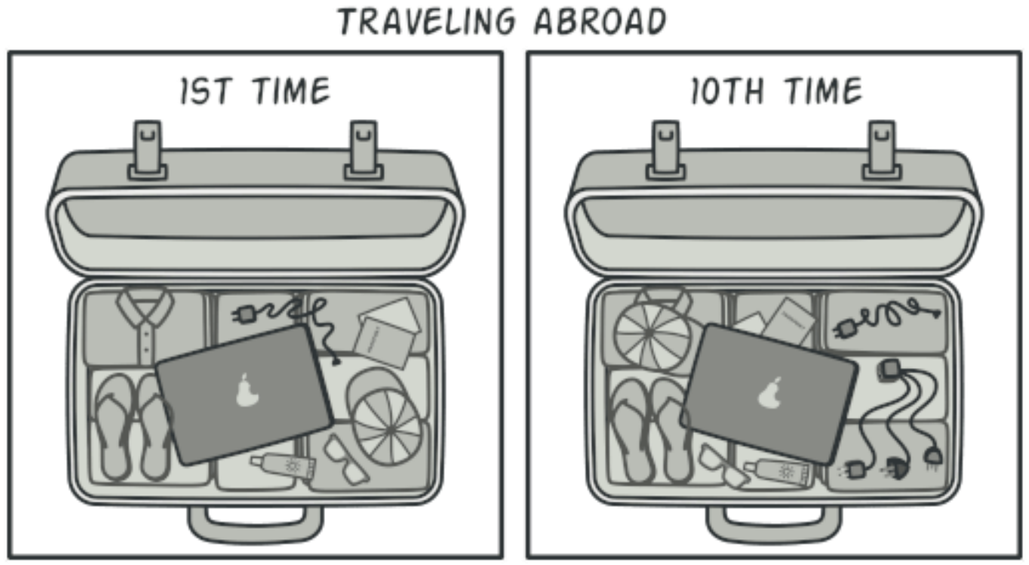


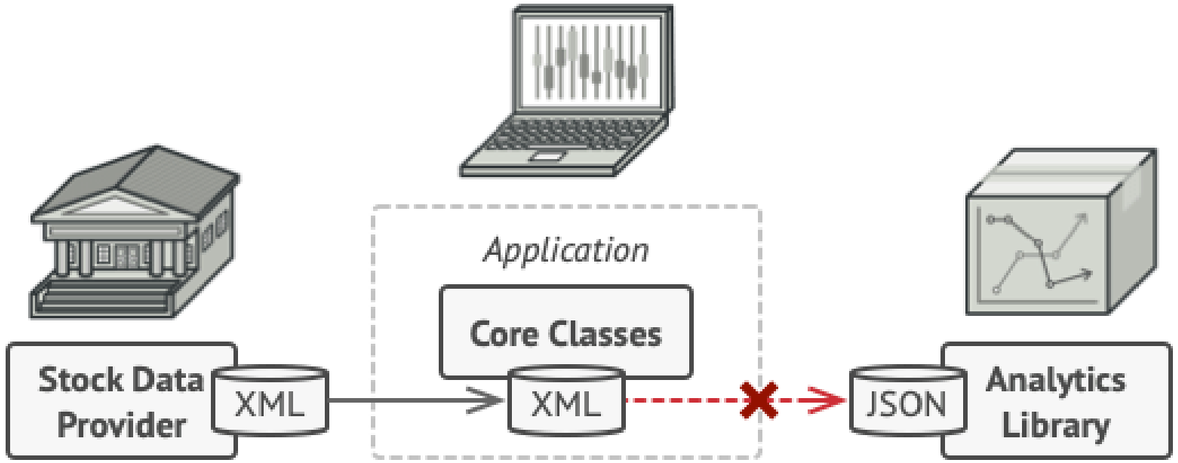
THE SCHEMATIC: ADAPTER



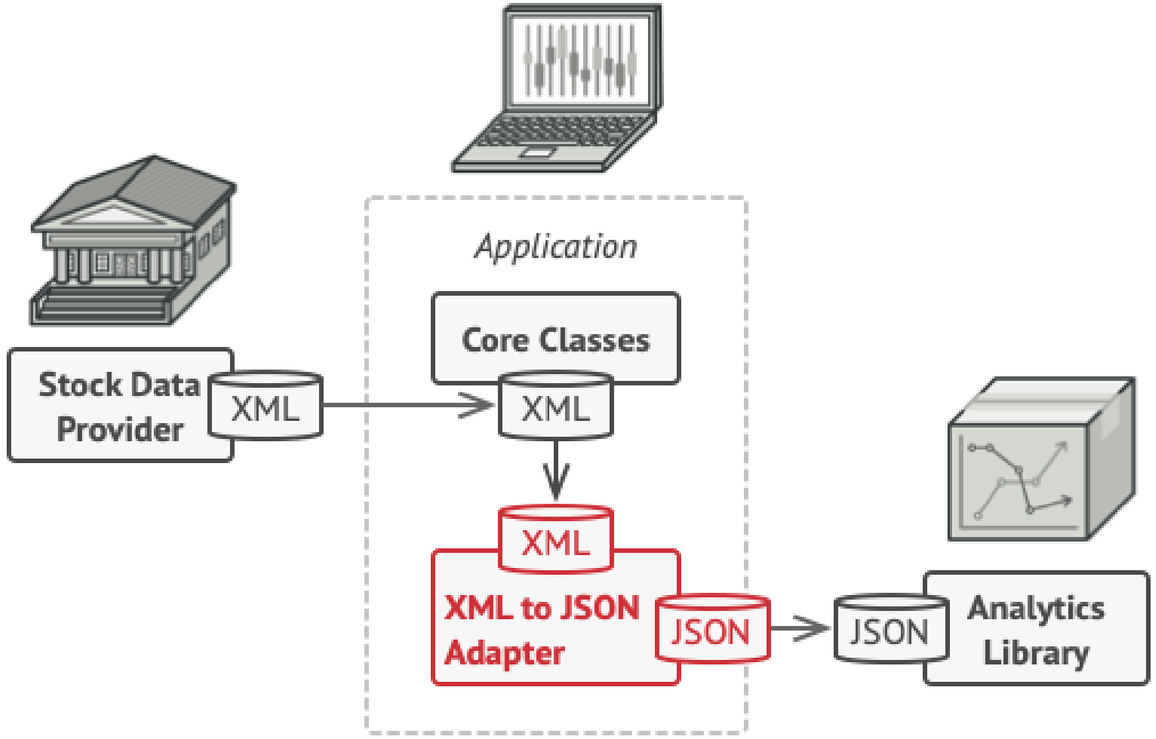
Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.

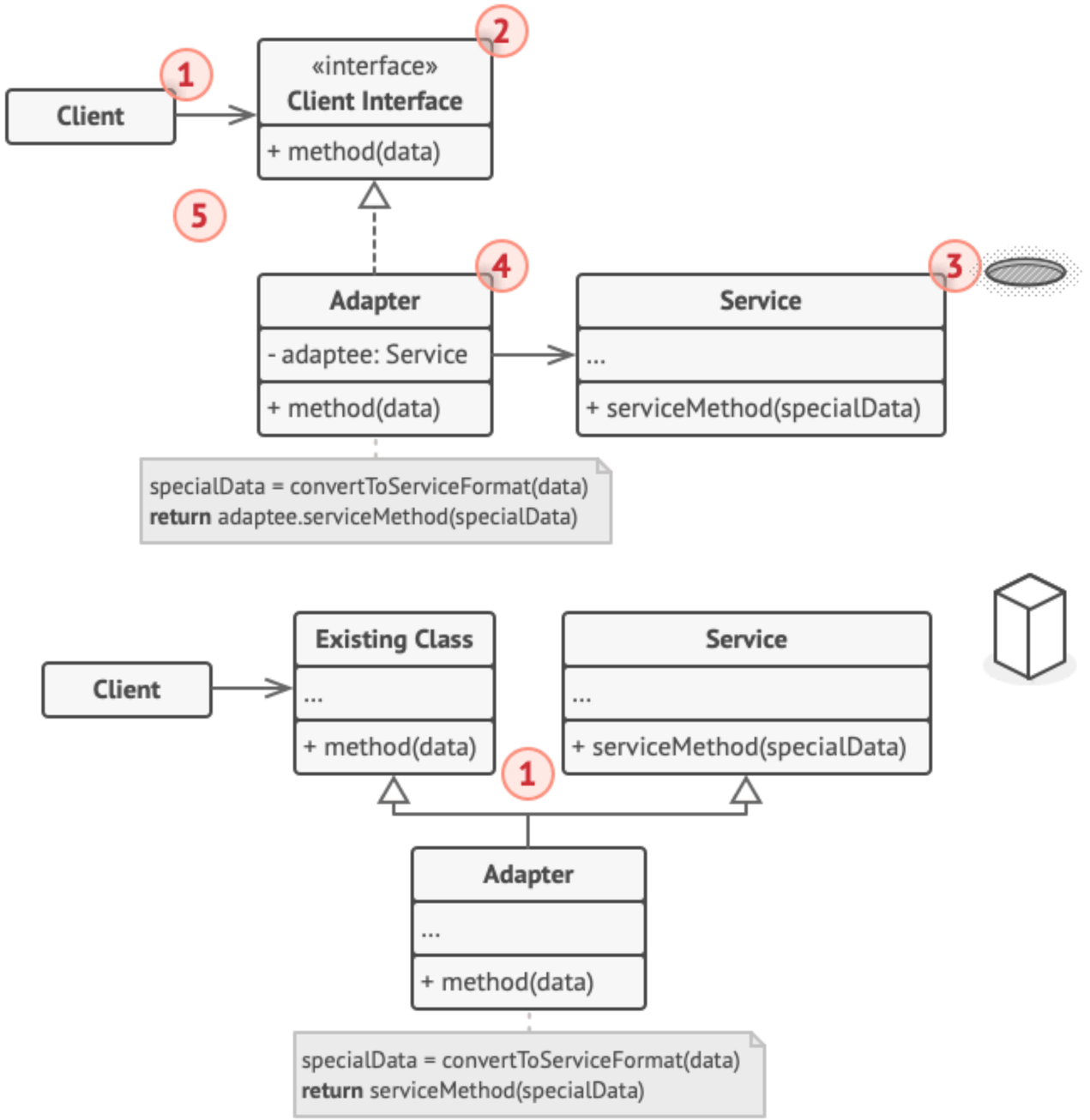


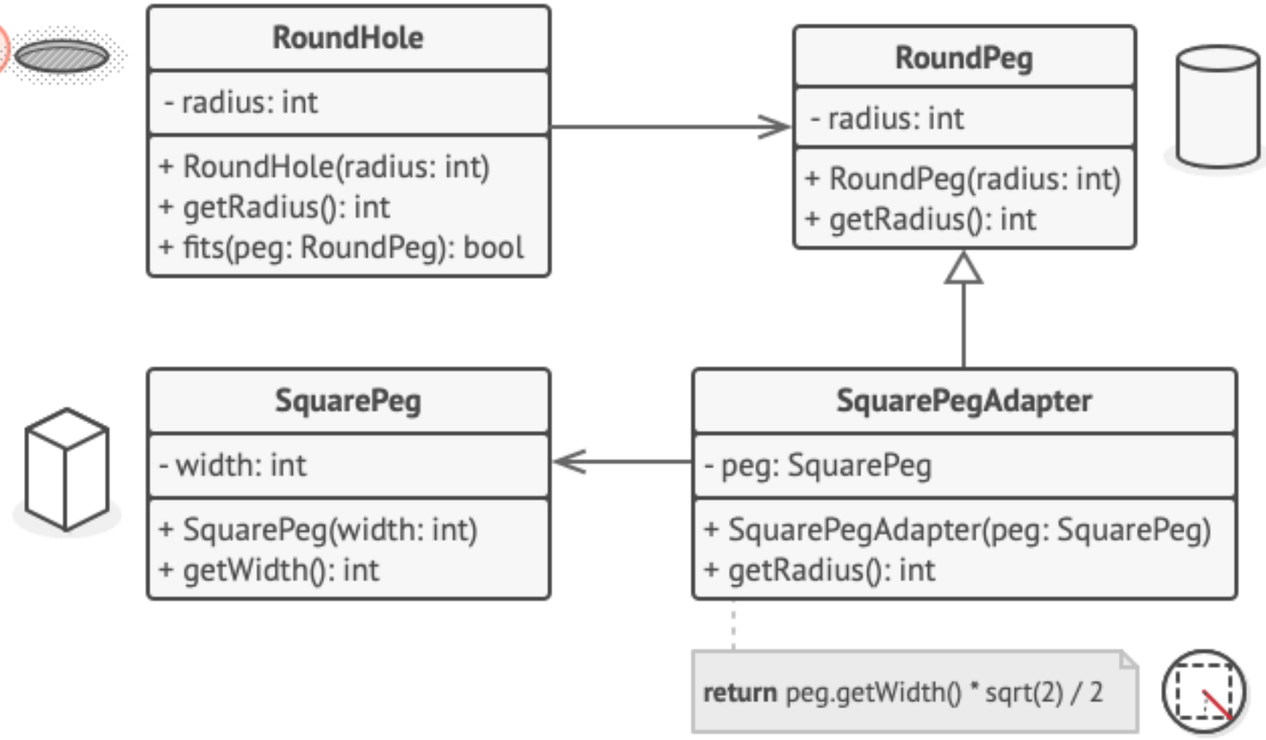




You can’t use the analytics library “as is” because it expects the data in a format that’s incompatible with your app.





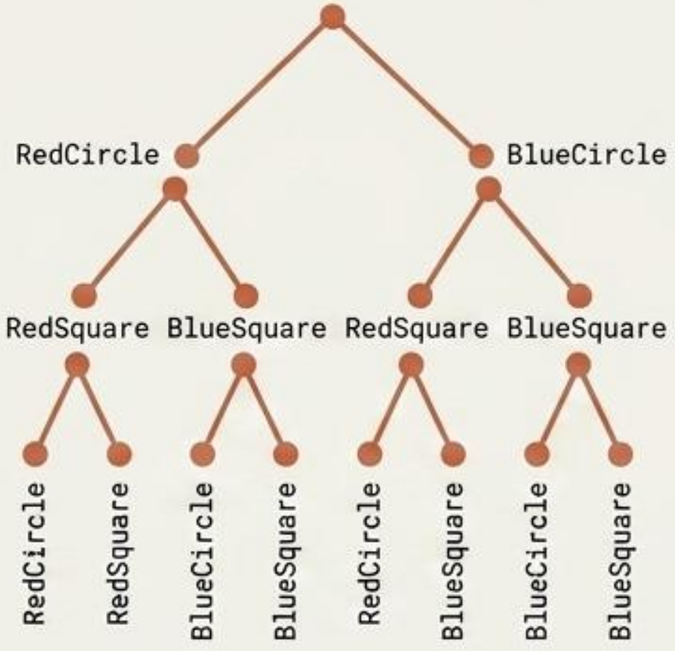


Adapting square pegs to round holes.

```
// Say you have two classes with compatible interfaces:
// RoundHole and RoundPeg.
class RoundHole is
    constructor RoundHole(radius) { ... }

    method getRadius() is
        // Return the radius of the hole.

    method fits(peg: RoundPeg) is
        return this.getRadius() >= peg.getRadius()

class RoundPeg is
    constructor RoundPeg(radius) { ... }

    method getRadius() is
        // Return the radius of the peg.


// But there's an incompatible class: SquarePeg.
class SquarePeg is
    constructor SquarePeg(width) { ... }

    method getWidth() is
        // Return the square peg width.


// An adapter class lets you fit square pegs into round holes.
// It extends the RoundPeg class to let the adapter objects act
// as round pegs.
class SquarePegAdapter extends RoundPeg is
    // In reality, the adapter contains an instance of the
    // SquarePeg class.
    private field peg: SquarePeg

    constructor SquarePegAdapter(peg: SquarePeg) is
        this.peg = peg

    method getRadius() is
        // The adapter pretends that it's a round peg with a
        // radius that could fit the square peg that the
adapter
        // actually wraps.
        return peg.getWidth() * Math.sqrt(2) / 2


// Somewhere in client code.
hole = new RoundHole(5)
rpeg = new RoundPeg(5)
hole.fits(rpeg) // true

small_sqpeg = new SquarePeg(5)
large_sqpeg = new SquarePeg(10)
hole.fits(small_sqpeg) // this won't compile (incompatible
types)

small_sqpeg_adapter = new SquarePegAdapter(small_sqpeg)
large_sqpeg_adapter = new SquarePegAdapter(large_sqpeg)
hole.fits(small_sqpeg_adapter) // true
hole.fits(large_sqpeg_adapter) // false
```

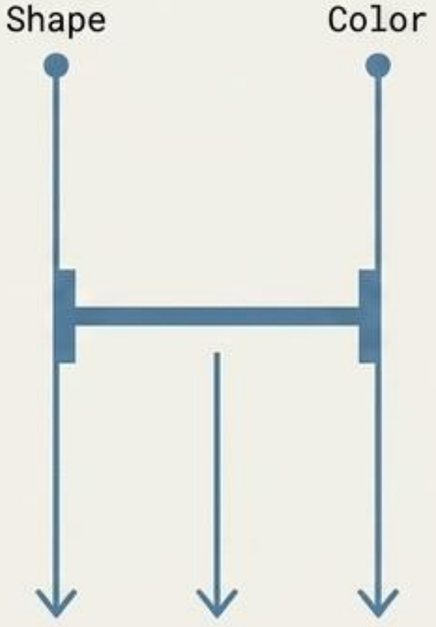
THE FRICTION

Cartesian explosion: hierarchy grows exponentially when extending both shapes and colors.

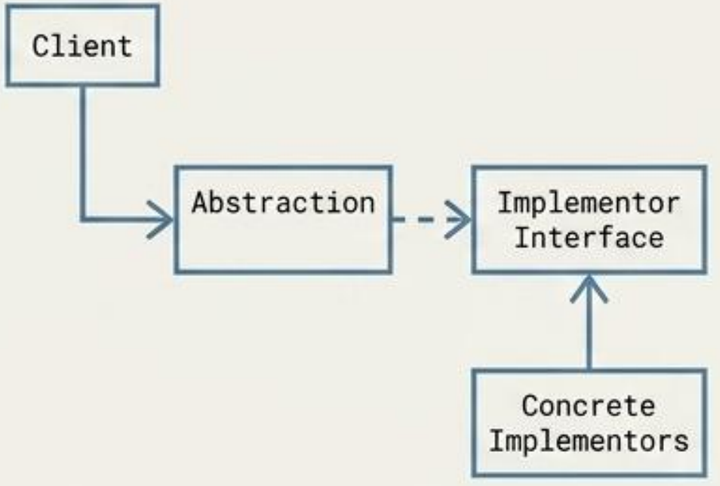


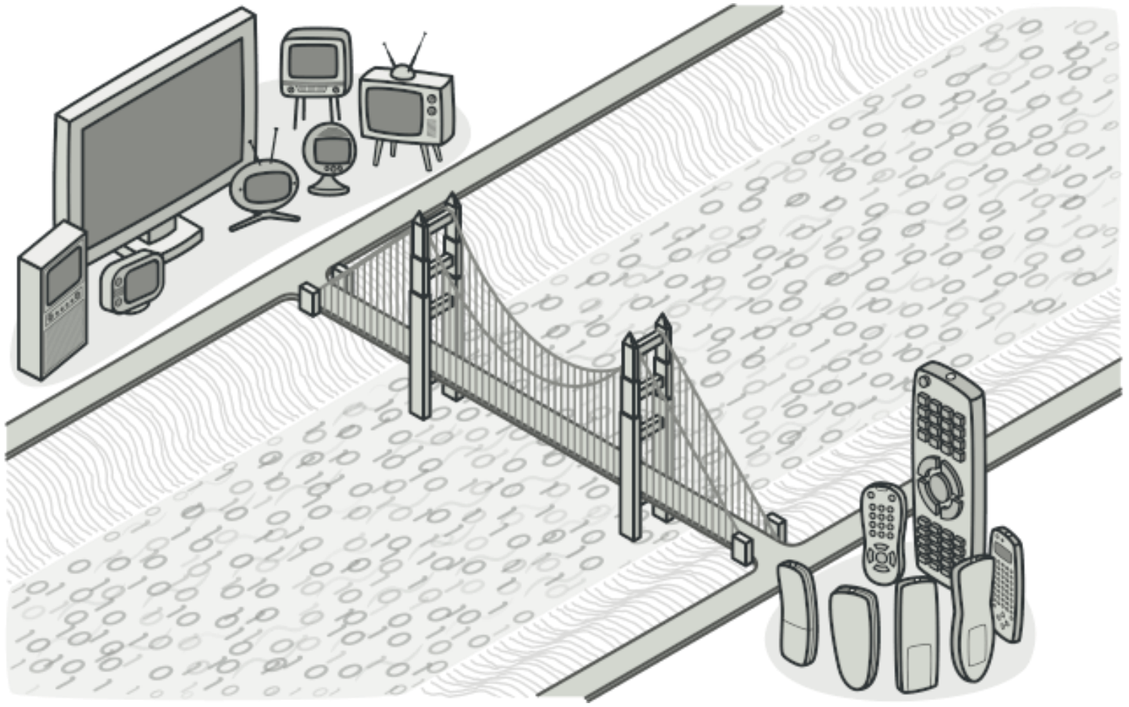
THE MECHANISM

Split a monolithic class into two separate hierarchies—Abstraction and Implementation—allowing them to evolve independently.

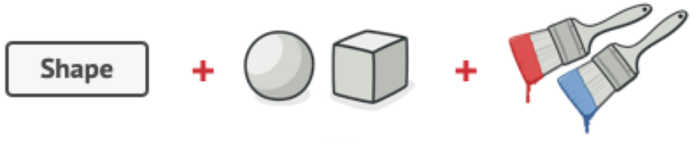


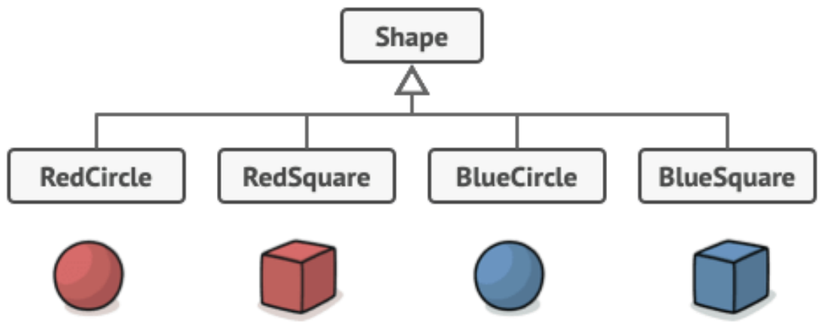
THE SCHEMATIC: BRIDGE

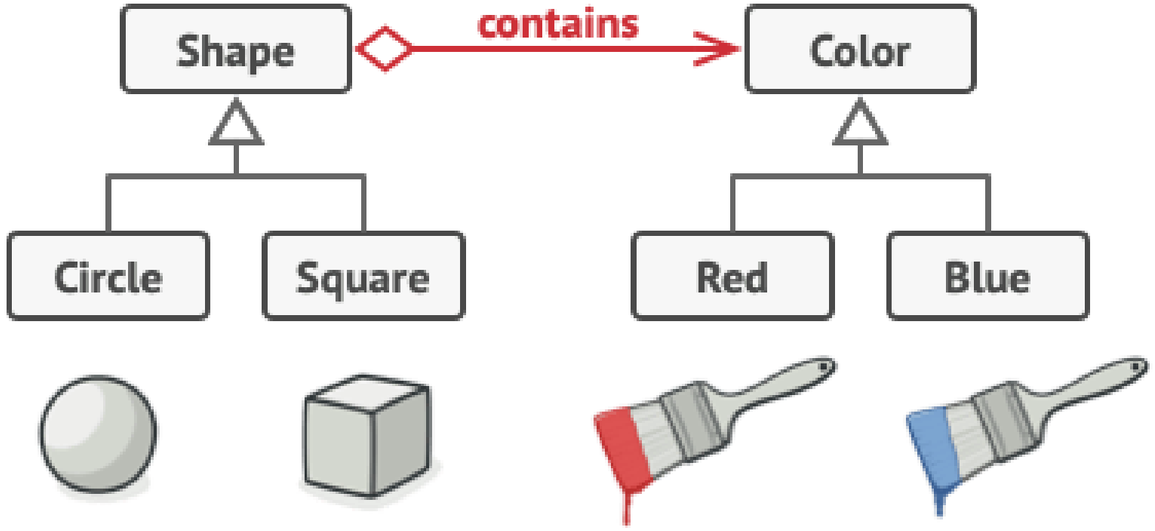


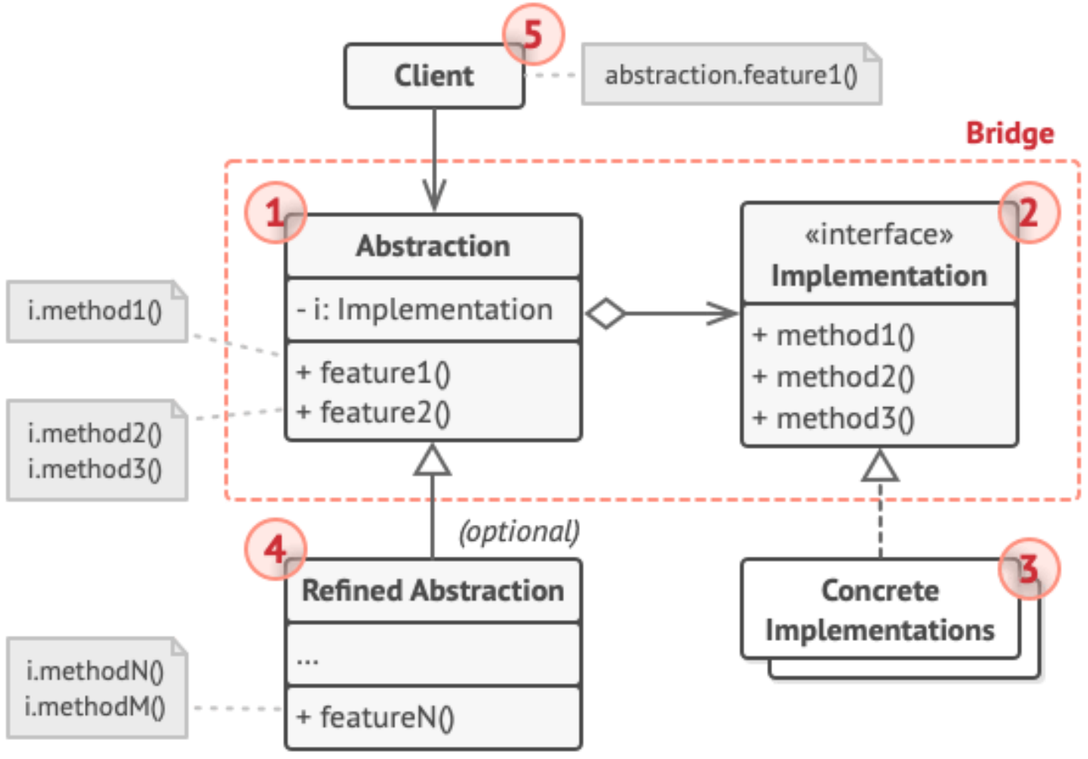


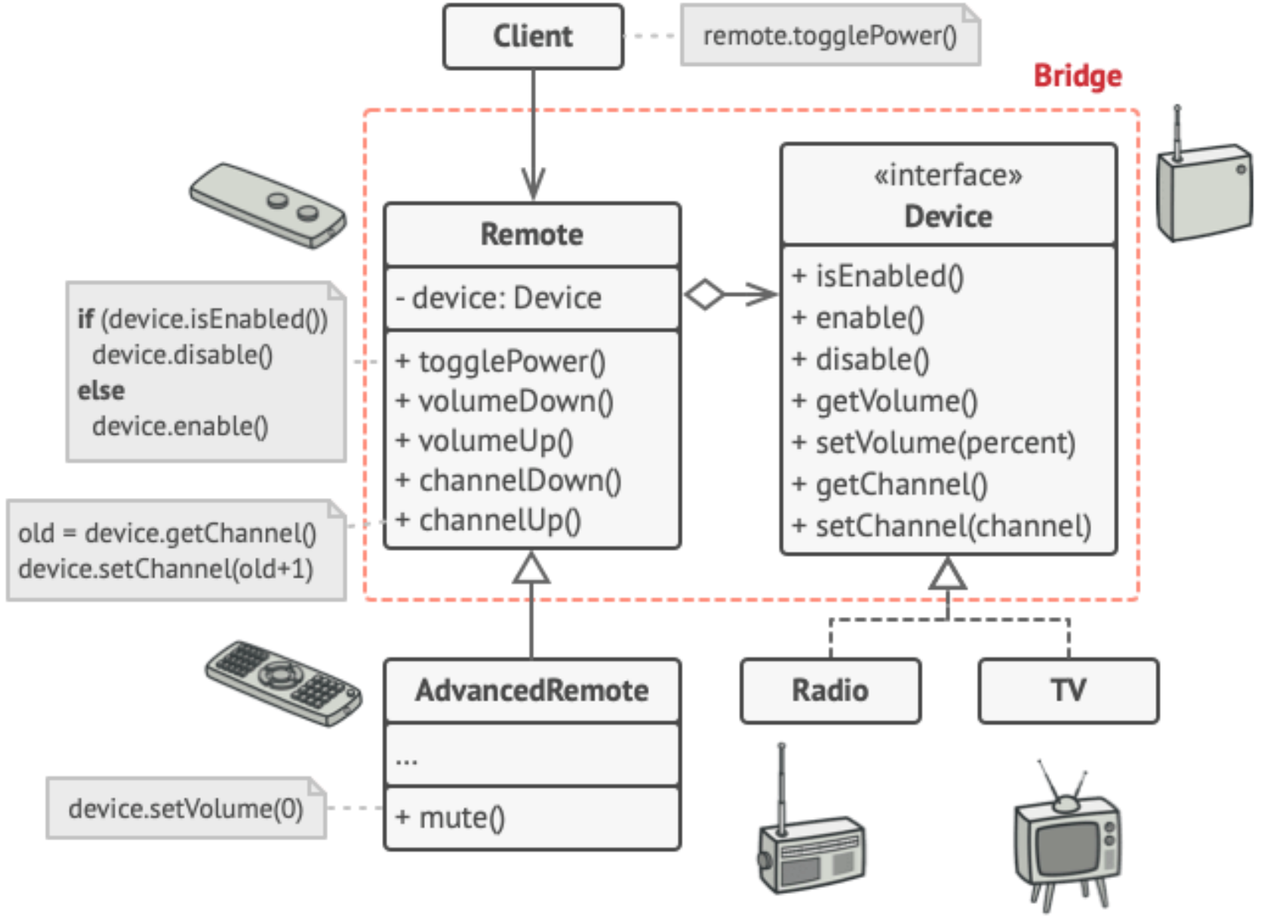
Bridge is a structural design pattern that lets you split a large class or a set of closely related classes into two separate hierarchies—abstraction and implementation—which can be developed independently of each other.







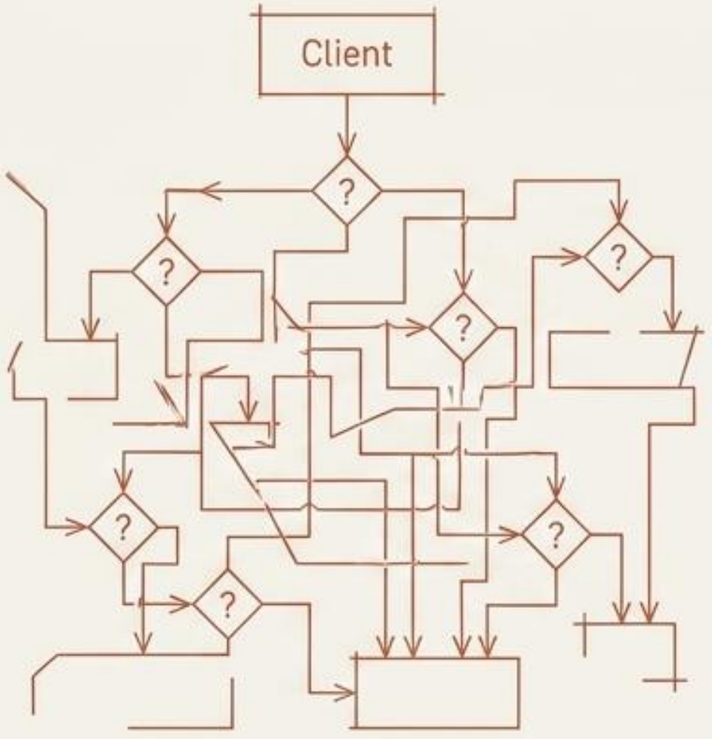

Client
5
abstraction.feature1()
Bridge
1
Abstraction
- i: Implementation
+ feature1()
+ feature2()
2
«interface»
Implementation
+ method1()
+ method2()
+ method3()
i.method1()
i.method2()
i.method3()
(optional)
4
Refined Abstraction
...
+ featureN()
3
Concrete
Implementations
i.methodN()
i.methodM()


Client
remote.togglePower()
Bridge
Remote
- device: Device
+ togglePower()
+ volumeDown()
+ volumeUp()
+ channelDown()
+ channelUp()
«interface»
Device
+ isEnabled()
+ enable()
+ disable()
+ getVolume()
+ setVolume(percent)
+ getChannel()
+ setChannel(channel)
if (device.isEnabled())
device.disable()
else
device.enable()
old = device.getChannel()
device.setChannel(old+1)
AdvancedRemote
...
+ mute()
device.setVolume(0)
Radio
TV

```
// The "abstraction" defines the interface for the "control"
// part of the two class hierarchies. It maintains a reference
// to an object of the "implementation" hierarchy and delegates
// all of the real work to this object.
class RemoteControl is
    protected field device: Device
    constructor RemoteControl(device: Device) is
        this.device = device
    method togglePower() is
        if (device.isEnabled()) then
            device.disable()
        else
            device.enable()
    method volumeDown() is
        device.setVolume(device.getVolume() - 10)
    method volumeUp() is
        device.setVolume(device.getVolume() + 10)
    method channelDown() is
        device.setChannel(device.getChannel() - 1)
    method channelUp() is
        device.setChannel(device.getChannel() + 1)


// You can extend classes from the abstraction hierarchy
// independently from device classes.
class AdvancedRemoteControl extends RemoteControl is
    method mute() is
        device.setVolume(0)


// The "implementation" interface declares methods common to
all
// concrete implementation classes. It doesn't have to match
the
// abstraction's interface. In fact, the two interfaces can be
// entirely different. Typically the implementation interface
// provides only primitive operations, while the abstraction
// defines higher-level operations based on those primitives.
interface Device is
    method isEnabled()
    method enable()
    method disable()
    method getVolume()
    method setVolume(percent)
    method getChannel()
    method setChannel(channel)


// All devices follow the same interface.
class Tv implements Device is
    // ...

class Radio implements Device is
    // ...


// Somewhere in client code.
tv = new Tv()
remote = new RemoteControl(tv)
remote.togglePower()

radio = new Radio()
remote = new AdvancedRemoteControl(radio)
```

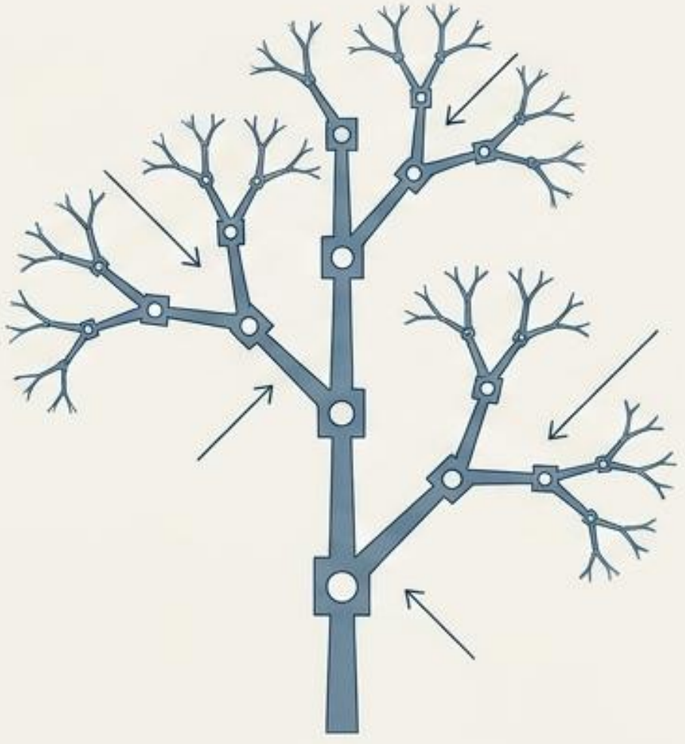
THE FRICTION

Clients must write complex conditional checks to differentiate between single items and groups of items.

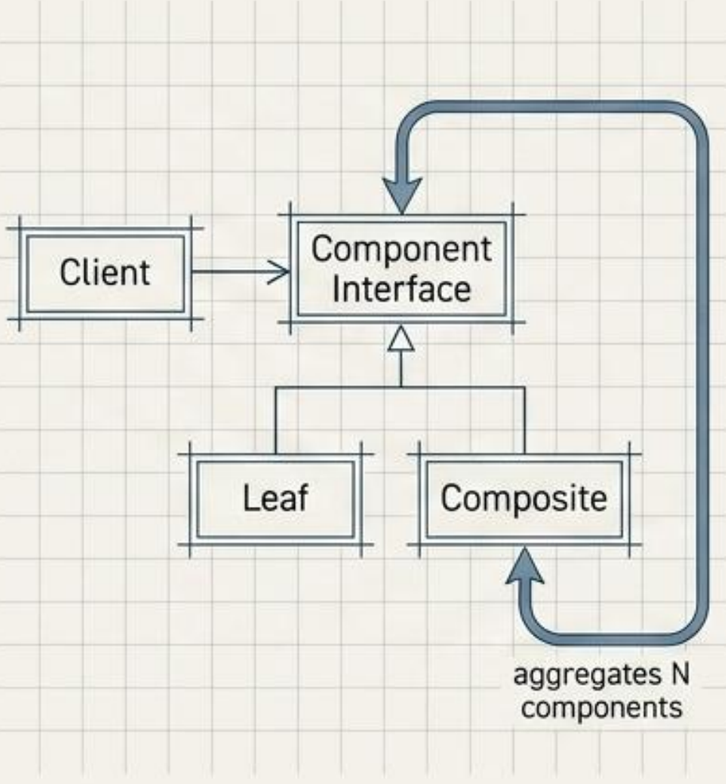


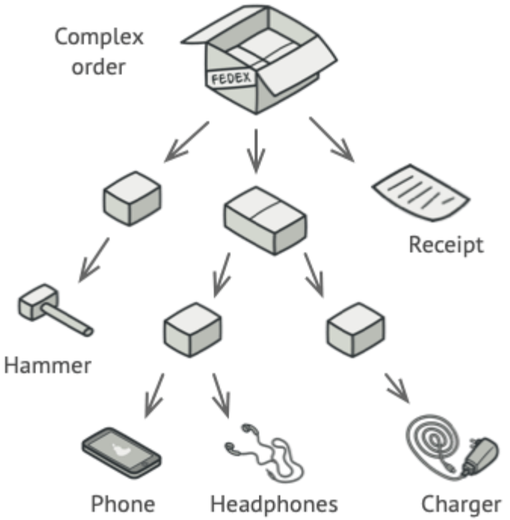
THE MECHANISM

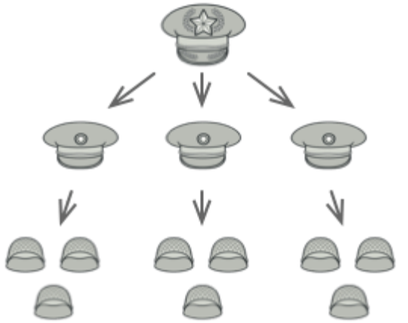
Compose objects into tree structures. Clients treat individual objects (leaves) and compositions (branches) uniformly.

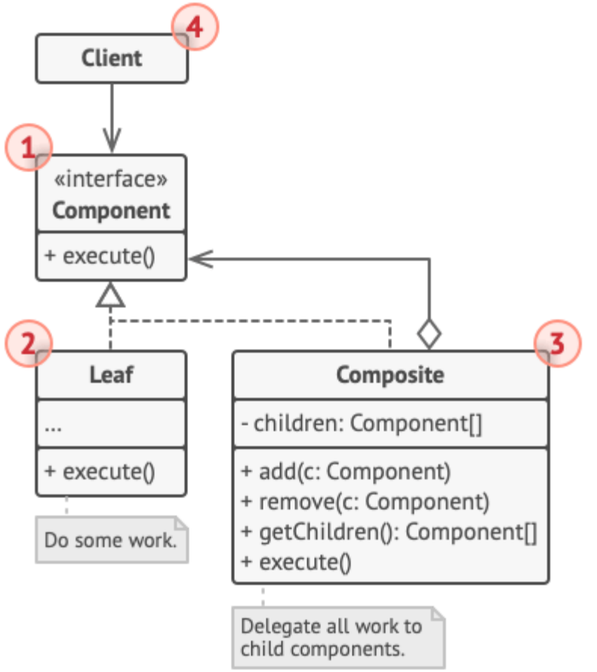


THE SCHEMATIC: COMPOSITE

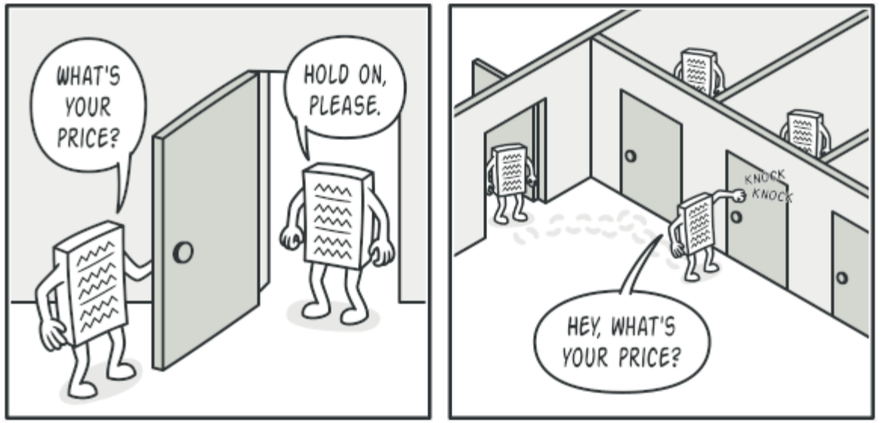


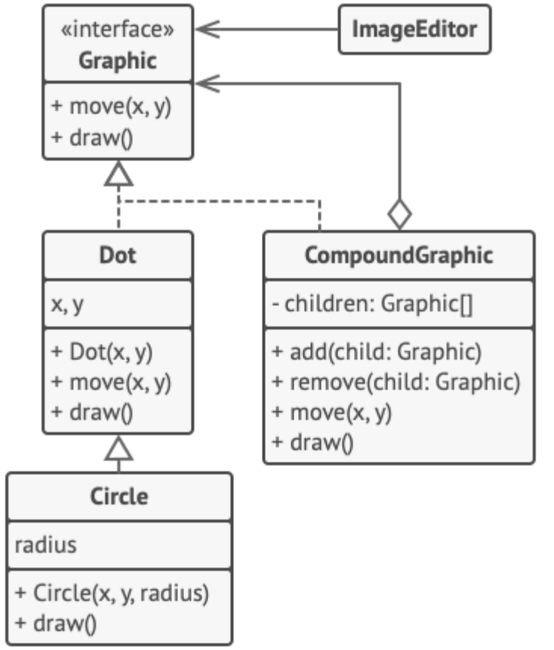






Composite is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.

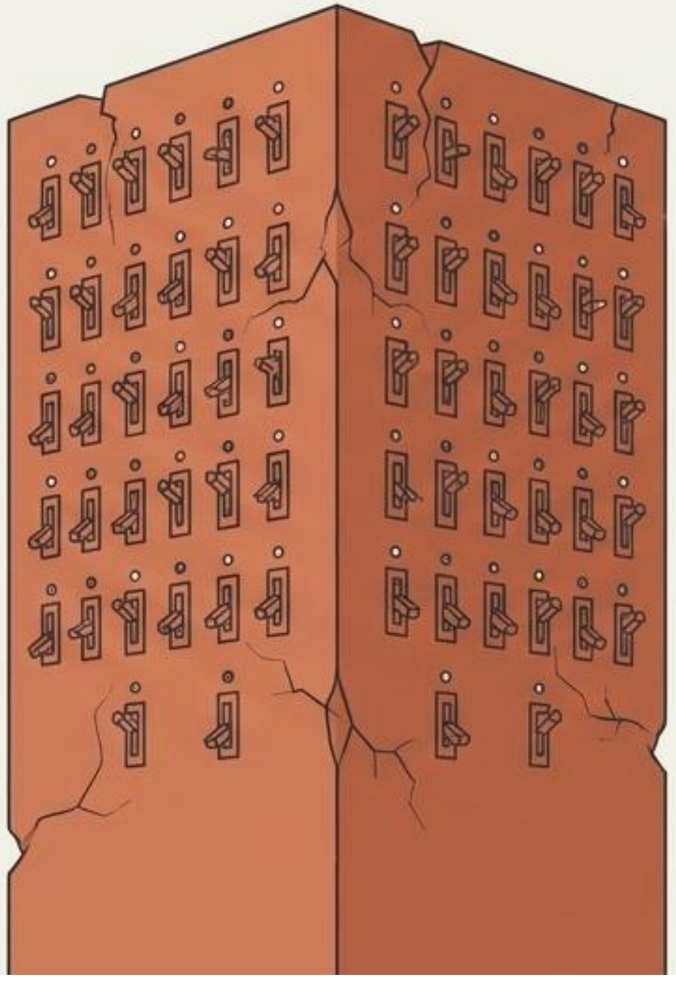




```
// The component interface declares common operations for both
// simple and complex objects of a composition.
interface Graphic is
    method move(x, y)
    method draw()

// The leaf class represents end objects of a composition. A
// leaf object can't have any sub-objects. Usually, it's leaf
// objects that do the actual work, while composite objects only
// delegate to their sub-components.
class Dot implements Graphic is
    field x, y

    constructor Dot(x, y) { ... }

    method move(x, y) is
        this.x += x, this.y += y

    method draw() is
        // Draw a dot at X and Y.

// All component classes can extend other components.
class Circle extends Dot is
    field radius

    constructor Circle(x, y, radius) { ... }

    method draw() is
        // Draw a circle at X and Y with radius R.

// The composite class represents complex components that may
// have children. Composite objects usually delegate the actual
// work to their children and then "sum up" the result.
class CompoundGraphic implements Graphic is
    field children: array of Graphic

    // A composite object can add or remove other components
    // (both simple or complex) to or from its child list.
    method add(child: Graphic) is
        // Add a child to the array of children.

    method remove(child: Graphic) is
        // Remove a child from the array of children.

    method move(x, y) is
        foreach (child in children) do
            child.move(x, y)

    // A composite executes its primary logic in a particular
    // way. It traverses recursively through all its children,
    // collecting and summing up their results. Since the
    // composite's children pass these calls to their own
    // children and so forth, the whole object tree is traversed
    // as a result.
    method draw() is
        // 1. For each child component:
        //     - Draw the component.
        //     - Update the bounding rectangle.
        // 2. Draw a dashed rectangle using the bounding
        // coordinates.


// The client code works with all the components via their base
// interface. This way the client code can support simple leaf
// components as well as complex composites.
class ImageEditor is
    field all: CompoundGraphic

    method load() is
        all = new CompoundGraphic()
        all.add(new Dot(1, 2))
        all.add(new Circle(5, 3, 10))
        // ...

    // Combine selected components into one complex composite
    // component.
    method groupSelected(components: array of Graphic) is
        group = new CompoundGraphic()
        foreach (component in components) do
            group.add(component)
            all.remove(component)
        all.add(group)
        // All components will be drawn.
        all.draw()
```

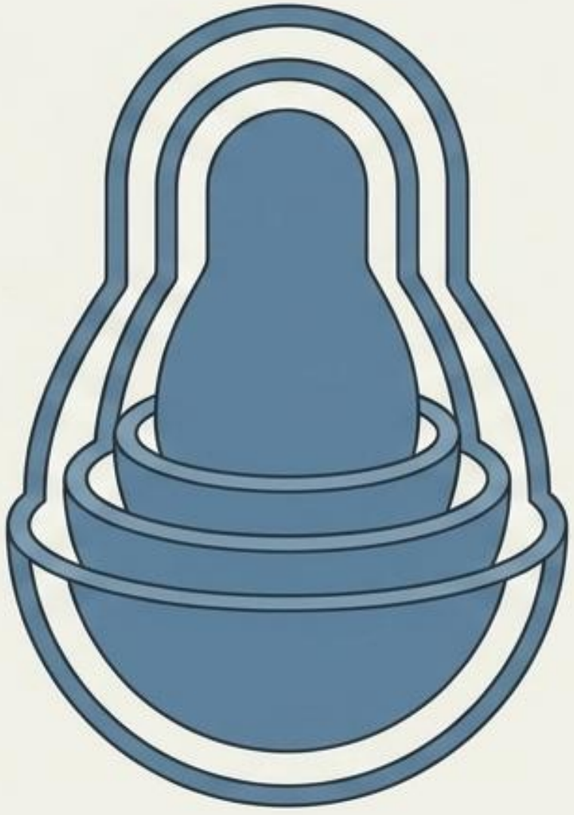
THE FRICTION

Bloated monolithic classes attempting to hardcode every possible combination of optional behaviors.

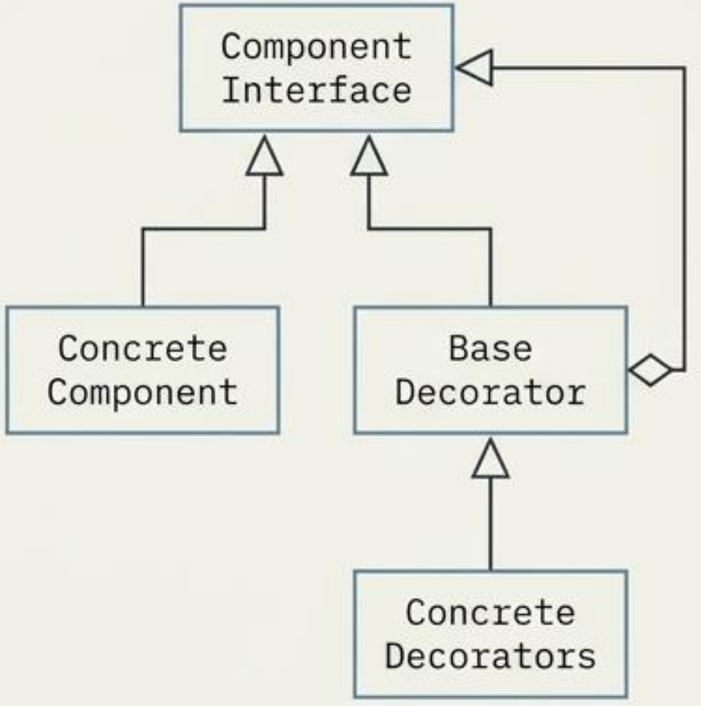


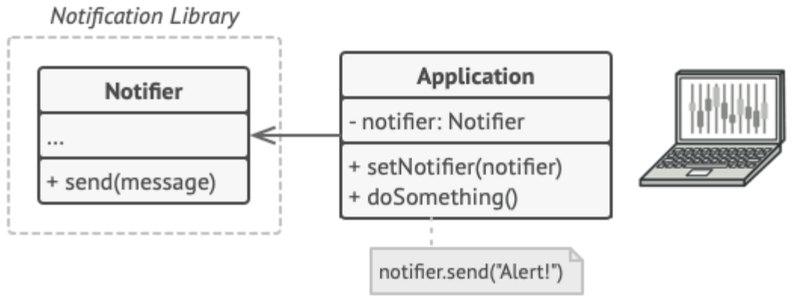
THE MECHANISM

Attach new behaviors dynamically by placing objects inside special wrapper objects. Stack wrappers infinitely.



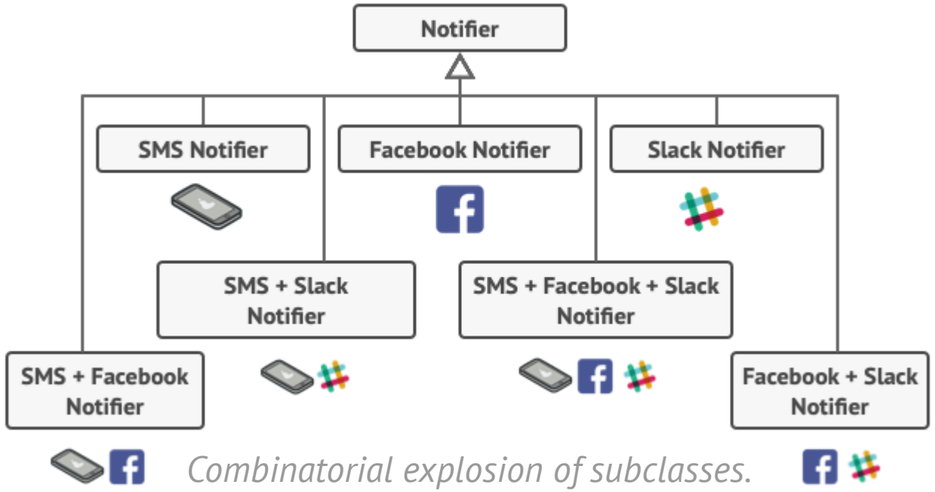
THE SCHEMATIC: DECORATOR



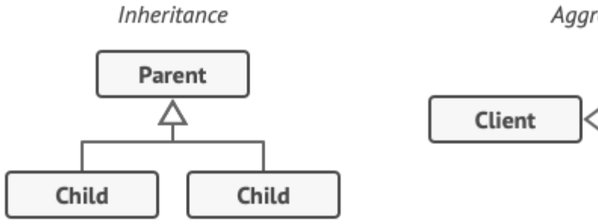


A program could use the notifier class to send notifications about important events to a predefined set of emails.

Each notification type is implemented as a notifier's subclass

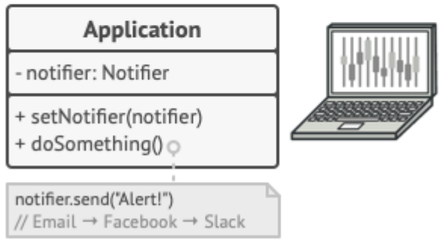


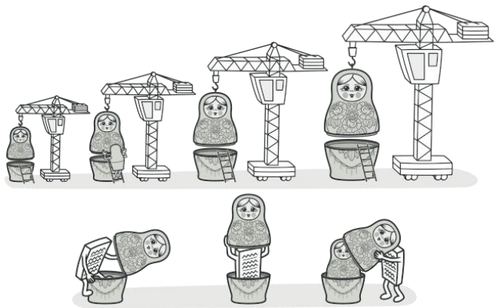
Combinatorial explosion of subclasses.



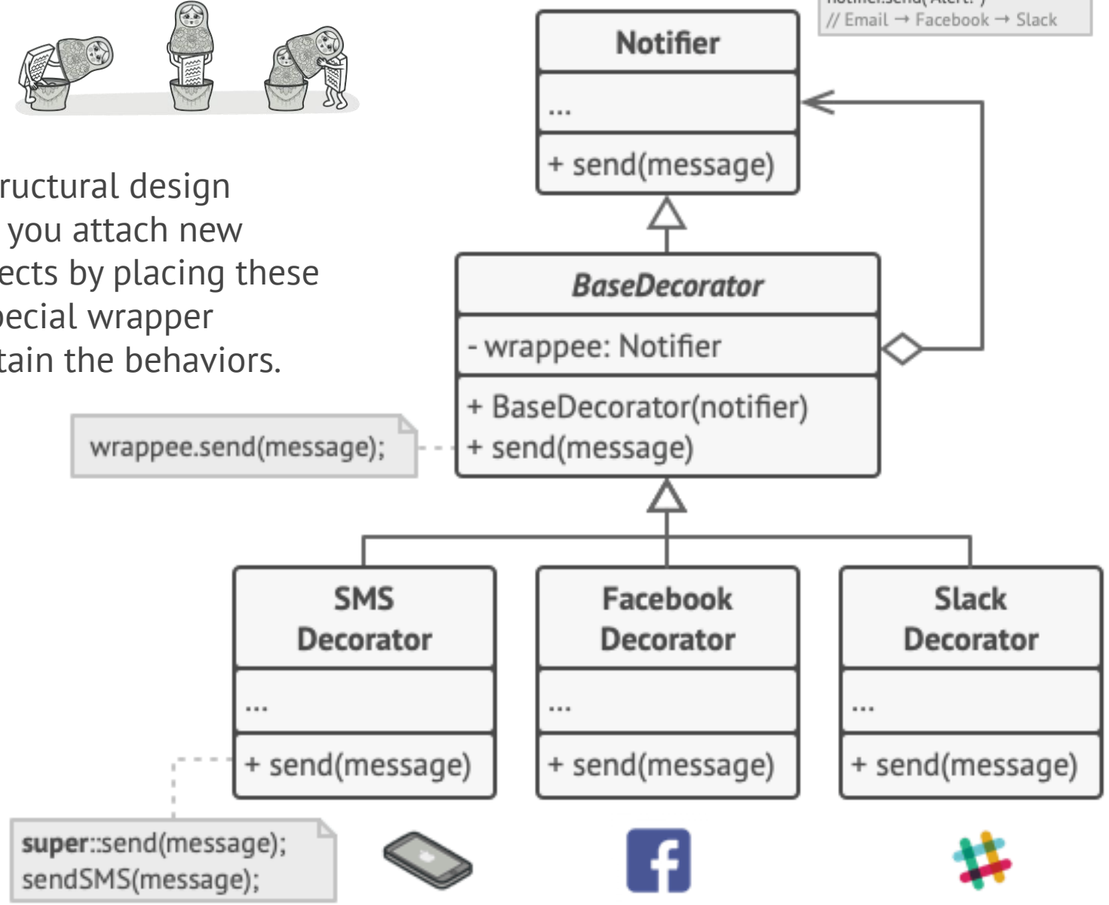
Apps might configure complex stacks of notification decorators.

```
stack = new Notifier()
if (facebookEnabled)
    stack = new FacebookDecorator(stack)
if (slackEnabled)
    stack = new SlackDecorator(stack)

app.setNotifier(stack)
```

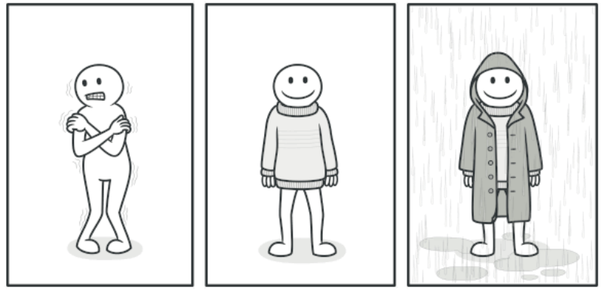




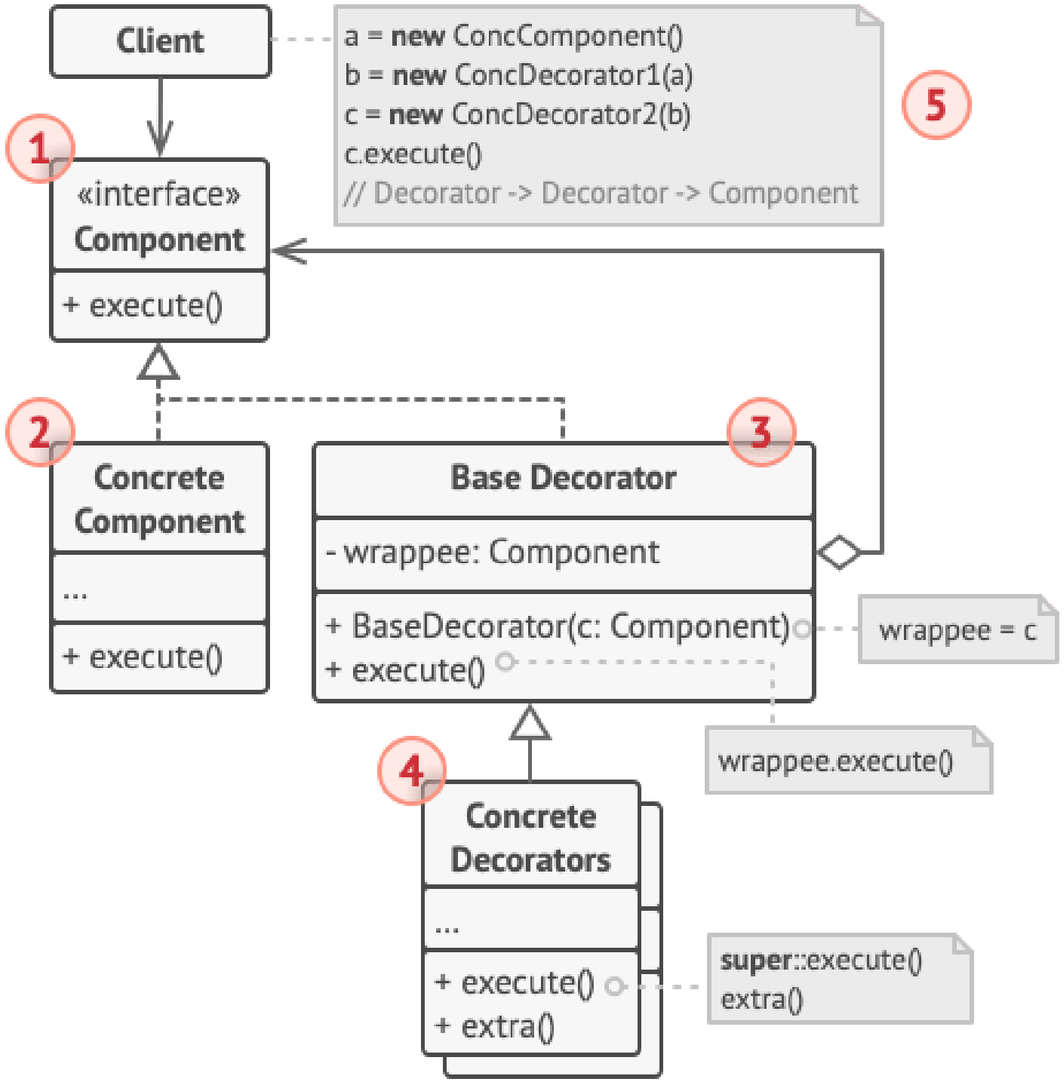
Decorator is a structural design pattern that lets you attach new behaviors to objects by placing these objects inside special wrapper objects that contain the behaviors.

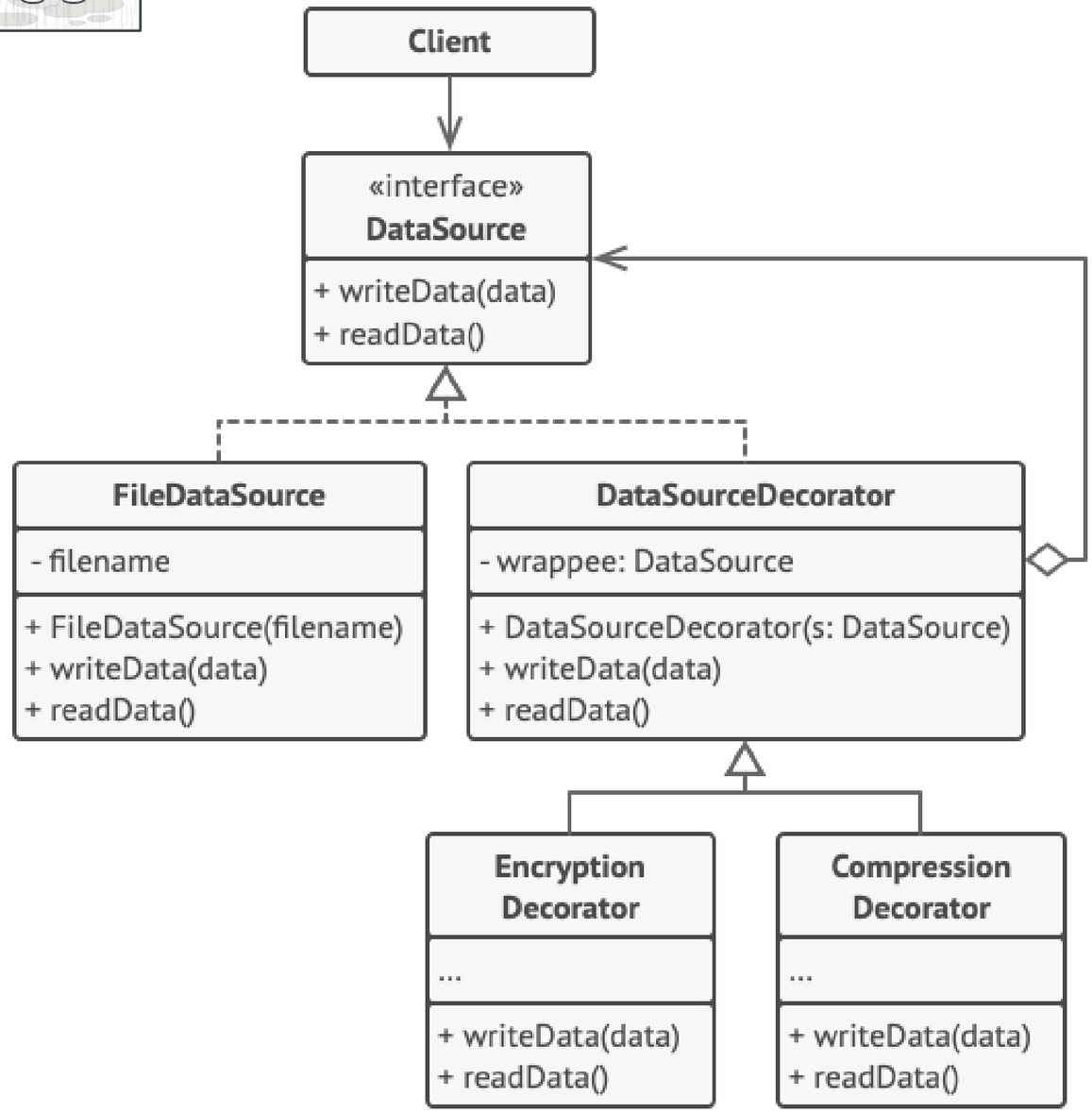


Various notification methods become decorators.



You get a combined effect from wearing multiple pieces of clothing.

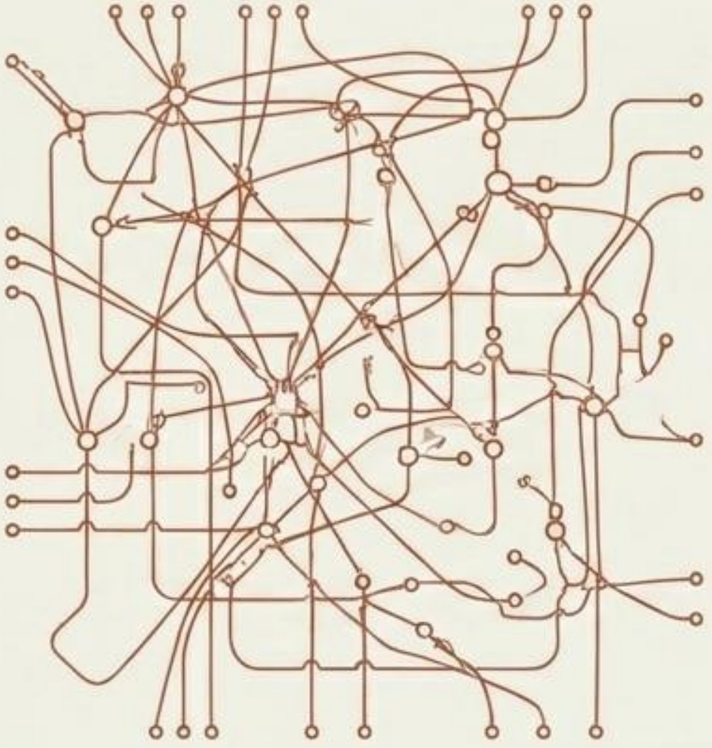




```
// The component interface defines operations that can be
// altered by decorators.
interface DataSource is
    method writeData(data)
    method readData():data

// Concrete components provide default implementations for the
// operations. There might be several variations of these
// classes in a program.
class FileDataSource implements DataSource is
    constructor FileDataSource(filename) { ... }

    method writeData(data) is
        // Write data to file.

    method readData():data is
        // Read data from file.

// The base decorator class follows the same interface as the
// other components. The primary purpose of this class is to
// define the wrapping interface for all concrete decorators.
// The default implementation of the wrapping code might
include
// a field for storing a wrapped component and the means to
// initialize it.
class DataSourceDecorator implements DataSource is
    protected field wrappee: DataSource

    constructor DataSourceDecorator(source: DataSource) is
        wrappee = source

    // The base decorator simply delegates all work to the
    // wrapped component. Extra behaviors can be added in
    // concrete decorators.
    method writeData(data) is
        wrappee.writeData(data)

    // Concrete decorators may call the parent implementation
of
    // the operation instead of calling the wrapped object
    // directly. This approach simplifies extension of
decorator
    // classes.
    method readData():data is
        return wrappee.readData()

// Concrete decorators must call methods on the wrapped
object,
// but may add something of their own to the result.
Decorators
// can execute the added behavior either before or after the
// call to a wrapped object.
class EncryptionDecorator extends DataSourceDecorator is
    method writeData(data) is
        // 1. Encrypt passed data.
        // 2. Pass encrypted data to the wrappee's writeData
        // method.

    method readData():data is
        // 1. Get data from the wrappee's readData method.
        // 2. Try to decrypt it if it's encrypted.
        // 3. Return the result.

// You can wrap objects in several layers of decorators.
class CompressionDecorator extends DataSourceDecorator is
    method writeData(data) is
        // 1. Compress passed data.
        // 2. Pass compressed data to the wrappee's writeData
        // method.

    method readData():data is
        // 1. Get data from the wrappee's readData method.
        // 2. Try to decompress it if it's compressed.
        // 3. Return the result.


// Option 1. A simple example of a decorator assembly.
class Application is
    method dumbUsageExample() is
        source = new FileDataSource("somefile.dat")
        source.writeData(salaryRecords)
        // The target file has been written with plain data.

        source = new CompressionDecorator(source)
        source.writeData(salaryRecords)
        // The target file has been written with compressed
        // data.

        source = new EncryptionDecorator(source)
        // The source variable now contains this:
        // Encryption > Compression > FileDataSource
        source.writeData(salaryRecords)
        // The file has been written with compressed and
        // encrypted data.

// Option 2. Client code that uses an external data source.
// SalaryManager objects neither know nor care about data
// storage specifics. They work with a pre-configured data
// source received from the app configurator.
class SalaryManager is
    field source: DataSource

    constructor SalaryManager(source: DataSource) { ... }

    method load() is
        return source.readData()

    method save() is
        source.writeData(salaryRecords)
    // ...Other useful methods...


// The app can assemble different stacks of decorators at
// runtime, depending on the configuration or environment.
class ApplicationConfigurator is
    method configurationExample() is
        source = new FileDataSource("salary.dat")
        if (enabledEncryption)
            source = new EncryptionDecorator(source)
        if (enabledCompression)
            source = new CompressionDecorator(source)

        logger = new SalaryManager(source)
        salary = logger.load()
    // ...
```

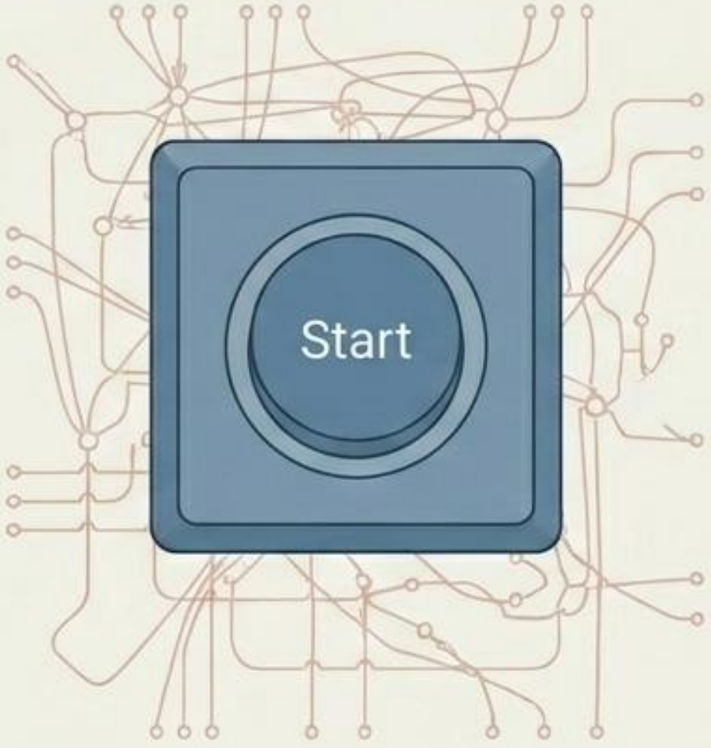
THE FRICTION

Clients are forced to manage the intricate, fragile dependencies of a highly complex subsystem.

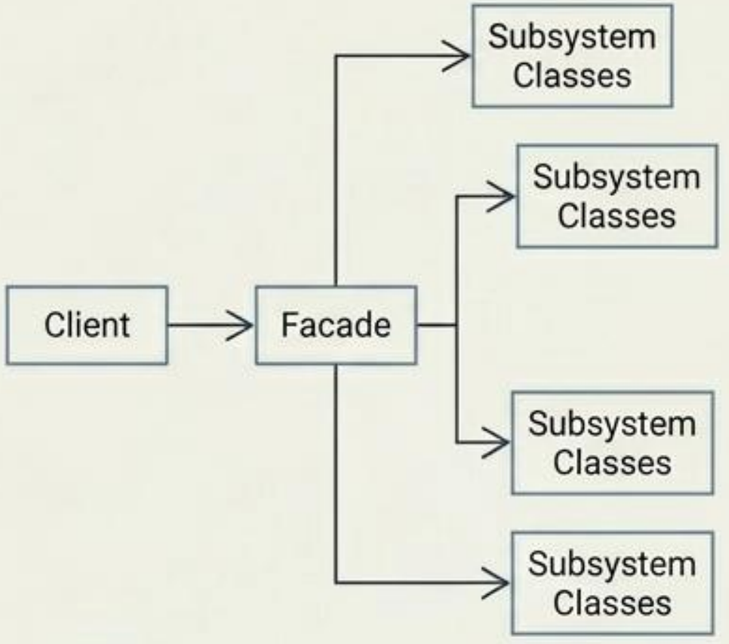


THE MECHANISM

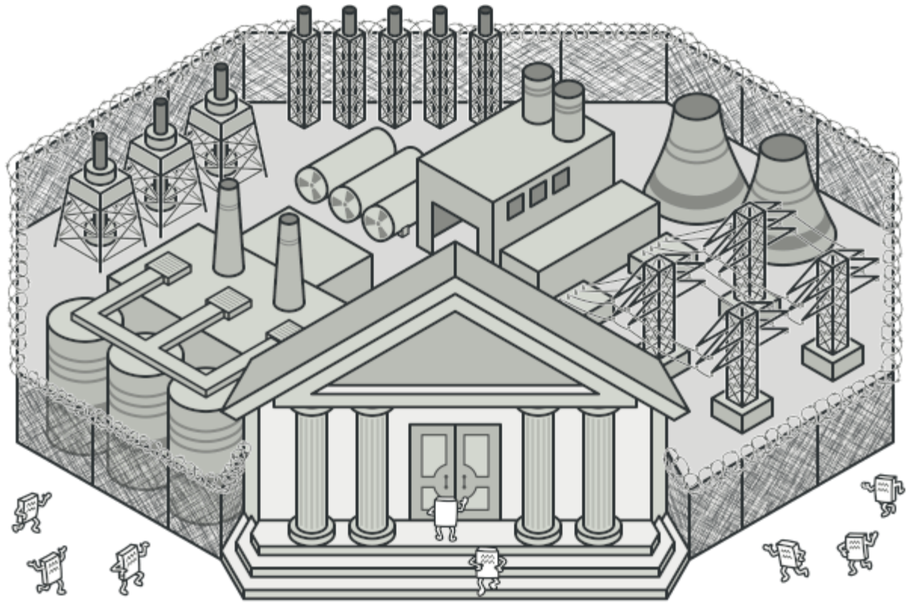
Provide a simplified, higher-level interface to shield the client from the underlying architectural mess.

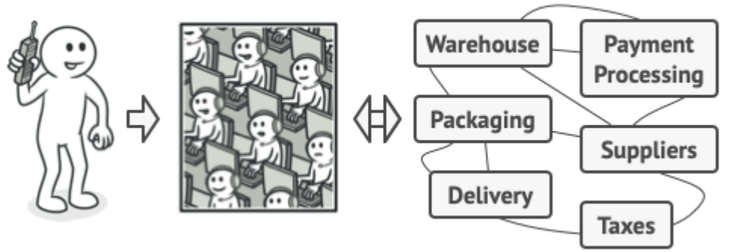


THE SCHEMATIC: FACADE

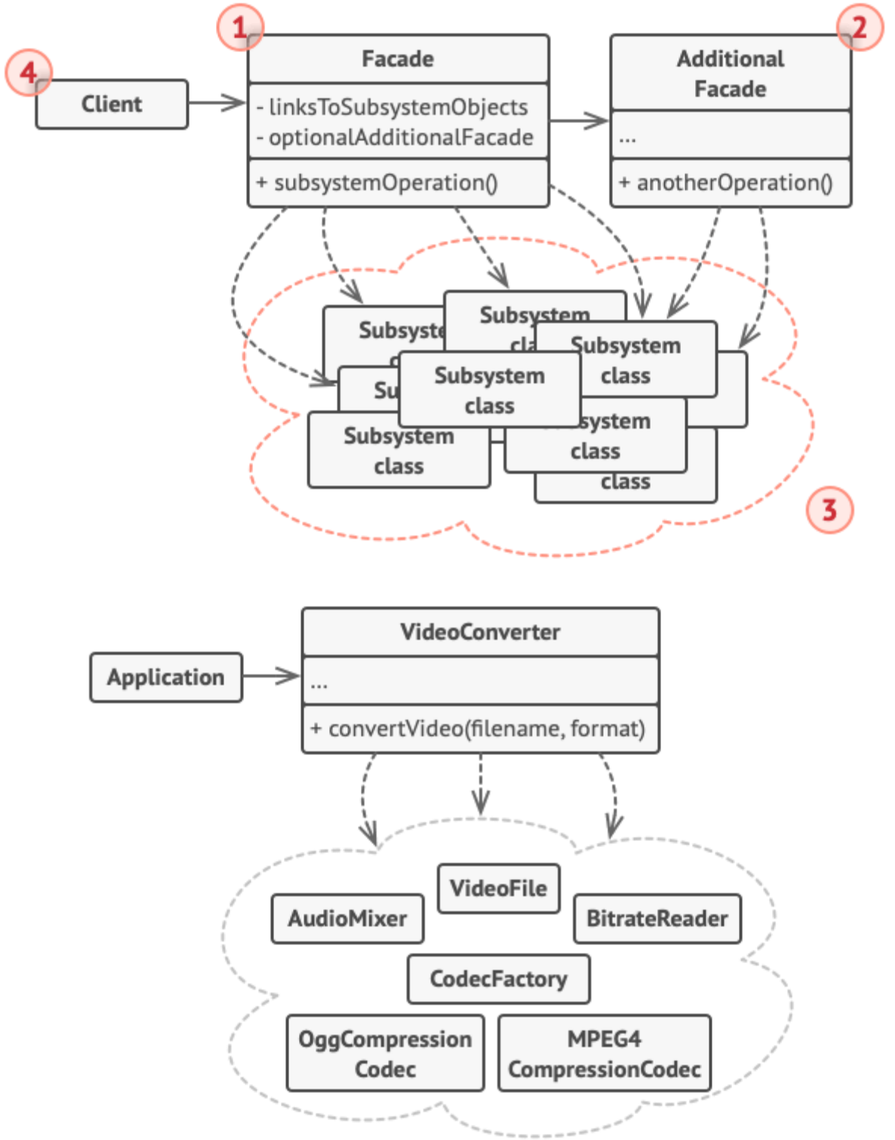


Facade is a structural design pattern that provides a simplified interface to a library, a framework, or any other complex set of classes.





Placing orders by phone.



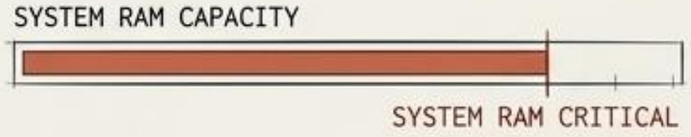
An example of isolating multiple dependencies within a single facade class.

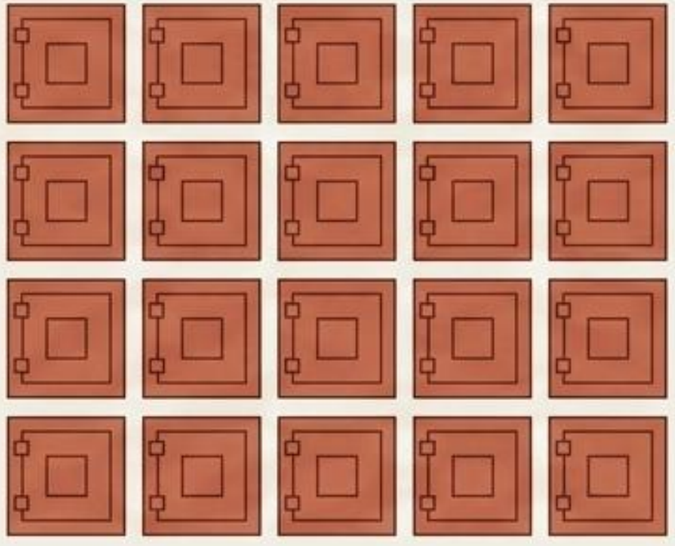
```
// These are some of the classes of a complex 3rd-party video
// conversion framework. We don't control that code, therefore
// can't simplify it.

class VideoFile
// ...

class OggCompressionCodec
// ...

class MPEG4CompressionCodec
// ...

class CodecFactory
// ...

class BitrateReader
// ...

class AudioMixer
// ...


// We create a facade class to hide the framework's complexity
// behind a simple interface. It's a trade-off between
// functionality and simplicity.
```

```
class VideoConverter is
    method convert(filename, format):File is
        file = new VideoFile(filename)
        sourceCodec = (new CodecFactory).extract(file)
        if (format == "mp4")
            destinationCodec = new MPEG4CompressionCodec()
        else
            destinationCodec = new OggCompressionCodec()
        buffer = BitrateReader.read(filename, sourceCodec)
        result = BitrateReader.convert(buffer, destinationCodec)
        result = (new AudioMixer()).fix(result)
        return new File(result)

// Application classes don't depend on a billion classes
// provided by the complex framework. Also, if you decide to
// switch frameworks, you only need to rewrite the facade class.
class Application is
    method main() is
        convertor = new VideoConverter()
        mp4 = convertor.convert("funny-cats-video.ogg", "mp4")
        mp4.save()
```

THE FRICTION

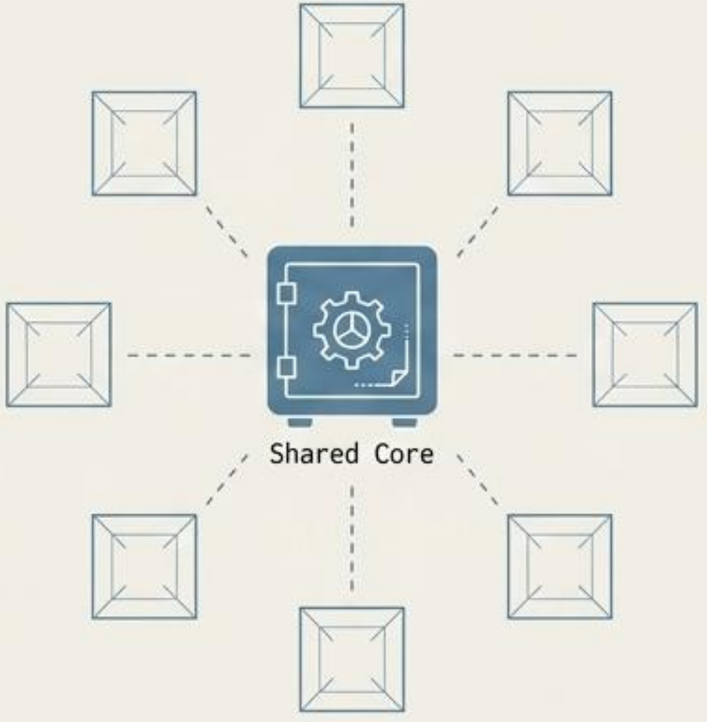
System memory exhausts rapidly when millions of identical objects are instantiated with duplicate state data.



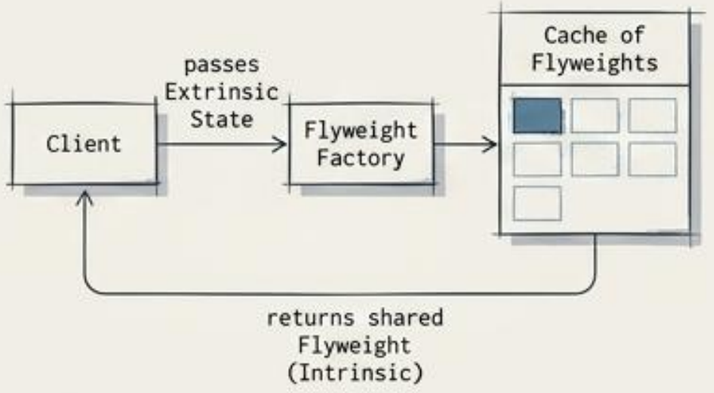


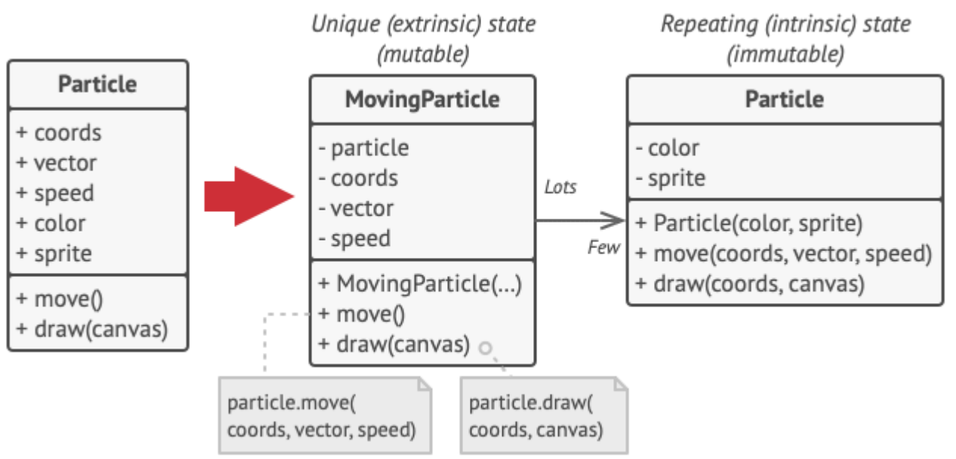
THE MECHANISM

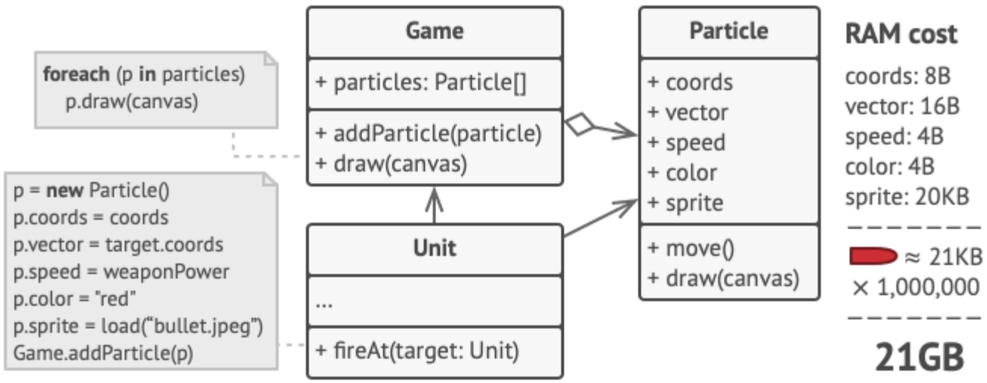
Separate intrinsic (shared) state from extrinsic (contextual) state. Share the core blueprint to reduce memory footprint.



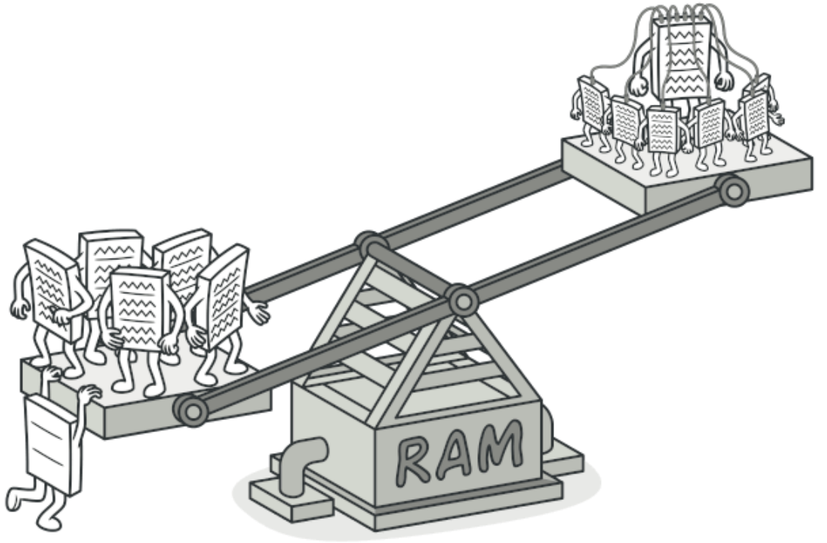
THE SCHEMATIC: FLYWEIGHT

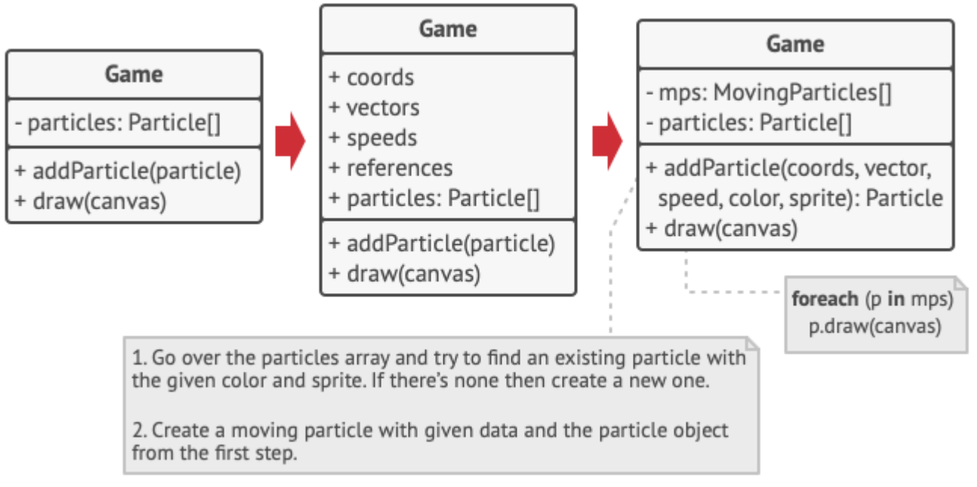


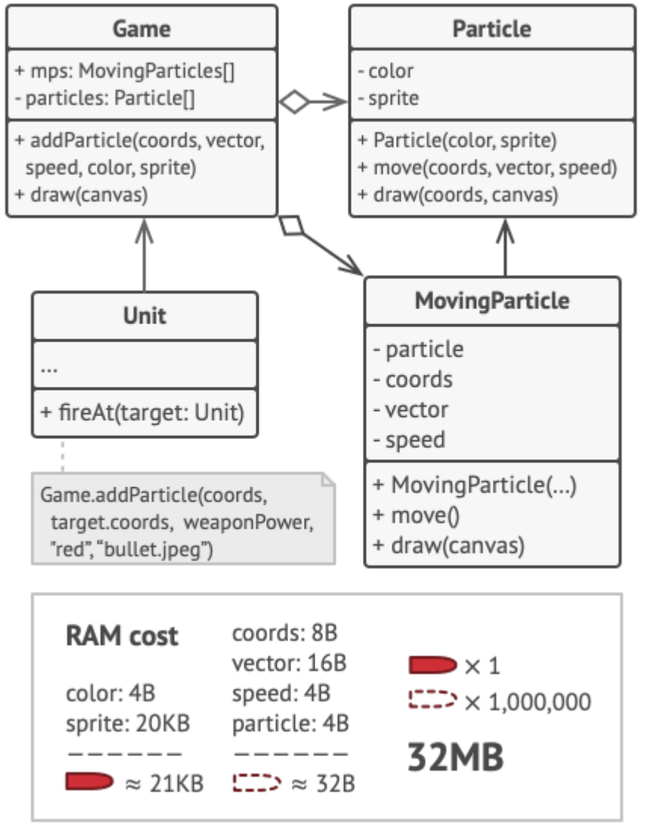


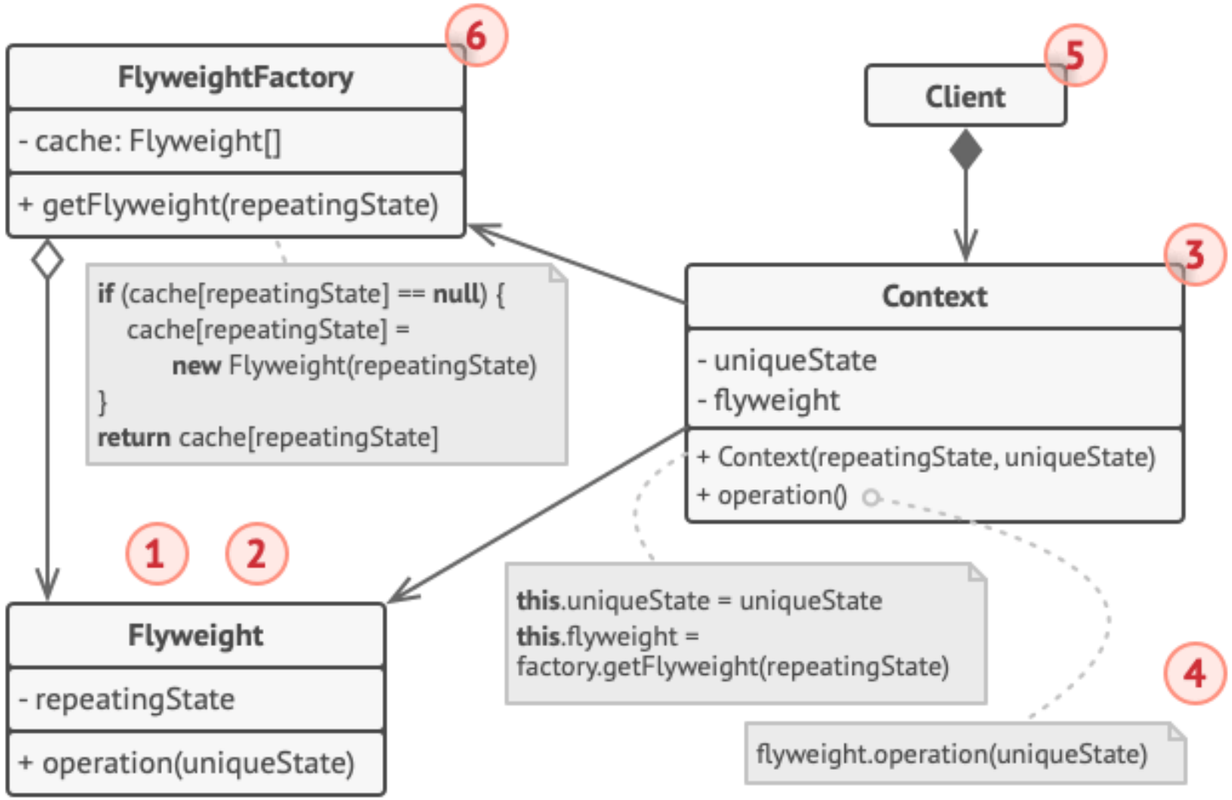


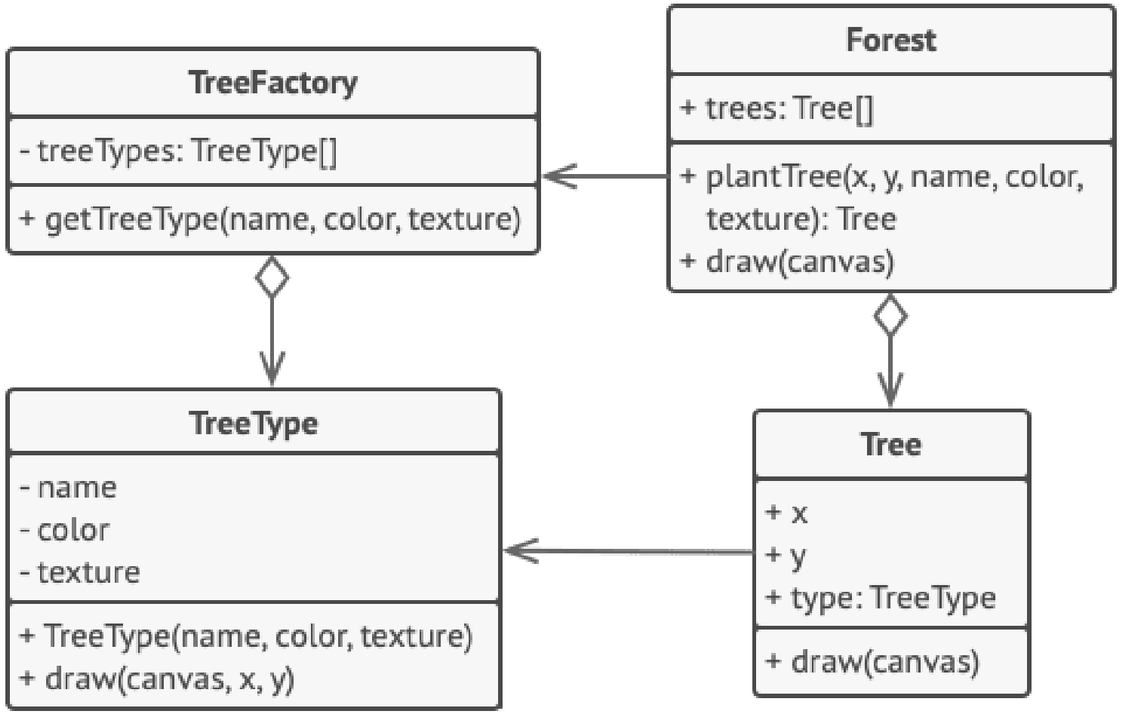
Flyweight is a structural design pattern that lets you fit more objects into the available amount of RAM by sharing common parts of state between multiple objects instead of keeping all of the data in each object.








6
FlyweightFactory
- cache: Flyweight[]
+ getFlyweight(repeatingState)
5
Client
3
Context
- uniqueState
- flyweight
+ Context(repeatingState, uniqueState)
+ operation()
if (cache[repeatingState] == null) {
cache[repeatingState] =
new Flyweight(repeatingState)
}
return cache[repeatingState]
1
2
Flyweight
- repeatingState
+ operation(uniqueState)
this.uniqueState = uniqueState
this.flyweight =
factory.getFlyweight(repeatingState)
4
flyweight.operation(uniqueState)


TreeFactory
- treeTypes: TreeType[]
+ getTreeType(name, color, texture)
Forest
+ trees: Tree[]
+ plantTree(x, y, name, color,
texture): Tree
+ draw(canvas)
TreeType
- name
- color
- texture
+ TreeType(name, color, texture)
+ draw(canvas, x, y)
Tree
+ x
+ y
+ type: TreeType
+ draw(canvas)

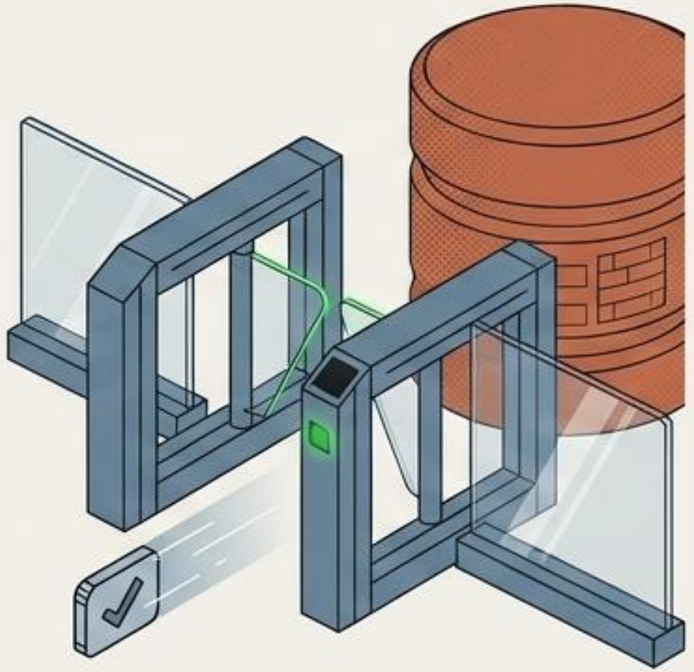
```
// The flyweight class contains a portion of the state of a
// tree. These fields store values that are unique for each
// particular tree. For instance, you won't find here the tree
// coordinates. But the texture and colors shared between many
// trees are here. Since this data is usually BIG, you'd waste
a
// lot of memory by keeping it in each tree object. Instead, we
// can extract texture, color and other repeating data into a
// separate object which lots of individual tree objects can
// reference.
class TreeType is
    field name
    field color
    field texture
    constructor TreeType(name, color, texture) { ... }
    method draw(canvas, x, y) is
        // 1. Create a bitmap of a given type, color & texture.
        // 2. Draw the bitmap on the canvas at X and Y coords.

// Flyweight factory decides whether to re-use existing
// flyweight or to create a new object.
class TreeFactory is
    static field treeTypes: collection of tree types
    static method getTreeType(name, color, texture) is
        type = treeTypes.find(name, color, texture)
        if (type == null)
            type = new TreeType(name, color, texture)
            treeTypes.add(type)
        return type

// The contextual object contains the extrinsic part of the
tree
// state. An application can create billions of these since
they
// are pretty small: just two integer coordinates and one
// reference field.
class Tree is
    field x,y
    field type: TreeType
    constructor Tree(x, y, type) { ... }
    method draw(canvas) is
        type.draw(canvas, this.x, this.y)

// The Tree and the Forest classes are the flyweight's clients.
// You can merge them if you don't plan to develop the Tree
// class any further.
class Forest is
    field trees: collection of Trees

    method plantTree(x, y, name, color, texture) is
        type = TreeFactory.getTreeType(name, color, texture)
        tree = new Tree(x, y, type)
        trees.add(tree)

    method draw(canvas) is
        foreach (tree in trees) do
            tree.draw(canvas)
```

THE FRICTION

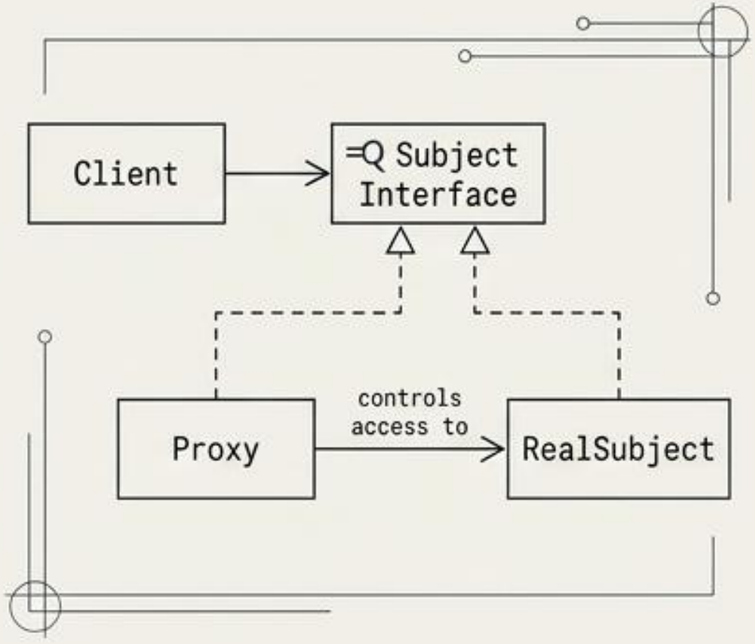
Massive, resource-heavy objects initialize immediately upon system boot, bottlenecking performance before they are even needed.

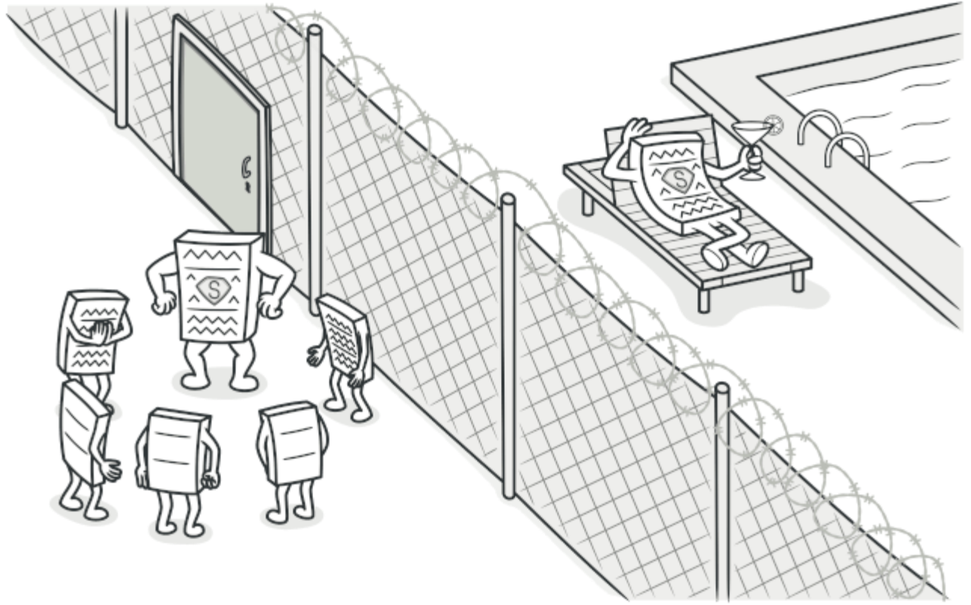
THE MECHANISM

Provide a placeholder or surrogate object to control access, delay initialization, or handle security checks.

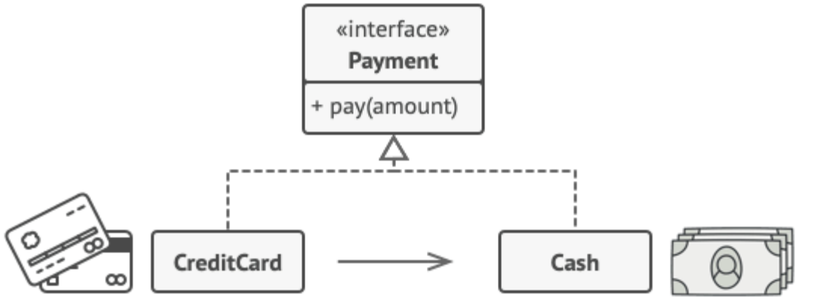


THE SCHEMATIC: PROXY

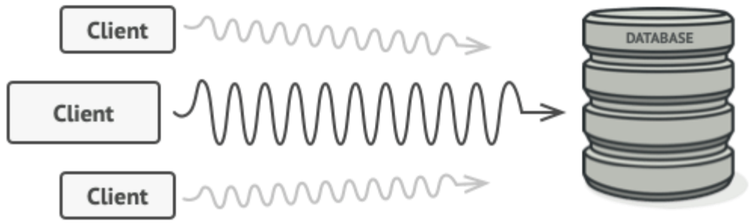


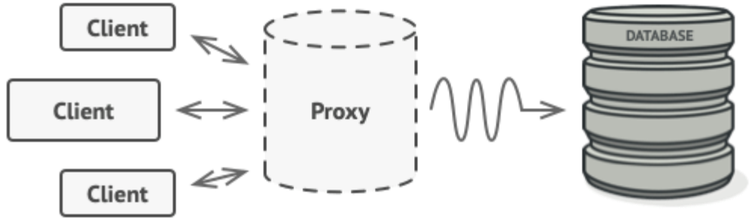


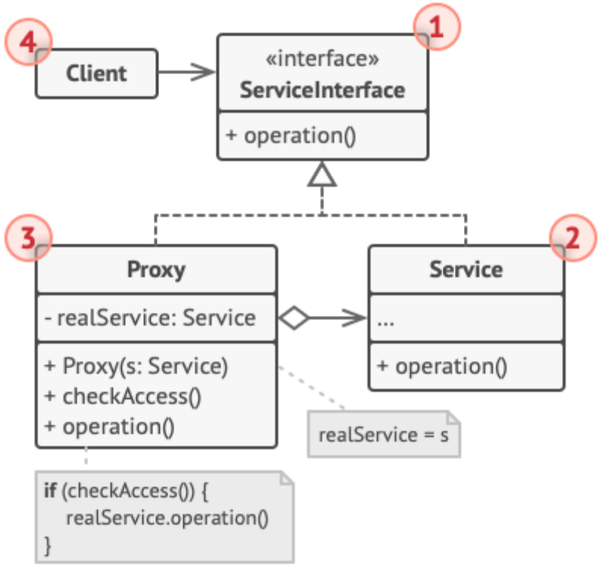
Proxy is a structural design pattern that lets you provide a substitute or placeholder for another object. A proxy controls access to the original object, allowing you to perform something either before or after the request gets through to the original object.

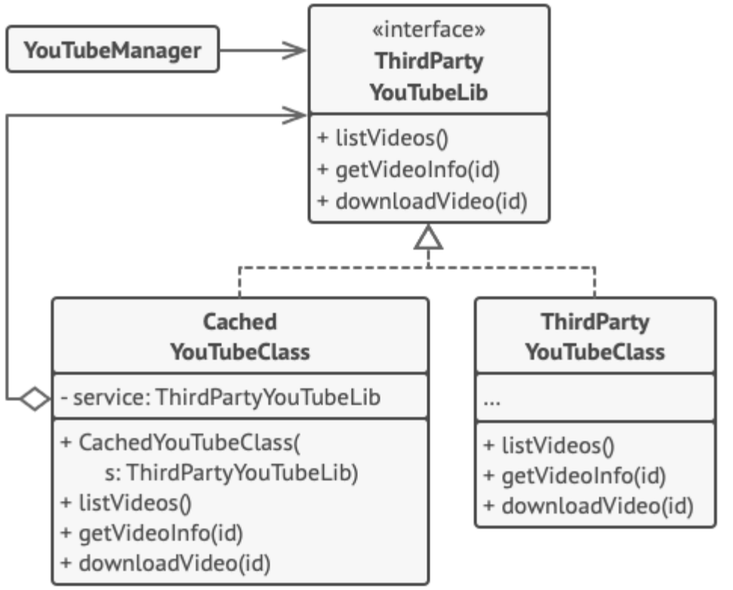


Credit cards can be used for payments just the same as cash.









Caching results of a service with a proxy.

```
// The interface of a remote service.
interface ThirdPartyYouTubeLib is
    method listVideos()
    method getVideoInfo(id)
    method downloadVideo(id)

// The concrete implementation of a service
connector. Methods
// of this class can request information from
YouTube. The speed
// of the request depends on a user's internet
connection as
// well as YouTube's. The application will slow
down if a lot of
// requests are fired at the same time, even if
they all request
// the same information.
class ThirdPartyYouTubeClass implements
ThirdPartyYouTubeLib is
    method listVideos() is
        // Send an API request to YouTube.

    method getVideoInfo(id) is
        // Get metadata about some video.

    method downloadVideo(id) is
        // Download a video file from YouTube.

// To save some bandwidth, we can cache request
results and keep
// them for some time. But it may be impossible
to put such code
// directly into the service class. For
example, it could have
// been provided as part of a third party
library and/or defined
// as `final`. That's why we put the caching
code into a new
// proxy class which implements the same
interface as the
// service class. It delegates to the service
object only when
// the real requests have to be sent.
class CachedYouTubeClass implements
ThirdPartyYouTubeLib is
    private field service: ThirdPartyYouTubeLib
    private field listCache, videoCache
    field needReset

    constructor CachedYouTubeClass(service:
ThirdPartyYouTubeLib) is
        this.service = service

    method listVideos() is
        if (listCache == null || needReset)
            listCache = service.listVideos()
        return listCache

    method getVideoInfo(id) is
        if (videoCache == null || needReset)
            videoCache =
service.getVideoInfo(id)
        return videoCache

    method downloadVideo(id) is
        if (!downloadExists(id) || needReset)
            service.downloadVideo(id)

// The GUI class, which used to work directly
with a service
// object, stays unchanged as long as it works
with the service
// object through an interface. We can safely
pass a proxy
// object instead of a real service object
since they both
// implement the same interface.
class YouTubeManager is
    protected field service:
ThirdPartyYouTubeLib

    constructor YouTubeManager(service:
ThirdPartyYouTubeLib) is
        this.service = service

    method renderVideoPage(id) is
        info = service.getVideoInfo(id)
        // Render the video page.

    method renderListPanel() is
        list = service.listVideos()
        // Render the list of video thumbnails.

    method reactOnUserInput() is
        renderVideoPage()
        renderListPanel()

// The application can configure proxies on the
fly.
class Application is
    method init() is
        aYouTubeService = new
ThirdPartyYouTubeClass()
        aYouTubeProxy = new
CachedYouTubeClass(aYouTubeService)
        manager = new
YouTubeManager(aYouTubeProxy)
        manager.reactOnUserInput()
```

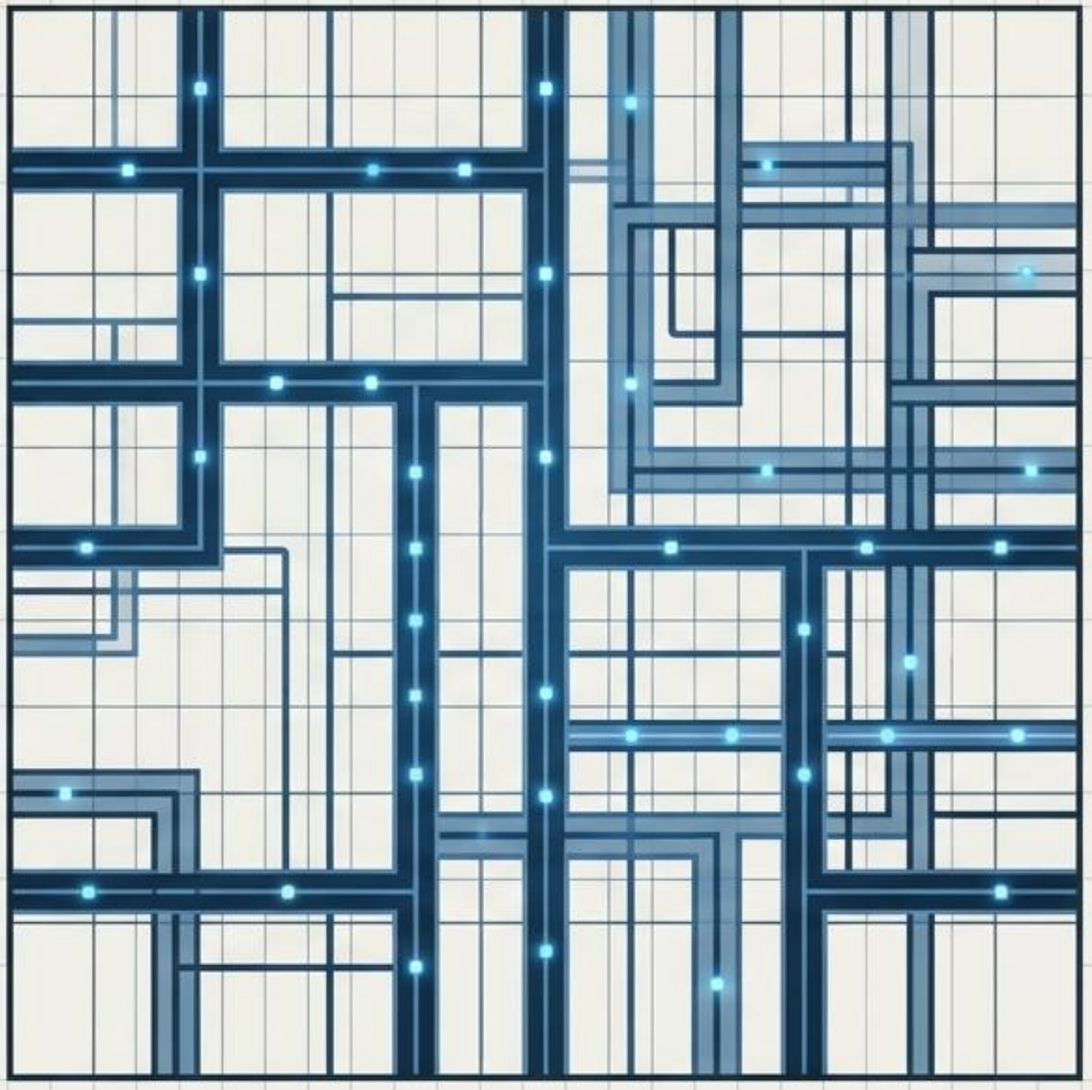
THE ARCHITECT'S DIAGNOSTIC MATRIX

PATTERN	THE CORE INTENT	PRIMARY ANALOGY	STRUCTURAL SIGNATURE
Adapter	**Translates** incompatible interfaces	Power Plug	`Wrapper translating A into B.`
Bridge	**Decouples** orthogonal dimensions	Remote & TV	`Two parallel, connected hierarchies.`
Composite	**Unifies** tree structures	Fractal Branches	`Recursive interface aggregation.`
Decorator	**Adds** dynamic behavior	Nesting Dolls	`Intercepting, stackable wrapper.`
Facade	**Simplifies** subsystem complexity	Control Panel	`Single entry point to many.`
Flyweight	**Conserves** memory footprint	Shared Blueprint	`Intrinsic vs. Extrinsic state cache.`
Proxy	**Controls** object access	The Gatekeeper	`Surrogate intercepting specific requests.`

Master the Blueprint, Build the Future

Design patterns are not rigid laws; they are diagnostic tools. Real architectural mastery lies in accurately identifying the friction within your system and applying the precise structural pattern required to resolve it—without introducing unnecessary complexity.

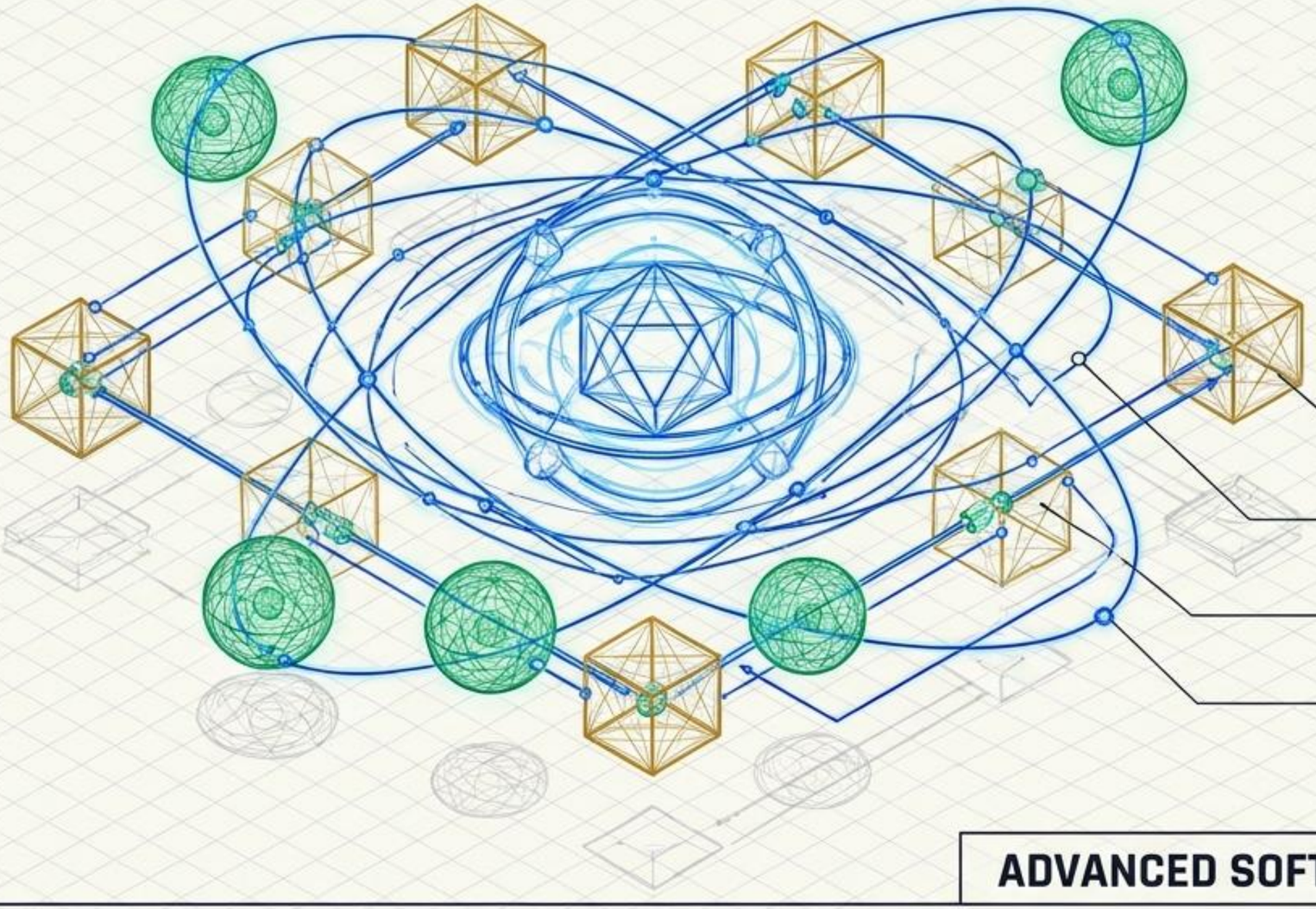
Are interfaces colliding?	Apply an Adapter
Is subclassing exploding?	Build a Bridge
Is memory exhausted?	Extract a Flyweight



> Apply structure to simplify reality.

BEHAVIORAL DESIGN PATTERNS

The Blueprint for Object Communication, Choreography, and Control



While Creational patterns instantiate and Structural patterns assemble, Behavioral patterns govern the dynamic interactions and distribution of responsibility. They abstract the control flow, transforming rigid, hard-coded dependencies into flexible, decentralized networks.

DECENTRALIZED NETWORKS

ORCHESTRATED COMMUNICATION

FLEXIBLE CONTROL FLOW

DYNAMIC INTERACTIONS

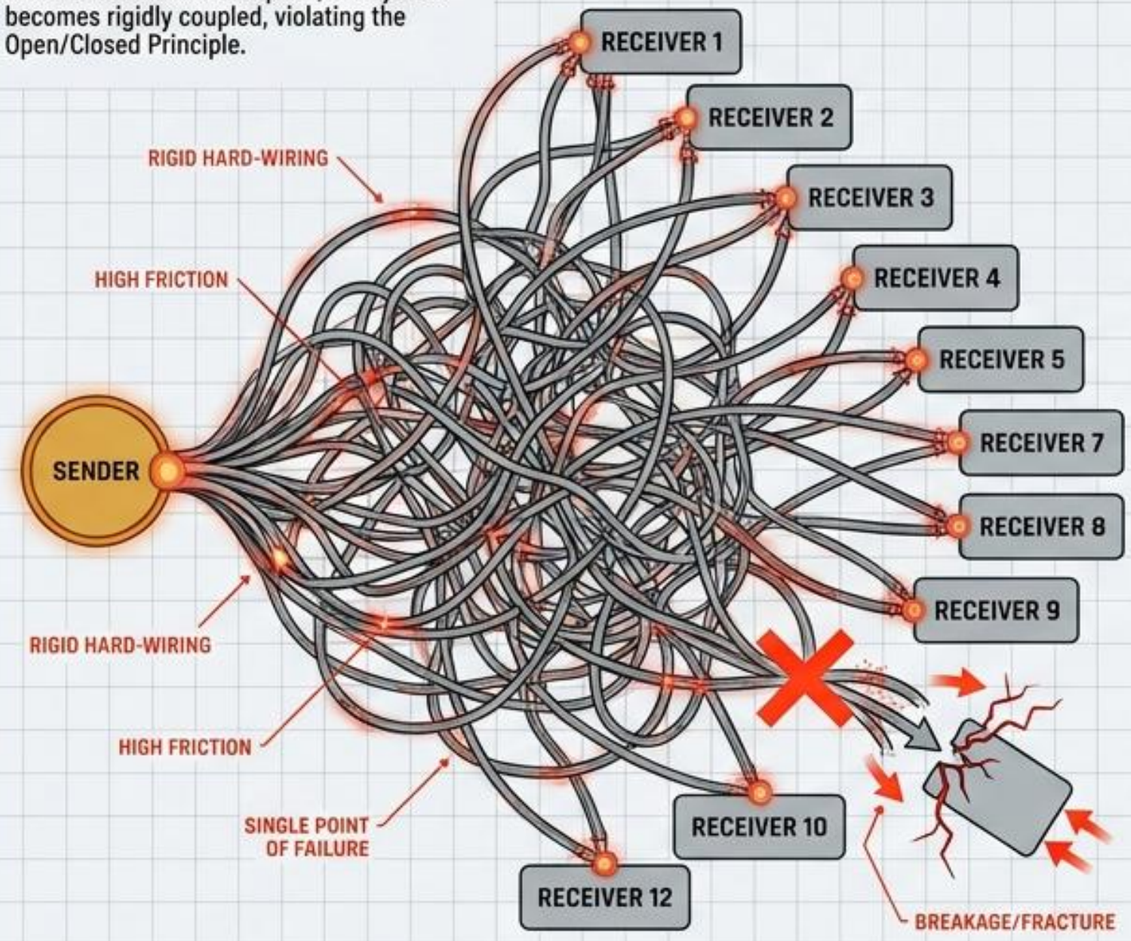
ADVANCED SOFTWARE ARCHITECTURE DEEP-DIVE

CHAIN OF RESPONSIBILITY // TIGHT COUPLING VS. SEQUENTIAL RELAY

TIGHT COUPLING IN REQUEST HANDLING

SPECIFICATION: 16:9
TOLERANCIONS - R00 mm
TOLERANCE: ± 1.5 - ±3.5

When a sender must explicitly know which receiver can handle a request, the system becomes rigidly coupled, violating the Open/Closed Principle.



GLOBAL SYSTEMS ENGINEERING

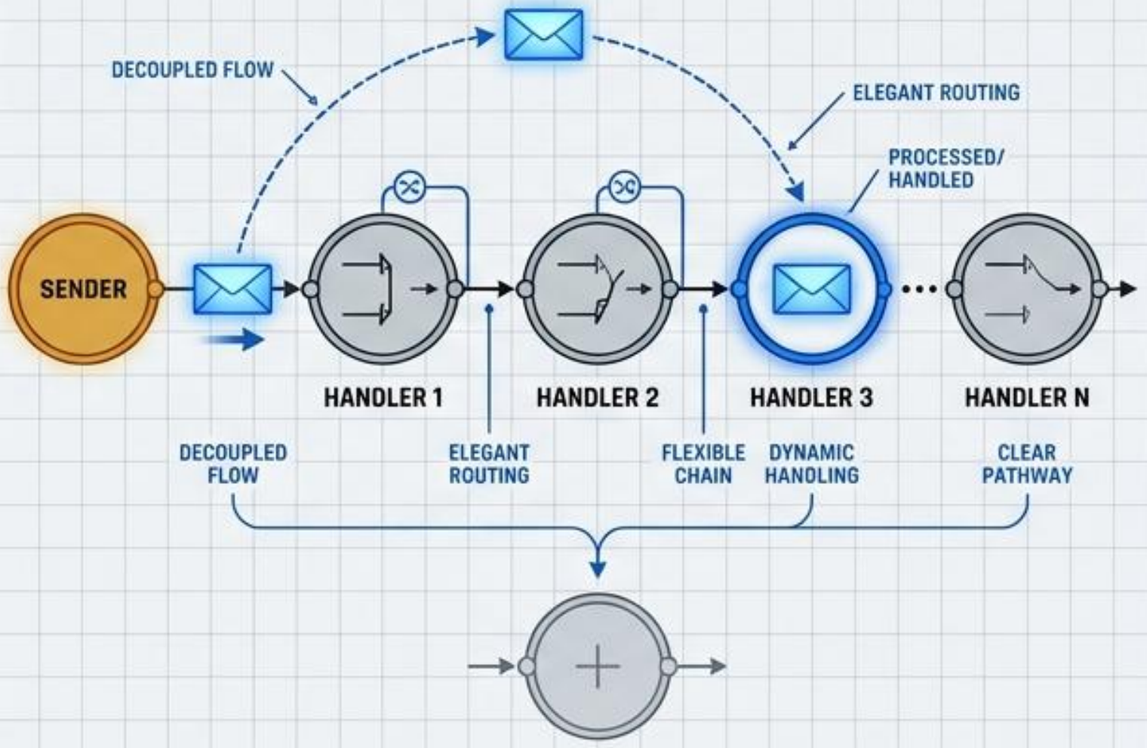
TECHNICATIONS: 16:9"
TOLERANCE: 16-8
TOLERANCE: -0.00

2005.2020 REV. 10.4

THE SEQUENTIAL RELAY

SPECIFICATION: 16:9
TOLERANCIONS - 0.00 mm
TOLERANCE: - 0.5 - ±1.5

Decouple senders and receivers by creating a chain of handler objects. The request travels along the chain until a handler recognizes it, processes it, or passes it to the next.

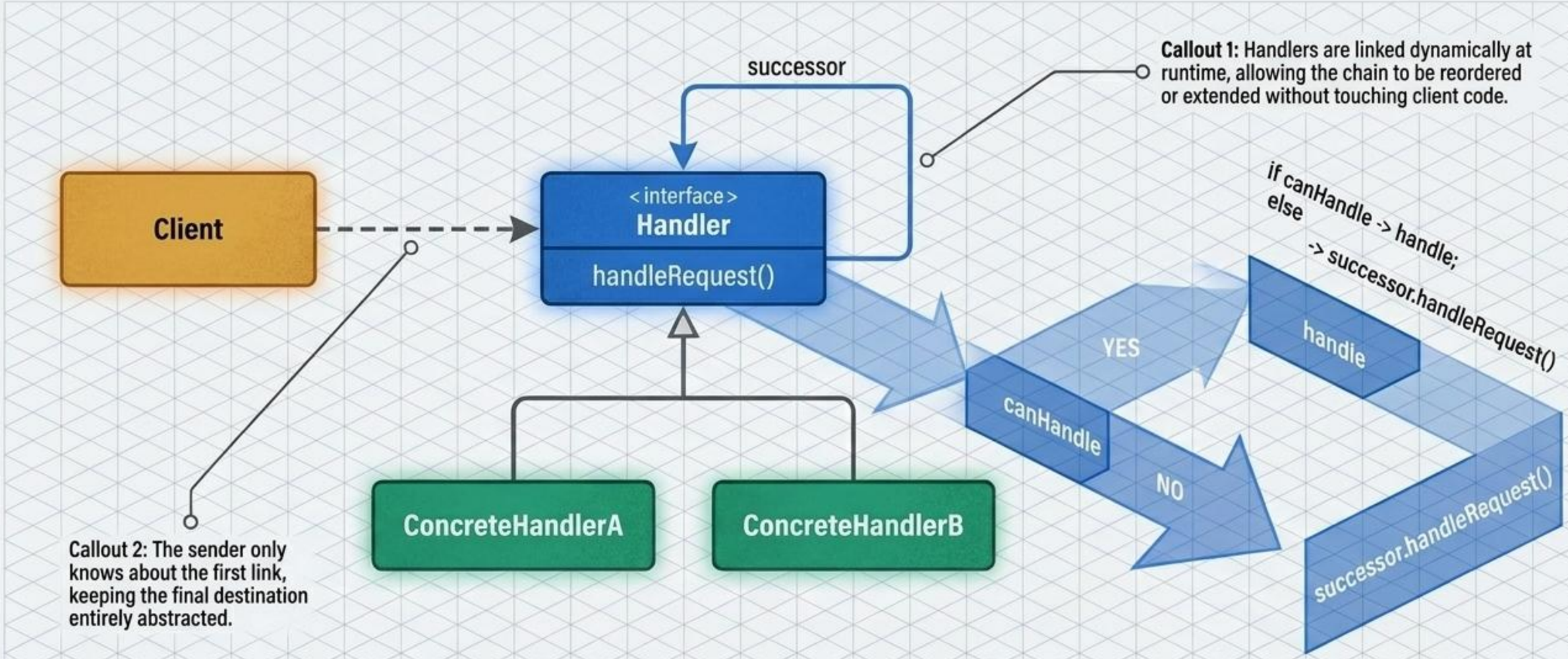


TECHNICATIONS: 16/9"
TOLERANCE: 16:0
TOLERANCE: -0.08

GLOBAL SYSTEMS ENGINEERING

2025.0020 REV. 16.8

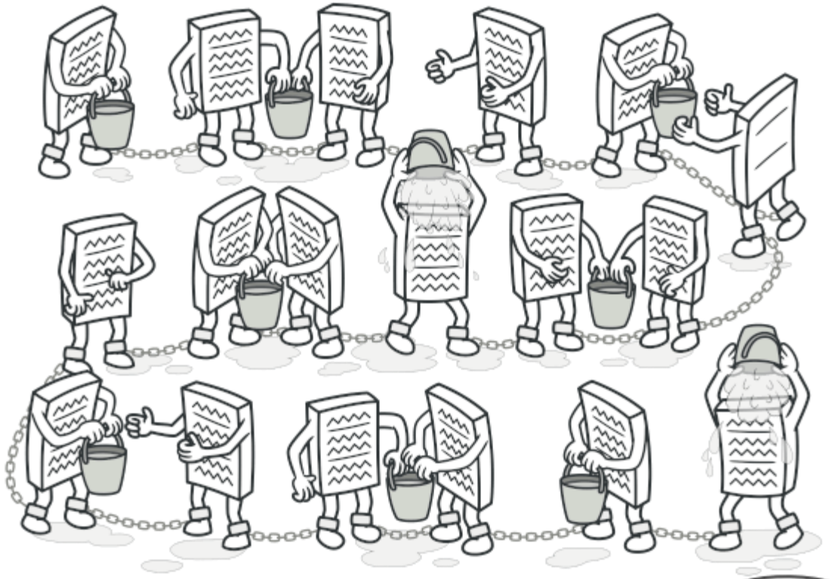
CHAIN OF RESPONSIBILITY // INTERNAL STRUCTURE



GLOBAL SYSTEMS ENGINEERING

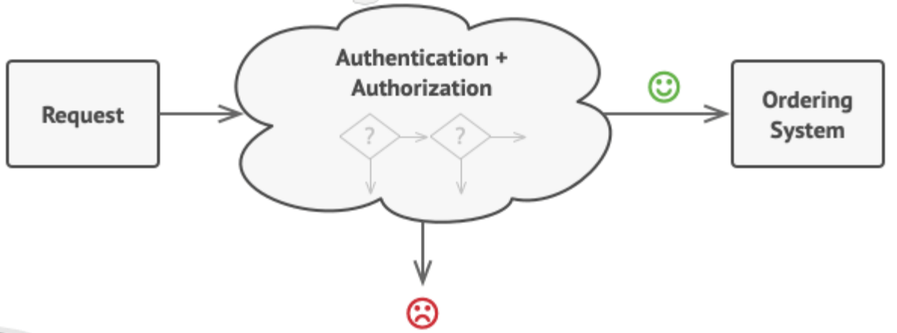
TECHNICATIONS: 16:9"
TOLERANCE: 16-8
TOLERANCE: -0.00

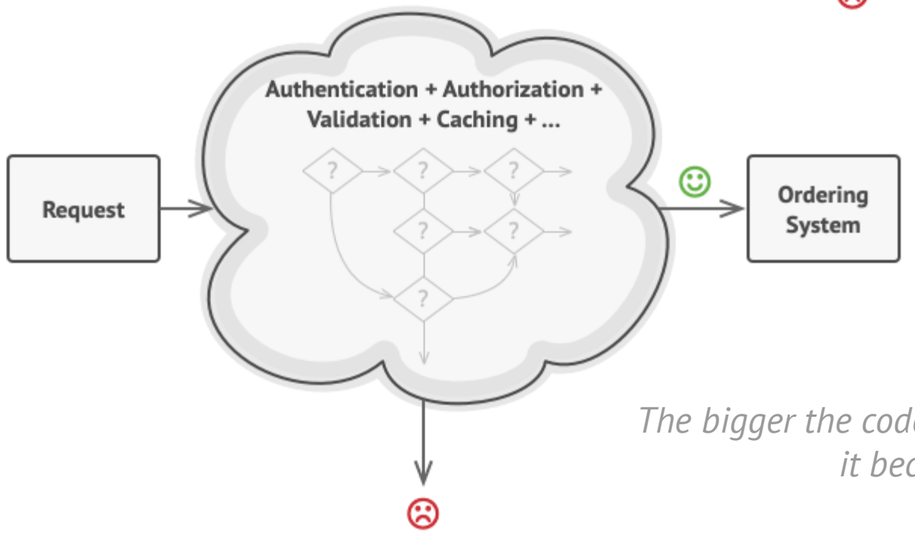
GLOBAL SYSTEMS ENGINEERING



Chain of Responsibility is a behavioral design pattern that lets you pass requests along a chain of handlers. Upon receiving a request, each handler decides either to process the request or to pass it to the next handler in the chain.

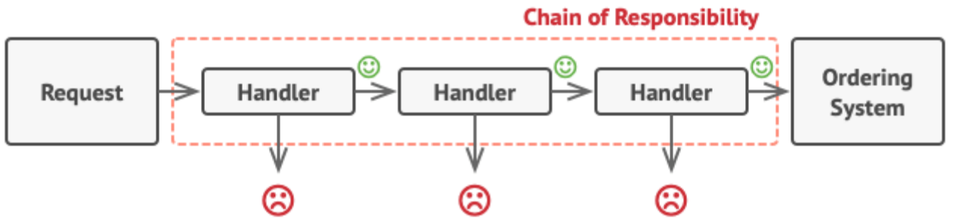
The request must pass a series of checks before the ordering system itself can handle it.



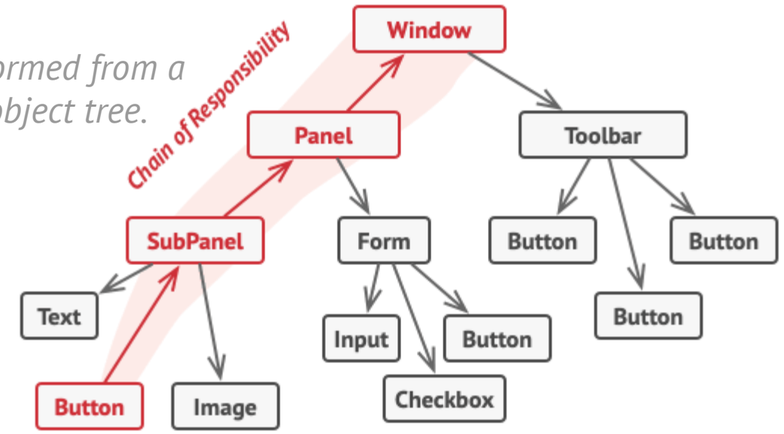


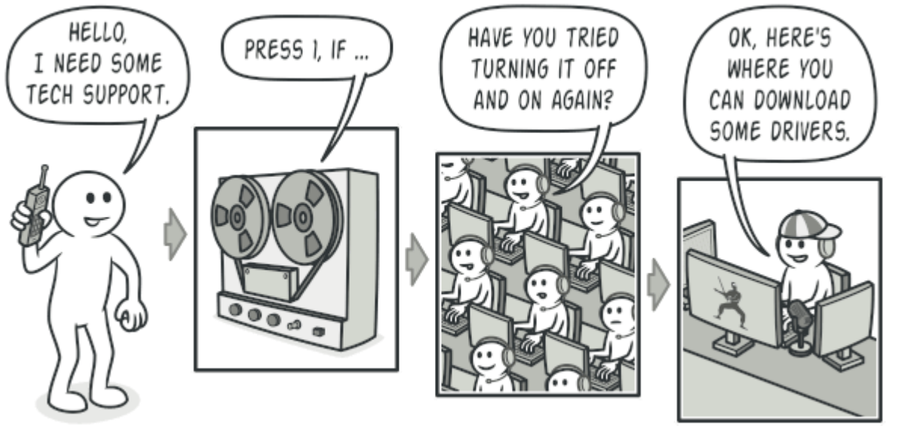
The bigger the code grew, the messier it became.

Handlers are lined up one by one, forming a chain.

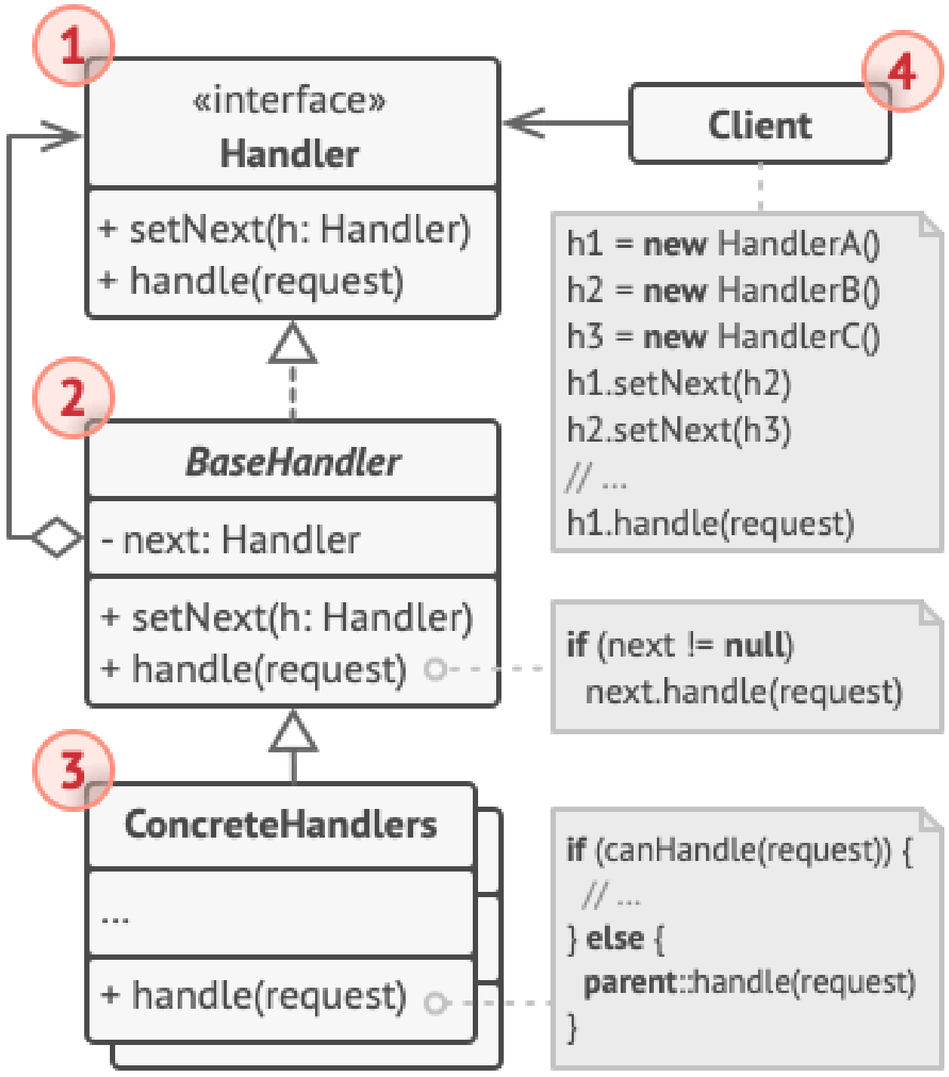


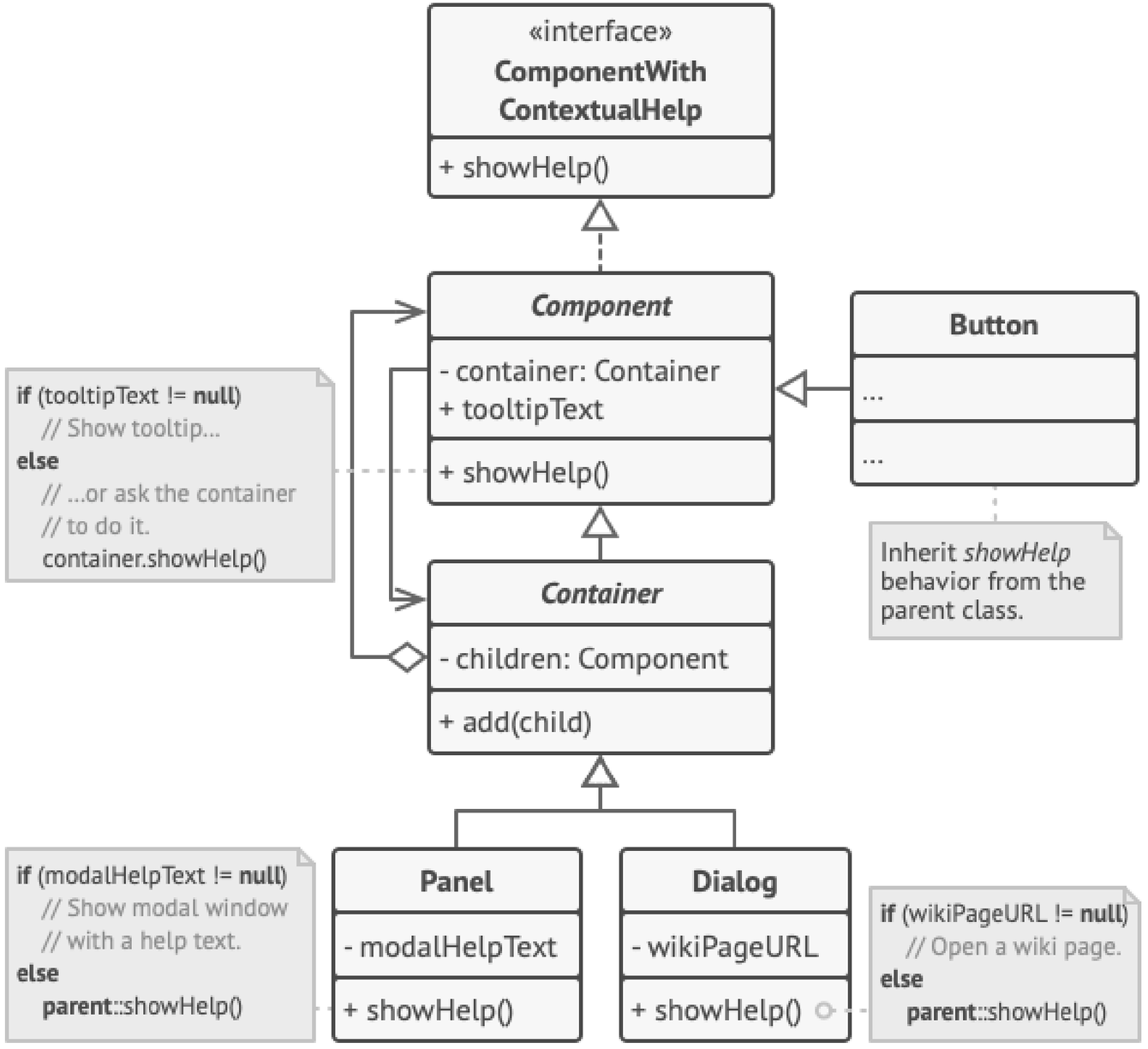
A chain can be formed from a branch of an object tree.





A call to tech support can go through multiple operators.





The GUI classes are built with the Composite pattern. Each element is linked to its container element. At any point, you can build a chain of elements that starts with the element itself and goes through all of its container elements.

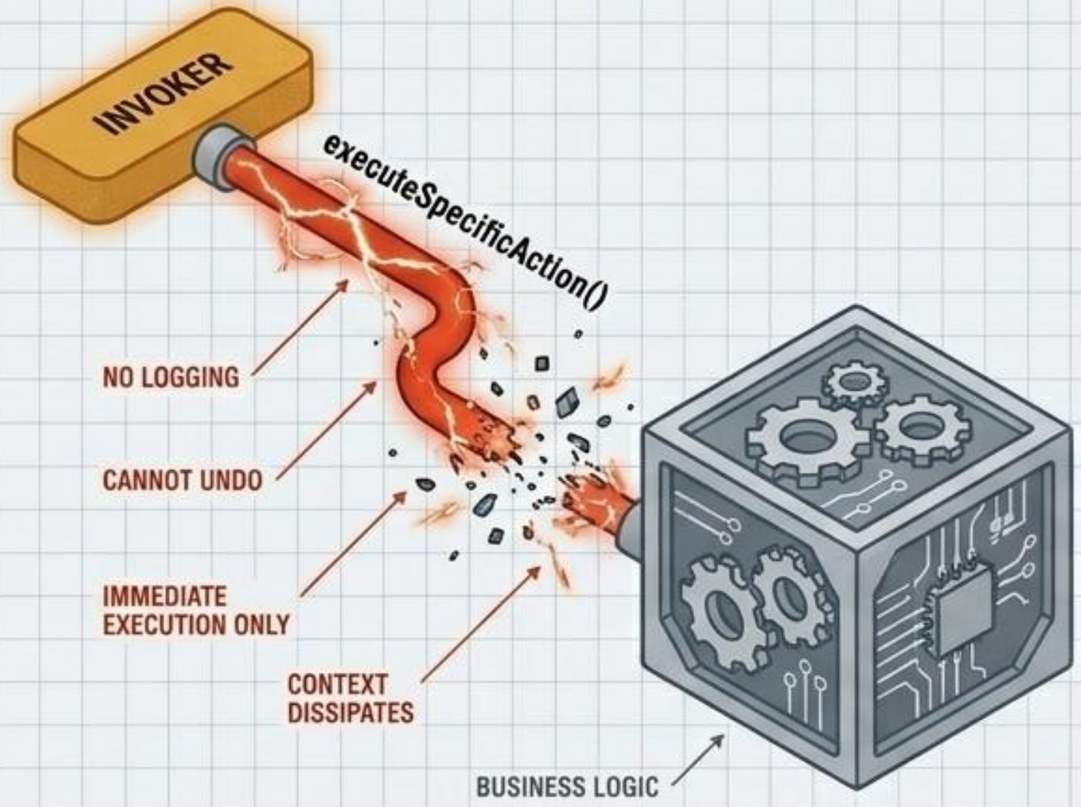
```
// The handler interface declares a method for executing a
// request.
interface ComponentWithContextualHelp is
    method showHelp()


// The base class for simple components.
abstract class Component implements ComponentWithContextualHelp is
    field tooltipText: string

    // The component's container acts as the next link in the
    // chain of handlers.
    protected field container: Container

    // The component shows a tooltip if there's help text
    // assigned to it. Otherwise it forwards the call to the
    // container, if it exists.
    method showHelp() is
        if (tooltipText != null)
            // Show tooltip.
        else
            container.showHelp()


// Containers can contain both simple components and other
// containers as children. The chain relationships are
// established here. The class inherits showHelp behavior from
// its parent.
abstract class Container extends Component is
    protected field children: array of Component

    method add(child) is
        children.add(child)
        child.container = this


// Primitive components may be fine with default help
// implementation...
class Button extends Component is
    // ...

// But complex components may override the default
// implementation. If the help text can't be provided in a new
// way, the component can always call the base implementation
// (see Component class).
class Panel extends Container is
    field modalHelpText: string

    method showHelp() is
        if (modalHelpText != null)
            // Show a modal window with the help text.
        else
            super.showHelp()

// ...same as above...
class Dialog extends Container is
    field wikiPageURL: string

    method showHelp() is
        if (wikiPageURL != null)
            // Open the wiki help page.
        else
            super.showHelp()


// Client code.
class Application is
    // Every application configures the chain differently.
    method createUI() is
        dialog = new Dialog("Budget Reports")
        dialog.wikiPageURL = "http://..."
        panel = new Panel(0, 0, 400, 800)
        panel.modalHelpText = "This panel does..."
        ok = new Button(250, 760, 50, 20, "OK")
        ok.tooltipText = "This is an OK button that..."
        cancel = new Button(320, 760, 50, 20, "Cancel")
        // ...
        panel.add(ok)
        panel.add(cancel)
        dialog.add(panel)

    // Imagine what happens here.
    method onF1KeyPress() is
        component = this.getComponentAtMouseCoords()
        component.showHelp()
```

COMMAND // HARDWIRED REQUESTS VS. OBJECTIFIED INTENT

LOST INTENT

SPECIFICATION: 16:9
TOLERANCIONS - 830 mm
TOLERANCE: ± 1.5 - 13.8

Directly coupling an invoker to a receiver makes it impossible to queue, log, undo, or parameterize requests. Once fired, the context vanishes.



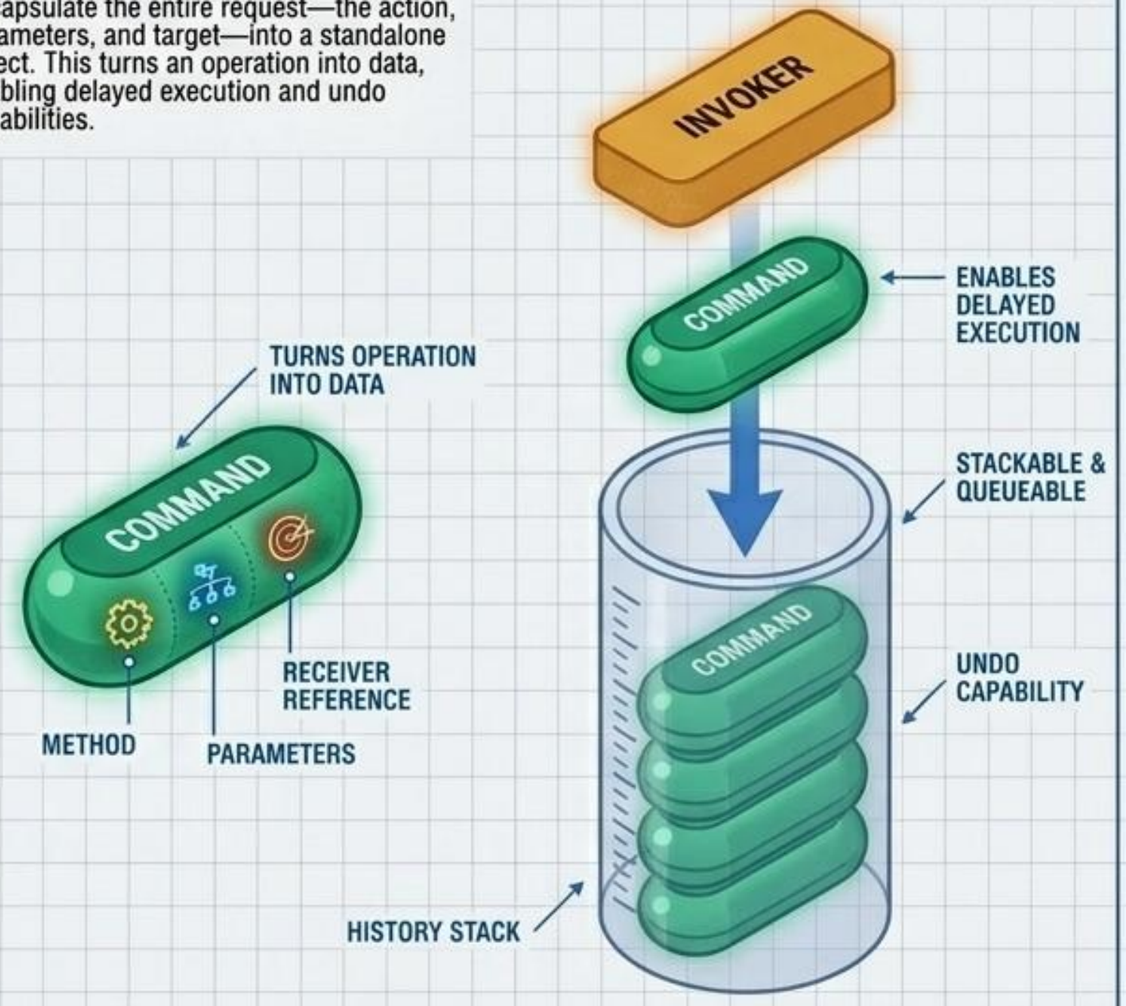
GLOBAL SYSTEMS ENGINEERING

TECHNICATIONS: 16:9"
TOLERANCE: 16-8
TOLERANGE: -0.00

OBJECTIFYING THE REQUEST

SPECIFICATIONS. 10-9
TOLERANCRCNS - B,BD mm
TOLERANCE: - 0.5 - ±H.5

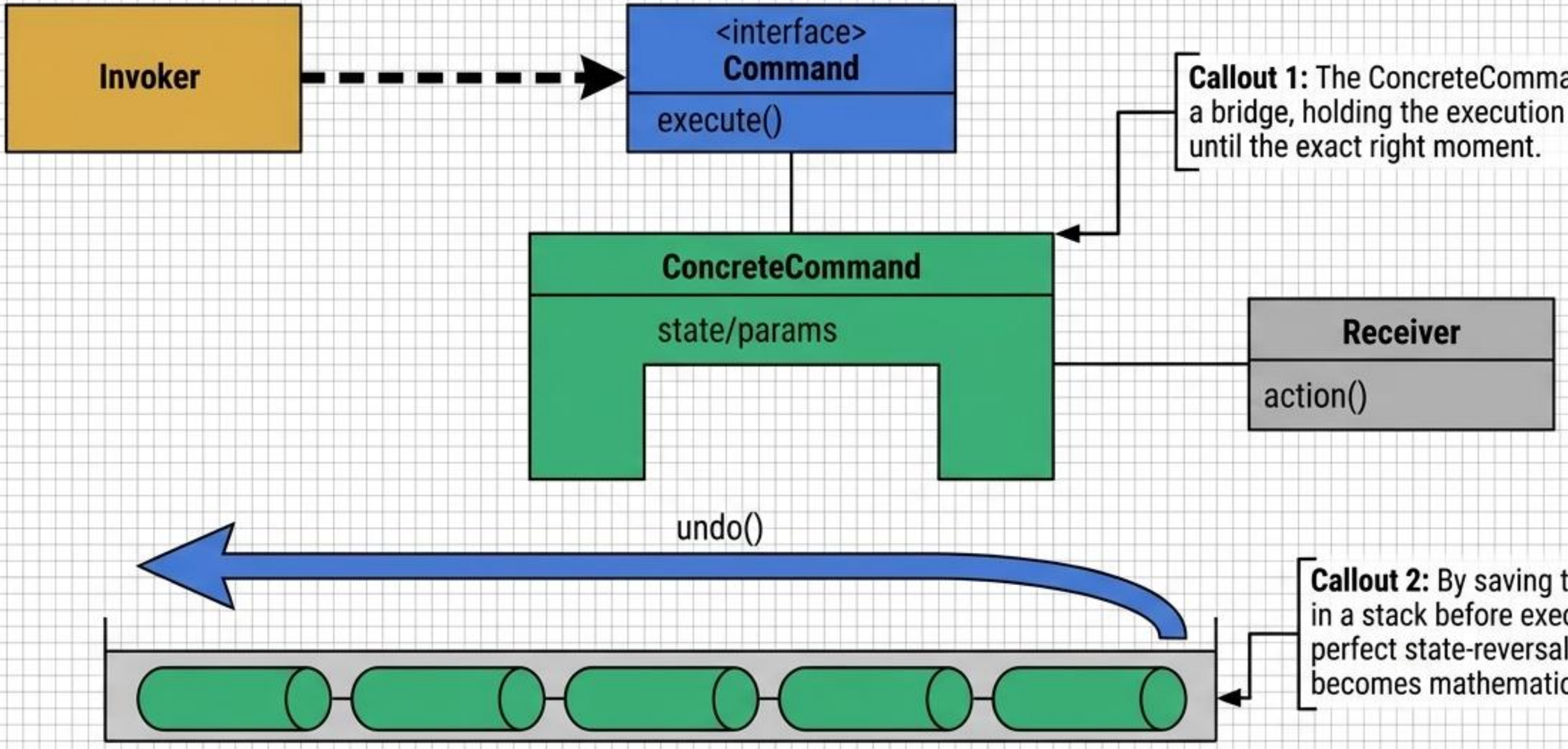
Encapsulate the entire request—the action, parameters, and target—into a standalone object. This turns an operation into data, enabling delayed execution and undo capabilities.



TECHNICATIONS: 16/9"
TOLERANCE- 16.0
TOLERANCE: -0.08

GLOBAL SYSTEMS ENGINEERING

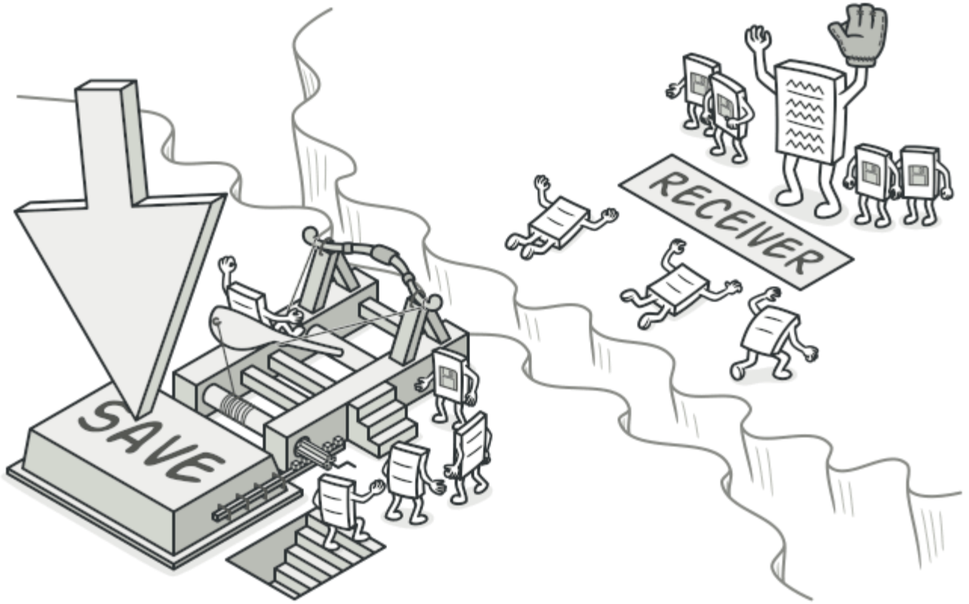
COMMAND // INTERNAL STRUCTURE

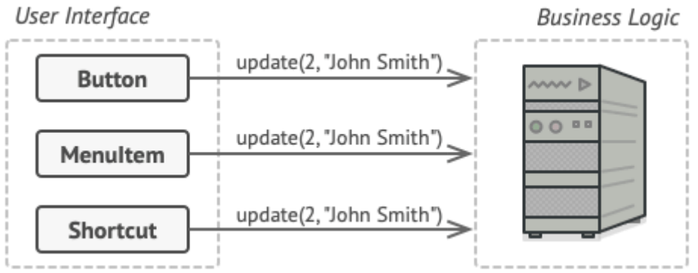


Callout 1: The ConcreteCommand forms a bridge, holding the execution context until the exact right moment.

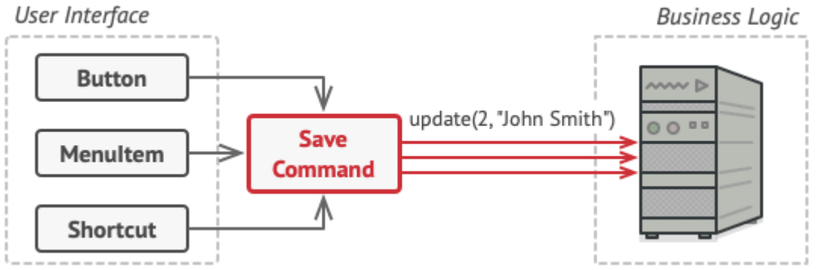
Callout 2: By saving these objects in a stack before execution, perfect state-reversal (Undo) becomes mathematically trivial.

GLOBAL SYSTEMS ENGINEERING | TECHNICAL SPECIFICATION: 16:9 | TOLERANCE: -0.00 | REV 1.0 | P.O.C. 2023-0022



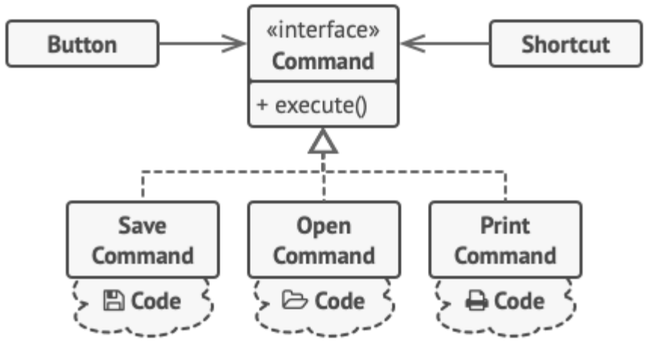


The GUI objects may access the business logic objects directly.

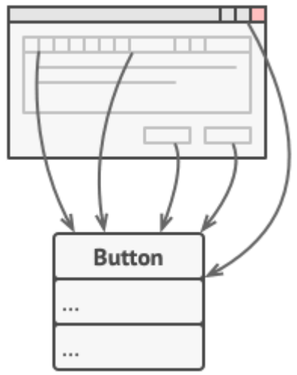


Accessing the business logic layer via a command.

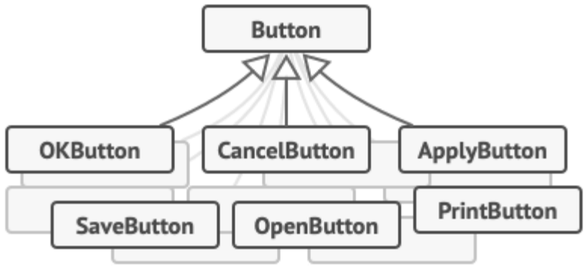
Command is a behavioral design pattern that turns a request into a stand-alone object that contains all information about the request. This transformation lets you pass requests as a method arguments, delay or queue a request's execution, and support undoable operations.



The GUI objects delegate the work to commands.

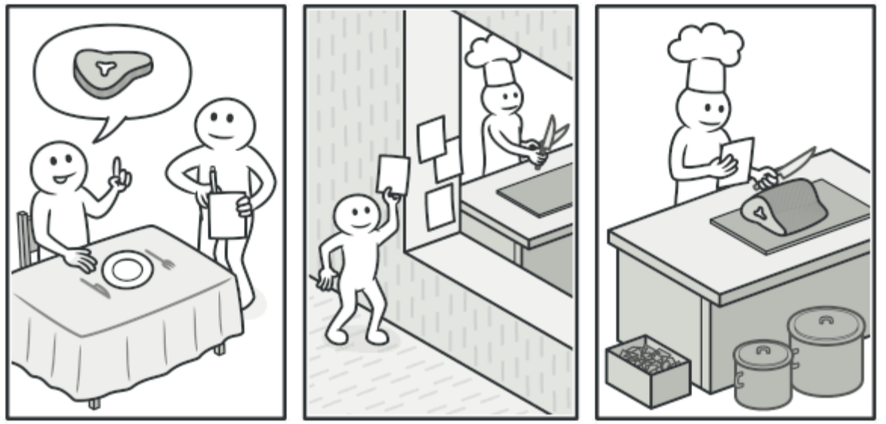


All buttons of the app are derived from the same class.

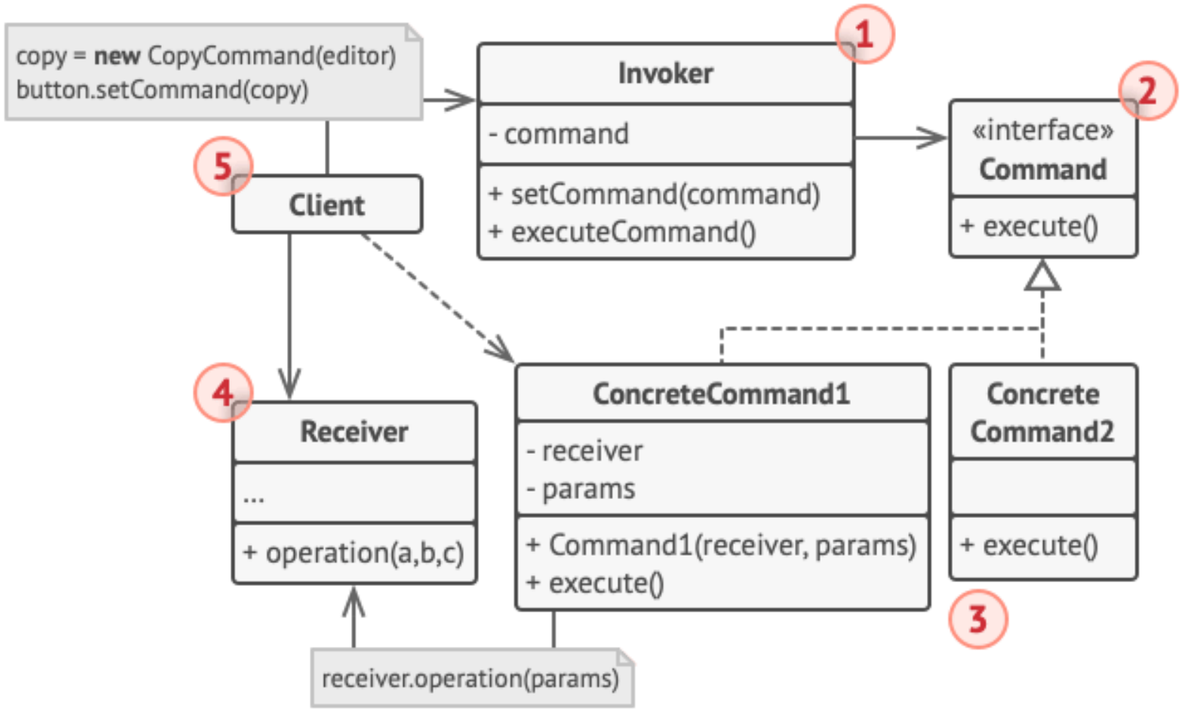


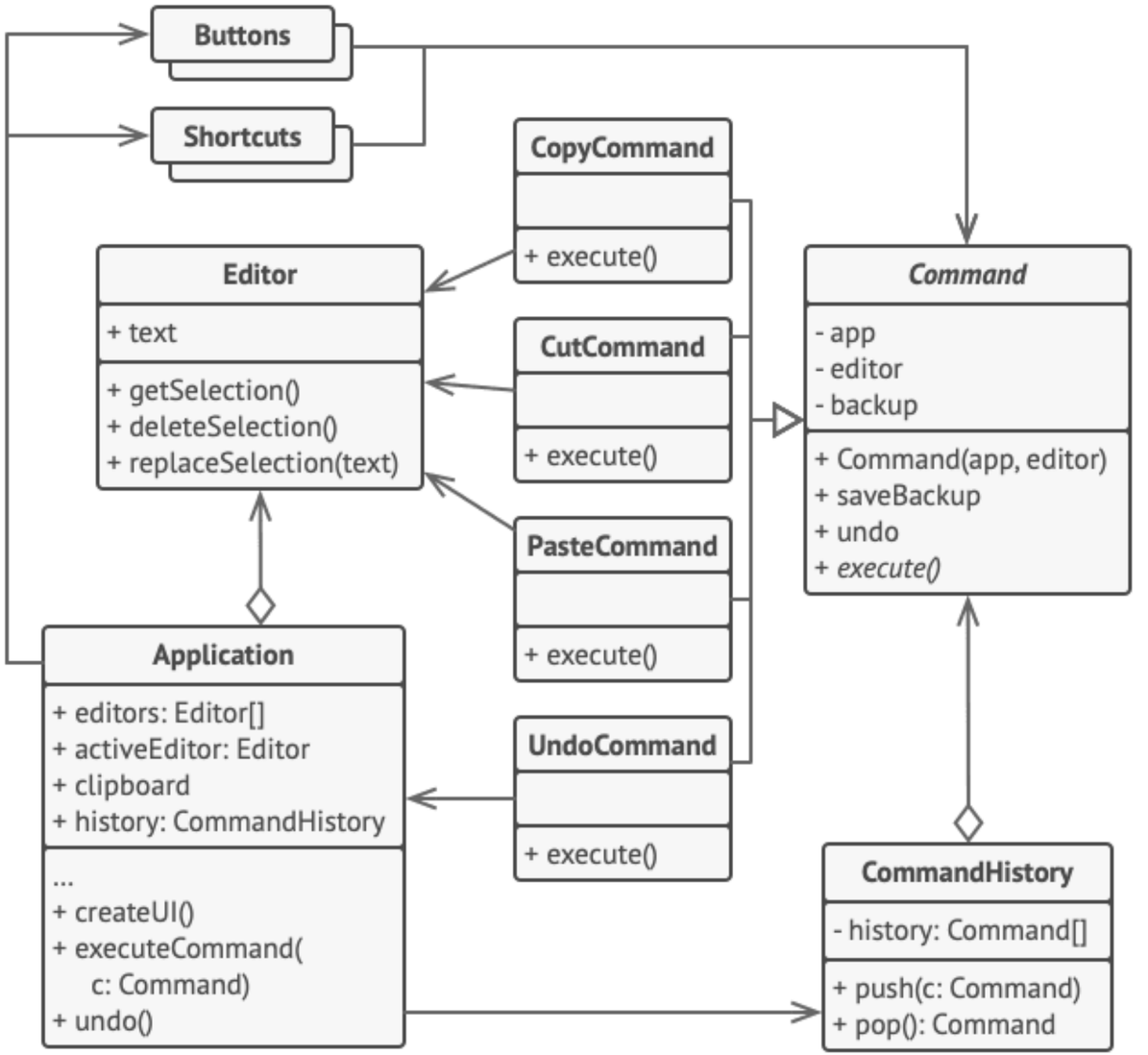
Lots of button subclasses. What can go wrong?

Several classes implement the same functionality.



Making an order in a restaurant.





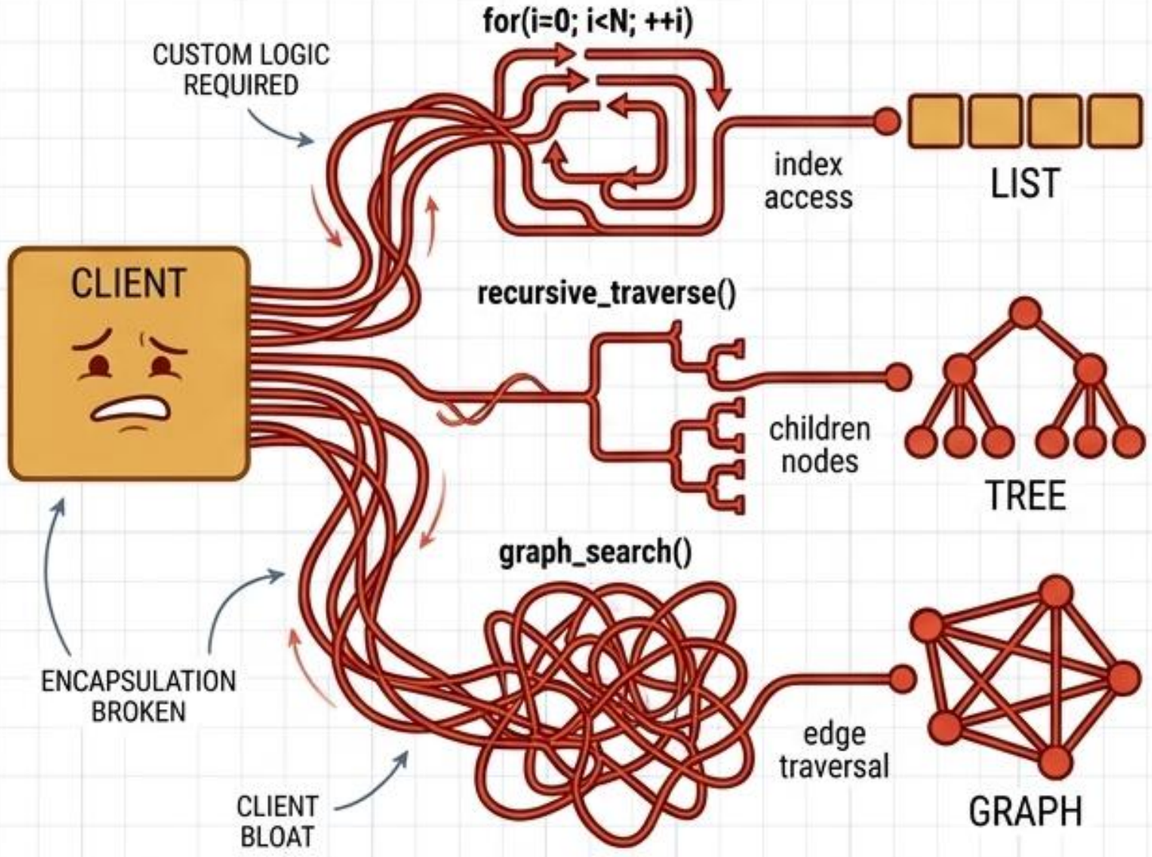
Undoable operations in a text editor.

```
// The base command class defines the common interface for all
// concrete commands.
abstract class Command is
    protected field app: Application
    protected field editor: Editor
    protected field backup: text

    constructor Command(app: Application, editor: Editor) is
        this.app = app
        this.editor = editor

    // Make a backup of the editor's state.
    method saveBackup() is
        backup = editor.text

    // Restore the editor's state.
    method undo() is
        editor.text = backup

    // The execution method is declared abstract to force all
    // concrete commands to provide their own implementations.
    // The method must return true or false depending on
whether
    // the command changes the editor's state.
    abstract method execute()


// The concrete commands go here.
class CopyCommand extends Command is
    // The copy command isn't saved to the history since it
    // doesn't change the editor's state.
    method execute() is
        app.clipboard = editor.getSelection()
        return false

class CutCommand extends Command is
    // The cut command does change the editor's state,
therefore
    // it must be saved to the history. And it'll be saved as
    // long as the method returns true.
    method execute() is
        saveBackup()
        app.clipboard = editor.getSelection()
        editor.deleteSelection()
        return true

class PasteCommand extends Command is
    method execute() is
        saveBackup()
        editor.replaceSelection(app.clipboard)
        return true

// The undo operation is also a command.
class UndoCommand extends Command is
    method execute() is
        app.undo()
        return false


// The global command history is just a stack.
class CommandHistory is
    private field history: array of Command

    // Last in...
    method push(c: Command) is
        // Push the command to the end of the history array.

    // ...first out
    method pop():Command is
        // Get the most recent command from the history.


// The editor class has actual text editing operations. It
plays
// the role of a receiver: all commands end up delegating
// execution to the editor's methods.
class Editor is
    field text: string

    method getSelection() is
        // Return selected text.

    method deleteSelection() is
        // Delete selected text.

    method replaceSelection(text) is
        // Insert the clipboard's contents at the current
        // position.


// The application class sets up object relations. It acts as
a
// sender: when something needs to be done, it creates a
command
// object and executes it.
class Application is
    field clipboard: string
    field editors: array of Editors
    field activeEditor: Editor
    field history: CommandHistory

    // The code which assigns commands to UI objects may look
    // like this.
    method createUI() is
        // ...
        copy = function() { executeCommand(
            new CopyCommand(this, activeEditor)) }
        copyButton.setCommand(copy)
        shortcuts.onKeyPress("Ctrl+C", copy)

        cut = function() { executeCommand(
            new CutCommand(this, activeEditor)) }
        cutButton.setCommand(cut)
        shortcuts.onKeyPress("Ctrl+X", cut)

        paste = function() { executeCommand(
            new PasteCommand(this, activeEditor)) }
        pasteButton.setCommand(paste)
        shortcuts.onKeyPress("Ctrl+V", paste)

        undo = function() { executeCommand(
            new UndoCommand(this, activeEditor)) }
        undoButton.setCommand(undo)
        shortcuts.onKeyPress("Ctrl+Z", undo)

    // Execute a command and check whether it has to be added
to
    // the history.
    method executeCommand(command) is
        if (command.execute())
            history.push(command)

    // Take the most recent command from the history and run
its
    // undo method. Note that we don't know the class of that
    // command. But we don't have to, since the command knows
    // how to undo its own action.
    method undo() is
        command = history.pop()
        if (command != null)
            command.undo()
```

ITERATOR // EXPOSED STRUCTURES VS. ABSTRACTED TRAVERSAL

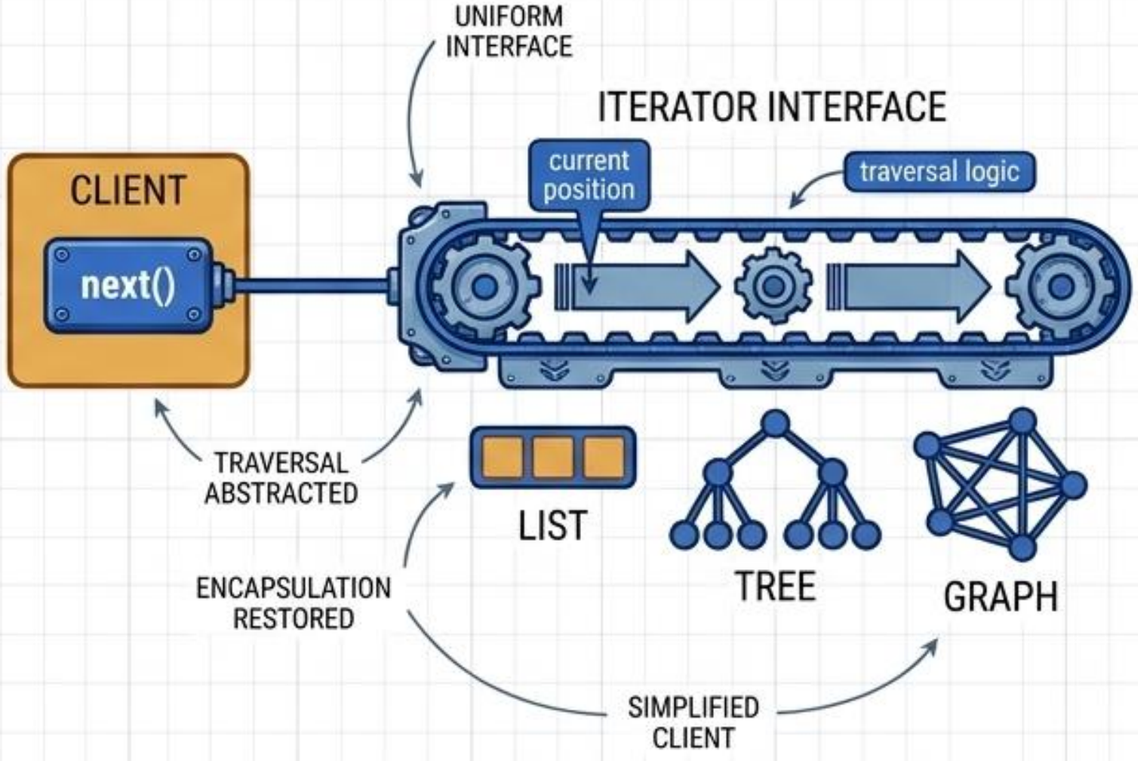
EXPOSED INTERNAL STRUCTURES

Forcing clients to implement distinct traversal logic for lists, trees, and graphs breaks encapsulation and bloats client code.



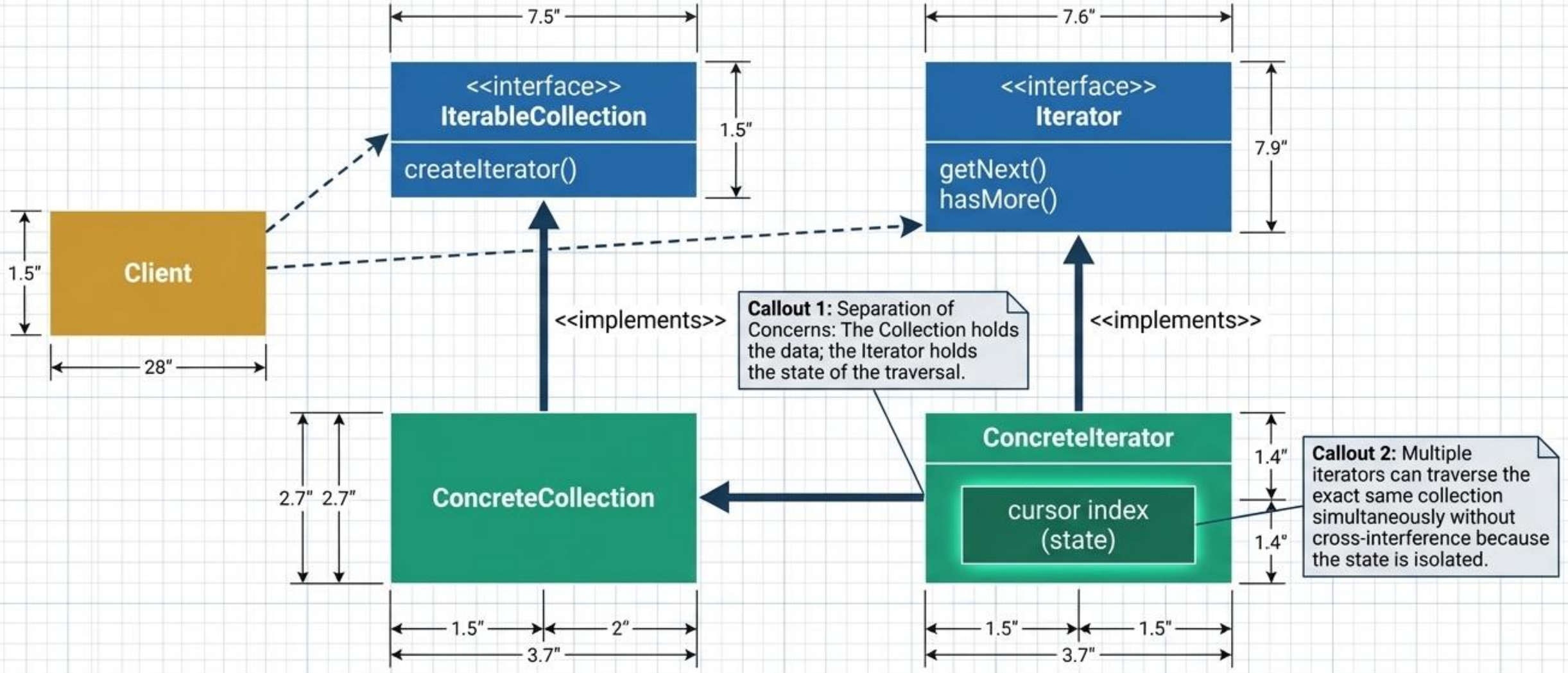
ABSTRACTING THE TRAVERSAL

Extract traversal behavior into a separate iterator object. It tracks position and handles mechanics, providing a uniform interface while hiding the collection's inner workings.

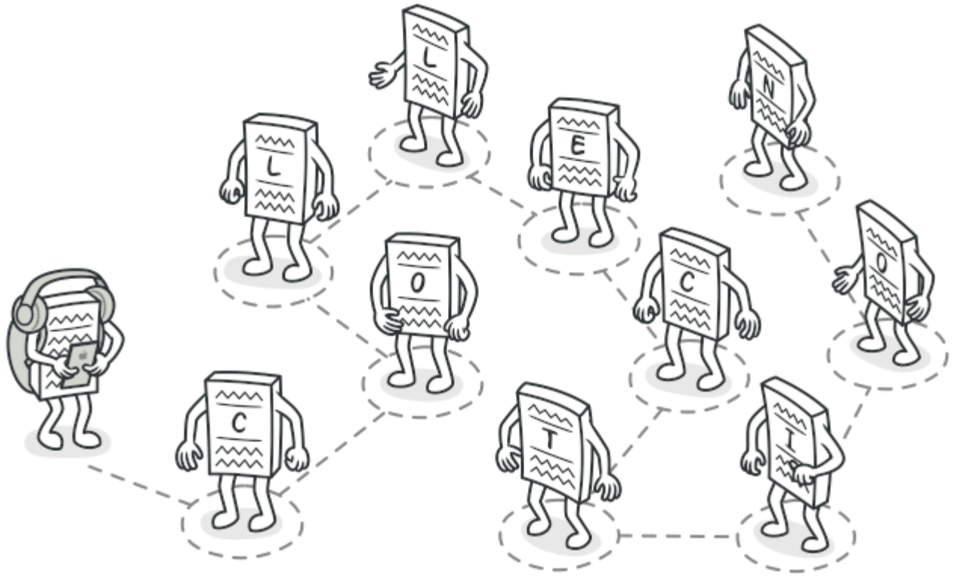


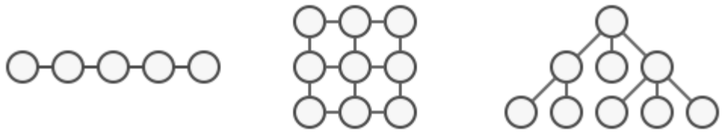
ITERATOR // INTERNAL STRUCTURE

Ut no.	Specifications	Revision	Rev.
1	[illegible]	Stord cont	1

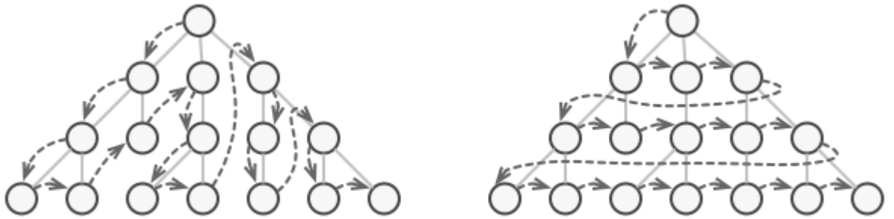


GLOBAL SYSTEMS ENGINEERING	Specification GEHHA SYSTEMS	Specification Specification: DCPI/BISTAE, MISN and BRVIM2007..40:2021	Description	Rev.	Revs. 1



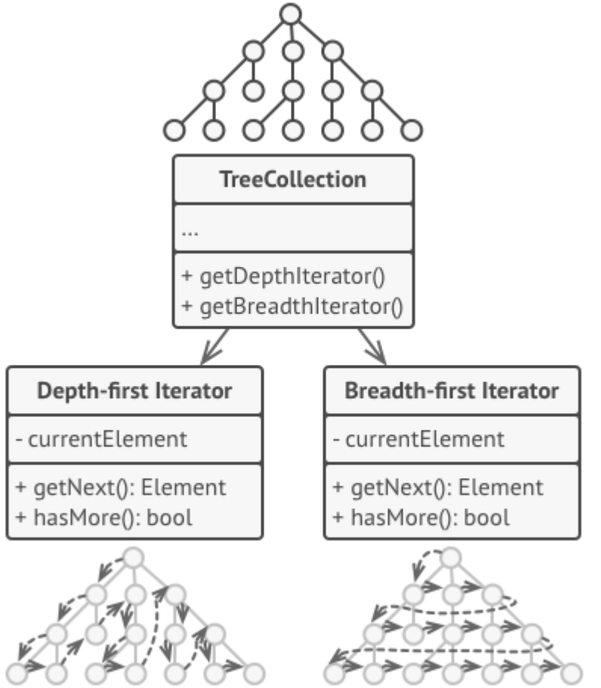


Various types of collections.

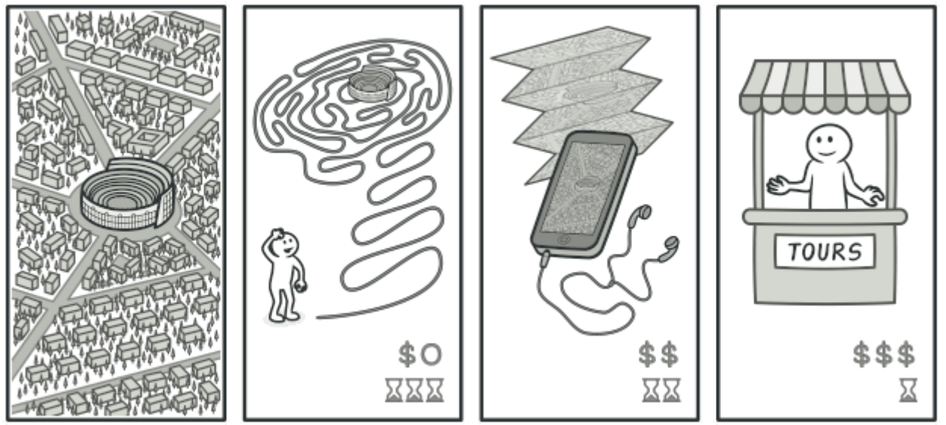


The same collection can be traversed in several different ways.

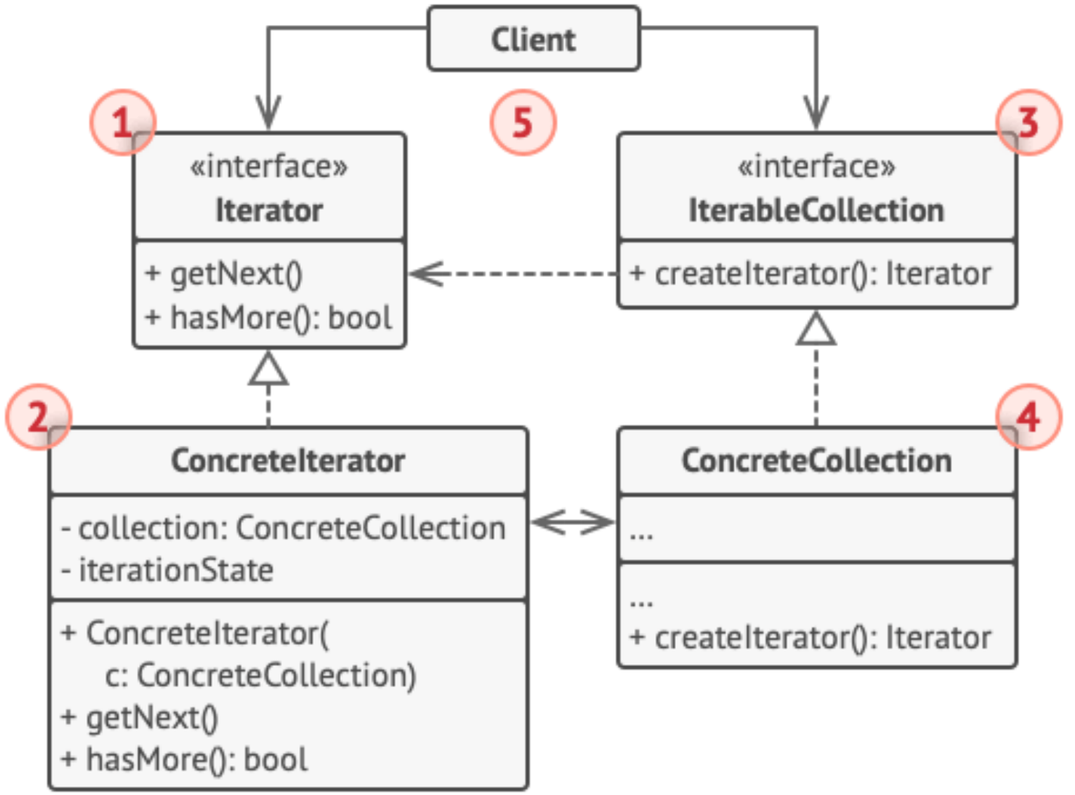
Iterator is a behavioral design pattern that lets you traverse elements of a collection without exposing its underlying representation (list, stack, tree, etc.).

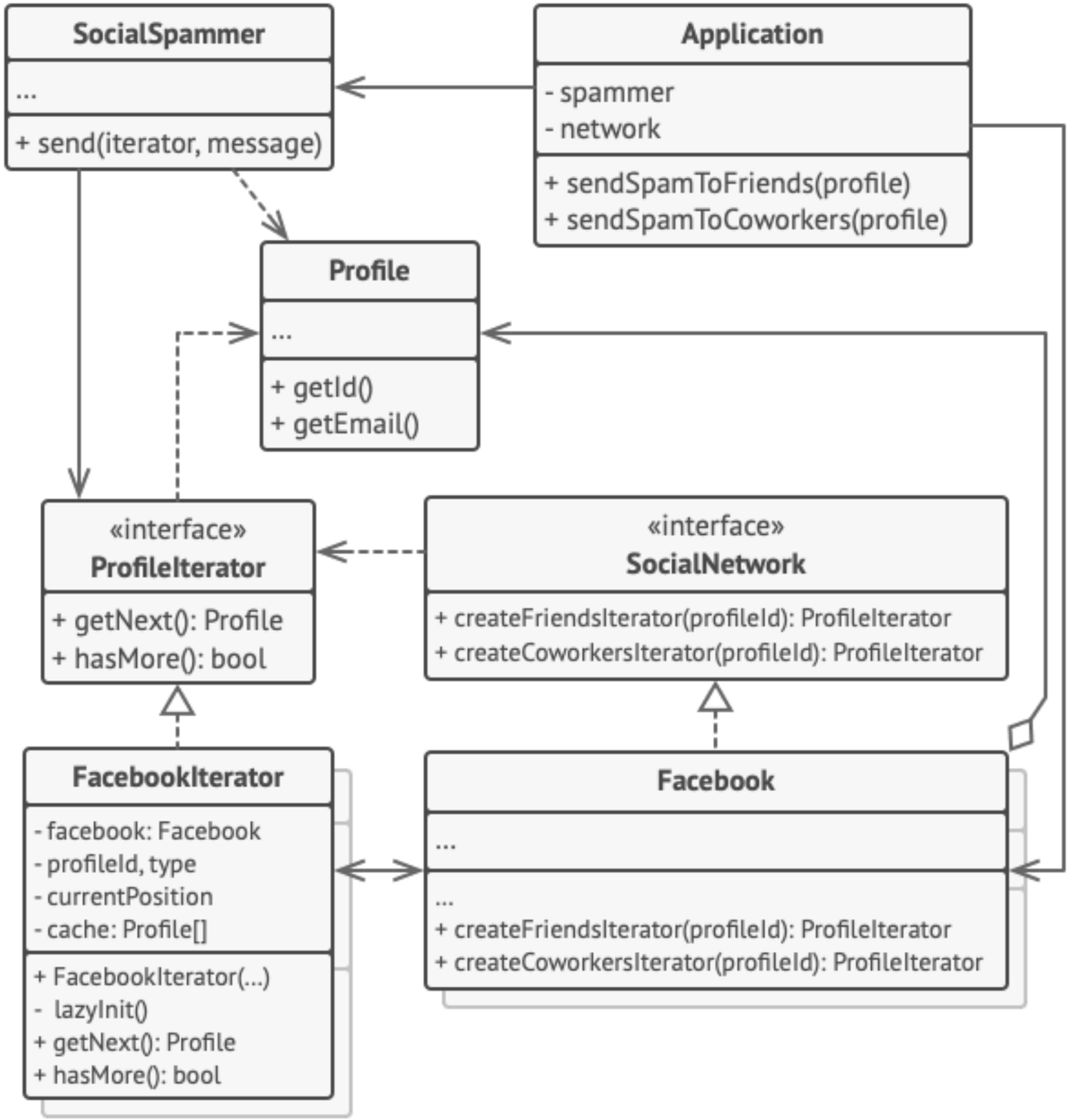


Iterators implement various traversal algorithms. Several iterator objects can traverse the same collection at the same time.



Various ways to walk around Rome.





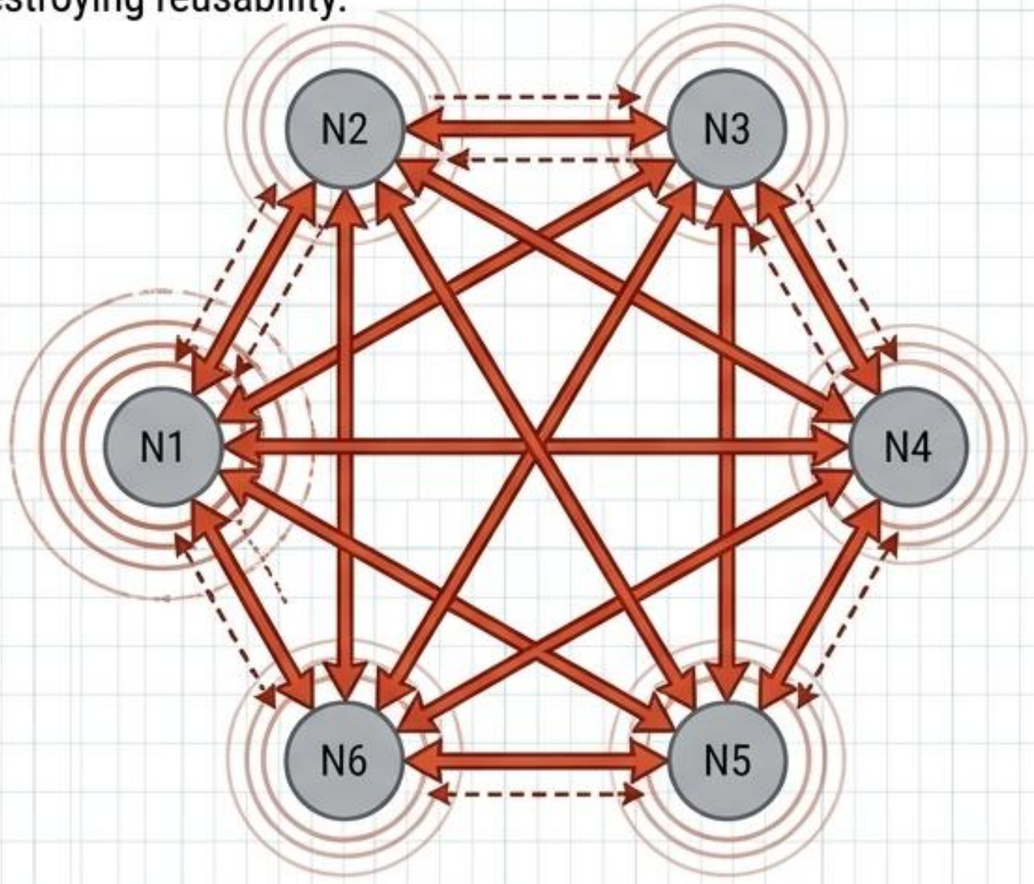
Example of iterating over social profiles.

```
// The collection interface must declare a factory
method for
// producing iterators. You can declare several
methods if there
// are different kinds of iteration available in your
program.
interface SocialNetwork is
    method
createFriendsIterator(profileId):ProfileIterator
    method
createCoworkersIterator(profileId):ProfileIterator


// Each concrete collection is coupled to a set of
concrete
// iterator classes it returns. But the client isn't,
since the
// signature of these methods returns iterator
interfaces.
class Facebook implements SocialNetwork is
    // ... The bulk of the collection's code should
go here ...

    // Iterator creation code.
    method createFriendsIterator(profileId) is
        return new FacebookIterator(this, profileId,
"friends")
    method createCoworkersIterator(profileId) is
        return new FacebookIterator(this, profileId,
"coworkers")


// The common interface for all iterators.
interface ProfileIterator is
    method getNext():Profile
    method hasMore():bool


// The concrete iterator class.
class FacebookIterator implements ProfileIterator is
    // The iterator needs a reference to the
collection that it
    // traverses
    private field facebook: Facebook
    private field profileId, type: string

    // An iterator object traverses the collection
independently
    // from other iterators. Therefore it has to
store the
    // iteration state.
    private field currentPosition
    private field cache: array of Profile

    constructor FacebookIterator(facebook, profileId,
type) is
        this.facebook = facebook
        this.profileId = profileId
        this.type = type

    private method lazyInit() is
        if (cache == null)
            cache =
facebook.socialGraphRequest(profileId, type)

    // Each concrete iterator class has its own
implementation
    // of the common iterator interface.
    method getNext() is
        if (hasMore())
            result = cache[currentPosition]
            currentPosition++
            return result

    method hasMore() is
        lazyInit()
        return currentPosition < cache.length


// Here is another useful trick: you can pass an
iterator to a
// client class instead of giving it access to a
whole
// collection. This way, you don't expose the
collection to the
// client.
//
// And there's another benefit: you can change the
way the
// client works with the collection at runtime by
passing it a
// different iterator. This is possible because the
client code
// isn't coupled to concrete iterator classes.
class SocialSpammer is
    method send(iterator: ProfileIterator, message:
string) is
        while (iterator.hasMore())
            profile = iterator.getNext()
            System.sendEmail(profile.getEmail(),
message)


// The application class configures collections and
iterators
// and then passes them to the client code.
class Application is
    field network: SocialNetwork
    field spammer: SocialSpammer

    method config() is
        if working with Facebook
            this.network = new Facebook()
        if working with LinkedIn
            this.network = new LinkedIn()
        this.spammer = new SocialSpammer()

    method sendSpamToFriends(profile) is
        iterator =
network.createFriendsIterator(profile.getId())
        spammer.send(iterator, "Very important
message")

    method sendSpamToCoworkers(profile) is
        iterator =
network.createCoworkersIterator(profile.getId())
        spammer.send(iterator, "Very important
message")
```

MEDIATOR // THE SPAGHETTI WEB VS. THE CENTRALIZED HUB

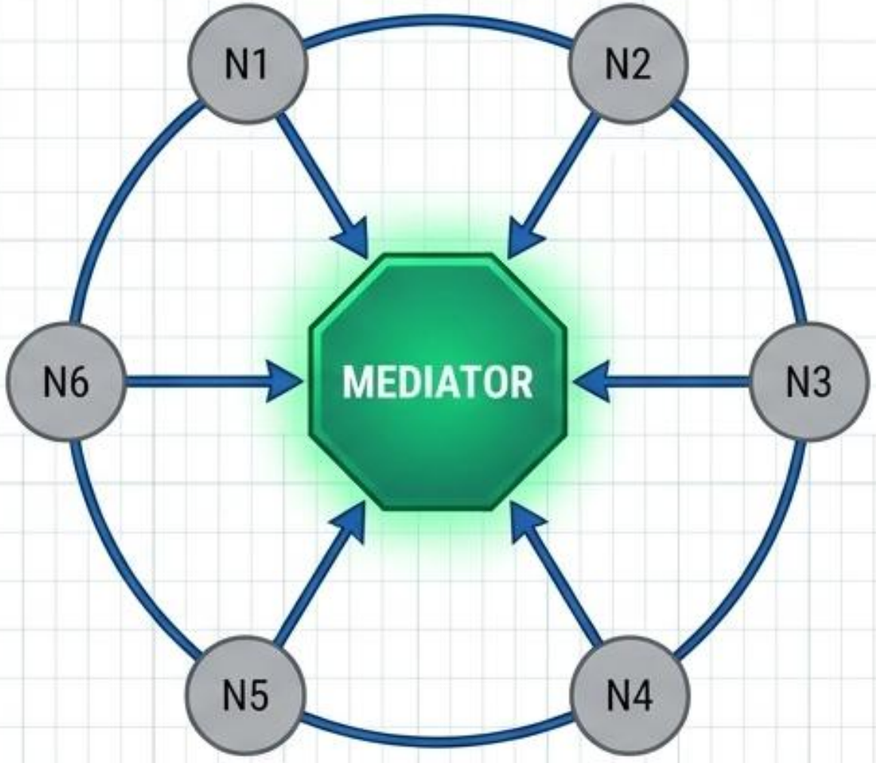
UNMAINTAINABLE INTERDEPENDENCIES

When objects communicate directly, the system devolves into a dense web. Changing one component risks breaking them all, destroying reusability.



THE CENTRAL HUB

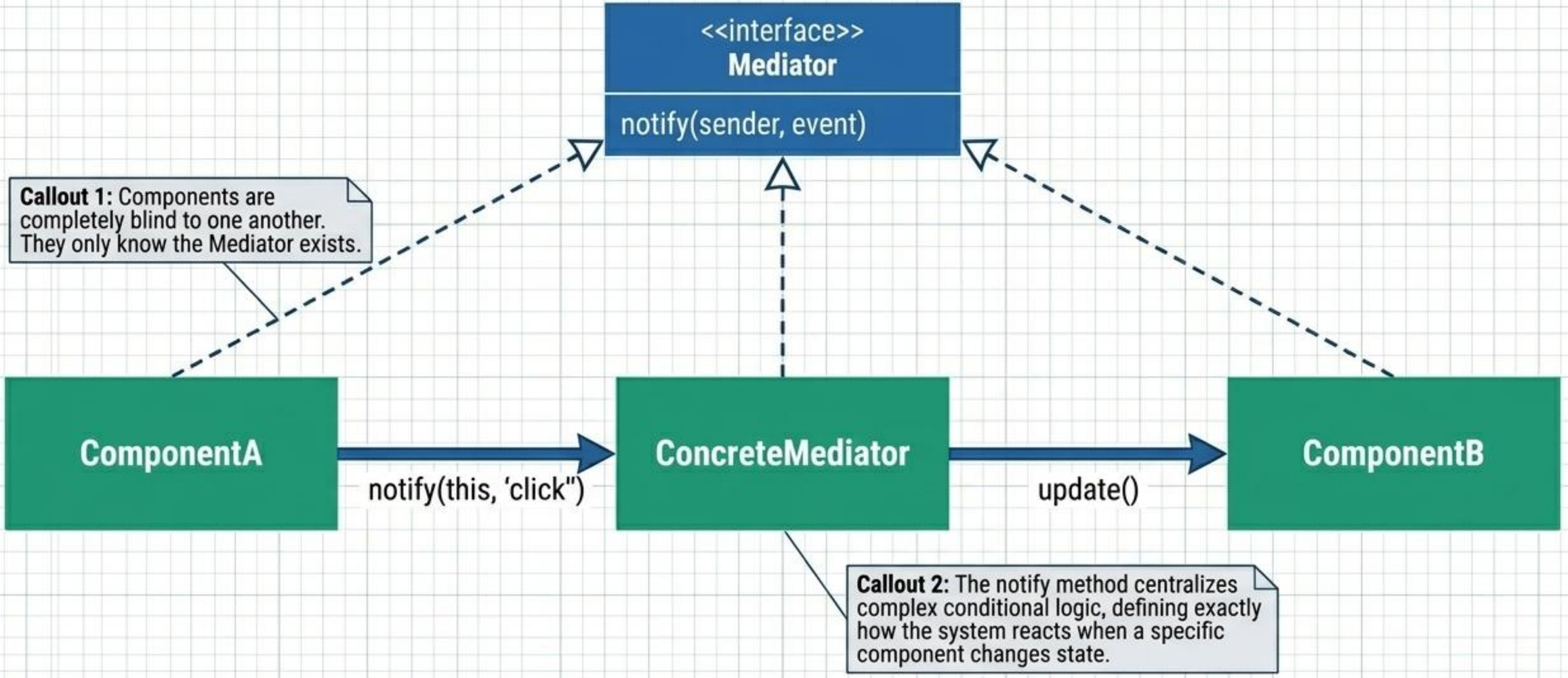
Restrict direct communications. Force components to collaborate only via a central mediator, turning chaotic many-to-many relationships into manageable one-to-many logic.



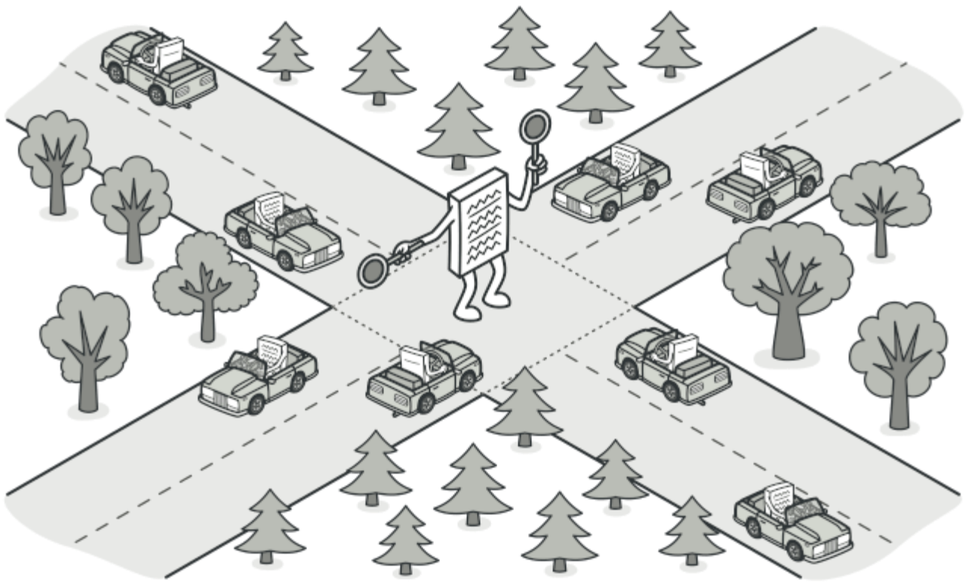
GLOBAL SYSTEMS ENGINEERING	Specification	Specification	Description	Rev.	Revs.
	GEHHA SYSTEMS	Specification: DCPI/BISTAE, MISN and BRVIM2007.:43·2021			1

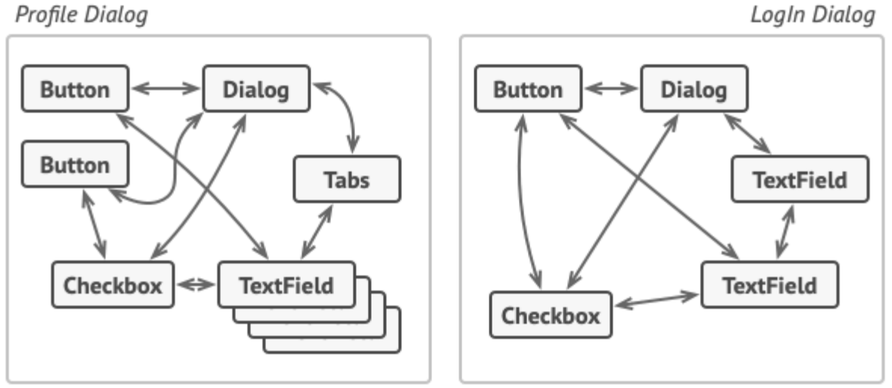
Ut no.	Specifications	Revision	Rev.
1	[illegible]	Stard cont	1

MEDIATOR // INTERNAL STRUCTURE



GLOBAL SYSTEMS ENGINEERING	Specification GEHHA SYSTEMS	Specification	Description	Rev.	Revs.
		Specification: DCPI/BISTAE, MISN and BRVIM2007.:43:2021			1

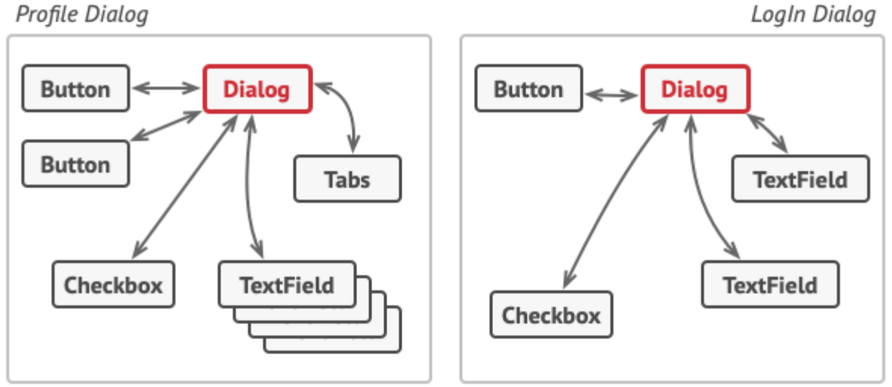




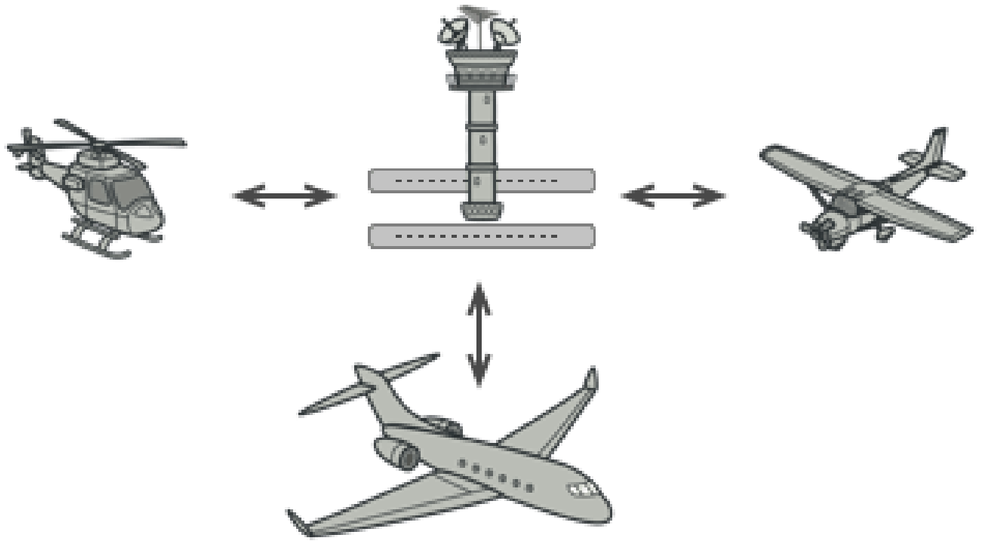
Relations between elements of the user interface can become chaotic as the application evolves.

Elements can have lots of relations with other elements. Hence, changes to some elements may affect the others.

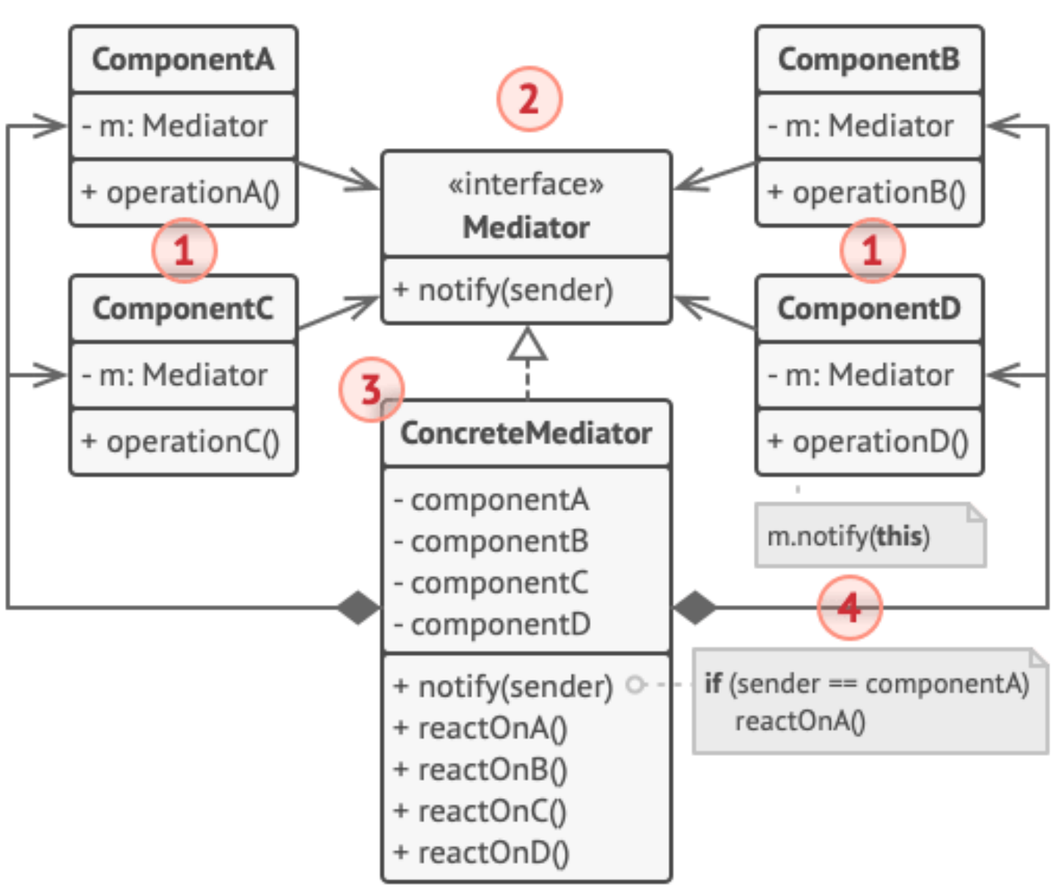
Mediator is a behavioral design pattern that lets you reduce chaotic dependencies between objects. The pattern restricts direct communications between the objects and forces them to collaborate only via a mediator object.

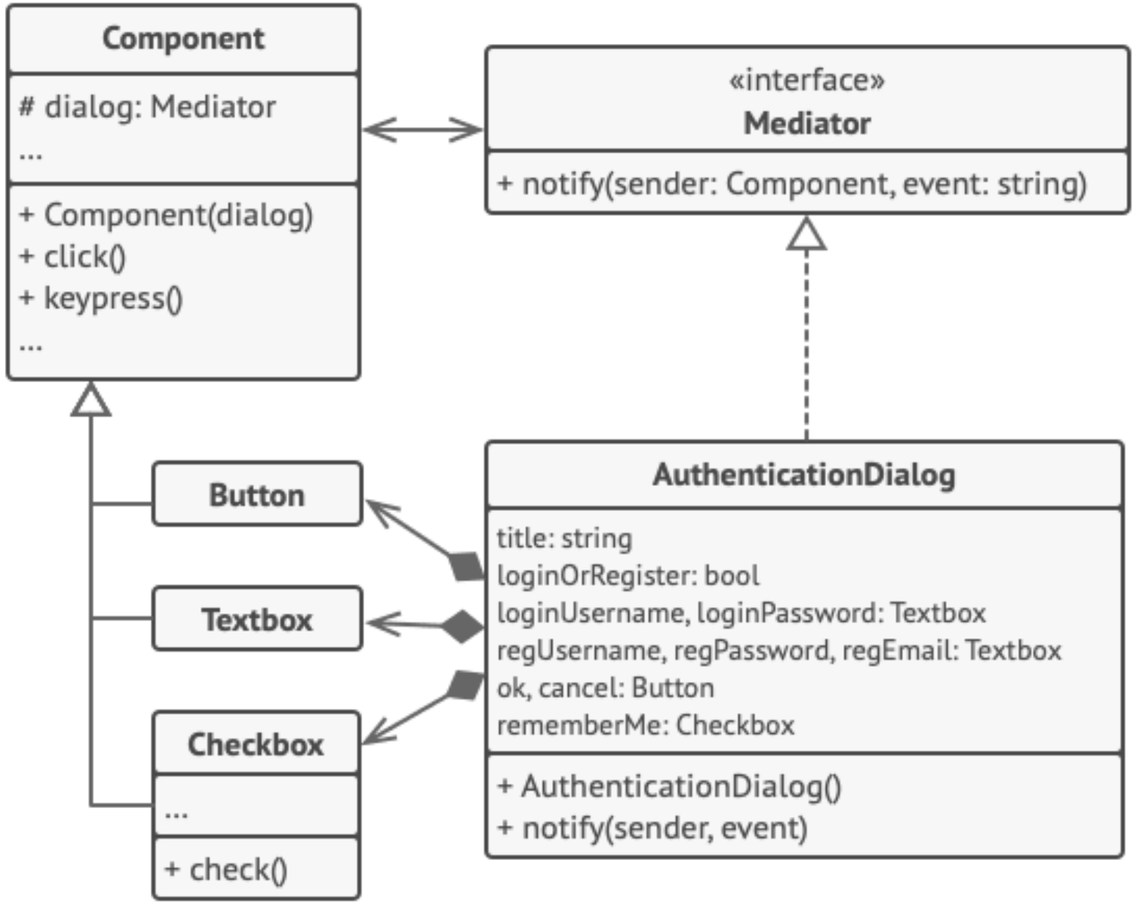


UI elements should communicate indirectly, via the mediator object.



Aircraft pilots don't talk to each other directly when deciding who gets to land their plane next. All communication goes through the control tower.





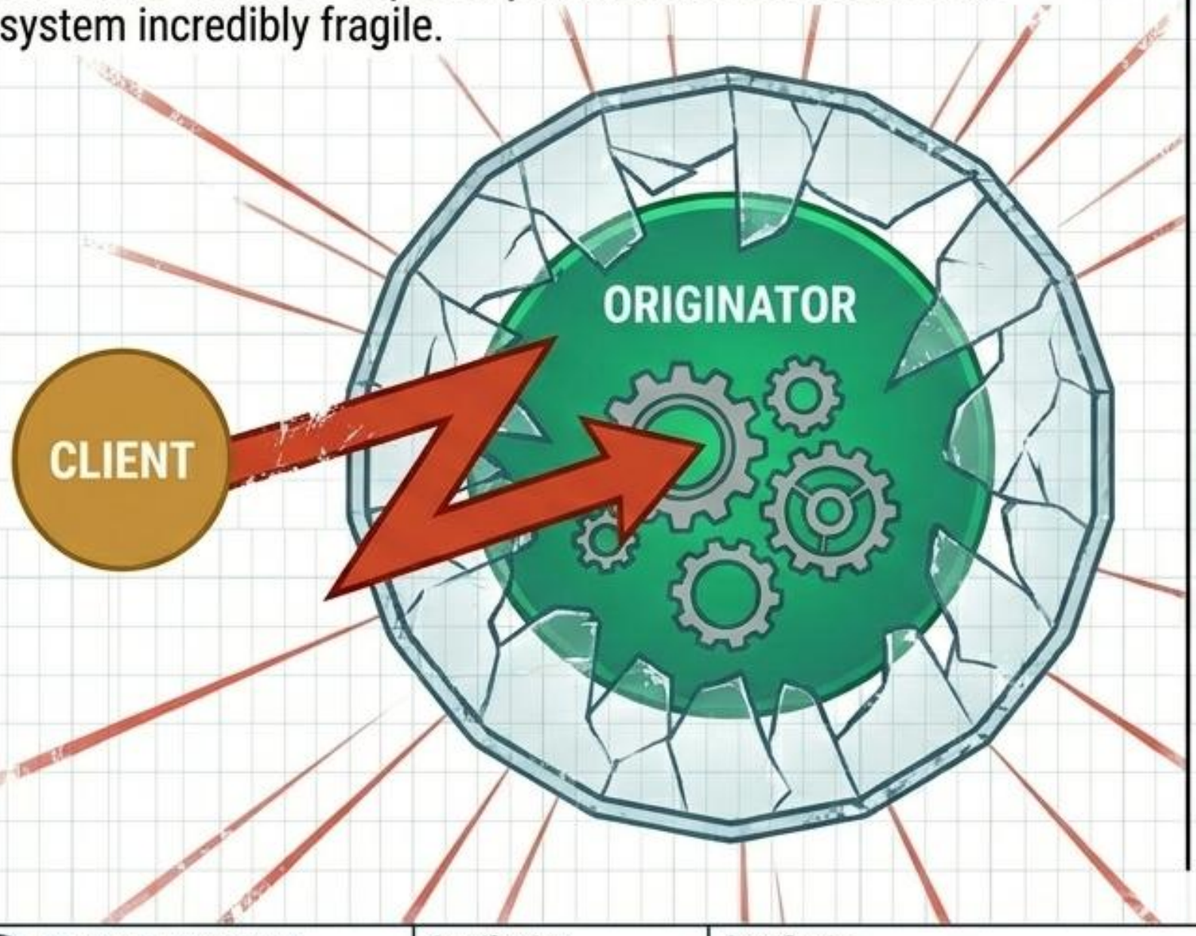
Structure of the UI dialog classes.

```
// The mediator interface declares a method used by components
// to notify the mediator about various events. The mediator may
// react to these events and pass the execution to other
// components.
interface Mediator is
    method notify(sender: Component, event: string)


// The concrete mediator class. The intertwined web of
// connections between individual components has been untangled
// and moved into the mediator.
class AuthenticationDialog implements Mediator is
    private field title: string
    private field loginOrRegisterChkBx: Checkbox
    private field loginUsername, loginPassword: Textbox
    private field registrationUsername, registrationPassword,
                  registrationEmail: Textbox
    private field okBtn, cancelBtn: Button

    constructor AuthenticationDialog() is
        // Create all component objects by passing the current
        // mediator into their constructors to establish links.

    // When something happens with a component, it notifies the
    // mediator. Upon receiving a notification, the mediator may
    // do something on its own or pass the request to another
    // component.
    method notify(sender, event) is
        if (sender == loginOrRegisterChkBx and event == "check")
            if (loginOrRegisterChkBx.checked)
                title = "Log in"
                // 1. Show login form components.
                // 2. Hide registration form components.
            else
                title = "Register"
                // 1. Show registration form components.
                // 2. Hide login form components

        if (sender == okBtn && event == "click")
            if (loginOrRegister.checked)
                // Try to find a user using login credentials.
                if (!found)
                    // Show an error message above the login
                    // field.
            else
                // 1. Create a user account using data from the
                // registration fields.
                // 2. Log that user in.
                // ...


// Components communicate with a mediator using the mediator
// interface. Thanks to that, you can use the same components in
// other contexts by linking them with different mediator
// objects.
class Component is
    field dialog: Mediator

    constructor Component(dialog) is
        this.dialog = dialog

    method click() is
        dialog.notify(this, "click")

    method keypress() is
        dialog.notify(this, "keypress")

// Concrete components don't talk to each other. They have only
// one communication channel, which is sending notifications to
// the mediator.
class Button extends Component is
    // ...

class Textbox extends Component is
    // ...

class Checkbox extends Component is
    method check() is
        dialog.notify(this, "check")
    // ...
```

MEMENTO // THE ENCAPSULATION BREACH VS. THE STATE SNAPSHOT

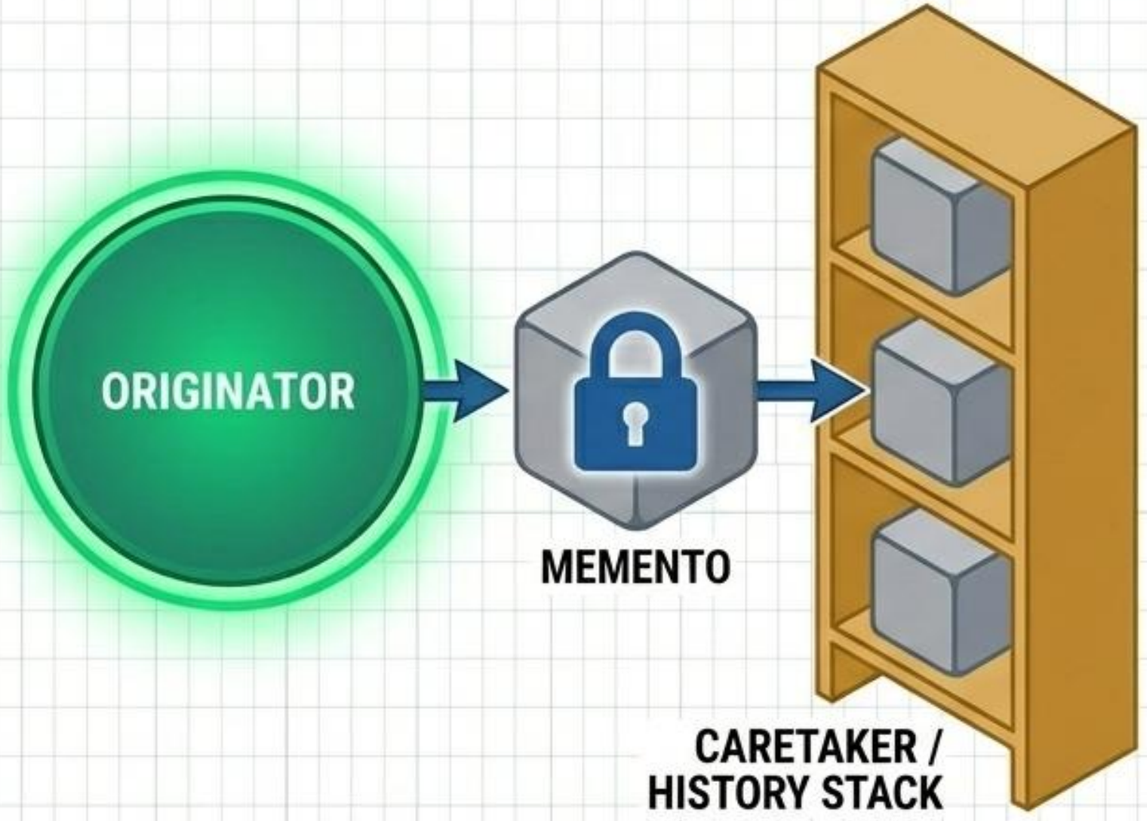
BREAKING THE SHIELD

Allowing external objects to access and record internal state for “Undo” features exposes private fields and makes the system incredibly fragile.



THE OPAQUE SNAPSHOT

The object creates a snapshot of its state (Memento) and hands it to a caretaker. The caretaker can store and return the snapshot, but cannot inspect or modify its contents.



GLOBAL SYSTEMS ENGINEERING	Specification GEHHA SYSTEMS	Specification	Description	Rev.	Revs. 1
		Specification: DCPI/BISTAE, MISN and BRVIM2007.:43-2021			

Ut no.	Specifications	Revision	Rev.
1	(M3	Stard cont	1

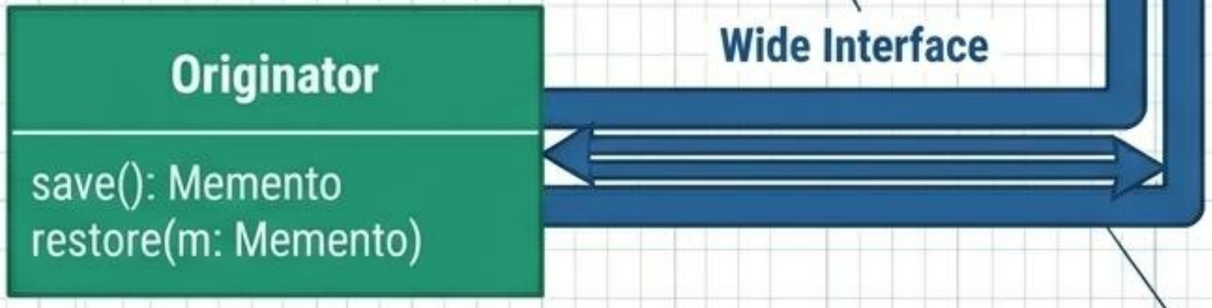
MEMENTO // INTERNAL STRUCTURE

Callout 1: Narrow vs. Wide Interfaces: The Caretaker sees a 'narrow' interface (locked, no access to state).

The Originator sees a "wide" interface (full access to restore itself).

Narrow Interface

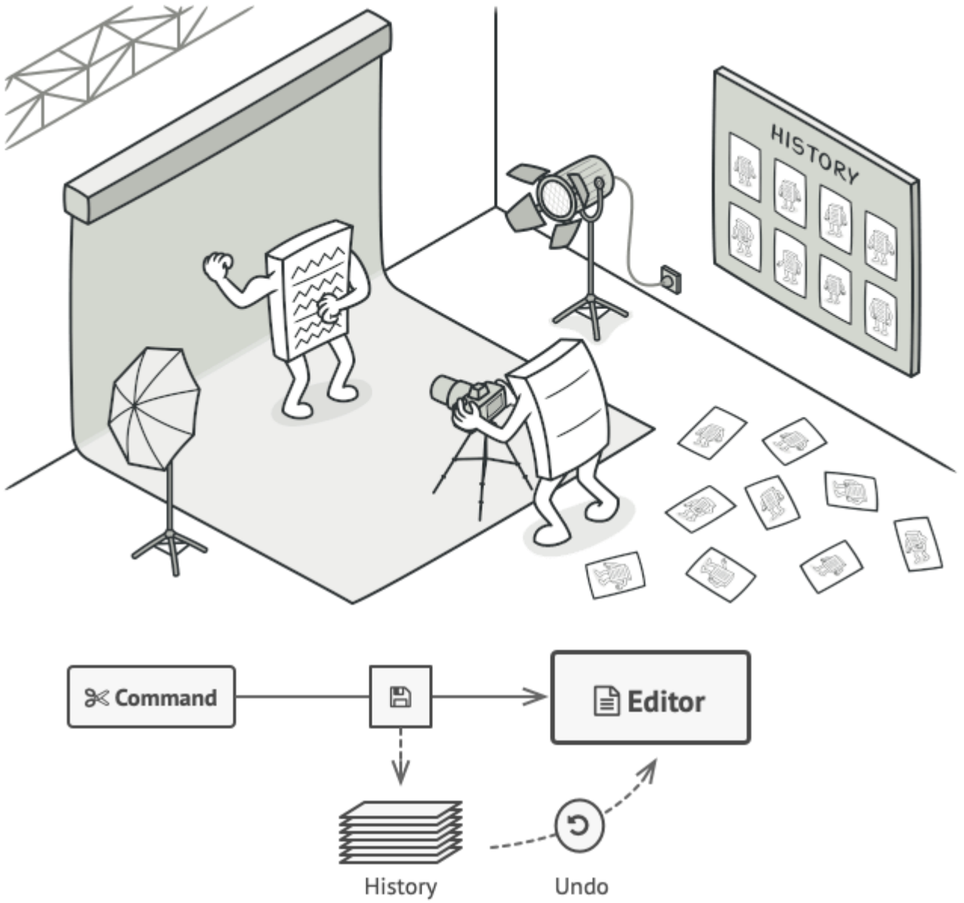
Wide Interface



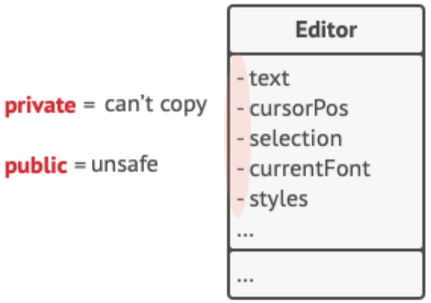
Callout 2: Perfect encapsulation is preserved while still granting the system infinite rollback capabilities.

GLOBAL SYSTEMS ENGINEERING	Specification GEHHA SYSTEMS	Specification Specification: DCPI/BISTAE, MISN and BRVIM2007.:43-2021	Description	Rev.	Revs. 1

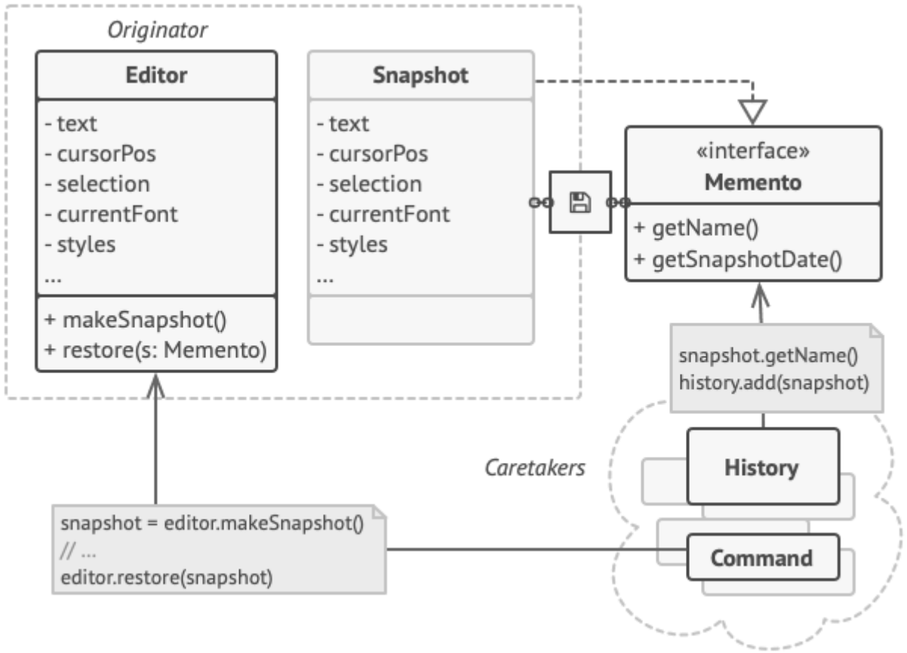
Memento is a behavioral design pattern that lets you save and restore the previous state of an object without revealing the details of its implementation.



Before executing an operation, the app saves a snapshot of the objects' state, which can later be used to restore objects to their previous state.

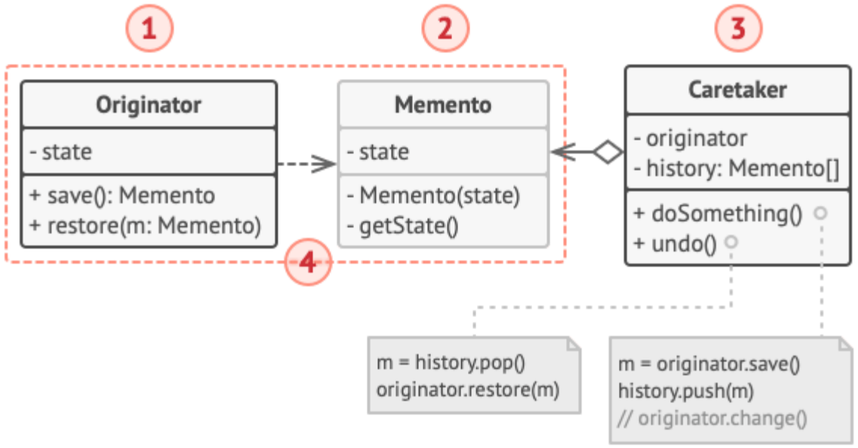


How to make a copy of the object's private state?

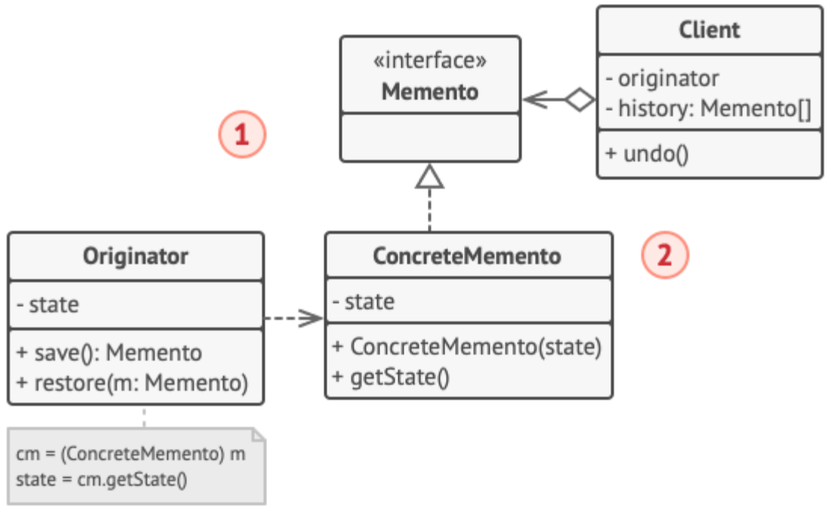


The originator has full access to the memento, whereas the caretaker can only access the metadata.

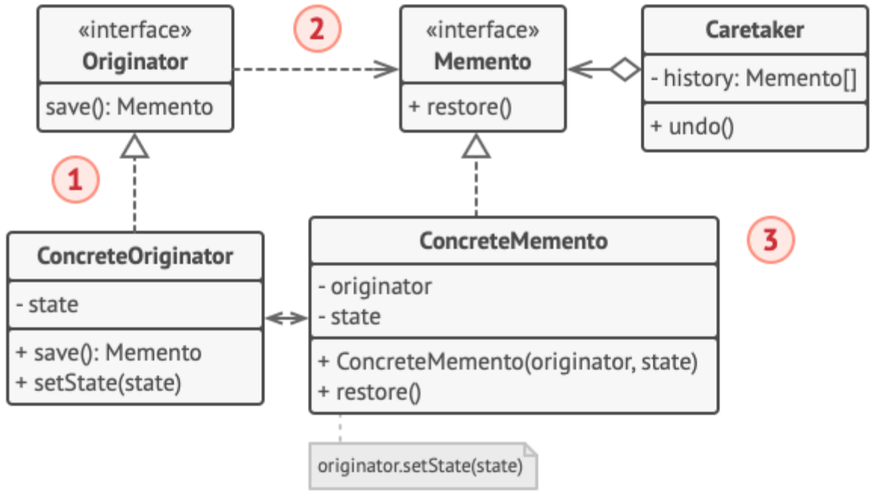
Implementation based on nested classes

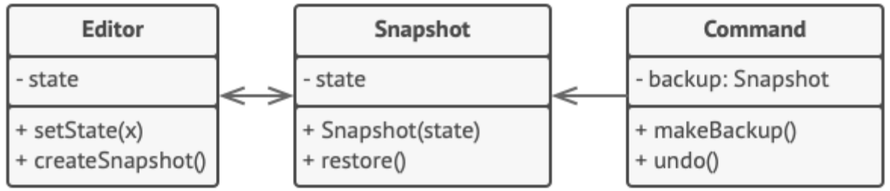


Implementation based on an intermediate interface



Implementation with even stricter encapsulation





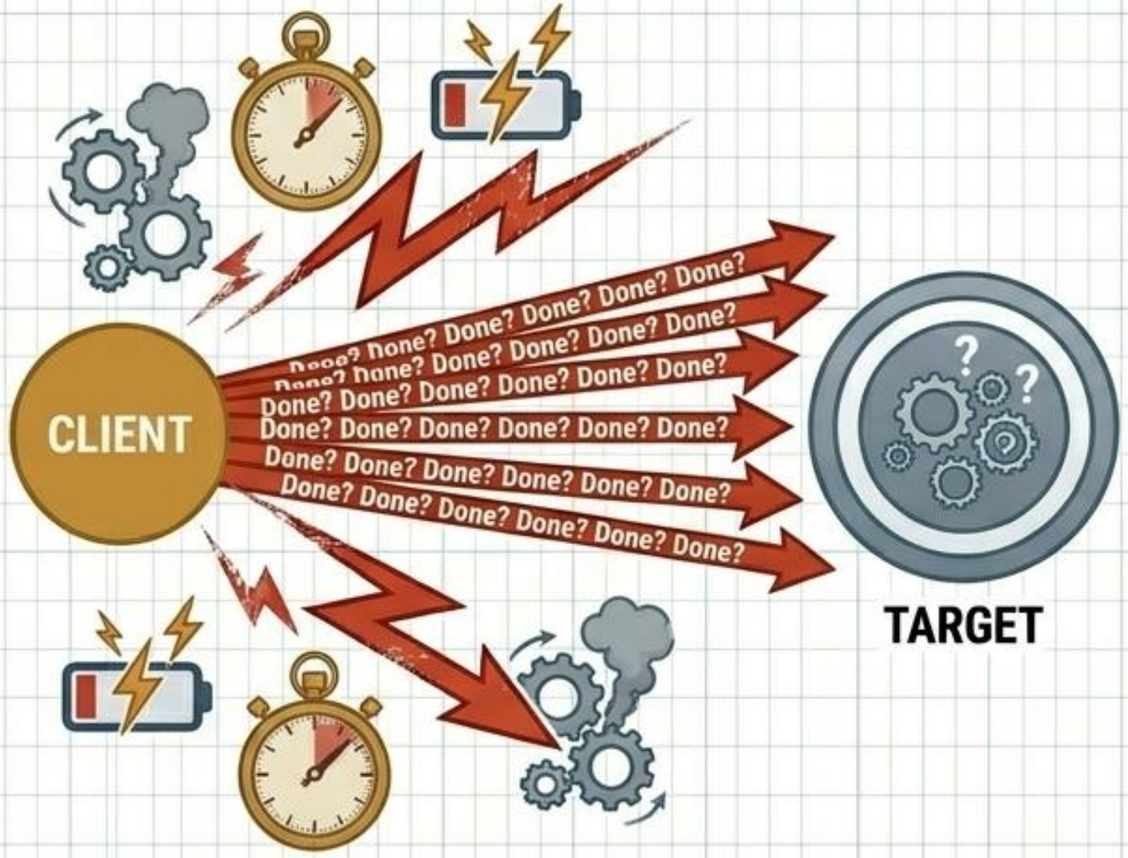
Saving snapshots of the text editor's state.

```
// The originator holds some important data that may change
over
// time. It also defines a method for saving its state
inside a
// memento and another method for restoring the state from
it.
class Editor is
    private field text, curX, curY, selectionWidth

    method setText(text) is
        this.text = text

    method setCursor(x, y) is
        this.curX = x
        this.curY = y

    method setSelectionWidth(width) is
        this.selectionWidth = width

    // Saves the current state inside a memento.
    method createSnapshot():Snapshot is
        // Memento is an immutable object; that's why the
        // originator passes its state to the memento's
        // constructor parameters.
        return new Snapshot(this, text, curX, curY,
selectionWidth)

// The memento class stores the past state of the editor.
class Snapshot is
    private field editor: Editor
    private field text, curX, curY, selectionWidth

    constructor Snapshot(editor, text, curX, curY,
selectionWidth) is
        this.editor = editor
        this.text = text
        this.curX = x
        this.curY = y
        this.selectionWidth = selectionWidth

    // At some point, a previous state of the editor can be
    // restored using a memento object.
    method restore() is
        editor.setText(text)
        editor.setCursor(curX, curY)
        editor.setSelectionWidth(selectionWidth)

// A command object can act as a caretaker. In that case,
the
// command gets a memento just before it changes the
// originator's state. When undo is requested, it restores
the
// originator's state from a memento.
class Command is
    private field backup: Snapshot

    method makeBackup() is
        backup = editor.createSnapshot()

    method undo() is
        if (backup != null)
            backup.restore()
    // ...
```

OBSERVER // INEFFICIENT POLLING VS. PUBLISH-SUBSCRIBE MECHANICS

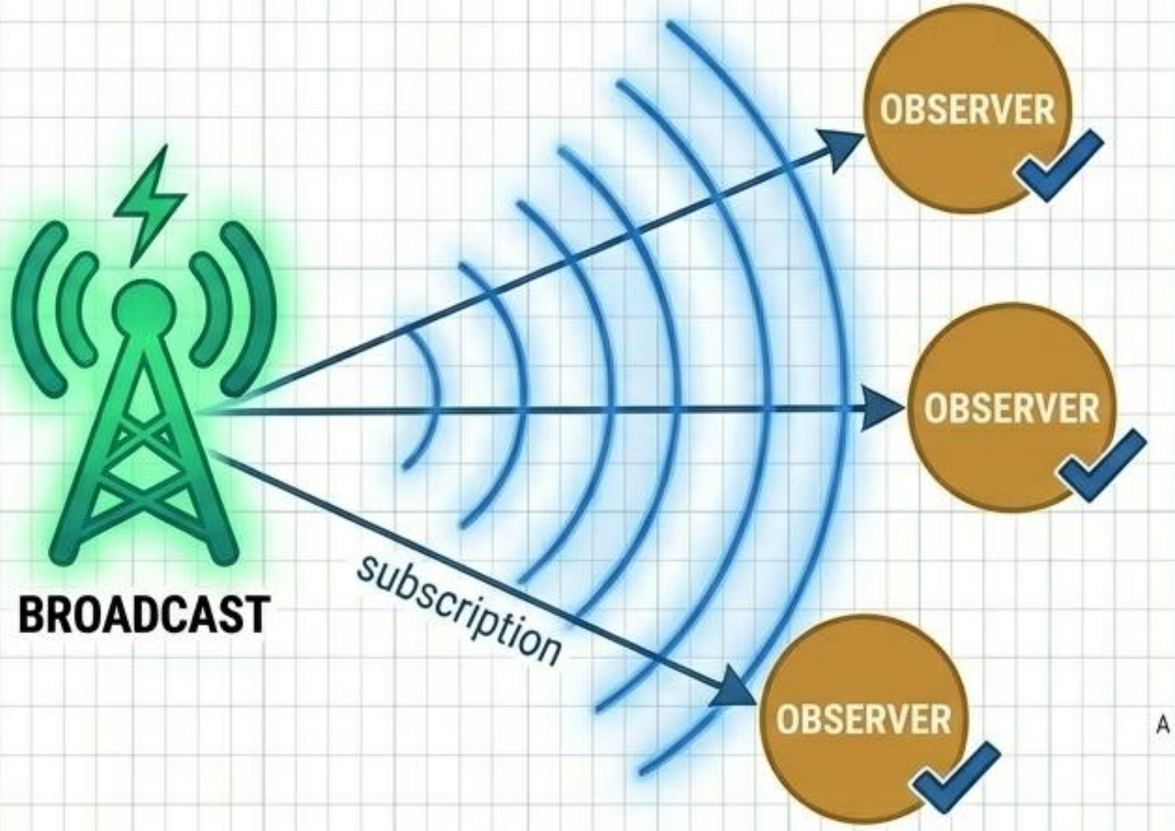
THE POLLING TAX

Constantly checking another object for state changes burns resources, creates tight coupling, and scales terribly.



BROADCAST SUBSCRIPTIONS

The publisher maintains a list of subscribers. When its state changes, it instantly pushes a broadcast outward. Objects react dynamically with zero wasted resources.

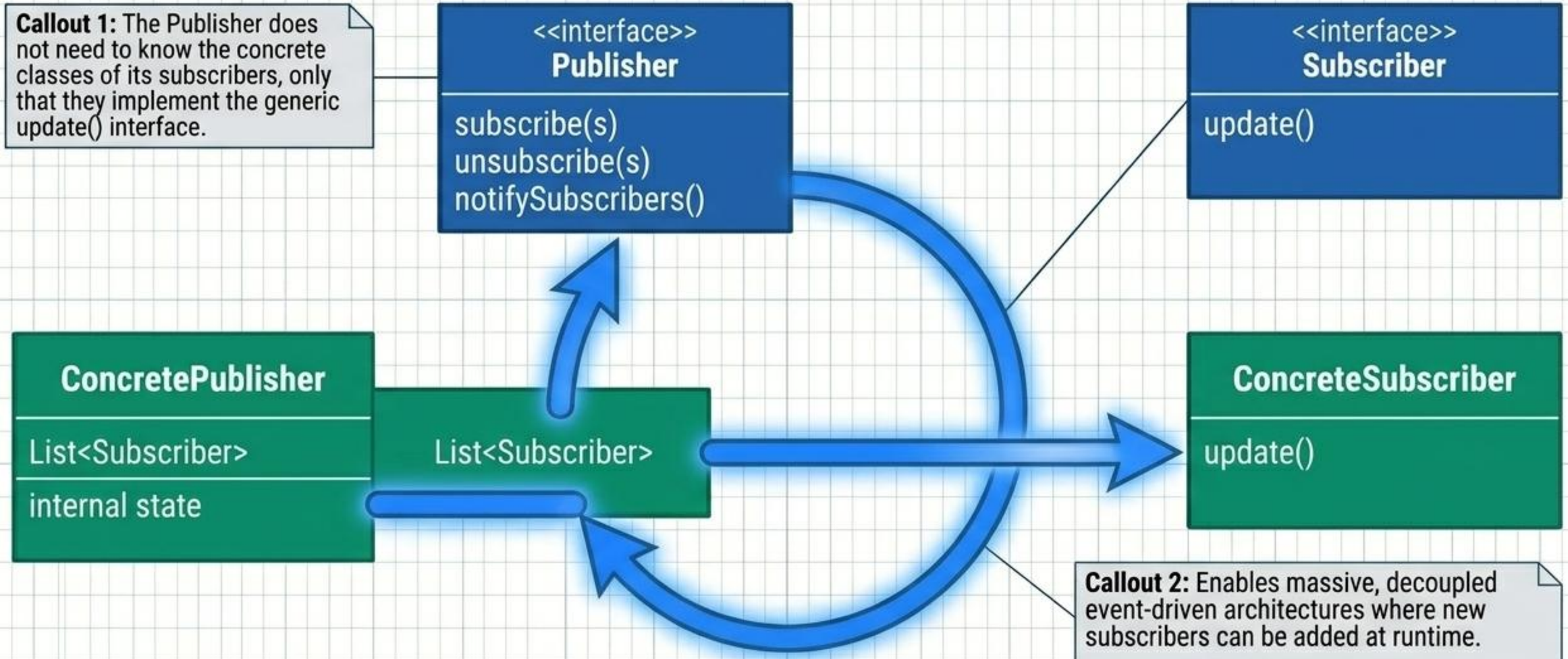


GLOBAL SYSTEMS ENGINEERING	Specification GEHHA SYSTEMS	Specification Specification: DCPI/BISTAE, MISN and BRVIM2007.:43-2021	Description	Rev.	Revs. 1

Ut no.	Specifications	Revision	Rev.
1	M3	Stard cont	1

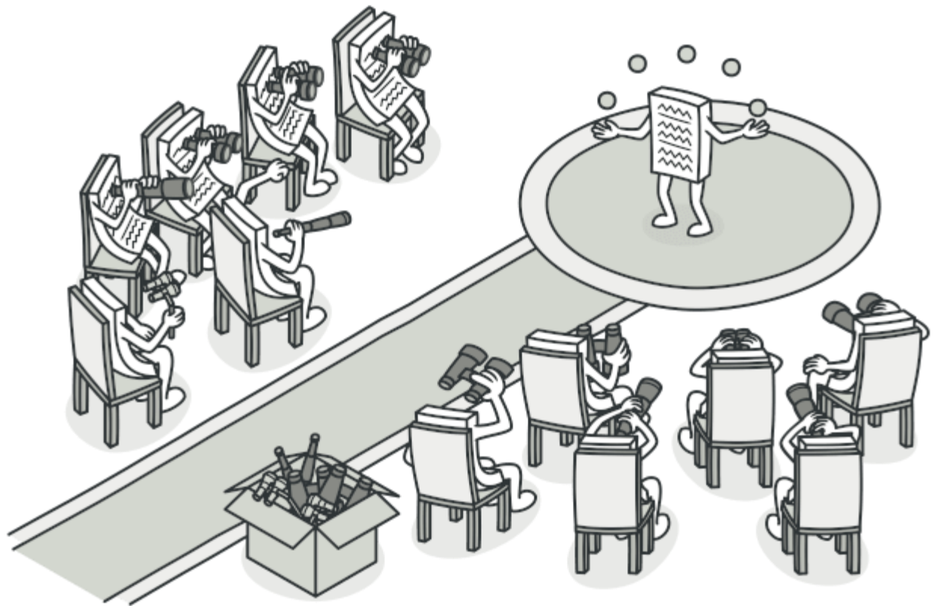
OBSERVER // INTERNAL STRUCTURE

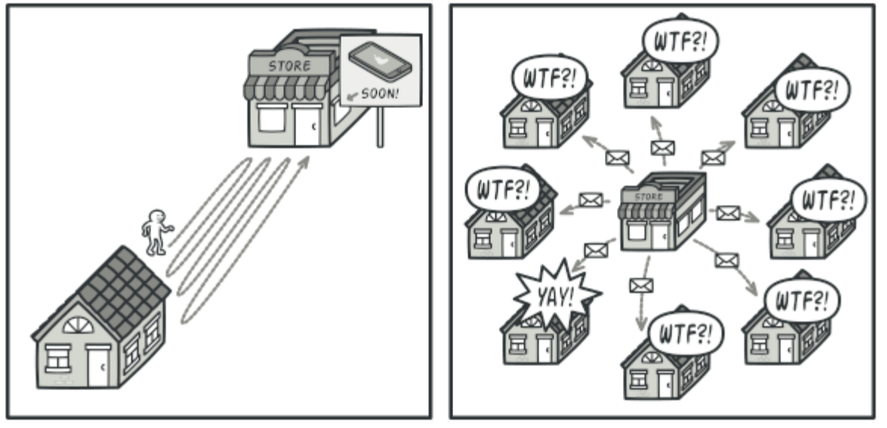
Callout 1: The Publisher does not need to know the concrete classes of its subscribers, only that they implement the generic update() interface.



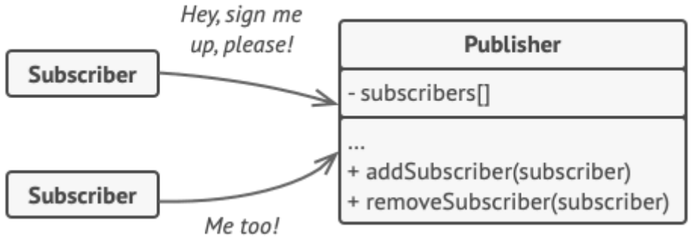
Callout 2: Enables massive, decoupled event-driven architectures where new subscribers can be added at runtime.

GLOBAL SYSTEMS ENGINEERING	Specification	Specification	Description	Rev.	Revs.
	GEHHA SYSTEMS	Specification: DCPI/BISTAE, MISN and BRVIM2007.:43-2021			1



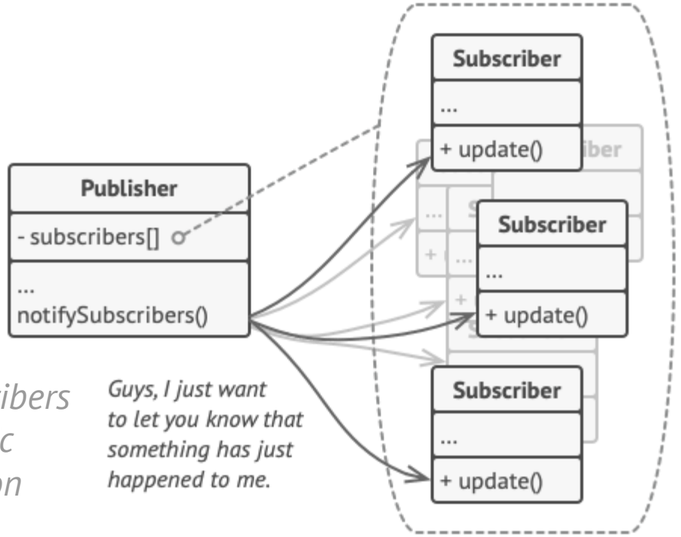


Visiting the store vs. sending spam



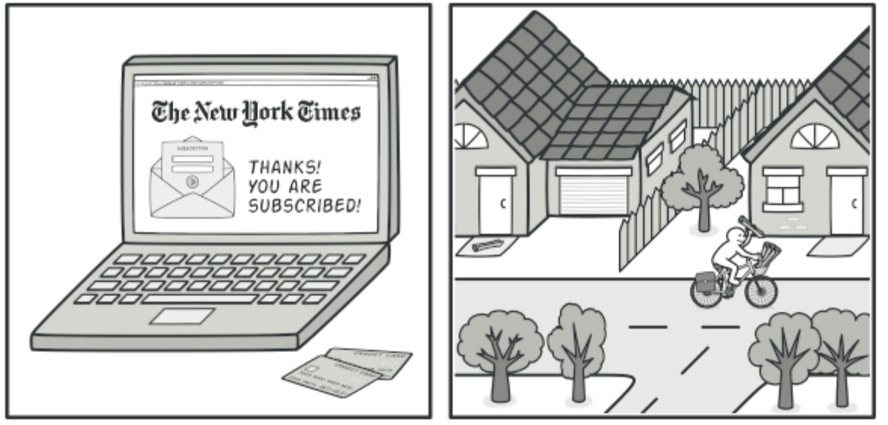
A subscription mechanism lets individual objects subscribe to event notifications.

Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.

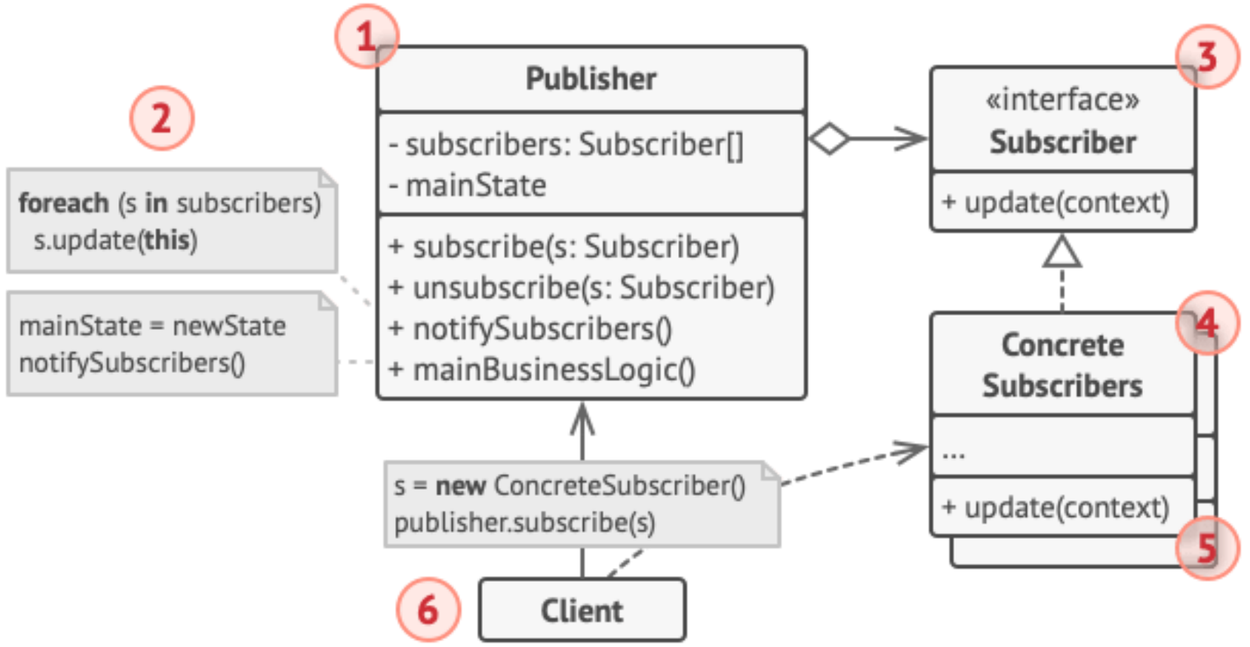


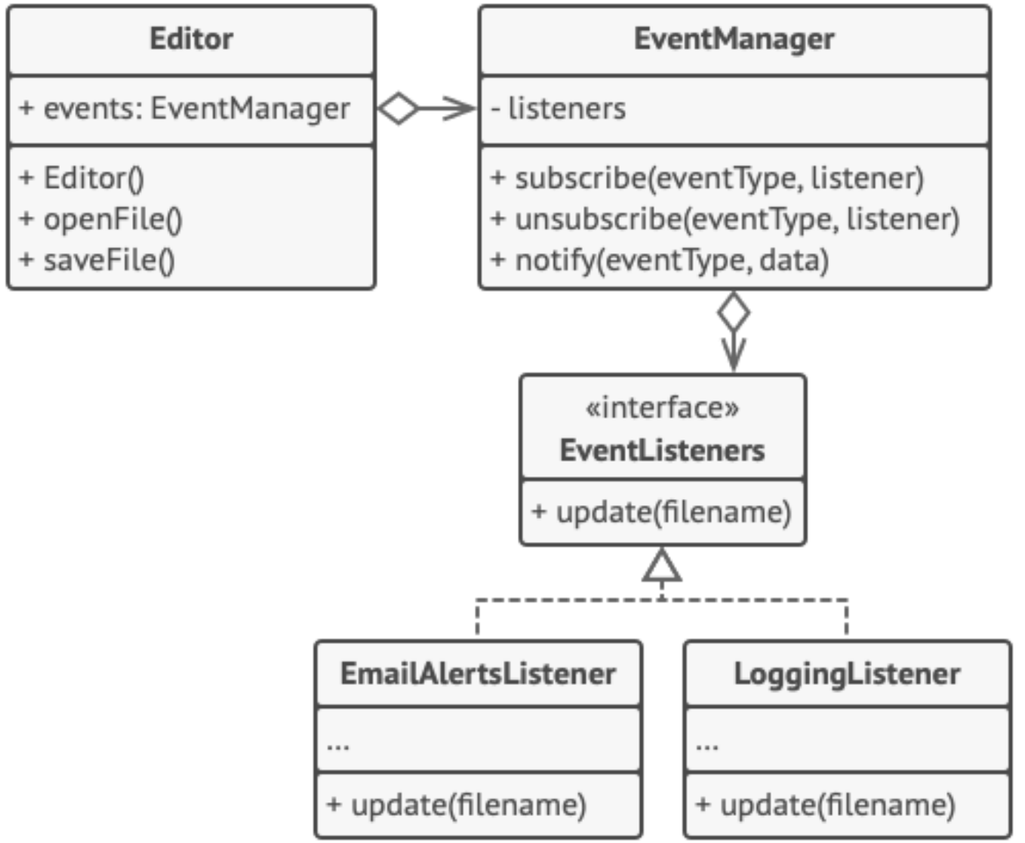
Publisher notifies subscribers by calling the specific notification method on their objects.

Guys, I just want to let you know that something has just happened to me.



Magazine and newspaper subscriptions.





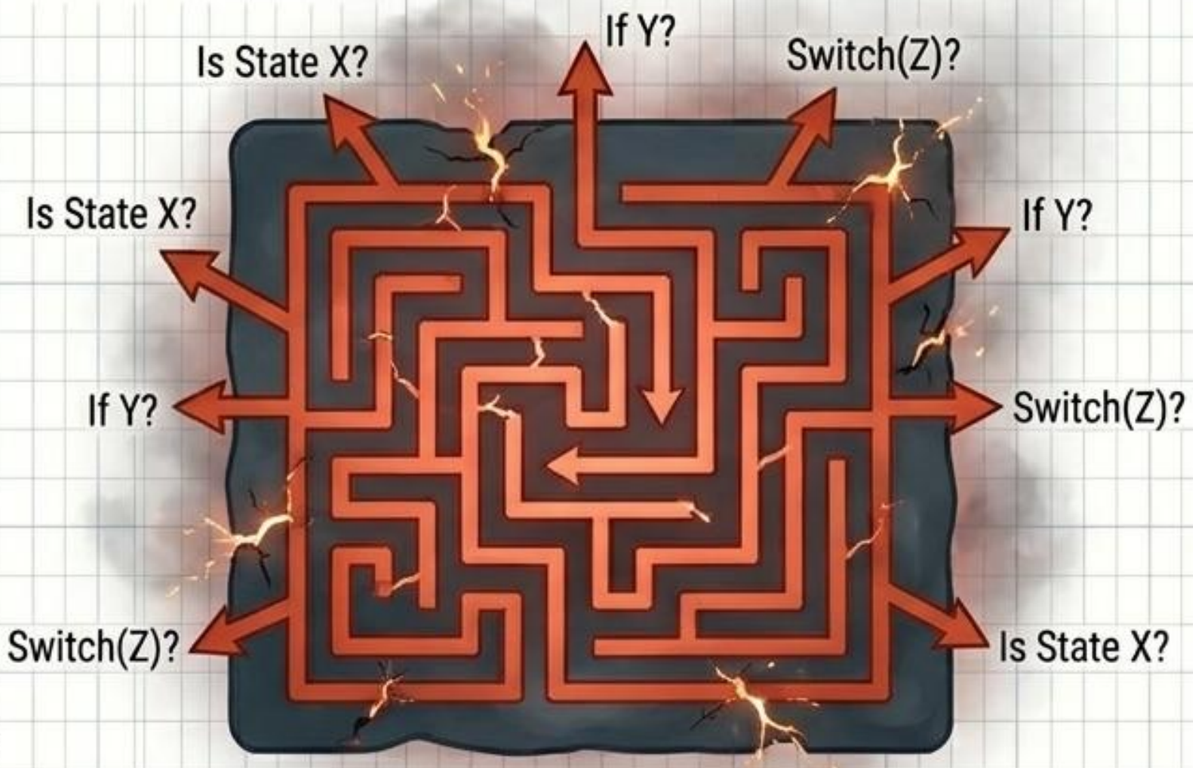
Notifying objects about events that happen to other objects.

```
// The base publisher class includes subscription management
// code and notification methods.
class EventManager is
    private field listeners: hash map of event types and listeners

    method subscribe(eventType, listener) is
        listeners.add(eventType, listener)

    method unsubscribe(eventType, listener) is
        listeners.remove(eventType, listener)

    method notify(eventType, data) is
        foreach (listener in listeners.of(eventType)) do
            listener.update(data)

// The concrete publisher contains real business logic that's
// interesting for some subscribers. We could derive this class
// from the base publisher, but that isn't always possible in
// real life because the concrete publisher might already be a
// subclass. In this case, you can patch the subscription logic
// in with composition, as we did here.
class Editor is
    public field events: EventManager
    private field file: File

    constructor Editor() is
        events = new EventManager()

    // Methods of business logic can notify subscribers about
    // changes.
    method openFile(path) is
        this.file = new File(path)
        events.notify("open", file.name)

    method saveFile() is
        file.write()
        events.notify("save", file.name
    // ...
// Here's the subscriber interface. If your programming language
// supports functional types, you can replace the whole
// subscriber hierarchy with a set of functions.
interface EventListener is
    method update(filename)

// Concrete subscribers react to updates issued by the publisher
// they are attached to.
class LoggingListener implements EventListener is
    private field log: File
    private field message: string

    constructor LoggingListener(log_filename, message) is
        this.log = new File(log_filename)
        this.message = message

    method update(filename) is
        log.write(replace('%s',filename,message))

class EmailAlertsListener implements EventListener is
    private field email: string
    private field message: string

    constructor EmailAlertsListener(email, message) is
        this.email = email
        this.message = message

    method update(filename) is
        system.email(email, replace('%s',filename,message))


// An application can configure publishers and subscribers at
// runtime.
class Application is
    method config() is
        editor = new Editor()

        logger = new LoggingListener(
            "/path/to/log.txt",
            "Someone has opened the file: %s")
        editor.events.subscribe("open", logger)

        emailAlerts = new EmailAlertsListener(
            "admin@example.com",
            "Someone has changed the file: %s")
        editor.events.subscribe("save", emailAlerts)
```

STATE // MONSTROUS CONDITIONALS VS. OBJECTIFYING CONTEXT

CONDITIONAL LABYRINTHS

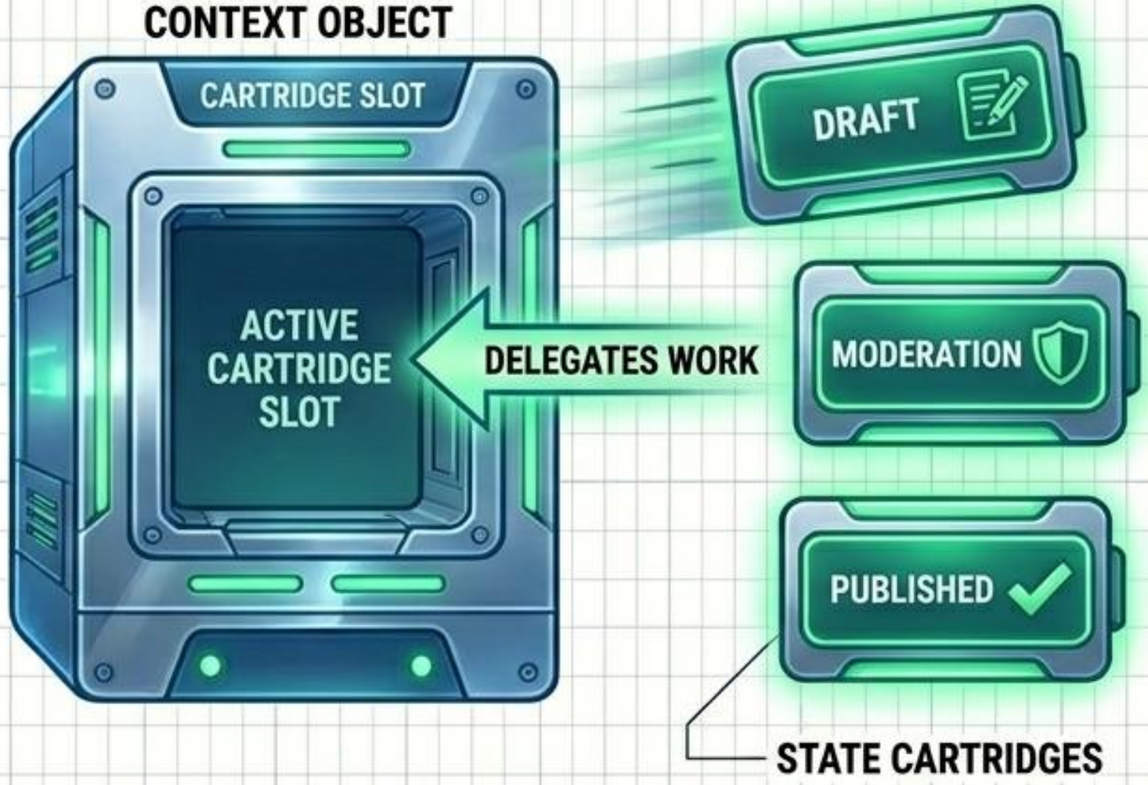
When behavior heavily depends on internal state, relying on massive conditional statements creates fragile, unmaintainable monoliths.



MONOLITHIC CLASS

SWAPPABLE STATE CARTRIDGES

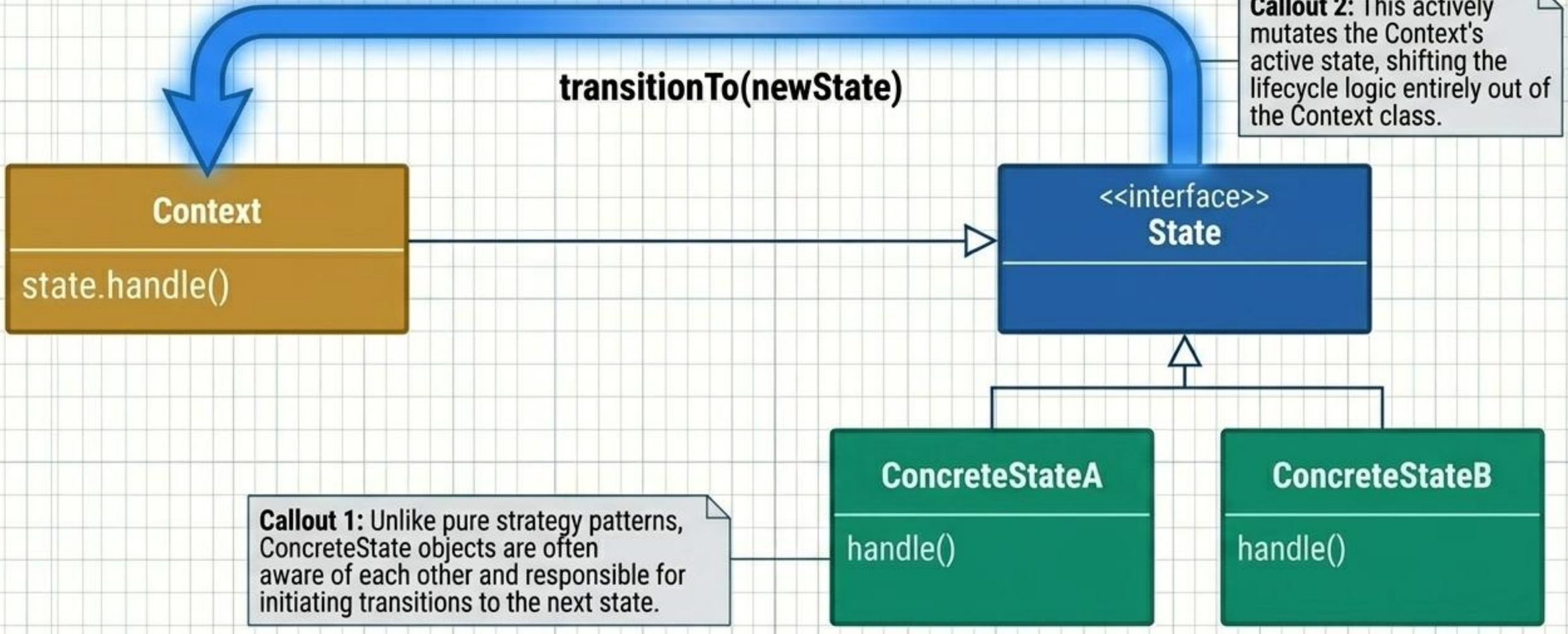
Extract state-specific behaviors into distinct classes. The Context object delegates work to the active state cartridge. Changing state simply means swapping the cartridge.



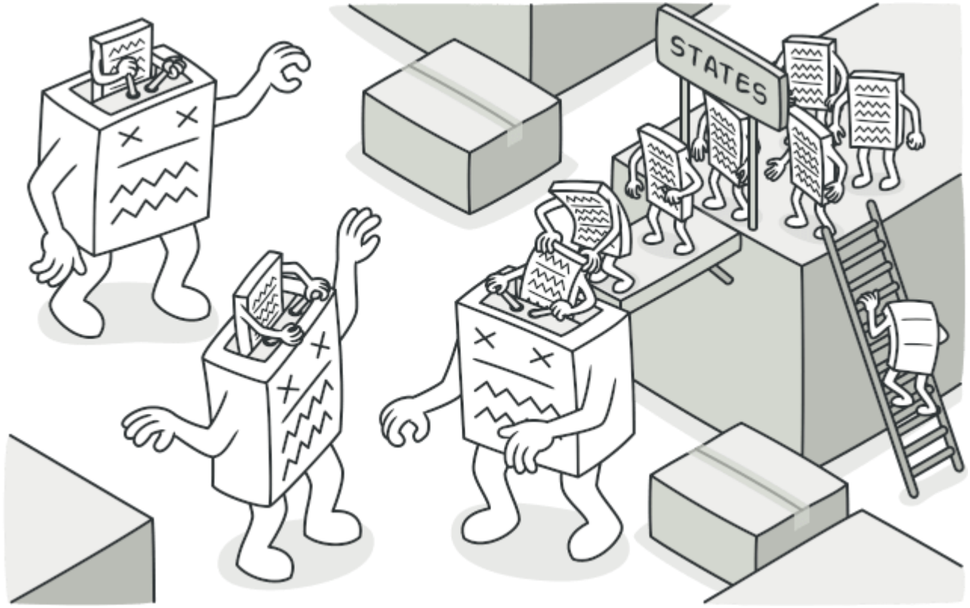
GLOBAL SYSTEMS ENGINEERING	Specification	Specification	Description	Rev.	Revs.
	GEHHA SYSTEMS	Speoification: DCPI/BISTAE, MISN and BRVIM2007.:43-2021			1

STATE // INTERNAL STRUCTURE

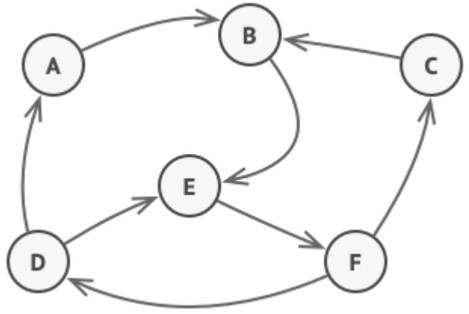
Ut no.	Specifications	Revision	Rev.
1	M3	Stard cont	1



GLOBAL SYSTEMS ENGINEERING	Specification	Specification	Description	Rev.	Reve.
	GEHHA SYSTEMS	Speoification: DCPI/BISTAE, MISN and BRVIMC007.:43'-2021			1



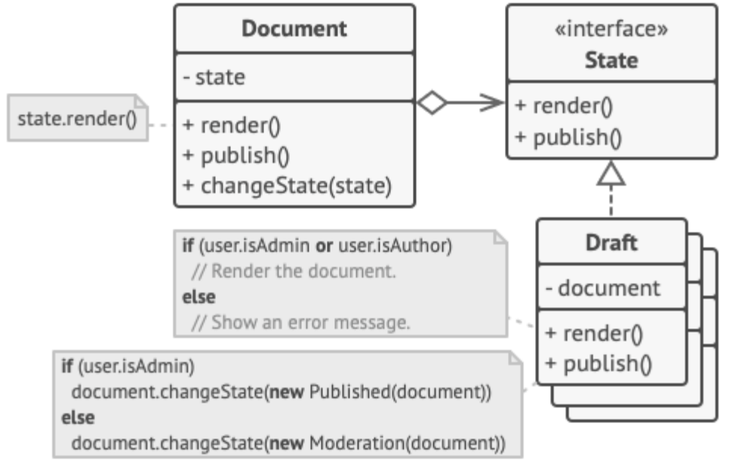
State is a behavioral design pattern that lets an object alter its behavior when its internal state changes. It appears as if the object changed its class.



Finite-State Machine.

```
class Document is
    field state: string
    // ...
    method publish() is
        switch (state)
            "draft":
                state = "moderation"
                break
            "moderation":
                if (currentUser.role == "admin")
                    state = "published"
                break
            "published":
                // Do nothing.
                break
    // ...
```

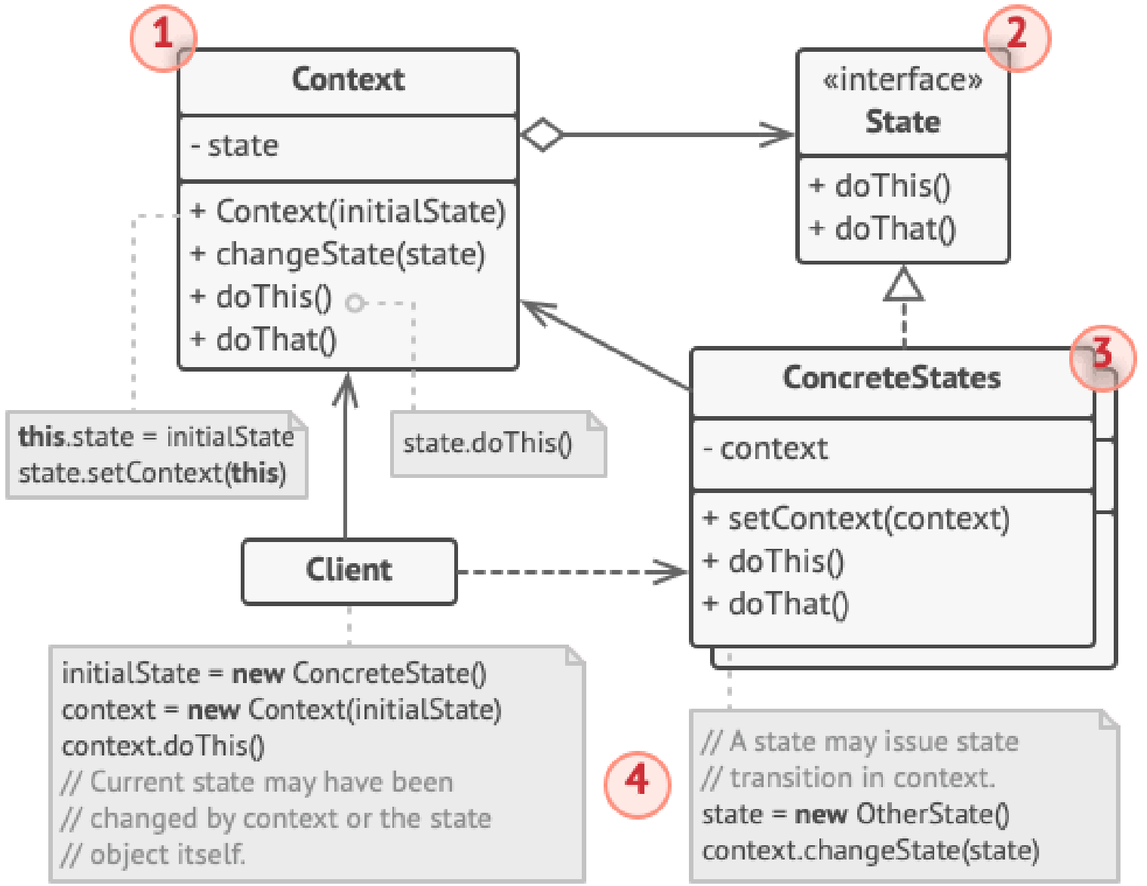
Possible states and transitions of a document object.

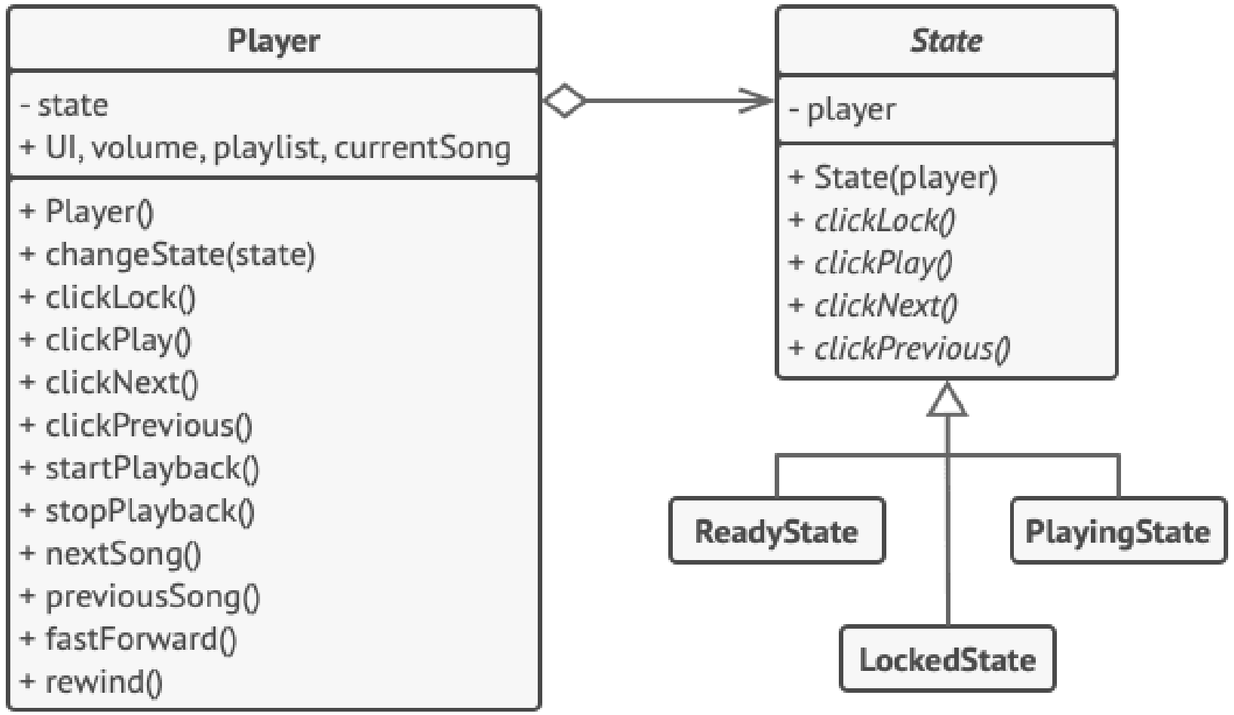


Real-World Analogy

The buttons and switches in your smartphone behave differently depending on the current state of the device:

- When the phone is unlocked, pressing buttons leads to executing various functions.
- When the phone is locked, pressing any button leads to the unlock screen.
- When the phone's charge is low, pressing any button shows the charging screen.





Example of changing object behavior with state objects.

```
// The AudioPlayer class acts as a context. It also maintains
a
// reference to an instance of one of the state classes that
// represents the current state of the audio player.
class AudioPlayer is
    field state: State
    field UI, volume, playlist, currentSong

    constructor AudioPlayer() is
        this.state = new ReadyState(this)

        // Context delegates handling user input to a state
        // object. Naturally, the outcome depends on what
state
        // is currently active, since each state can handle
the
        // input differently.
        UI = new UserInterface()
        UI.lockButton.onClick(this.clickLock)
        UI.playButton.onClick(this.clickPlay)
        UI.nextButton.onClick(this.clickNext)
        UI.prevButton.onClick(this.clickPrevious)

    // Other objects must be able to switch the audio player's
    // active state.
    method changeState(state: State) is
        this.state = state

    // UI methods delegate execution to the active state.
    method clickLock() is
        state.clickLock()
    method clickPlay() is
        state.clickPlay()
    method clickNext() is
        state.clickNext()
    method clickPrevious() is
        state.clickPrevious()

    // A state may call some service methods on the context.
    method startPlayback() is
        // ...
    method stopPlayback() is
        // ...
    method nextSong() is
        // ...
    method previousSong() is
        // ...
    method fastForward(time) is
        // ...
    method rewind(time) is
        // ...


// The base state class declares methods that all concrete
// states should implement and also provides a backreference
to
// the context object associated with the state. States can
use
// the backreference to transition the context to another
state.
abstract class State is
    protected field player: AudioPlayer

    // Context passes itself through the state constructor.
This
    // may help a state fetch some useful context data if it's
    // needed.
    constructor State(player) is
        this.player = player

    abstract method clickLock()
    abstract method clickPlay()
    abstract method clickNext()
    abstract method clickPrevious()


// Concrete states implement various behaviors associated with
a
// state of the context.
class LockedState extends State is

    // When you unlock a locked player, it may assume one of
two
    // states.
    method clickLock() is
        if (player.playing)
            player.changeState(new PlayingState(player))
        else
            player.changeState(new ReadyState(player))

    method clickPlay() is
        // Locked, so do nothing.

    method clickNext() is
        // Locked, so do nothing.

    method clickPrevious() is
        // Locked, so do nothing.


// They can also trigger state transitions in the context.
class ReadyState extends State is
    method clickLock() is
        player.changeState(new LockedState(player))

    method clickPlay() is
        player.startPlayback()
        player.changeState(new PlayingState(player))

    method clickNext() is
        player.nextSong()

    method clickPrevious() is
        player.previousSong()


class PlayingState extends State is
    method clickLock() is
        player.changeState(new LockedState(player))

    method clickPlay() is
        player.stopPlayback()
        player.changeState(new ReadyState(player))

    method clickNext() is
        if (event.doubleclick)
            player.nextSong()
        else
            player.fastForward(5)

    method clickPrevious() is
        if (event.doubleclick)
            player.previous()
        else
            player.rewind(5)
```

STRATEGY // ALGORITHMIC CLUTTER VS. SWAPPABLE ALGORITHMS

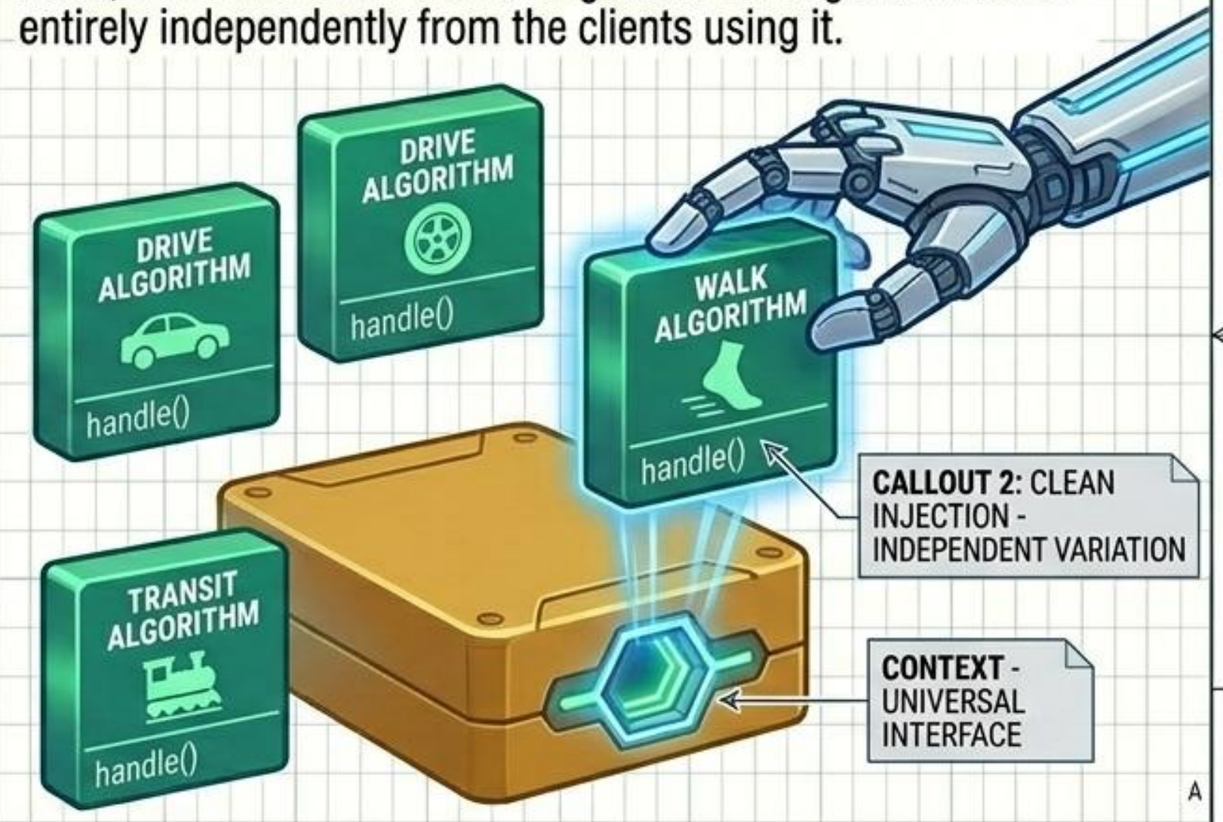
ALGORITHMIC CLUTTER

Housing multiple variations of an algorithm inside a single class leads to bloated codebases and severe merge conflicts.

MONOLITHIC CLASS

MODULAR INJECTION

Define a family of algorithms, encapsulate each in a separate class, and make them interchangeable. The algorithm varies entirely independently from the clients using it.

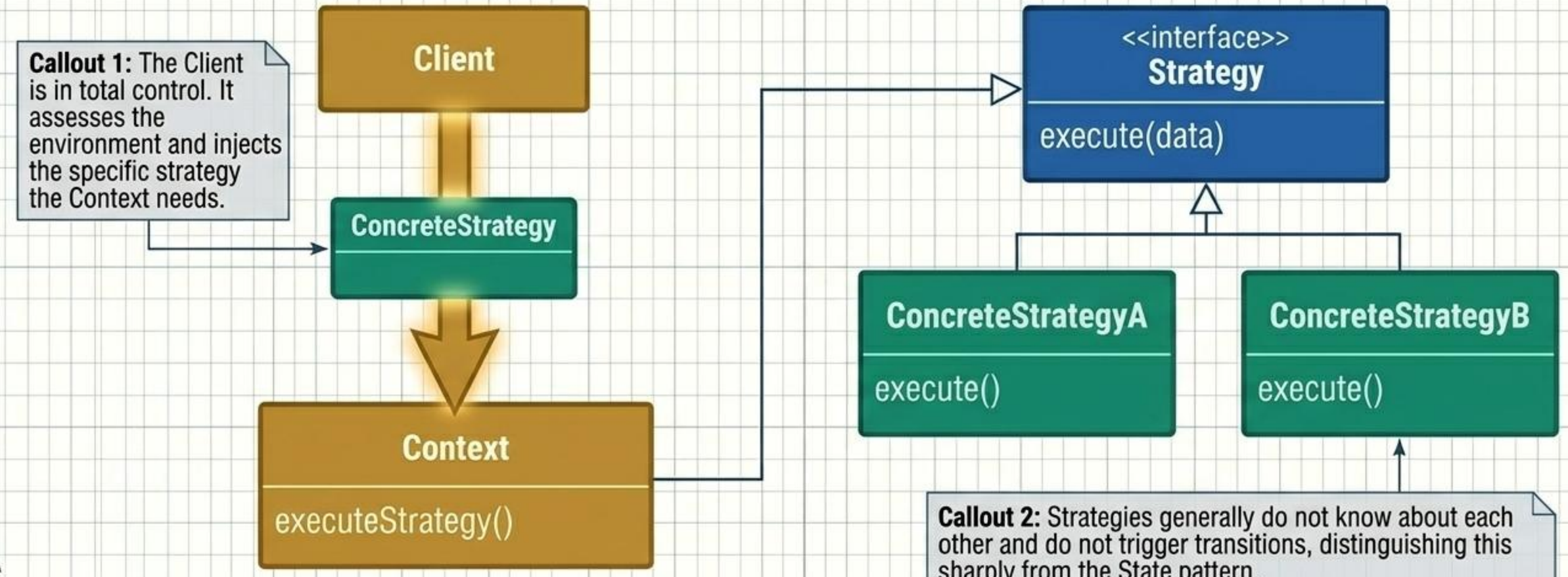


NAVIGATION CONTEXT

GLOBAL SYSTEMS ENGINEERING	Specification GEHHA SYSTEMS	Specification Specification: DCPI/BISTAE, MISN and BRVIMC007..43'-2021	Description	Rev.	Reve. 1

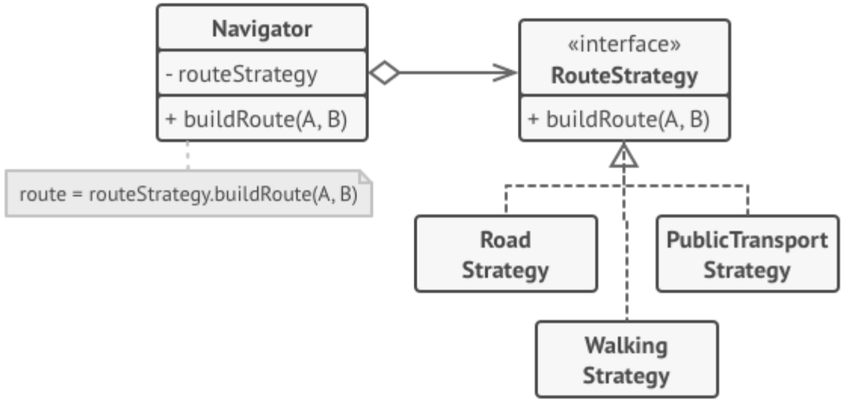
STRATEGY // INTERNAL STRUCTURE

Ut no.	Specifications	Revision	Rev.
1	/03	Stard cont	1

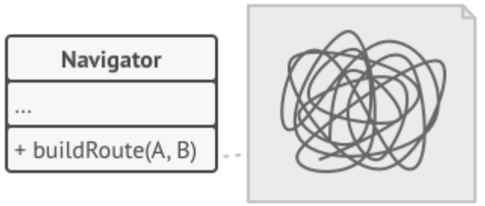


GLOBAL SYSTEMS ENGINEERING	Specification	Specification	Description	Rev.	Reve.
	GEHHA SYSTEMS	Specification: DCPI/BISTAE, MISN and BRVIMC007..43'-2021			1

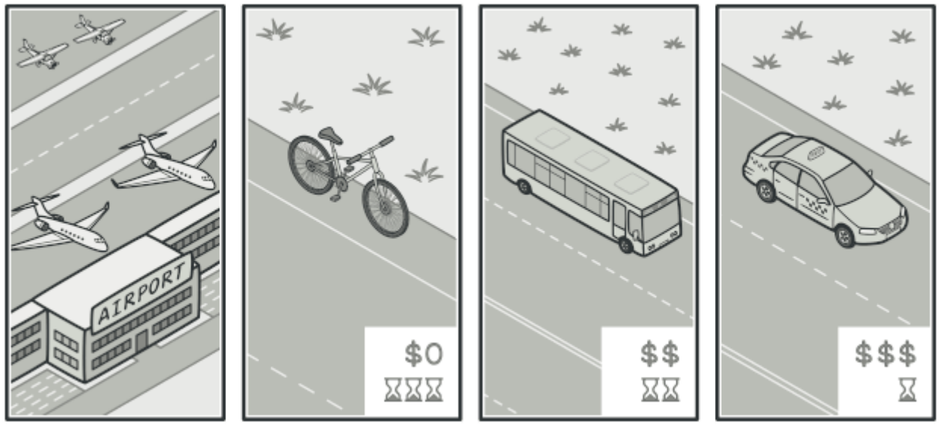
Strategy is a behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable.



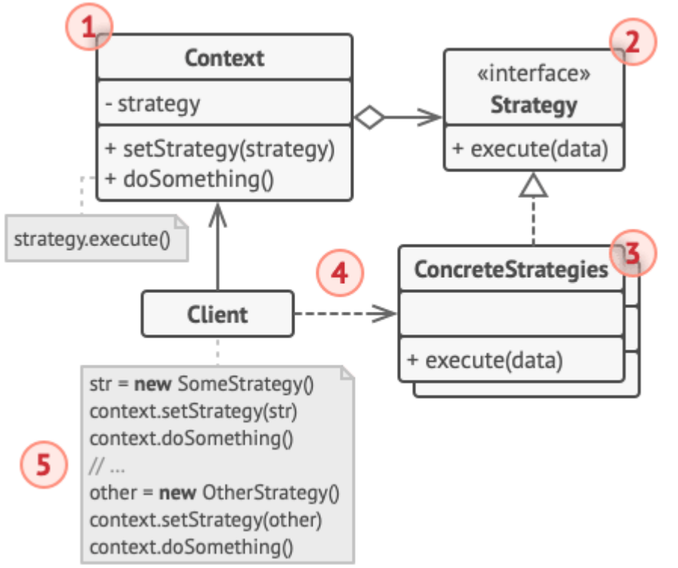
Route planning strategies.



The code of the navigator became bloated.



Various strategies for getting to the airport.

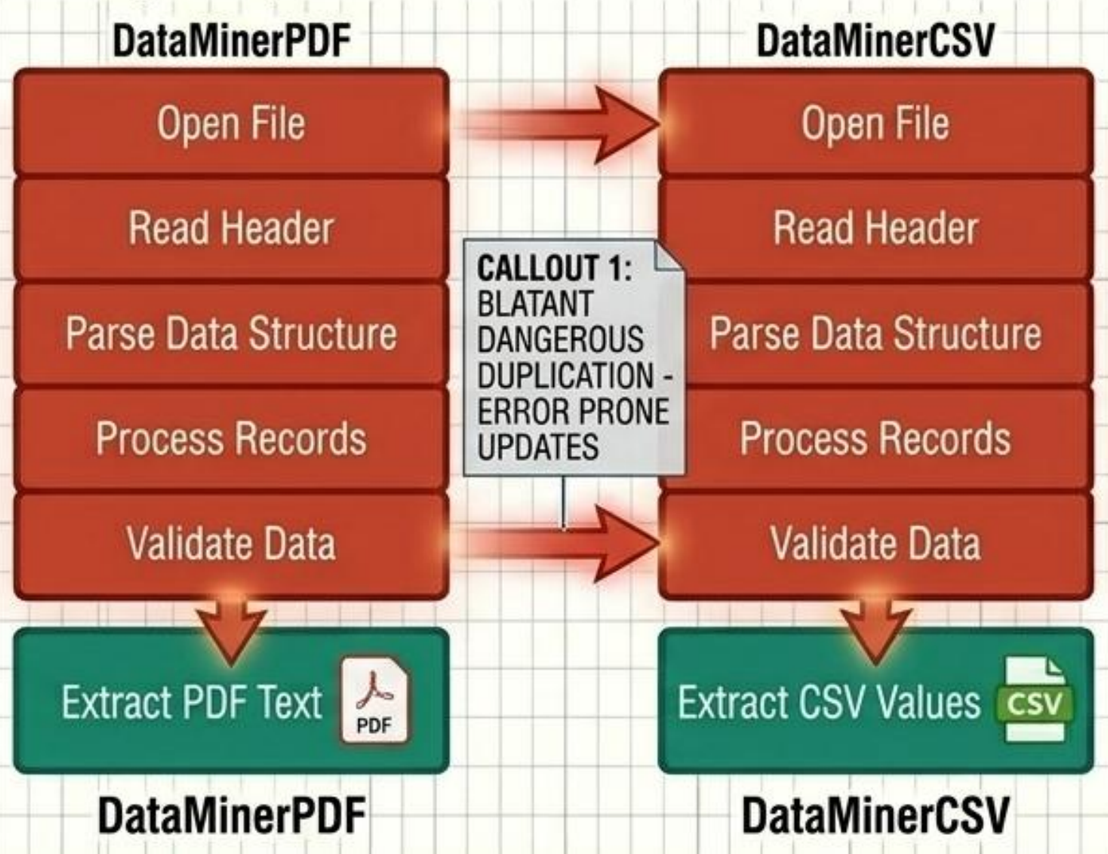


```
// The strategy interface declares operations common to all
// supported versions of some algorithm. The context uses this
// interface to call the algorithm defined by the concrete
// strategies.
interface Strategy is
    method execute(a, b)

// Concrete strategies implement the algorithm while following
// the base strategy interface. The interface makes them
// interchangeable in the context.
class ConcreteStrategyAdd implements Strategy is
    method execute(a, b) is
        return a + b

class ConcreteStrategySubtract implements Strategy is
    method execute(a, b) is
        return a - b

class ConcreteStrategyMultiply implements Strategy is
    method execute(a, b) is
        return a * b

// The context defines the interface of interest to clients.
class Context is
    // The context maintains a reference to one of the strategy
    // objects. The context doesn't know the concrete class of a
    // strategy. It should work with all strategies via the
    // strategy interface.
    private strategy: Strategy

    // Usually the context accepts a strategy through the
    // constructor, and also provides a setter so that the
    // strategy can be switched at runtime.
    method setStrategy(Strategy strategy) is
        this.strategy = strategy

    // The context delegates some work to the strategy object
    // instead of implementing multiple versions of the
    // algorithm on its own.
    method executeStrategy(int a, int b) is
        return strategy.execute(a, b)


// The client code picks a concrete strategy and passes it to
// the context. The client should be aware of the differences
// between strategies in order to make the right choice.
class ExampleApplication is
    method main() is
        Create context object.

        Read first number.
        Read last number.
        Read the desired action from user input.

        if (action == addition) then
            context.setStrategy(new ConcreteStrategyAdd())

        if (action == subtraction) then
            context.setStrategy(new ConcreteStrategySubtract())

        if (action == multiplication) then
            context.setStrategy(new ConcreteStrategyMultiply())

        result = context.executeStrategy(First number, Second number)

        Print result.
```

TEMPLATE METHOD // DUPLICATED LOGIC VS. THE ALGORITHMIC SKELETON

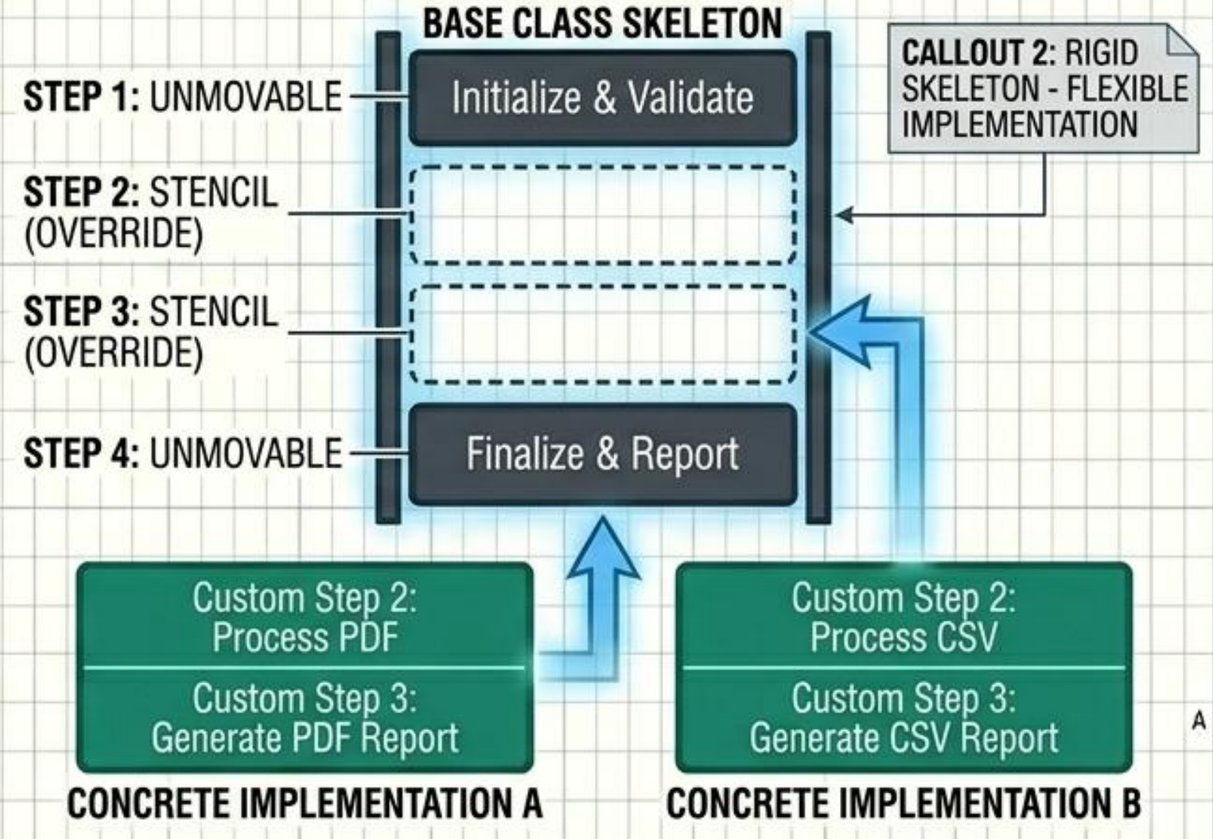
STRUCTURAL DUPLICATION

When several classes share identical overarching steps but differ in details, structural code is duplicated, making systemic updates incredibly error-prone.



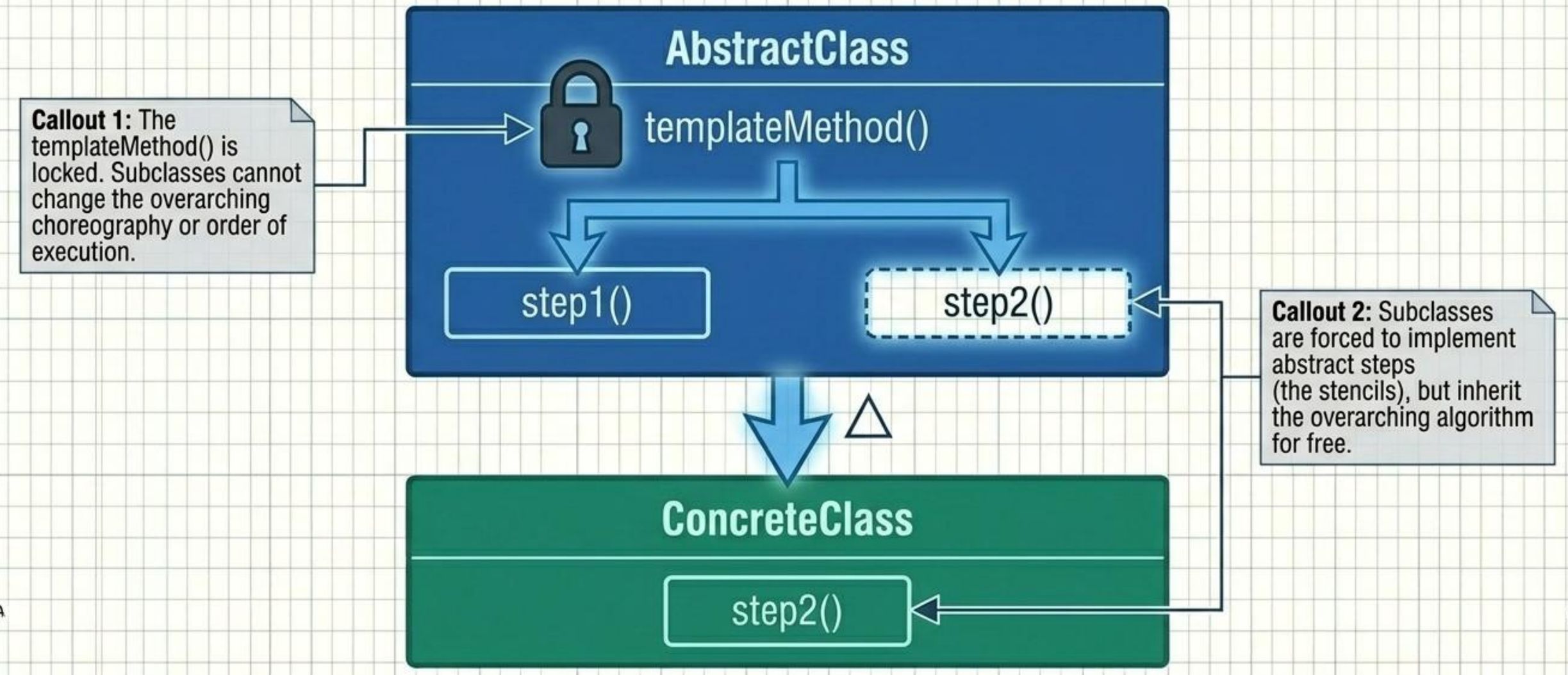
THE SKELETON OUTLINE

Define the skeleton of an algorithm in a base class method. Let subclasses override specific steps to customize behavior while the base class firmly dictates the sequence.

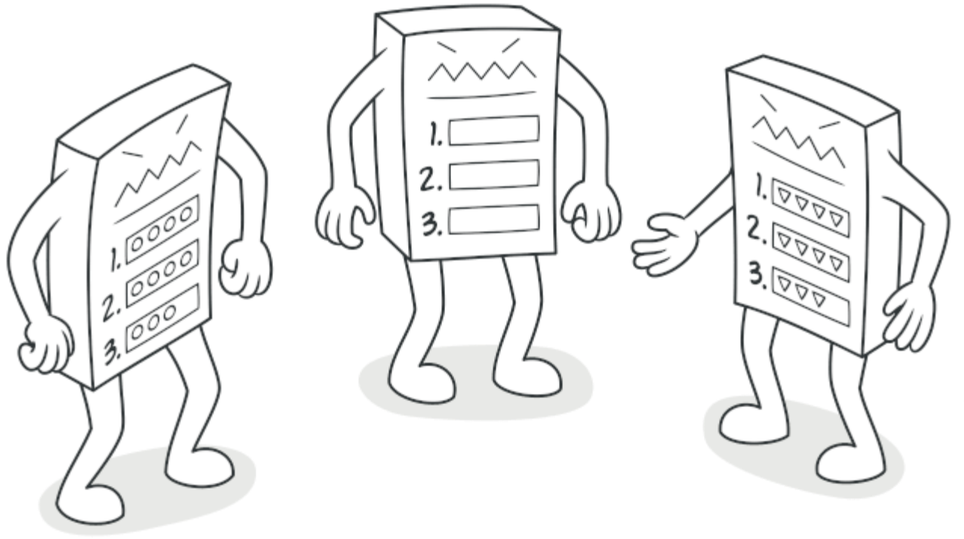


GLOBAL SYSTEMS ENGINEERING	Specification	Specification	Description	Rev.	Reve.
	GEHHA SYSTEMS	Specification: DCPI/BISTAE, MISN and BRVIMC007..43'-2021			1

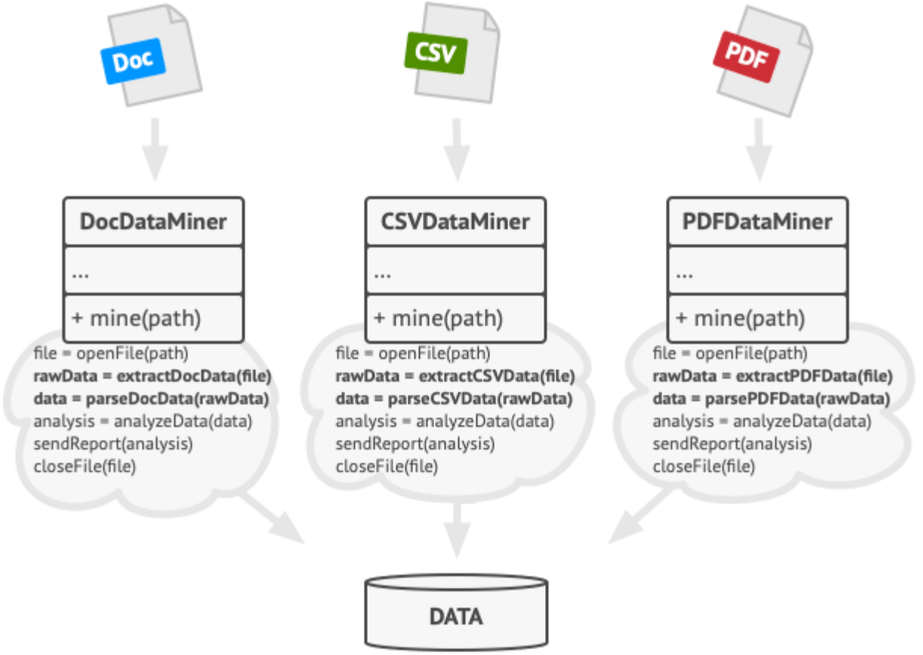
TEMPLATE METHOD // INTERNAL STRUCTURE



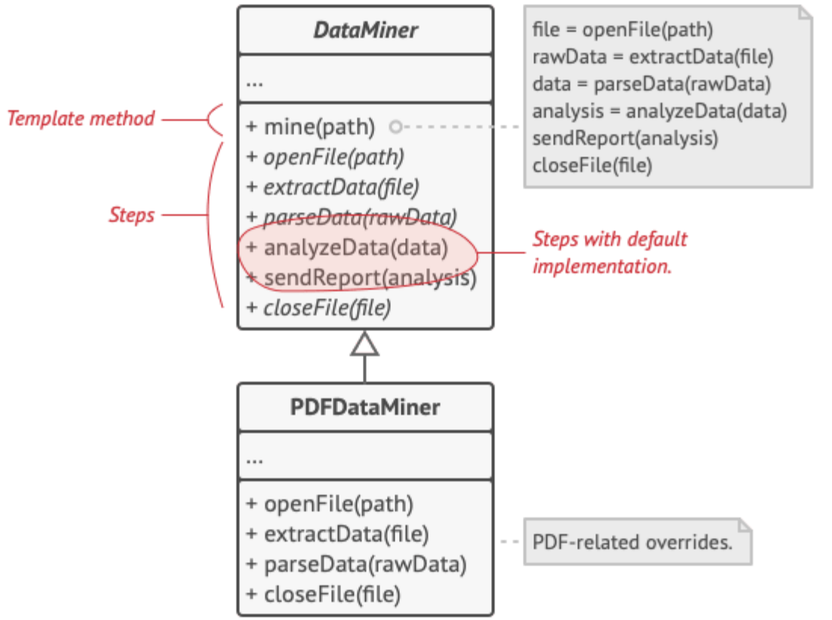
GLOBAL SYSTEMS ENGINEERING	Specification GEHHA SYSTEMS	Specification Specification: DCPI/BISTAE, MISN and BRVIMC007..43'-2021	Description	Rev.	Rave. 1



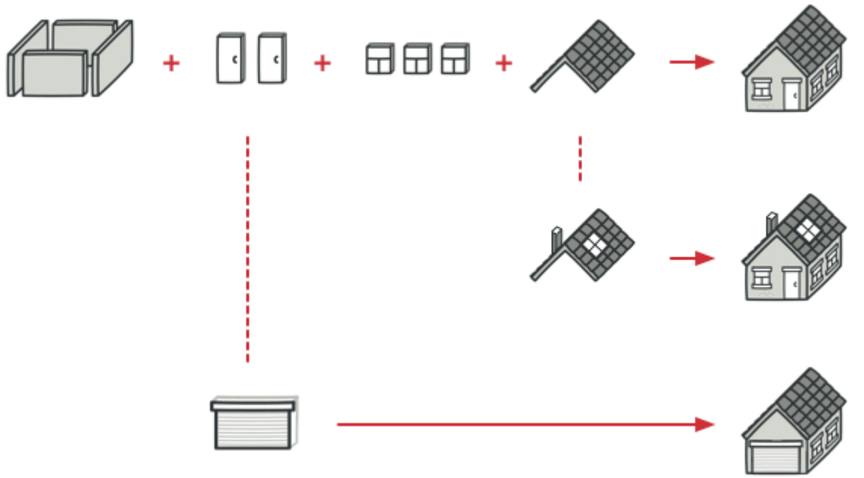
Template Method is a behavioral design pattern that defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure.



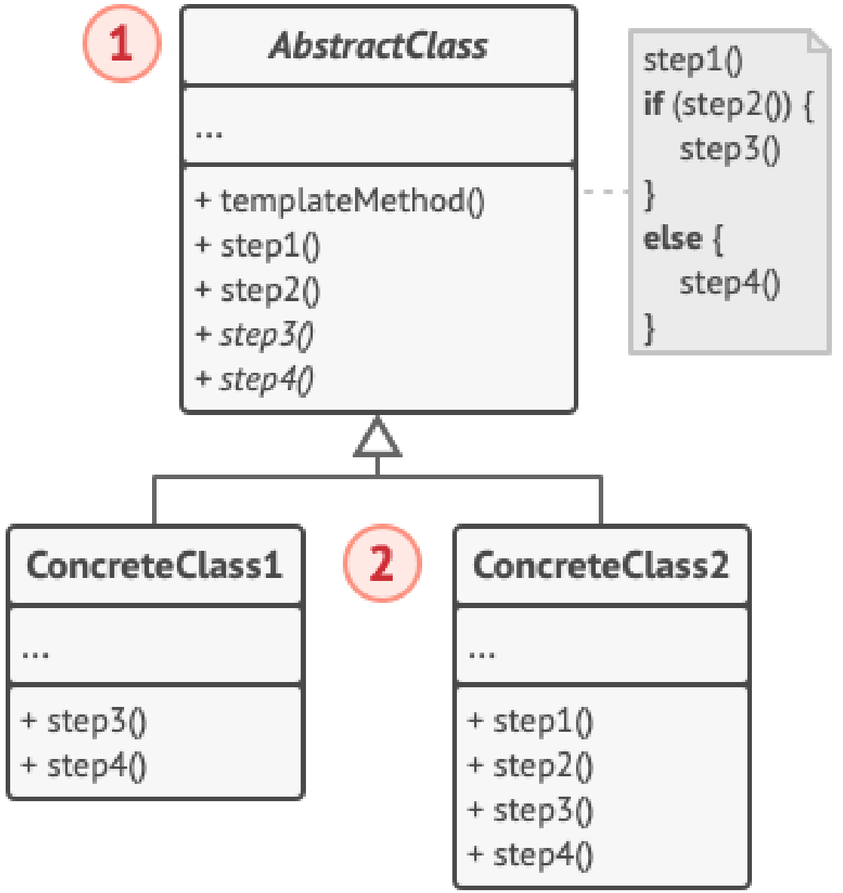
Data mining classes contained a lot of duplicate code.

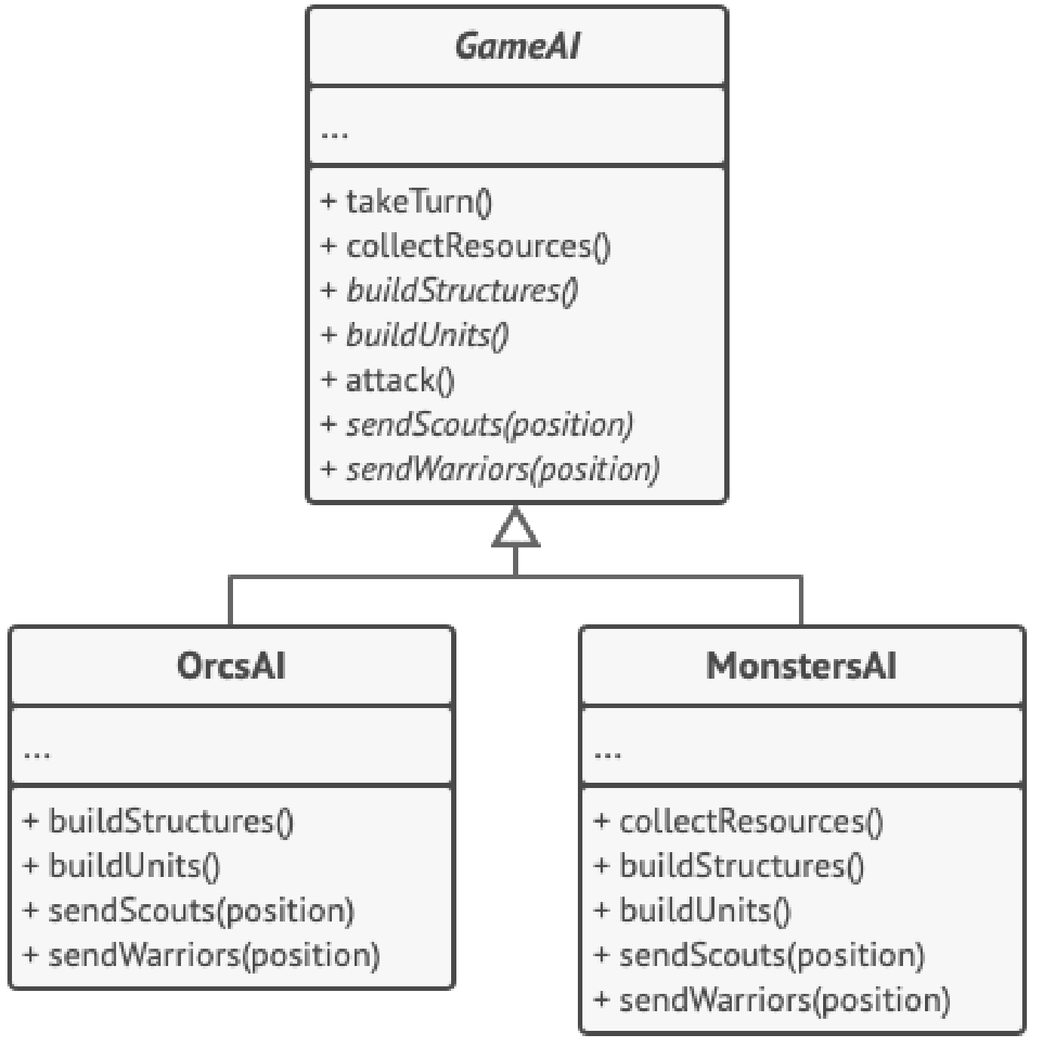


Template method breaks the algorithm into steps, allowing subclasses to override these steps but not the actual method.



A typical architectural plan can be slightly altered to better fit the client's needs.



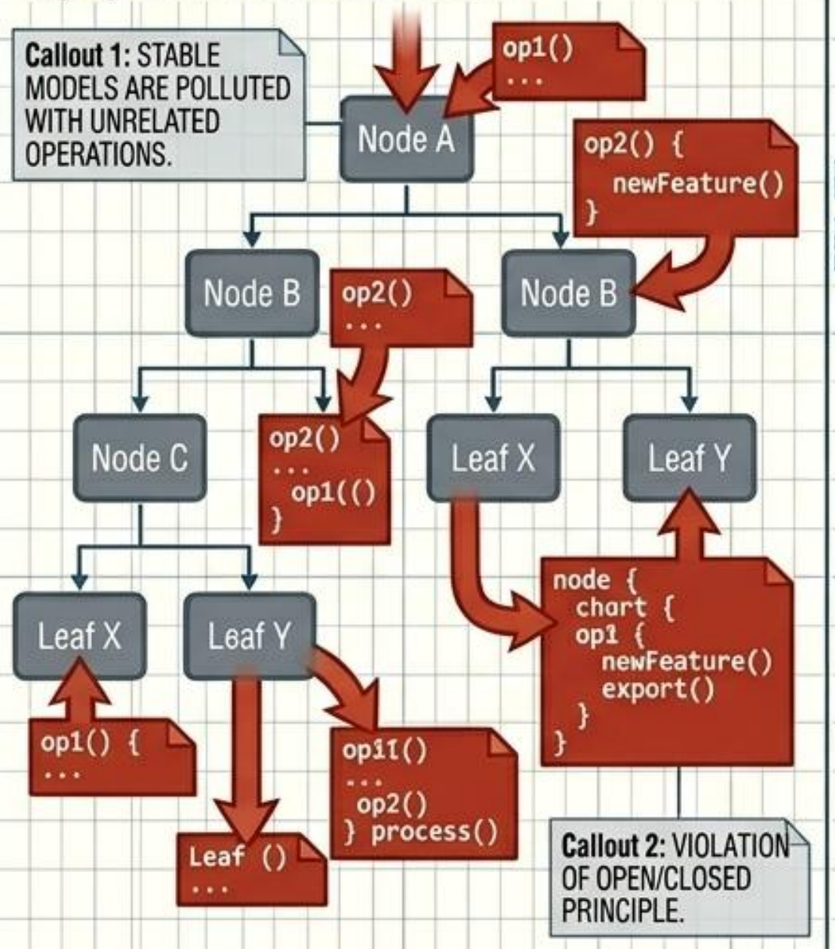


AI classes of a simple video game.

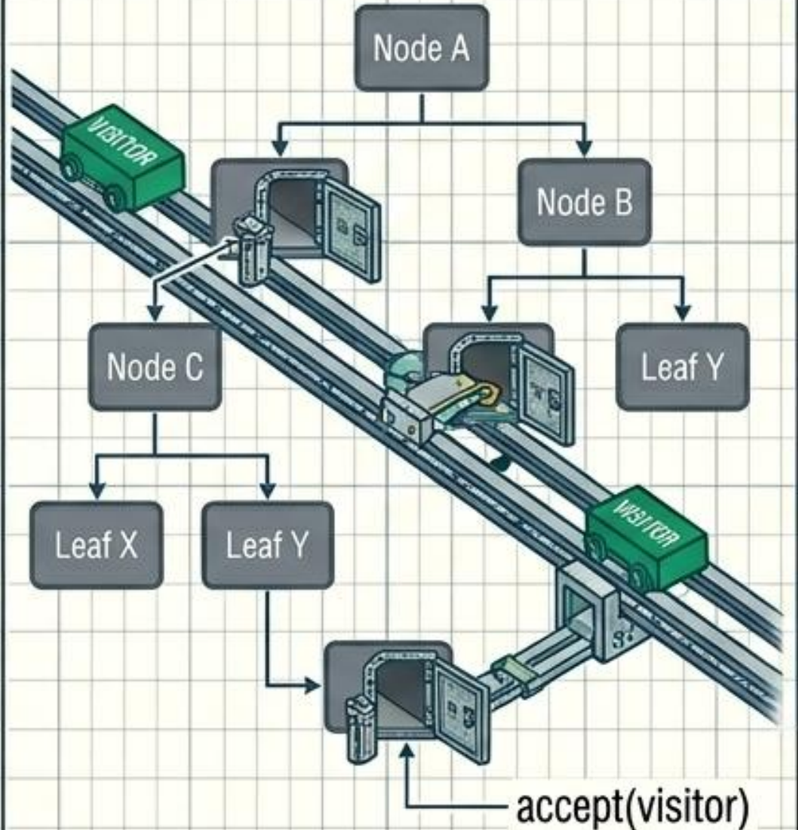
```
// The abstract class defines a template method that contains a
// skeleton of some algorithm composed of calls, usually to
// abstract primitive operations. Concrete subclasses implement
// these operations, but leave the template method itself
// intact.
class GameAI is
    // The template method defines the skeleton of an algorithm.
    method turn() is
        collectResources()
        buildStructures()
        buildUnits()
        attack()

    // Some of the steps may be implemented right in a base
    // class.
    method collectResources() is
        foreach (s in this.builtStructures) do
            s.collect()

    // And some of them may be defined as abstract.
    abstract method buildStructures()
    abstract method buildUnits()

    // A class can have several template methods.
    method attack() is
        enemy = closestEnemy()
        if (enemy == null)
            sendScouts(map.center)
        else
            sendWarriors(enemy.position)

    abstract method sendScouts(position)
    abstract method sendWarriors(position)

// Concrete classes have to implement all abstract operations of
// the base class but they must not override the template method
// itself.
class OrcsAI extends GameAI is
    method buildStructures() is
        if (there are some resources) then
            // Build farms, then barracks, then stronghold.

    method buildUnits() is
        if (there are plenty of resources) then
            if (there are no scouts)
                // Build peon, add it to scouts group.
            else
                // Build grunt, add it to warriors group.

    // ...

    method sendScouts(position) is
        if (scouts.length > 0) then
            // Send scouts to position.

    method sendWarriors(position) is
        if (warriors.length > 5) then
            // Send warriors to position.

// Subclasses can also override some operations with a default
// implementation.
class MonstersAI extends GameAI is
    method collectResources() is
        // Monsters don't collect resources.

    method buildStructures() is
        // Monsters don't build structures.

    method buildUnits() is
        // Monsters don't build units.
```

VISITOR // POLLUTING STRUCTURES VS. DOUBLE DISPATCH ROUTING

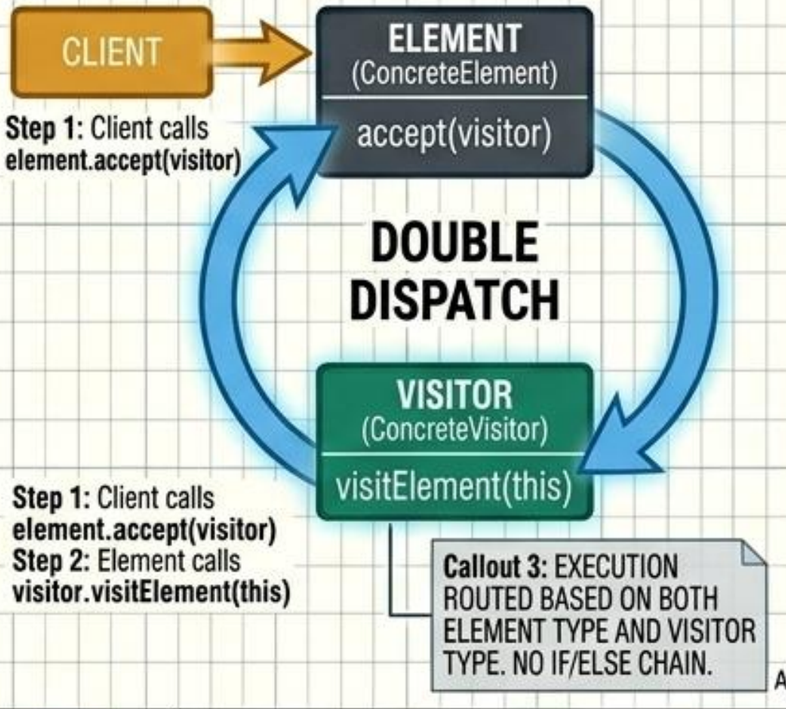
THE PROBLEM: Adding new operations to a complex object structure pollutes highly stable domain models.



THE SOLUTION: Extract the behavior into a Visitor object. Elements implement an accept(visitor) method.



DOUBLE DISPATCH: Step 1: Client routes to the Element. Step 2: Element routes back to the Visitor's specific method, guaranteeing the correct type is processed without massive if/else type-checking.

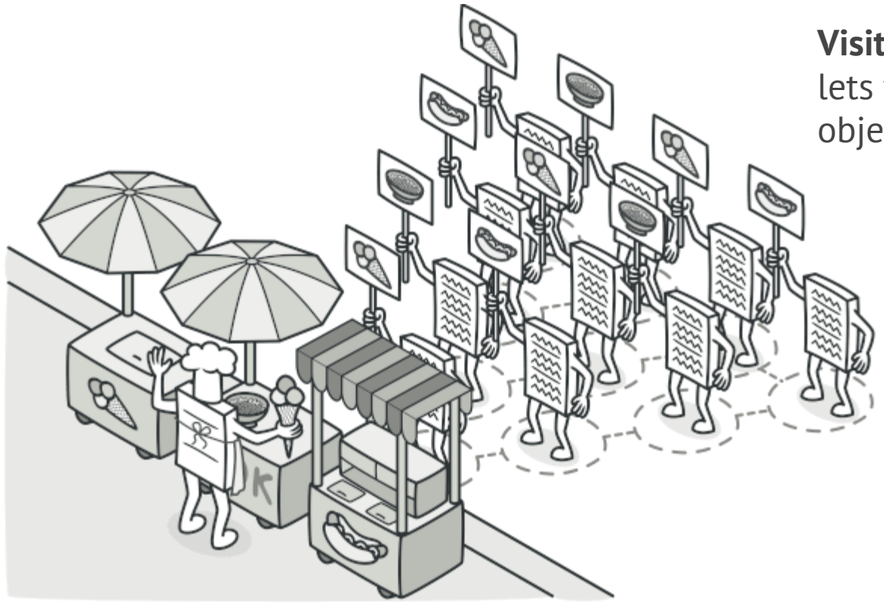


GLOBAL SYSTEMS ENGINEERING

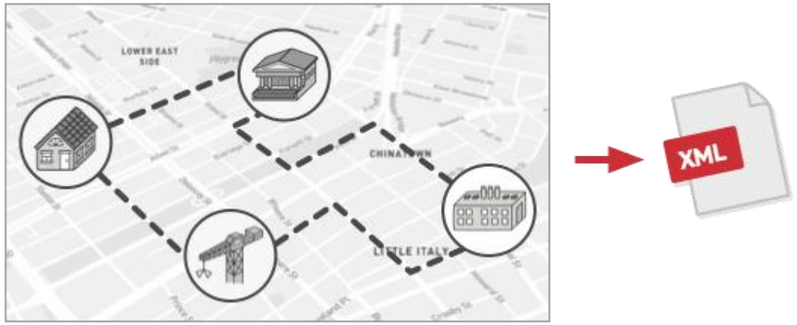
Specification: DESIGN PATTERNS // VISITOR
Specification Number: VSTR/DOUBDISP.REV01.2024

Description:
VISITOR PATTERN AND DOUBLE DISPATCH MECHANISM

Rev.	Page.
A	1



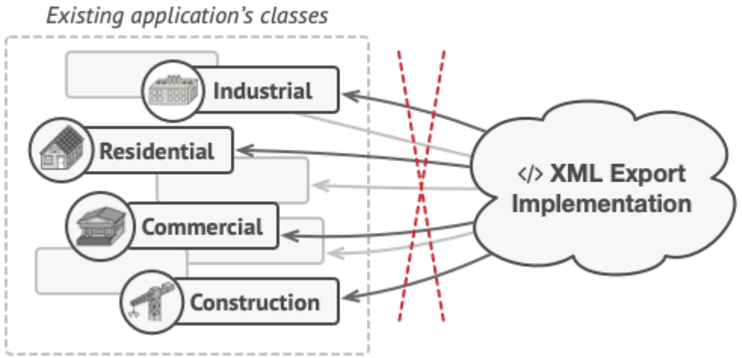
Visitor is a behavioral design pattern that lets you separate algorithms from the objects on which they operate.



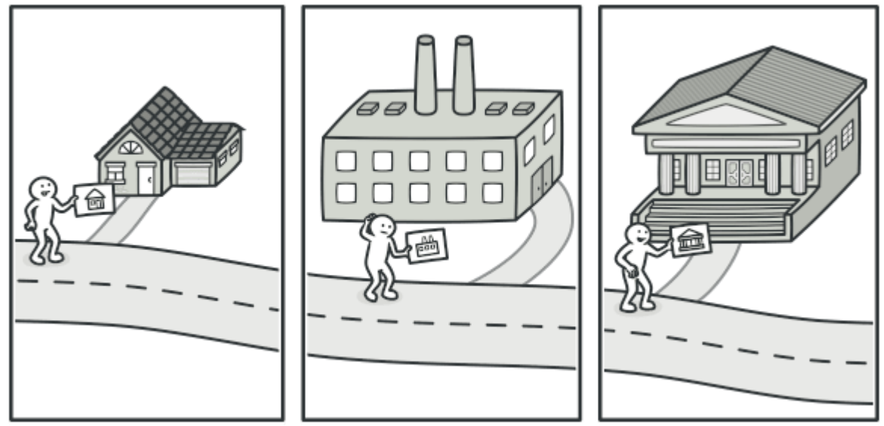
Exporting the graph into XML.

```
class ExportVisitor implements Visitor is
    method doForCity(City c) { ... }
    method doForIndustry(Industry f) { ... }
    method doForSightSeeing(SightSeeing ss) { ... }
    // ...

foreach (Node node in graph)
    if (node instanceof City)
        exportVisitor.doForCity((City) node)
    if (node instanceof Industry)
        exportVisitor.doForIndustry((Industry) node)
    // ...
}
```

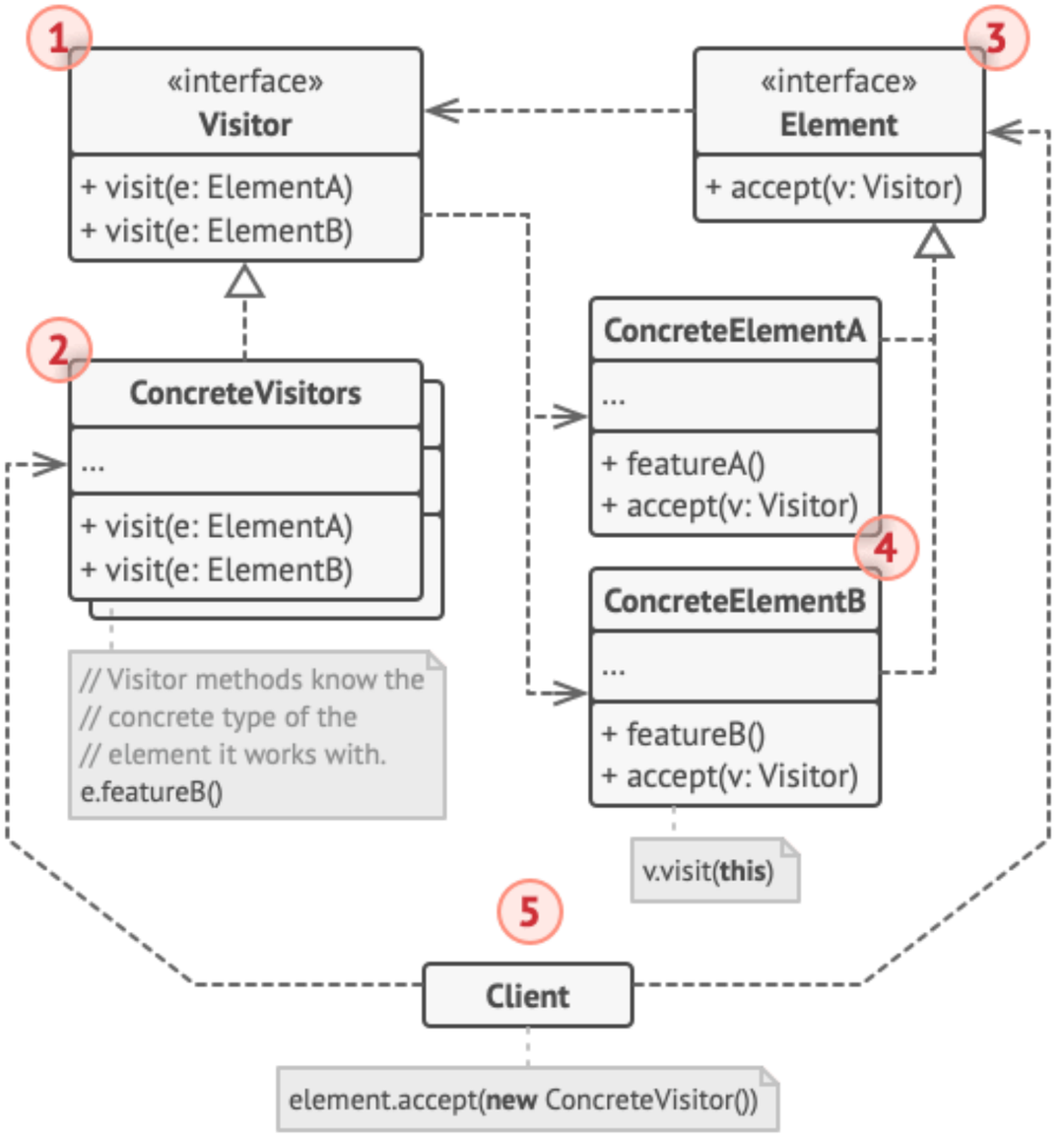
```
// Client code
foreach (Node node in graph)
    node.accept(exportVisitor)

// City
class City is
    method accept(Visitor v) is
        v.doForCity(this)
    // ...

// Industry
class Industry is
    method accept(Visitor v) is
        v.doForIndustry(this)
    // ...
```

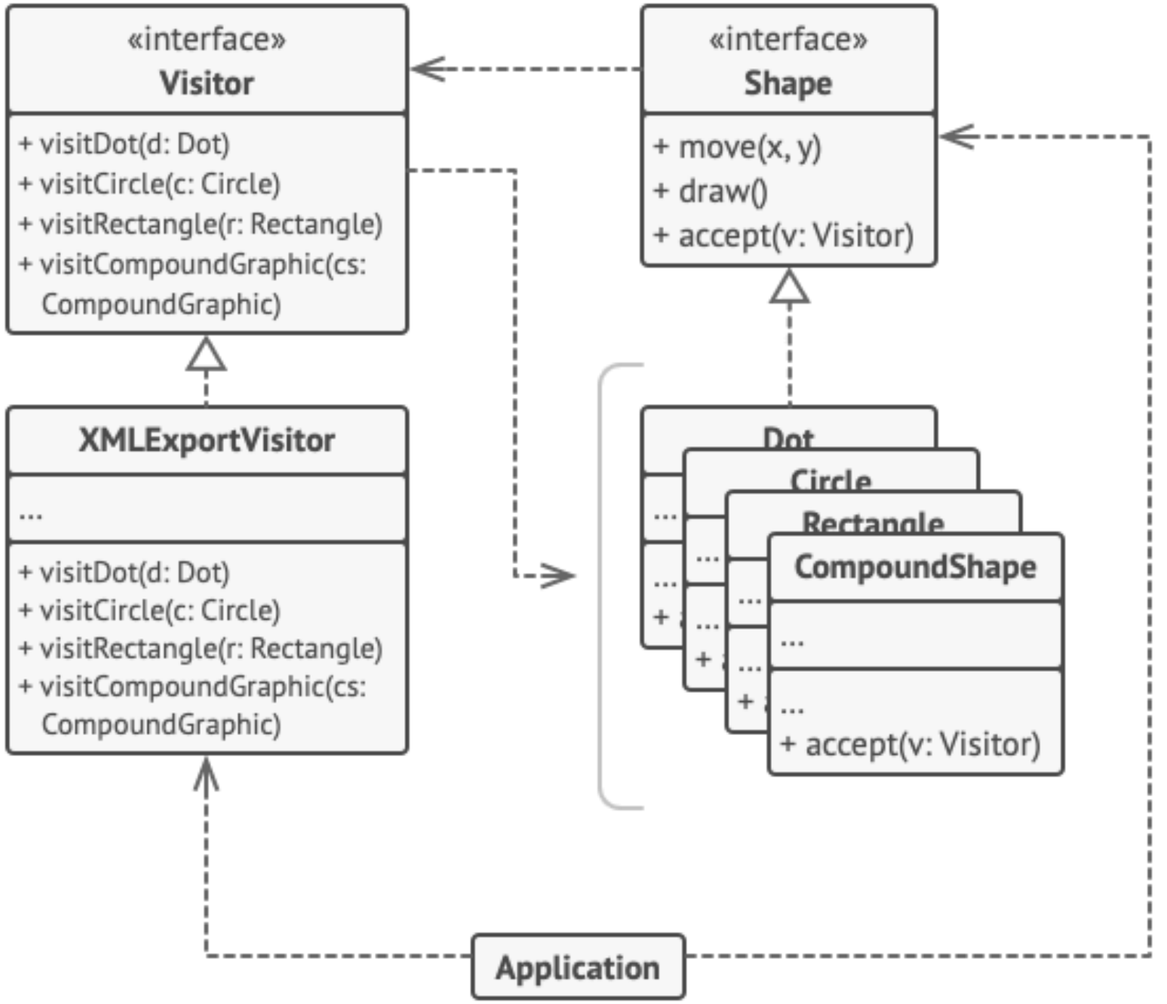


The XML export method had to be added into all node classes, which bore the risk of breaking the whole application if any bugs slipped through along with the change.



A good insurance agent is always ready to offer different policies to various types of organizations.



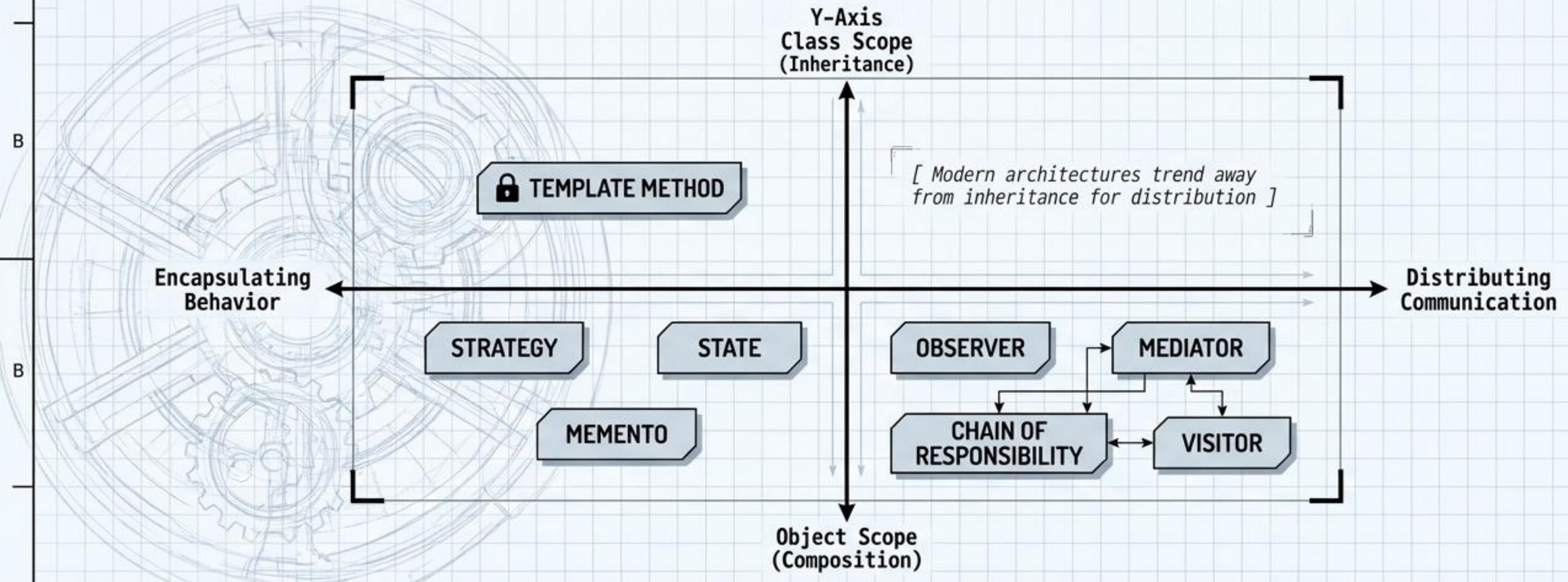


Exporting various types of objects into XML format via a visitor object.

```
// The element interface declares an `accept`
method that takes
// the base visitor interface as an argument.
interface Shape is
    method move(x, y)
    method draw()
    method accept(v: Visitor)

// Each concrete element class must implement
the `accept`
// method in such a way that it calls the
visitor's method that
// corresponds to the element's class.
class Dot implements Shape is
    // ...

    // Note that we're calling `visitDot`,
which matches the
    // current class name. This way we let the
visitor know the
    // class of the element it works with.
    method accept(v: Visitor) is
        v.visitDot(this)

class Circle implements Shape is
    // ...
    method accept(v: Visitor) is
        v.visitCircle(this)

class Rectangle implements Shape is
    // ...
    method accept(v: Visitor) is
        v.visitRectangle(this)

class CompoundShape implements Shape is
    // ...
    method accept(v: Visitor) is
        v.visitCompoundShape(this)


// The Visitor interface declares a set of
visiting methods that
// correspond to element classes. The signature
of a visiting
// method lets the visitor identify the exact
class of the
// element that it's dealing with.
interface Visitor is
    method visitDot(d: Dot)
    method visitCircle(c: Circle)
    method visitRectangle(r: Rectangle)
    method visitCompoundShape(cs:
CompoundShape)

// Concrete visitors implement several versions
of the same
// algorithm, which can work with all concrete
element classes.
//
// You can experience the biggest benefit of
the Visitor pattern
// when using it with a complex object
structure such as a
// Composite tree. In this case, it might be
helpful to store
// some intermediate state of the algorithm
while executing the
// visitor's methods over various objects of
the structure.
class XMLExportVisitor implements Visitor is
    method visitDot(d: Dot) is
        // Export the dot's ID and center
coordinates.
    method visitCircle(c: Circle) is
        // Export the circle's ID, center
coordinates and
        // radius.

    method visitRectangle(r: Rectangle) is
        // Export the rectangle's ID, left-top
coordinates,
        // width and height.

    method visitCompoundShape(cs:
CompoundShape) is
        // Export the shape's ID as well as the
list of its
        // children's IDs.


// The client code can run visitor operations
over any set of
// elements without figuring out their concrete
classes. The
// accept operation directs a call to the
appropriate operation
// in the visitor object.
class Application is
    field allShapes: array of Shapes

    method export() is
        exportVisitor = new XMLExportVisitor()

        foreach (shape in allShapes) do
            shape.accept(exportVisitor)
```

SYNTHESIZING CHOREOGRAPHY // THE BEHAVIORAL DIAGNOSTIC MATRIX

Master this matrix to choose the precise architectural choreography your system requires. While Template Method locks behavior at compile-time via inheritance, most modern solutions leverage object composition for dynamic runtime flexibility.



"Refactoring transforms a mess into clean code and simple design."

— Alexander Shvets, Refactoring.Guru

GLOBAL SYSTEMS ENGINEERING

Specification:
BEHAVIORAL DIAGNOSTIC MATRIX

Specification Number:
BDM/CHOREO.REV03.2024

Rev. A

Page. 1